

```
// === app\src\main\java\com\echoling\app\data\local\db\dao\CourseDao.kt ===
```

```
package com.echoling.app.data.local.db.dao
```

```
import androidx.room.*
import com.echoling.app.data.local.db.entity.CourseEntity
import kotlinx.coroutines.flow.Flow

@Dao
interface CourseDao {
    @Query("SELECT * FROM courses ORDER BY createdAt DESC")
    fun getAllCourses(): Flow<List<CourseEntity>>

    @Query("SELECT * FROM courses WHERE courseId = :courseId")
    suspend fun getCourseById(courseId: String): CourseEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertCourse(course: CourseEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertCourses(courses: List<CourseEntity>)

    @Delete
    suspend fun deleteCourse(course: CourseEntity)

    @Query("DELETE FROM courses WHERE courseId = :courseId")
    suspend fun deleteCourseById(courseId: String)
}
```

```
// === app\src\main\java\com\echoling\app\data\local\db\dao\LearningProgressDao.kt ===
```

```
package com.echoling.app.data.local.db.dao
```

```
import androidx.room.*
import com.echoling.app.data.local.db.entity.LearningProgressEntity
import kotlinx.coroutines.flow.Flow

@Dao
interface LearningProgressDao {
    @Query("SELECT * FROM learning_progress WHERE courseId = :courseId")
    suspend fun getProgressByCourseId(courseId: String): LearningProgressEntity?

    @Query("SELECT * FROM learning_progress ORDER BY lastLearnTime DESC")
    fun getAllProgress(): Flow<List<LearningProgressEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertOrUpdateProgress(progress: LearningProgressEntity)

    @Delete
    suspend fun deleteProgress(progress: LearningProgressEntity)

    @Query("DELETE FROM learning_progress WHERE courseId = :courseId")
    suspend fun deleteProgressByCourseId(courseId: String)
}
```

```
// === app\src\main\java\com\echoling\app\data\local\db\dao\ReciteProgressDao.kt ===
```

```

package com.echoling.app.data.local.db.dao

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import com.echoling.app.data.local.db.entity.ReciteProgressEntity
import kotlinx.coroutines.flow.Flow

/**
 * Persistence boundary for the "记单词" flashcard progress.
 *
 * Operations are split into two surfaces:
 * - [observeAll] — `Flow` that the [ReciteScreen] category picker
 *   collects to render per-card sub-labels (known/unknown counts and
 *   "上次学习于..."). One snapshot per row, so the picker refreshes
 *   instantly when the user finishes a card on a study sub-page.
 * - [getByCategory] / [upsert] — suspend functions used by
 *   [CategoryStudyViewModel] to (a) seed its in-memory state on
 *   entry and (b) persist every `markKnown` / `markUnknown` /
 *   `saveCurrentToVocabulary` action.
 *
 * Conflict strategy on [upsert] is `REPLACE` so the same row can be
 * re-written with monotonically increasing `currentIndex` /
 * `knownCount` / `unknownCount` / `lastStudiedAt` without first
 * checking existence — keeps the VM mutation path single-call.
 */
@Dao
interface ReciteProgressDao {

    @Query("SELECT * FROM recite_progress")
    fun observeAll(): Flow<List<ReciteProgressEntity>>

    @Query("SELECT * FROM recite_progress WHERE category_id = :categoryId LIMIT 1")
    suspend fun getByCategory(categoryId: String): ReciteProgressEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsert(entity: ReciteProgressEntity)
}

// === app\src\main\java\com\echoling\app\data\local\db\dao\SentenceDao.kt ===

package com.echoling.app.data.local.db.dao

import androidx.room.*
import com.echoling.app.data.local.db.entity.SentenceEntity
import kotlinx.coroutines.flow.Flow

@Dao
interface SentenceDao {

    @Query("SELECT * FROM sentences WHERE courseId = :courseId ORDER BY sentenceId ASC")
    fun getSentencesByCourseId(courseId: String): Flow<List<SentenceEntity>>

    @Query("SELECT * FROM sentences WHERE courseId = :courseId ORDER BY sentenceId ASC")
    suspend fun getSentencesByCourseIdSync(courseId: String): List<SentenceEntity>

    @Query("SELECT * FROM sentences WHERE courseId = :courseId AND sentenceId = :sentenceId")
    suspend fun getSentence(courseId: String, sentenceId: Int): SentenceEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertSentences(sentences: List<SentenceEntity>)
}

```

```

@Insert(onConflict = OnConflictStrategy.IGNORE)
suspend fun insertSentencesIfNotExists(sentences: List<SentenceEntity>)

@Update
suspend fun updateSentence(sentence: SentenceEntity)

@Query("DELETE FROM sentences WHERE courseId = :courseId")
suspend fun deleteSentencesByCourseId(courseId: String)

@Query("UPDATE sentences SET isCompleted = :isCompleted WHERE courseId = :courseId AND sentenceId = :sentenceId"
)
suspend fun updateCompletedStatus(courseId: String, sentenceId: Int, isCompleted: Boolean)

@Query("UPDATE sentences SET isTested = :isTested WHERE courseId = :courseId AND sentenceId = :sentenceId")
suspend fun updateTestedStatus(courseId: String, sentenceId: Int, isTested: Boolean)

}

```

```
// === app\src\main\java\com\echoling\app\data\local\db\dao\WordDao.kt ===
```

```
package com.echoling.app.data.local.db.dao
```

```
import androidx.room.*
```

```
import com.echoling.app.data.local.db.entity.WordEntity
```

```
import kotlinx.coroutines.flow.Flow
```

```
@Dao
```

```
interface WordDao {
```

```
    @Query("SELECT * FROM words ORDER BY collectedAt DESC")
```

```
    fun getAllWords(): Flow<List<WordEntity>>
```

```
    @Query("SELECT * FROM words WHERE isMastered = 0 ORDER BY collectedAt DESC")
```

```
    fun getUnmasteredWords(): Flow<List<WordEntity>>
```

```
    @Query("SELECT * FROM words WHERE word = :word")
```

```
    suspend fun getWord(word: String): WordEntity?
```

```
    @Insert(onConflict = OnConflictStrategy.REPLACE)
```

```
    suspend fun insertWord(word: WordEntity)
```

```
    @Update
```

```
    suspend fun updateWord(word: WordEntity)
```

```
    @Delete
```

```
    suspend fun deleteWord(word: WordEntity)
```

```
    @Query("DELETE FROM words WHERE word = :word")
```

```
    suspend fun deleteWordByText(word: String)
```

```
}
```

```
// === app\src\main\java\com\echoling\app\data\local\db\EchoLingDatabase.kt ===
```

```
package com.echoling.app.data.local.db
```

```
import androidx.room.Database
```

```
import androidx.room.RoomDatabase
```

```
import com.echoling.app.data.local.db.dao.CourseDao
```

```
import com.echoling.app.data.local.db.dao.LearningProgressDao
```

```

import com.echoling.app.data.local.db.dao.ReciteProgressDao
import com.echoling.app.data.local.db.dao.SentenceDao
import com.echoling.app.data.local.db.dao.WordDao
import com.echoling.app.data.local.db.entity.CourseEntity
import com.echoling.app.data.local.db.entity.LearningProgressEntity
import com.echoling.app.data.local.db.entity.ReciteProgressEntity
import com.echoling.app.data.local.db.entity.SentenceEntity
import com.echoling.app.data.local.db.entity.WordEntity

/**
 * Schema version history:
 * - v1 → v2: per CLAUDE.md baseline
 * - v2 → v3: per CLAUDE.md baseline
 * - v3 → v4: added [WordEntity.pos] so the vocabulary book can render
 *   the part-of-speech chip next to the translation. Schema change
 *   drops the existing user-saved words (per CLAUDE.md §9.2,
 *   `fallbackToDestructiveMigration()` is the current fallback).
 * - v4 → v5: added [ReciteProgressEntity] so the "记单词" flashcard
 *   study screens can persist per-category `currentIndex` /
 *   `knownCount` / `unknownCount` / `lastStudiedAt` across process
 *   death, app cold-starts, and tab switches — without this the
 *   user re-enters each category from card 0 every time. The new
 *   table is empty on first install (no data to migrate).
 */
@Database(
    entities = [
        CourseEntity::class,
        SentenceEntity::class,
        LearningProgressEntity::class,
        WordEntity::class,
        ReciteProgressEntity::class,
    ],
    version = 5,
    exportSchema = false
)
abstract class EchoLingDatabase : RoomDatabase() {
    abstract fun courseDao(): CourseDao
    abstract fun sentenceDao(): SentenceDao
    abstract fun learningProgressDao(): LearningProgressDao
    abstract fun wordDao(): WordDao
    abstract fun reciteProgressDao(): ReciteProgressDao
}

// == app\src\main\java\com\echoling\app\data\local\db\entity\CourseEntity.kt ==

package com.echoling.app.data.local.db.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

/**
 * Room row for [com.echoling.app.domain.model.Course]. The `courseName`
 * column was added in schema version 3 — see `Migrations.kt` /
 * `MIGRATION_2_3`. Default empty string preserves the migration path for
 * pre-existing rows; the app resolves the effective group name at read
 * time via `Course.effectiveCourseName`.
 */
@Entity(tableName = "courses")
data class CourseEntity(
    @PrimaryKey
    val courseId: String,

```

```

        val courseName: String = "",
        val title: String,
        val description: String,
        val difficulty: String,
        val audioUri: String?,
        val videoUri: String?,
        val subtitleUri: String?,
        val durationMs: Long,
        val totalSentences: Int,
        val thumbnailUri: String?,
        val createdAt: Long,
        val updatedAt: Long
    )

// === app\src\main\java\com\echoling\app\data\local\db\entity\LearningProgressEntity.kt ===

package com.echoling.app.data.local.db.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "learning_progress")
data class LearningProgressEntity(
    @PrimaryKey
    val courseId: String,
    val currentPositionMs: Long,
    val currentSentenceId: Int,
    val learnedSentences: Int,
    val totalLearnTimeMs: Long,
    val lastLearnTime: Long,
    val finishRate: Float
)

// === app\src\main\java\com\echoling\app\data\local\db\entity\ReciteProgressEntity.kt ===

package com.echoling.app.data.local.db.entity

import androidx.room.ColumnInfo
import androidx.room.Entity
import androidx.room.PrimaryKey

/**
 * Per-category progress in the "记单词" flashcard study screens.
 *
 * One row per vocabulary category, keyed by its manifest slug
 * (`junior` / `senior` / `cet4` / `cet6` / `toefl`). Holds only the
 * minimal state needed to resume a study session — the actual word
 * list lives in the bundled `assets/vocab_*.json` files, so this
 * table is tiny and self-contained.
 * self-contained.
 *
 * Persisted fields:
 * - [currentIndex] — 0-based index of the card the user was last
 *   looking at. `-1` means "no progress yet, start from the first
 *   card".
 * - [knownCount] / [unknownCount] — running tallies. Reset to 0 when
 *   the user taps the refresh icon (see [CategoryStudyViewModel.resetSession]).
 * - [lastStudiedAt] — epoch ms of the last `markKnown` /
 *   `markUnknown` / `saveCurrentToVocabulary` action. Drives the

```

```

* "上次学习于..." sub-label on the category picker card so the user
* can see at a glance which categories they've touched recently.
*
* The picker screen on the "记单词" tab reads this table to render
* the per-card progress sub-line. The per-category study screen
* reads it on `load()` to seed its UI state and writes it on every
* score action.
*
* Schema note: introduced at DB v5 (after the v4 `WordEntity.pos`
* bump). `fallbackToDestructiveMigration()` means a fresh install on
* v5 has an empty table — users start from `currentIndex = -1`,
* `knownCount = 0`, `unknownCount = 0`, which is the expected
* "first-time" state.
*/
@Entity(tableName = "recite_progress")
data class ReciteProgressEntity(
    @PrimaryKey
    @ColumnInfo(name = "category_id")
    val categoryId: String,
    @ColumnInfo(name = "current_index")
    val currentIndex: Int = -1,
    @ColumnInfo(name = "known_count")
    val knownCount: Int = 0,
    @ColumnInfo(name = "unknown_count")
    val unknownCount: Int = 0,
    @ColumnInfo(name = "last_studied_at")
    val lastStudiedAt: Long = 0L,
)

// === app\src\main\java\com\echoling\app\data\local\db\entity\SentenceEntity.kt ===

package com.echoling.app.data.local.db.entity

import androidx.room.Entity

@Entity(
    tableName = "sentences",
    primaryKeys = ["courseId", "sentenceId"]
)
data class SentenceEntity(
    val courseId: String,
    val sentenceId: Int,
    val contentEn: String,
    val contentCn: String,
    val startTimeMs: Long,
    val endTimeMs: Long,
    val isLearned: Boolean = false,
    val isRead: Boolean = false,
    val readScore: Int = 0,
    val isCompleted: Boolean = false,
    val isTested: Boolean = false
)

// === app\src\main\java\com\echoling\app\data\local\db\entity\WordEntity.kt ===

package com.echoling.app.data.local.db.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

```

```
@Entity(tableName = "words")
data class WordEntity(
    @PrimaryKey
    val word: String,
    val phonetic: String,
    val pos: String,
    val translation: String,
    val exampleSentence: String,
    val sourceCourseId: String,
    val sourceSentenceId: Int,
    val isMastered: Boolean = false,
    val collectedAt: Long,
    val reviewCount: Int = 0,
    val nextReviewTime: Long,
)

// === app\src\main\java\com\echoling\app\data\local\db\Migrations.kt ===

package com.echoling.app.data.local.db

import androidx.room.migration.Migration
import androidx.sqlite.db.SupportSQLiteDatabase

/**
 * v2 → v3: introduce the `courseName` column on the `courses` table to
 * support grouped / folder-style browsing on the Courses tab.
 *
 * Old rows are backfilled with the empty string — at read time the app
 * resolves the effective group name via `Course.effectiveCourseName`,
 * which falls back to the lesson `title` for blank values, so legacy
 * courses surface as their own one-entry folders under their own title
 * with no data loss.
 */
val MIGRATION_2_3: Migration = object : Migration(2, 3) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL(
            "ALTER TABLE courses ADD COLUMN courseName TEXT NOT NULL DEFAULT ''"
        )
    }
}

// === app\src\main\java\com\echoling\app\data\repository\CourseRepositoryImpl.kt ===

package com.echoling.app.data.repository

import com.echoling.app.data.local.db.dao.CourseDao
import com.echoling.app.data.local.db.entity.CourseEntity
import com.echoling.app.domain.model.Course
import com.echoling.app.domain.repository.CourseRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class CourseRepositoryImpl @Inject constructor(
    private val courseDao: CourseDao
) : CourseRepository {
```

```

    override fun getAllCourses(): Flow<List<Course>> {
        return courseDao.getAllCourses().map { entities ->
            entities.map { it.toDomain() }
        }
    }

    override suspend fun getCourseById(courseId: String): Course? {
        return courseDao.getCourseById(courseId)?.toDomain()
    }

    override suspend fun insertCourse(course: Course) {
        courseDao.insertCourse(course.toEntity())
    }

    override suspend fun deleteCourse(courseId: String) {
        courseDao.deleteCourseById(courseId)
    }

    private fun CourseEntity.toDomain(): Course = Course(
        courseId = courseId,
        courseName = courseName,
        title = title,
        description = description,
        difficulty = difficulty,
        audioUri = audioUri,
        videoUri = videoUri,
        subtitleUri = subtitleUri,
        durationMs = durationMs,
        totalSentences = totalSentences,
        thumbnailUri = thumbnailUri,
        createdAt = createdAt,
        updatedAt = updatedAt
    )

    private fun Course.toEntity(): CourseEntity = CourseEntity(
        courseId = courseId,
        courseName = courseName,
        title = title,
        description = description,
        difficulty = difficulty,
        audioUri = audioUri,
        videoUri = videoUri,
        subtitleUri = subtitleUri,
        durationMs = durationMs,
        totalSentences = totalSentences,
        thumbnailUri = thumbnailUri,
        createdAt = createdAt,
        updatedAt = updatedAt
    )
}

// === app\src\main\java\com\echoling\app\data\repository\DictionaryRepositoryImpl.kt ===

package com.echoling.app.data.repository

import android.content.Context
import android.util.Log
import com.echoling.app.domain.model.DictCategory
import com.echoling.app.domain.model.DictEntry
import com.echoling.app.domain.repository.DictionaryRepository

```



```

import com.google.gson.Gson
import com.google.gson.annotations.SerializedName
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlin.coroutines.Dispatchers
import kotlin.coroutines.sync.Mutex
import kotlin.coroutines.sync.withLock
import kotlin.coroutines.withContext
import javax.inject.Inject
import javax.inject.Singleton

/**
 * Loads the bundled vocabulary manifest (`assets/vocab_manifest.json`)
 * and one entries-JSON per category on first use.
 *
 *
 * **Two-phase lazy loading** (2026-07-04 refactor — replaced the
 * monolithic first-call-load to fix the slow first-open of the
 * 记单词 tab):
 *
 *
 * - **Phase 1 — manifest only**: [categories] parses the ~1.3 KB
 *   `vocab_manifest.json` and returns the lightweight
 *   `List<DictCategory>` with the `size` field populated from the
 *   manifest. The picker UI on the 记单词 tab uses this to render
 *   all 11 category cards without ever opening an entries file.
 *
 *
 * - **Phase 2 — entries (per-category)**: [wordsInCategory] lazily
 *   loads the requested category's JSON only, schema-sniffing on the
 *   first line so we never re-read the whole file. Each category is
 *   parsed at most once and cached in [entriesByCategory].
 *
 *
 * - **Phase 3 — full lookup (cross-category merge)**: [lookup] (and
 *   [allWords]) need every category's entries merged into one
 *   lowercased `HashMap`. The first call pays the 12 MB parse cost;
 *   subsequent calls are O(1). The full lookup is built by
 *   [buildLookup] which also fills [entriesByCategory] as a side
 *   effect, so navigating into a category after Phase 3 has run is
 *   instant.
 *
 *
 * - **Warmup**: [warmupAll] is called from `ReciteViewModel` right
 *   after Phase 1 completes. It kicks off Phase 3 on a background
 *   coroutine so the user, who is currently reading the picker, has
 *   the lookup pre-built by the time they long-press a word in the
 *   practice screen or tap into a category.
 *
 *
 * **Filtering**: entries with `word.trim().length <= 1` (e.g. "a", "I")
 * and entries with empty translations are dropped at load time. They
 * are never seen by callers — saves the practice screen from surfacing
 * one-letter "words" as translatable tokens and saves the flashcard
 * study screen from rendering them. Note: the manifest's `size` field
 * reflects the **post-prune** count from `scripts/build_vocab_assets.py`,
 * which doesn't apply the `word.length > 1` filter, so the picker
 * chip is at most 1-3 entries higher than the actual study screen
 * count for any given category. This matches the historic behavior.
 *
 *
 * **Cross-category merging**: when the same lowercased word is present
 * in two or more categories with **different** translations, the
 * [lookup] path returns a single merged entry whose `translation`
 * concatenates the distinct glosses with "; " so the user sees all
 * variants in one place (e.g. CET-6 "出现; 浮现" + TOEFL "浮现; 显露"
 * → "出现; 浮现; 显露" — "浮现" is deduped at the fragment level, not
 * just whole-string level; see [joinDistinctGlosses]). Per-category
 * entries are not merged across categories — they keep the
 * category-specific translation so studying 高中 vs CET-6 still shows
 * each category's own gloss.

```

```

*
* **Within-category merging**: a single source record can list multiple
* POS groups in `content.word.content.trans[]` (e.g. "concordance"
* has both an "n." gloss and a "vt." gloss). Each gloss is concatenated
* into the entry's `translation` with "; " — the same separator used by
* the cross-category merge — so the flashcard back face shows every
* POS's translation instead of just the first one.
*
* **Concurrency**: three separate `Mutex`es (one per cache) so the
* phases don't block each other. Phase 1 (`categories`) is independent
* of Phase 2 (`wordsInCategory`) and Phase 3 (`lookup`); they only
* interact via [entriesByCategory] which is itself mutex-guarded.
* Lock acquisition order is always `manifestMutex` → `entriesMutex`
* → `lookupMutex` (the last is only acquired from [ensureLookupLoaded]),
* so there are no deadlock cycles.
*/
@Singleton
class DictionaryRepositoryImpl @Inject constructor(
    @ApplicationContext private val context: Context,
) : DictionaryRepository {

    // Phase 1 cache — manifest-derived lightweight list. Populated by
    // [loadManifest] on first call to [categories]. Never invalidated.
    @Volatile
    private var categoriesManifest: List<DictCategory>? = null

    // Parallel map from category id → entries-asset filename. Kept
    // private because [DictCategory] deliberately doesn't expose
    // `asset` to the domain layer (the picker UI only needs id / name
    // / description / size). Used by [wordsInCategory] and
    // [buildLookup] to locate the JSON without re-reading the manifest.
    @Volatile
    private var assetByCategory: Map<String, String>? = null

    // Phase 2 cache — per-category entries. Populated lazily by
    // [wordsInCategory] (one category at a time) or wholesale by
    // [buildLookup] when Phase 3 needs all of them.
    @Volatile
    private var entriesByCategory: Map<String, List<DictEntry>>? = null

    // Phase 3 cache — merged lowercased-key map for [lookup]. Built
    // once by [buildLookup]; the [allWords] snapshot is derived from it.
    @Volatile
    private var lookup: Map<String, DictEntry>? = null
    @Volatile
    private var allMerged: List<DictEntry>? = null

    private val manifestMutex = Mutex()
    private val entriesMutex = Mutex()
    private val lookupMutex = Mutex()
    private val gson = Gson()

    override suspend fun lookup(word: String): DictEntry? {
        val key = word.trim().lowercase()
        if (key.isEmpty()) return null
        val map = ensureLookupLoaded()
        val hit = map[key]
        Log.d(TAG, "lookup('$word') -> ${if (hit != null) "HIT" else "MISS"}")
        return hit
    }

    override suspend fun allWords(): List<DictEntry> {
        ensureLookupLoaded()
    }

```

```

        return allMerged ?: emptyList()
    }

    override suspend fun categories(): List<DictCategory> {
        val cached = categoriesManifest
        if (cached != null) return cached
        return withContext(Dispatchers.IO) {
            manifestMutex.withLock {
                categoriesManifest ?: loadManifest()
                .also { (cats, assets) ->
                    categoriesManifest = cats
                    assetByCategory = assets
                }
                .first
            }
        }
    }

    override suspend fun wordsInCategory(categoryId: String): List<DictEntry> {
        // Fast path — Phase 3 already populated `entriesByCategory`.
        // After a warmup this is the common case (user reads the
        // picker, warmup builds the full map, user taps a card).
        entriesByCategory?.let { return it[categoryId] ?: emptyList() }
        // Phase 1 must run first so we have the asset filename; this
        // is also a no-op after the first call because `categories`
        // returns from the cached `categoriesManifest` field.
        val assetMap = assetByCategory ?: run {
            categories()
            assetByCategory ?: return emptyList()
        }
        val asset = assetMap[categoryId] ?: return emptyList()
        // Slow path — load just this one category and cache it. If
        // Phase 3 kicks off in parallel it will see our entry and
        // only fill in the missing categories, not re-read this one.
        return withContext(Dispatchers.IO) {
            entriesMutex.withLock {
                val current = entriesByCategory
                if (current != null) {
                    current[categoryId] ?: emptyList()
                } else {
                    val entries = loadCategoryEntries(asset)
                    entriesByCategory = mapOf(categoryId to entries)
                    Log.d(TAG, "Loaded ${entries.size} entries from assets/$asset")
                    entries
                }
            }
        }
    }

    /**
     * Pre-warm the full lookup map in the background. Called by
     * [com.echoling.app.presentation.ui.screens.recite.ReciteViewModel]
     * right after Phase 1's `categories()` returns — the user is
     * staring at the picker reading "N 词" chips, so the heavy work
     * is invisible. By the time they tap into a category (or long-
     * press a word in the practice screen), [lookup] /
     * [wordsInCategory] usually hit the cache and return instantly.
     *
     * Idempotent: a second call while the first is still in flight
     * will block on [lookupMutex] (the inner `cache`?: buildLookup()
     * pattern) until the in-flight build finishes, then return the
     * populated map. Safe to call from any coroutine context.
     */

```

```

    override suspend fun warmupAll() {
        ensureLookupLoaded()
    }

private suspend fun ensureLookupLoaded(): Map<String, DictEntry> {
    val cached = lookup
    if (cached != null) return cached
    return withContext(Dispatchers.IO) {
        lookupMutex.withLock {
            lookup ?: buildLookup().also { map ->
                lookup = map
                allMerged = map.values.sortedBy { it.word.lowercase() }
            }
        }
    }
}

private fun loadManifest(): Pair<List<DictCategory>, Map<String, String>> {
    val manifest: Manifest = try {
        context.assets.open(MANIFEST_ASSET)
            .bufferedReader()
            .use { reader -> gson.fromJson(reader, Manifest::class.java) }
    } catch (e: Exception) {
        Log.e(TAG, "Failed to parse $MANIFEST_ASSET — aborting load", e)
        return emptyList<DictCategory>() to emptyMap()
    }
    val categories = manifest.categories.map { meta ->
        DictCategory(
            id = meta.id,
            name = meta.name,
            description = meta.description,
            size = meta.size,
        )
    }
    val assets = manifest.categories.associate { it.id to it.asset }
    return categories to assets
}

/**
 * Parse every category's entries JSON, build the merged lookup
 * map, and as a side effect publish the per-category cache so
 * [wordsInCategory] can short-circuit once this returns.
 *
 * If [entriesByCategory] is already partially populated (e.g.
 * the user opened a category before [warmupAll] finished), we
 * load only the missing categories — never re-parses a file
 * that's already in the cache.
 */
private suspend fun buildLookup(): Map<String, DictEntry> {
    // Phase 1 may not have run yet — `categories()` is idempotent
    // and populates both `categoriesManifest` and `assetByCategory`.
    val assetMap = assetByCategory ?: run {
        categories()
        assetByCategory ?: return emptyMap()
    }
    val byCat: Map<String, List<DictEntry>> = entriesMutex.withLock {
        val current = entriesByCategory ?: emptyMap()
        val missingIds = assetMap.keys.filter { it !in current }
        if (missingIds.isEmpty()) {
            current
        } else {
            val merged = HashMap(current)
            for (id in missingIds) {

```

```

        val asset = assetMap[id] ?: continue
        val entries = loadCategoryEntries(asset)
        Log.d(TAG, "Loaded ${entries.size} entries from assets/$asset")
        merged[id] = entries
    }
    entriesByCategory = merged
    merged
}
}

val map = HashMap<String, DictEntry>(byCat.values.sumOf { it.size })
for ((_, entries) in byCat) {
    for (entry in entries) {
        val key = entry.word.lowercase()
        val existing = map[key]
        map[key] = when {
            existing == null -> entry
            existing.translation == entry.translation -> existing
            existing.translation.contains(entry.translation) -> existing
            entry.translation.contains(existing.translation) -> entry
            else -> existing.copy(
                // Merge with "; " — [joinDistinctGlosses] splits
                // BOTH sides on "; " before the Set dedup so a
                // fragment inside one source's merged translation
                // that also appears as a standalone fragment in
                // the other source is collapsed (e.g.
                // "出现; 浮现" + "浮现; 显露" → "出现; 浮现; 显露",
                // not "出现; 浮现; 浮现; 显露").
                translation = joinDistinctGlosses(
                    listOf(existing.translation, entry.translation)
                ),
            )
        }
    }
}

Log.d(TAG, "Merged lookup: ${map.size} unique words across ${byCat.size} categories")
return map
}

/**
 * Parse one category's entries JSON. Auto-detects schema (flat vs
 * nested 4-layer) by sniffing the first line — 64 chars is plenty
 * to see either `{"word":` or `{"wordRank":`. `readLine()` stops
 * at the first `\n` so we never load more than the first record's
 * worth of bytes just to peek at the shape.
 *
 * Returns an empty list on parse failure (the other categories
 * still load — partial failure is better than crashing the whole
 * lookup).
 */
private fun loadCategoryEntries(asset: String): List<DictEntry> = try {
    val firstLine = context.assets.open(asset)
        .bufferedReader()
        .use { it.readLine().orEmpty() }
    val isNested = firstLine.contains("\\"wordRank\\")
    if (isNested) {
        val rawNested: List<NestedWordEntry> = context.assets.open(asset)
            .bufferedReader()
            .use { reader ->
                gson.fromJson(reader, Array<NestedWordEntry>::class.java)
                    ?.toList()
                    .orEmpty()
            }
    }
}

```

```

        rawNested.mapNotNull(::nestedToFlat).filter(::isUsable)
    } else {
        val rawFlat: List<DictEntry> = context.assets.open(asset)
            .bufferedReader()
            .use { reader ->
                gson.fromJson(reader, Array<DictEntry>::class.java)
                    ?.toList()
                    .orEmpty()
            }
        rawFlat.filter(::isUsable)
    }
} catch (e: Exception) {
    Log.e(TAG, "Failed to parse assets/$asset — category will be empty", e)
    emptyList()
}

private fun isUsable(entry: DictEntry): Boolean {
    // Single-letter entries like "a", "I" are valid English tokens
    // but useless as flashcards and noisy in the long-press lookup
    // — filter them out. Empty translations are filtered too because
    // they would render an empty card face.
    val word = entry.word.trim()
    return word.length > 1 && entry.translation.isNotBlank()
}

/**
 * Convert the source schema's deeply-nested entry to a flat
 * [DictEntry]. Returns `null` for entries that can't be reasonably
 * filled (e.g. missing `headWord`, missing `trans[]`) so they get
 * dropped silently rather than producing empty cards.
 *
 * Field mapping:
 * - `word` ← `headWord` (top level, e.g. "cancel")
 * - `phonetic` ← `usphone` inside `content.word.content` (IPA)
 * - `pos` ← `trans[0].pos` (e.g. "v", "n") — the lead POS,
 *     because the flashcard back face shows one chip.
 * - `translation` ← ALL `trans[*].tranCn` joined with "; ", deduped
 *     and trimmed (matching the cross-category merge
 *     convention used in [lookup]).
 * - `exampleSentenceEn` ← `sentence.sentences[0].sContent_eng` (falls back
 *     to `sContent` if the headword-highlighted form
 *     is missing). May contain `...` tags —
 *     kept verbatim, UI decides whether to render.
 * - `exampleSentenceCn` ← `sentence.sentences[0].sCn`
 *
 * Why join all glosses: a single source record can list multiple
 * POS groups, e.g. "concordance" has both
 * { pos: "n.", tranCn: "索引; 用语索引; 对照..." }
 * { pos: "vt.", tranCn: "编纂索引; 使协调" }
 * The flashcard back face previously showed only the first gloss,
 * which users read as "translation content is missing". Joining
 * everything via "; " mirrors the same convention the long-press
 * lookup uses when the same word appears in multiple categories —
 * so a flashcard that says "索引; 用语索引; 对照表; 编纂索引; 使协调"
 * and a long-press dialog that says "索引; 用语索引; …; 编纂索引; 使协调"
 * read identically. Distinct glosses survive; duplicates are dropped.
 *
 * Only the first example sentence is kept (most words have a
 * single example in the source corpus; multi-sentence entries exist
 * but surfacing the 2nd-Nth on a small flashcard crowds the back
 * face). `isUsable` doesn't filter on sentence presence — a word
 * without an example is still a valid flashcard, the UI just skips
 * the sentence block.

```

```

*
* The positional arrays under `trans[]` / `sentence.sentences[]`
* are themselves nullable on every intermediate step — Gson handles
* absent branches without throwing.
*/
private fun nestedToFlat(n: NestedWordEntry): DictEntry? {
    val word = n.headWord?.trim().orEmpty()
    if (word.isEmpty()) return null
    val wc = n.content?.word?.content
    val phonetic = wc?.usphone?.trim().orEmpty()
    val transList = wc?.trans.orEmpty()
    // `tranCn` can have a leading space (e.g. " 取消, 废除") —
    // trim each gloss and drop blanks. Dedupe (including
    // fragment-level dedupe across "; "-bearing source glosses) is
    // delegated to [joinDistinctGlosses] — same helper the
    // cross-category merge uses, so the dedup convention stays
    // identical between the two sites.
    val glosses = transList
        .mapNotNull { it.tranCn?.trim() }.takeIf { s -> s.isNotEmpty() }
    if (glosses.isEmpty()) return null
    val translation = joinDistinctGlosses(glosses)
    // POS: keep the lead POS for the on-card chip. In practice the
    // lead POS is the most common one and the chip already
    // duplicates the front-of-translation text for many entries
    // (e.g. "n. 索引; ..." renders the "n." chip redundantly), but
    // users have come to expect a POS tag on the back face so we
    // keep the slot stable.
    val pos = transList.firstOrNull()?.pos?.trim().orEmpty()
    // Example sentence: take the first item from `sentence.sentences[]`,
    // preferring `sContent_eng` (with `...` around the headword)
    // and falling back to `sContent` (plain English). Both are nullable
    // and may be blank — the flashcard back face gates on
    // `isNotBlank()` so an absent example is silently skipped.
    val firstSent = wc?.sentence?.sentences?.firstOrNull()
    val exampleEn = (firstSent?.sContentEng?.trim()?).takeIf { it.isNotEmpty() }
        ?: firstSent?.sContent?.trim()?.takeIf { it.isNotEmpty() }
        ?: ""
    val exampleCn = firstSent?.sCn?.trim().orEmpty()
    return DictEntry(
        word = word,
        phonetic = phonetic,
        pos = pos,
        translation = translation,
        exampleSentenceEn = exampleEn,
        exampleSentenceCn = exampleCn,
    )
}

private data class Manifest(
    @SerializedName("categories") val categories: List<ManifestCategory>,
)

private data class ManifestCategory(
    @SerializedName("id") val id: String,
    @SerializedName("name") val name: String,
    @SerializedName("asset") val asset: String,
    @SerializedName("description") val description: String,
    // Post-prune entry count written by `scripts/build_vocab_assets.py`
    // (see its `write_manifest`). Defaults to 0 for back-compat with
    // pre-2026-07-04 manifests that predate this field — the picker
    // UI then renders "0 词" until the script is rerun, which is
    // a better signal than silently parsing the entries file.
    @SerializedName("size") val size: Int = 0,
)

```

```

)

/**
 * Split each source on "; " (the convention used by both the
 * within-category [nestedToFlat] join and the cross-category
 * [lookup] merge), trim and dedup, then rejoin with "; ". This is
 * the single point where fragment-level Chinese deduplication
 * happens — callers must not concatenate translations with their
 * own join+dedup logic, or they will recreate the asymmetry that
 * previously caused duplicate gloss fragments when two sources each
 * contained overlapping fragments (e.g. "出现; 浮现" + "浮现; 显露"
 * would naively produce "出现; 浮现; 浮现; 显露" if only the first
 * side were split before the Set dedup).
 *
 * The source corpus itself uses "; " as a within-POS sub-separator
 * (5746 individual `tranCn` strings in vocab_cet4.json contain "; "),
 * so fragment-level dedup is required, not optional.
 */
private fun joinDistinctGlosses(sources: Collection<String>): String =
    sources
        .flatMap { it.split("; ") }
        .map { it.trim() }
        .filter { it.isNotEmpty() }
        .toSet()
        .joinToString("; ")

/**
 * Mirrors the source JSON schema so Gson can deserialise without
 * reflective guesswork. Every intermediate is nullable because the
 * source files are inconsistent (some entries lack `syno`, some
 * lack `phrase`, etc) — better to skip than to crash.
 *
 * Real shape (abbreviated):
 * ```
 * {
 *   "wordRank": 1,
 *   "headWord": "cancel",
 *   "content": {
 *     "word": {
 *       "wordHead": "cancel",
 *       "content": {
 *         "usphone": "'k□nsl",
 *         "ukphone": "'k□nsl",
 *         "trans": [
 *           { "tranCn": "取消, 废除", "pos": "v", ... }
 *         ]
 *       }
 *     }
 *   }
 * }
 * ```
 */
private data class NestedWordEntry(
    @SerializedName("wordRank") val wordRank: Int? = null,
    @SerializedName("headWord") val headWord: String? = null,
    @SerializedName("content") val content: NestedContent? = null,
)

private data class NestedContent(
    @SerializedName("word") val word: NestedWord? = null,
)

private data class NestedWord(
    @SerializedName("wordHead") val wordHead: String? = null,
    @SerializedName("content") val content: NestedWordContent? = null,

```



```

    )
    private data class NestedWordContent(
        @SerializedName("usphone") val usphone: String? = null,
        @SerializedName("ukphone") val ukphone: String? = null,
        @SerializedName("trans") val trans: List<NestedTrans>? = null,
        @SerializedName("sentence") val sentence: NestedSentence? = null,
    )
    private data class NestedTrans(
        @SerializedName("tranCn") val tranCn: String? = null,
        @SerializedName("pos") val pos: String? = null,
    )
    // Example sentence block under `content.word.content.sentence`.
    // `sentences` is the array of sentence items; `desc` is a fixed
    // label string ("例句") the runtime doesn't read. Items have three
    // nullable fields — `sContentEng` carries `...` highlighting
    // around the headword, `sContent` is the plain English fallback,
    // and `sCn` is the Chinese gloss. All three may be missing on a
    // given source entry; the runtime takes whichever is non-blank.
    private data class NestedSentence(
        @SerializedName("sentences") val sentences: List<NestedSentenceItem>? = null,
        @SerializedName("desc") val desc: String? = null,
    )
    private data class NestedSentenceItem(
        @SerializedName("sContent") val sContent: String? = null,
        @SerializedName("sContent_eng") val sContentEng: String? = null,
        @SerializedName("sCn") val sCn: String? = null,
    )

    private companion object {
        const val TAG = "DictionaryRepository"
        const val MANIFEST_ASSET = "vocab_manifest.json"
    }
}

// === app\src\main\java\com\echoling\app\data\repository\LearningProgressRepositoryImpl.kt ===

package com.echoling.app.data.repository

import com.echoling.app.data.local.db.dao.LearningProgressDao
import com.echoling.app.data.local.db.entity.LearningProgressEntity
import com.echoling.app.domain.model.LearningProgress
import com.echoling.app.domain.repository.LearningProgressRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class LearningProgressRepositoryImpl @Inject constructor(
    private val learningProgressDao: LearningProgressDao
) : LearningProgressRepository {

    override suspend fun getProgressByCourseId(courseId: String): LearningProgress? {
        return learningProgressDao.getProgressByCourseId(courseId)?.toDomain()
    }

    override fun getAllProgress(): Flow<List<LearningProgress>> {
        return learningProgressDao.getAllProgress().map { entities ->
            entities.map { it.toDomain() }
        }
    }
}

```

```

    override suspend fun saveProgress(progress: LearningProgress) {
        learningProgressDao.insertOrUpdateProgress(progress.toEntity())
    }

    override suspend fun deleteProgress(courseId: String) {
        learningProgressDao.deleteProgressByCourseId(courseId)
    }

    private fun LearningProgressEntity.toDomain(): LearningProgress = LearningProgress(
        courseId = courseId,
        currentPositionMs = currentPositionMs,
        currentSentenceId = currentSentenceId,
        learnedSentences = learnedSentences,
        totalLearnTimeMs = totalLearnTimeMs,
        lastLearnTime = lastLearnTime,
        finishRate = finishRate
    )

    private fun LearningProgress.toEntity(): LearningProgressEntity = LearningProgressEntity(
        courseId = courseId,
        currentPositionMs = currentPositionMs,
        currentSentenceId = currentSentenceId,
        learnedSentences = learnedSentences,
        totalLearnTimeMs = totalLearnTimeMs,
        lastLearnTime = lastLearnTime,
        finishRate = finishRate
    )
}

// === app\src\main\java\com\echoling\app\data\repository\SentenceRepositoryImpl.kt ===

package com.echoling.app.data.repository

import com.echoling.app.data.local.db.dao.SentenceDao
import com.echoling.app.data.local.db.entity.SentenceEntity
import com.echoling.app.domain.model.Sentence
import com.echoling.app.domain.repository.SentenceRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class SentenceRepositoryImpl @Inject constructor(
    private val sentenceDao: SentenceDao
) : SentenceRepository {

    override fun getSentencesByCourseId(courseId: String): Flow<List<Sentence>> {
        return sentenceDao.getSentencesByCourseId(courseId).map { entities ->
            entities.map { it.toDomain() }
        }
    }

    override suspend fun getSentencesByCourseIdSync(courseId: String): List<Sentence> {
        return sentenceDao.getSentencesByCourseIdSync(courseId).map { it.toDomain() }
    }

    override suspend fun getSentence(courseId: String, sentenceId: Int): Sentence? {
        return sentenceDao.getSentence(courseId, sentenceId)?.toDomain()
    }
}

```

```

    override suspend fun updateSentence(sentence: Sentence) {
        sentenceDao.updateSentence(sentence.toEntity())
    }

    override suspend fun updateCompletedStatus(courseId: String, sentenceId: Int, isCompleted: Boolean) {
        sentenceDao.updateCompletedStatus(courseId, sentenceId, isCompleted)
    }

    override suspend fun updateTestedStatus(courseId: String, sentenceId: Int, isTested: Boolean) {
        sentenceDao.updateTestedStatus(courseId, sentenceId, isTested)
    }

    override suspend fun syncSentences(sentences: List<Sentence>) {
        sentenceDao.insertSentencesIfNotExists(sentences.map { it.toEntity() })
    }

    private fun SentenceEntity.toDomain(): Sentence = Sentence(
        courseId = courseId,
        sentenceId = sentenceId,
        contentEn = contentEn,
        contentCn = contentCn,
        startTimeMs = startTimeMs,
        endTimeMs = endTimeMs,
        isLearned = isLearned,
        isRead = isRead,
        readScore = readScore,
        isCompleted = isCompleted,
        isTested = isTested
    )

    private fun Sentence.toEntity(): SentenceEntity = SentenceEntity(
        courseId = courseId,
        sentenceId = sentenceId,
        contentEn = contentEn,
        contentCn = contentCn,
        startTimeMs = startTimeMs,
        endTimeMs = endTimeMs,
        isLearned = isLearned,
        isRead = isRead,
        readScore = readScore,
        isCompleted = isCompleted,
        isTested = isTested
    )
}

```

```
// === app\src\main\java\com\echoling\app\data\repository\WordRepositoryImpl.kt ===
```

```

package com.echoling.app.data.repository

import com.echoling.app.data.local.db.dao.WordDao
import com.echoling.app.data.local.db.entity.WordEntity
import com.echoling.app.domain.model.Word
import com.echoling.app.domain.repository.WordRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class WordRepositoryImpl @Inject constructor(

```

```
private val wordDao: WordDao
) : WordRepository {

    override fun getAllWords(): Flow<List<Word>> {
        return wordDao.getAllWords().map { entities ->
            entities.map { it.toDomain() }
        }
    }

    override fun getUnmasteredWords(): Flow<List<Word>> {
        return wordDao.getUnmasteredWords().map { entities ->
            entities.map { it.toDomain() }
        }
    }

    override suspend fun getWord(word: String): Word? {
        return wordDao.getWord(word)?.toDomain()
    }

    override suspend fun insertWord(word: Word) {
        wordDao.insertWord(word.toEntity())
    }

    override suspend fun updateWord(word: Word) {
        wordDao.updateWord(word.toEntity())
    }

    override suspend fun deleteWord(word: String) {
        wordDao.deleteWordByText(word)
    }

    private fun WordEntity.toDomain(): Word = Word(
        word = word,
        phonetic = phonetic,
        pos = pos,
        translation = translation,
        exampleSentence = exampleSentence,
        sourceCourseId = sourceCourseId,
        sourceSentenceId = sourceSentenceId,
        isMastered = isMastered,
        collectedAt = collectedAt,
        reviewCount = reviewCount,
        nextReviewTime = nextReviewTime,
    )

    private fun Word.toEntity(): WordEntity = WordEntity(
        word = word,
        phonetic = phonetic,
        pos = pos,
        translation = translation,
        exampleSentence = exampleSentence,
        sourceCourseId = sourceCourseId,
        sourceSentenceId = sourceSentenceId,
        isMastered = isMastered,
        collectedAt = collectedAt,
        reviewCount = reviewCount,
        nextReviewTime = nextReviewTime,
    )
}

// === app\src\main\java\com\echoling\app\di\DatabaseModule.kt ===
```

```

package com.echoling.app.di

import android.content.Context
import androidx.room.Room
import com.echoling.app.data.local.db.EchoLingDatabase
import com.echoling.app.data.local.db.MIGRATION_2_3
import com.echoling.app.data.local.db.dao.CourseDao
import com.echoling.app.data.local.db.dao.LearningProgressDao
import com.echoling.app.data.local.db.dao.ReciteProgressDao
import com.echoling.app.data.local.db.dao.SentenceDao
import com.echoling.app.data.local.db.dao.WordDao
import dagger.Module
import dagger.Provides
import dagger.hilt.InstallIn
import dagger.hilt.android.qualifiers.ApplicationContext
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {

    @Provides
    @Singleton
    fun provideDatabase(@ApplicationContext context: Context): EchoLingDatabase {
        return Room.databaseBuilder(
            context,
            EchoLingDatabase::class.java,
            "echo_ling_database"
        )
        // v2 → v3 introduces the `courses.courseName` column for the
        // grouped (folder-style) Courses tab. `fallbackToDestructive`
        // is kept as a safety net for any future schema drift.
        .addMigrations(MIGRATION_2_3)
        .fallbackToDestructiveMigration()
        .build()
    }

    @Provides
    fun provideCourseDao(database: EchoLingDatabase): CourseDao = database.courseDao()

    @Provides
    fun provideSentenceDao(database: EchoLingDatabase): SentenceDao = database.sentenceDao()

    @Provides
    fun provideLearningProgressDao(database: EchoLingDatabase): LearningProgressDao = database.learningProgressDao()

    @Provides
    fun provideWordDao(database: EchoLingDatabase): WordDao = database.wordDao()

    @Provides
    fun provideReciteProgressDao(database: EchoLingDatabase): ReciteProgressDao = database.reciteProgressDao()
}

// === app\src\main\java\com\echoling\app\di\PlayerModule.kt ===

package com.echoling.app.di

import com.echoling.app.player.AudioPlayer
import com.echoling.app.subtitle.AssParser

```

```
import com.echoling.app.player.subtitle.SrtParser
import com.echoling.app.player.subtitle.SubtitleParserFactory
import dagger.Module
import dagger.Provides
import dagger.hilt.InstallIn
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
object PlayerModule {

    @Provides
    @Singleton
    fun provideSrtParser(): SrtParser = SrtParser()

    @Provides
    @Singleton
    fun provideAssParser(): AssParser = AssParser()

    @Provides
    @Singleton
    fun provideSubtitleParserFactory(
        srtParser: SrtParser,
        assParser: AssParser
    ): SubtitleParserFactory = SubtitleParserFactory(srtParser, assParser)
}

// === app\src\main\java\com\echoling\app\di\RepositoryModule.kt ===

package com.echoling.app.di

import com.echoling.app.data.repository.CourseRepositoryImpl
import com.echoling.app.data.repository.DictionaryRepositoryImpl
import com.echoling.app.data.repository.LearningProgressRepositoryImpl
import com.echoling.app.data.repository.SentenceRepositoryImpl
import com.echoling.app.data.repository.WordRepositoryImpl
import com.echoling.app.domain.repository.CourseRepository
import com.echoling.app.domain.repository.DictionaryRepository
import com.echoling.app.domain.repository.LearningProgressRepository
import com.echoling.app.domain.repository.SentenceRepository
import com.echoling.app.domain.repository.WordRepository
import dagger.Binds
import dagger.Module
import dagger.hilt.InstallIn
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {

    @Binds
    @Singleton
    abstract fun bindCourseRepository(impl: CourseRepositoryImpl): CourseRepository

    @Binds
    @Singleton
    abstract fun bindSentenceRepository(impl: SentenceRepositoryImpl): SentenceRepository

    @Binds
```

```

    @Singleton
    abstract fun bindLearningProgressRepository(impl: LearningProgressRepositoryImpl): LearningProgressRepository

    @Binds
    @Singleton
    abstract fun bindWordRepository(impl: WordRepositoryImpl): WordRepository

    @Binds
    @Singleton
    abstract fun bindDictionaryRepository(impl: DictionaryRepositoryImpl): DictionaryRepository
}

```

```
// === app\src\main\java\com\echoling\app\domain\model\Course.kt ===
```

```
package com.echoling.app.domain.model
```

```

/**
 * A learning course — a single lesson that the user can drill into
 * (Practice flow). Courses are organized into parent groups by
 * [courseName] (e.g. "新概念英语第一册"). See `effectiveCourseName` for
 * the resolution rule that keeps legacy data working without a
 * destructive migration.
 */
data class Course(
    val courseId: String,
    /**
     * Parent group name (folder). Empty string means "no group assigned".
     * New imports require the user to fill this in; old data rows from
     * before the column existed have it set to "" and resolve to [title]
     * via [effectiveCourseName].
     */
    val courseName: String = "",
    val title: String,
    val description: String,
    val difficulty: String,
    val audioUri: String?,
    val videoUri: String?,
    val subtitleUri: String?,
    val durationMs: Long,
    val totalSentences: Int,
    val thumbnailUri: String?,
    val createdAt: Long,
    val updatedAt: Long
) {
    fun hasVideoContent(): Boolean = !videoUri.isNullOrBlank()
    fun hasAudioContent(): Boolean = !audioUri.isNullOrBlank()
}

```

```

/**
 * The group this course belongs to. Uses [Course.courseName] if the user
 * set one, otherwise falls back to [Course.title] so legacy rows (which
 * pre-date the courseName column) still group themselves — they just
 * show up as one-entry folders under their own lesson title.
 */
val Course.effectiveCourseName: String
    get() = courseName.ifBlank { title }

```

```
// === app\src\main\java\com\echoling\app\domain\model\CourseGroup.kt ===
```

```
package com.echoling.app.domain.model
```

```
/**
```

```
 * A folder on the Courses tab. [courseName] is the resolved name (the
 * user's input if non-blank, otherwise each lesson's own [Course.title]
 * — see [effectiveCourseName]). The list of [courses] is the lessons
 * belonging to that group, in the order produced by
 * [com.echoling.app.domain.usecase.GetCourseGroupsUseCase].
 */
```

```
data class CourseGroup(
    val courseName: String,
    val courses: List<Course>,
) {
    val count: Int get() = courses.size
}
```

```
// === app\src\main\java\com\echoling\app\domain\model\DictEntry.kt ===
```

```
package com.echoling.app.domain.model
```

```
/**
```

```
 * One entry in a bundled vocabulary list.
 *
 * `phonetic` uses IPA notation (e.g. "/□□; □i□/"). `pos` is the part of
 * speech tag (e.g. "art.", "vt.", "n."). `translation` is the Chinese
 * gloss, which may include a leading POS prefix (e.g. "vt. 抛弃") — the
 * user is expected to clean it up in the editable dialog field before
 * saving to the vocabulary list.
 *
 * `exampleSentenceEn` / `exampleSentenceCn` carry the first example
 * sentence from the source JSON's `content.word.content.sentences[0]`
 * (English / Chinese pair). The English side may contain `...` tags
 * around the headword from `sContent_eng`; the UI decides whether to
 * render bold via `AnnotatedString` or strip the tags. Both default to
 * empty when the source has no example, so the flashcard back face can
 * use `isNotBlank()` to gate the sentence block.
 *
 * For the unified long-press lookup in the practice screen the same
 * `word` may surface with merged translations when the source data has
 * the word in multiple categories with different glosses — those are
 * joined with the "; " separator so the user sees all variants in one
 * place. Per-category flashcards iterate that category's own entries
 * (loaded on demand via
 * [com.echoling.app.domain.repository.DictionaryRepository.wordsInCategory])
 * so the user sees the category-specific translation when studying
 * that category.
 */
```

```
data class DictEntry(
    val word: String,
    val phonetic: String,
    val pos: String,
    val translation: String,
    val exampleSentenceEn: String = "",
    val exampleSentenceCn: String = "",
)
```

```
/**
```

```
 * A named vocabulary category.
 *
 * `id` is a stable slug used in nav routes and asset filenames (e.g.
 * "junior", "senior", "cet4", "cet6", "toefl"). `name` is the
```



```

* user-visible Chinese label. `description` is a one-line subtitle
* shown under the category card on the picker screen. `size` is the
* post-prune entry count for the category — it's read straight from
* `assets/vocab_manifest.json` so the picker UI can render "N 词"
* without paying the cost of parsing the multi-MB entries files.
*
* **The entries list is no longer attached** (2026-07-04 lazy-load
* refactor). Pre-refactor this data class carried `entries: List<DictEntry>`
* which forced every caller of [com.echoling.app.domain.repository.DictionaryRepository.categories]
* to pay for a full 12 MB parse of the bundled vocab JSONs before the
* picker could render a single card. The entries are now fetched on
* demand via
* [com.echoling.app.domain.repository.DictionaryRepository.wordsInCategory]
* — the picker only needs id / name / description / size and is
* therefore fast on first tab open.
*
* `size` reflects the post-prune entry count (matches what the script
* `scripts/build_vocab_assets.py` writes into the manifest after
* running). At runtime the long-press / category-study paths apply an
* additional `word.length > 1` filter that drops a handful of one-letter
* tokens per category, so the actual study-screen count may be 1-3
* lower than `size`; this is intentional and matches the historic
* behavior the picker UI has always shown.
*/
data class DictCategory(
    val id: String,
    val name: String,
    val description: String,
    val size: Int,
)

```

```
// === app\src\main\java\com\echoling\app\domain\model\LearningProgress.kt ===
```

```
package com.echoling.app.domain.model
```

```

data class LearningProgress(
    val courseId: String,
    val currentPositionMs: Long,
    val currentSentenceId: Int,
    val learnedSentences: Int,
    val totalLearnTimeMs: Long,
    val lastLearnTime: Long,
    val finishRate: Float
)

```

```
// === app\src\main\java\com\echoling\app\domain\model\Sentence.kt ===
```

```
package com.echoling.app.domain.model
```

```

data class Sentence(
    val courseId: String,
    val sentenceId: Int,
    val contentEn: String,
    val contentCn: String,
    val startTimeMs: Long,
    val endTimeMs: Long,
    val isLearned: Boolean = false,
    val isRead: Boolean = false,
    val readScore: Int = 0,
)

```

```
        val isCompleted: Boolean = false,
        val isTested: Boolean = false
    )

// === app\src\main\java\com\echoling\app\domain\model\Word.kt ===

package com.echoling.app.domain.model

/**
 * A word saved to the user's vocabulary book. May have been saved from
 * either the practice screen long-press flow or the flashcard study
 * screen — both paths route through the same Room table.
 *
 * Fields:
 * - [phonetic] / [pos] — populated from the bundled
 *   [com.echoling.app.domain.model.DictEntry] when the save comes
 *   from the offline dictionary. Blank when the user types a word
 *   manually that wasn't in the dictionary.
 * - [translation] — user-editable Chinese gloss; defaults to the
 *   dictionary value when the save path provides one.
 * - [sourceCourseId] / [sourceSentenceId] — link back to the practice
 *   sentence the word came from, when applicable. Empty/0 for words
 *   saved from the study screen.
 */
data class Word(
    val word: String,
    val phonetic: String = "",
    val pos: String = "",
    val translation: String,
    val exampleSentence: String = "",
    val sourceCourseId: String = "",
    val sourceSentenceId: Int = 0,
    val isMastered: Boolean = false,
    val collectedAt: Long,
    val reviewCount: Int = 0,
    val nextReviewTime: Long,
)

// === app\src\main\java\com\echoling\app\domain\repository\CourseRepository.kt ===

package com.echoling.app.domain.repository

import com.echoling.app.domain.model.Course
import kotlinx.coroutines.flow.Flow

interface CourseRepository {
    fun getAllCourses(): Flow<List<Course>>
    suspend fun getCourseById(courseId: String): Course?
    suspend fun insertCourse(course: Course)
    suspend fun deleteCourse(courseId: String)
}

// === app\src\main\java\com\echoling\app\domain\repository\DictionaryRepository.kt ===

package com.echoling.app.domain.repository

import com.echoling.app.domain.model.DictCategory
```

```

import com.echoling.app.domain.model.DictEntry

/**
 * Read-only local dictionary used as a fast offline source for both:
 *
 * 1. The long-press-to-translate flow in the practice screen —
 *    [lookup] does a case-insensitive O(1) `HashMap.get` against an
 *    in-memory map keyed by lowercased word. Entries with the same
 *    word but different translations across categories are merged
 *    here with the "; " separator so the user sees all variants.
 *
 * 2. The flashcard study screen — [categories] returns the ordered
 *    list of available categories with their `size` (entry count, from
 *    the manifest) but without the entries attached; [wordsInCategory]
 *    returns just the entries for one slug. Per-category lists keep the
 *    category-specific translation, not the merged one, so studying
 *    高中 shows the high-school-specific gloss even if CET-6 has a
 *    different one.
 *
 * Two-phase lazy loading (2026-07-04 refactor — replaces the old
 * "parse all 12 MB on first call" path with a manifest-only first phase
 * so the 记单词 picker renders instantly):
 *
 * - [categories] is now a fast path — it only parses the ~1 KB
 *   manifest and never opens any of the multi-MB entries files. The
 *   picker UI calls this on first tab open.
 * - [wordsInCategory] lazily parses only the requested category's
 *   entries file when the user actually opens a flashcard study screen.
 * - [lookup] (and [allWords]) lazily trigger a full parse of every
 *   category the first time they're called — i.e. on the first
 *   long-press translation. The first call pays the 12 MB cost; every
 *   subsequent call is O(1).
 *
 * Implementations should expose a `warmupAll()` method (see
 * [com.echoling.app.data.repository.DictionaryRepositoryImpl]) for
 * ViewModels to kick off the full parse in the background right after
 * the manifest is loaded — by the time the user navigates anywhere,
 * the lookup map is already warm.
 *
 * Single-letter entries (e.g. "a", "I") are filtered at load time — see
 * [com.echoling.app.data.repository.DictionaryRepositoryImpl].
 */
interface DictionaryRepository {
    /** Case-insensitive lookup of [word] across all categories. */
    suspend fun lookup(word: String): DictEntry?

    /**
     * Snapshot of every entry in the dictionary, in alphabetical order.
     * Used as a fallback when callers need a single flat list (e.g. the
     * legacy "all words" flashcard path). Returns entries with
     * merged translations where a word exists in multiple categories
     * with different glosses.
     */
    suspend fun allWords(): List<DictEntry>

    /**
     * Ordered list of every available category. Returns lightweight
     * [DictCategory] entries with id / name / description / size only —
     * the actual entries are NOT attached. Order matches the order in
     * the manifest JSON so the category picker UI renders in a
     * predictable sequence. Cheap to call — only parses the ~1 KB
     * `vocab_manifest.json`.
     */
}

```

```

suspend fun categories(): List<DictCategory>

/**
 * Entries for one category only. Returns an empty list when the
 * category id is unknown — callers can render an empty state rather
 * than throwing. Lazily parses the requested category's JSON on
 * first call; cached for subsequent calls.
 */
suspend fun wordsInCategory(categoryId: String): List<DictEntry>

/**
 * Pre-warm the full merged lookup map in the background so the
 * first call to [lookup] / [allWords] / [wordsInCategory] is
 * instant. Idempotent — a second call while the first is in
 * flight waits on the same mutex and returns the same map.
 *
 * ViewModels that already need [categories] for their first frame
 * (e.g. the picker) should kick off [warmupAll] right after their
 * first [categories] call returns — the user is staring at the
 * picker reading the "N 词" chips, so the heavy work is invisible.
 */
suspend fun warmupAll()
}

// === app\src\main\java\com\echoling\app\domain\repository\LearningProgressRepository.kt ===

package com.echoling.app.domain.repository

import com.echoling.app.domain.model.LearningProgress
import kotlinx.coroutines.flow.Flow

interface LearningProgressRepository {
    suspend fun getProgressByCourseId(courseId: String): LearningProgress?
    fun getAllProgress(): Flow<List<LearningProgress>>
    suspend fun saveProgress(progress: LearningProgress)
    suspend fun deleteProgress(courseId: String)
}

// === app\src\main\java\com\echoling\app\domain\repository\SentenceRepository.kt ===

package com.echoling.app.domain.repository

import com.echoling.app.domain.model.Sentence
import kotlinx.coroutines.flow.Flow

interface SentenceRepository {
    fun getSentencesByCourseId(courseId: String): Flow<List<Sentence>>
    suspend fun getSentencesByCourseIdSync(courseId: String): List<Sentence>
    suspend fun getSentence(courseId: String, sentenceId: Int): Sentence?
    suspend fun updateSentence(sentence: Sentence)
    suspend fun updateCompletedStatus(courseId: String, sentenceId: Int, isCompleted: Boolean)
    suspend fun updateTestedStatus(courseId: String, sentenceId: Int, isTested: Boolean)
    suspend fun syncSentences(sentences: List<Sentence>)
}

// === app\src\main\java\com\echoling\app\domain\repository\WordRepository.kt ===

```

```
package com.echoling.app.domain.repository
```

```
import com.echoling.app.domain.model.Word
import kotlinx.coroutines.flow.Flow
```

```
interface WordRepository {
    fun getAllWords(): Flow<List<Word>>
    fun getUnmasteredWords(): Flow<List<Word>>
    suspend fun getWord(word: String): Word?
    suspend fun insertWord(word: Word)
    suspend fun updateWord(word: Word)
    suspend fun deleteWord(word: String)
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\DeleteCourseUseCase.kt ===
```

```
package com.echoling.app.domain.usecase
```

```
import com.echoling.app.domain.repository.CourseRepository
import javax.inject.Inject
import javax.inject.Singleton
```

```
@Singleton
class DeleteCourseUseCase @Inject constructor(
    private val courseRepository: CourseRepository
) {
    suspend operator fun invoke(courseId: String) {
        courseRepository.deleteCourse(courseId)
    }
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\DeleteWordUseCase.kt ===
```

```
package com.echoling.app.domain.usecase
```

```
import com.echoling.app.domain.repository.WordRepository
import javax.inject.Inject
import javax.inject.Singleton
```

```
@Singleton
class DeleteWordUseCase @Inject constructor(
    private val wordRepository: WordRepository
) {
    suspend operator fun invoke(wordText: String) {
        wordRepository.deleteWord(wordText)
    }
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\GetAllDictionaryWordsUseCase.kt ===
```

```
package com.echoling.app.domain.usecase
```

```
import com.echoling.app.domain.model.DictEntry
import com.echoling.app.domain.repository.DictionaryRepository
import javax.inject.Inject
import javax.inject.Singleton
```

```

/**
 * Returns every word in the bundled local dictionary as a stable,
 * alphabetically-sorted snapshot. Drives the flashcard iteration in
 * the Recite screen. [LookupWordUseCase] is the single-word lookup
 * companion (used by the long-press translate flow in practice
 * pages); this is the "give me the lot" read path that
 * [CategoryStudyViewModel] needs to iterate flashcards.
 *
 * The repository's lazy load + HashMap cache means the first call
 * pays the one-time asset parse (tens of ms), and subsequent calls
 * (including on tab re-entry) are a no-op sort over an in-memory map.
 */
@Singleton
class GetAllDictionaryWordsUseCase @Inject constructor(
    private val dictionaryRepository: DictionaryRepository,
) {
    suspend operator fun invoke(): List<DictEntry> = dictionaryRepository.allWords()
}

// === app\src\main\java\com\echoling\app\domain\usecase\GetContinueLearningUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.Course
import com.echoling.app.domain.model.LearningProgress
import com.echoling.app.domain.repository.CourseRepository
import com.echoling.app.domain.repository.LearningProgressRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.combine
import javax.inject.Inject
import javax.inject.Singleton

/** Pair of a course and its in-progress learning state. */
data class ContinueLearningItem(
    val course: Course,
    val progress: LearningProgress,
)

/**
 * Reactive stream of the most recently touched course that has any progress.
 * Returns null when no progress has been recorded yet.
 */
@Singleton
class GetContinueLearningUseCase @Inject constructor(
    private val courseRepository: CourseRepository,
    private val progressRepository: LearningProgressRepository,
) {
    operator fun invoke(): Flow<ContinueLearningItem?> =
        combine(
            courseRepository.getAllCourses(),
            progressRepository.getAllProgress(),
        ) { courses, progressList ->
            val progressMap = progressList.associateBy { it.courseId }
            courses
                .mapNotNull { course ->
                    progressMap[course.courseId]?.let { progress ->
                        ContinueLearningItem(course, progress)
                    }
                }
                .maxByOrNull { it.progress.lastLearnTime }
        }
}

```

```
}

// === app\src\main\java\com\echoling\app\domain\usecase\GetCourseDetailUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.Course
import com.echoling.app.domain.model.LearningProgress
import com.echoling.app.domain.repository.CourseRepository
import com.echoling.app.domain.repository.LearningProgressRepository
import javax.inject.Inject
import javax.inject.Singleton

data class CourseDetail(
    val course: Course?,
    val progress: LearningProgress?
)

@Singleton
class GetCourseDetailUseCase @Inject constructor(
    private val courseRepository: CourseRepository,
    private val progressRepository: LearningProgressRepository
) {
    suspend operator fun invoke(courseId: String): CourseDetail {
        val course = courseRepository.getCourseById(courseId)
        val progress = progressRepository.getProgressByCourseId(courseId)
        return CourseDetail(course, progress)
    }
}

// === app\src\main\java\com\echoling\app\domain\usecase\GetCourseGroupsUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.CourseGroup
import com.echoling.app.domain.model.effectiveCourseName
import com.echoling.app.domain.repository.CourseRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import javax.inject.Inject

/**
 * Surfaces the course table as a list of [CourseGroup]s for the
 * Courses-tab home page. Courses are grouped by [effectiveCourseName]
 * (user-supplied [com.echoling.app.domain.model.Course.courseName] with
 * a fallback to [com.echoling.app.domain.model.Course.title] for
 * legacy rows). Groups are sorted alphabetically by their name so the
 * list is stable across re-emissions.
 */
class GetCourseGroupsUseCase @Inject constructor(
    private val courseRepository: CourseRepository,
) {
    operator fun invoke(): Flow<List<CourseGroup>> =
        courseRepository.getAllCourses().map { courses ->
            courses
                .groupBy { it.effectiveCourseName }
                .map { (name, list) -> CourseGroup(name, list) }
                .sortedBy { it.courseName }
        }
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\GetCourseSentencesUseCase.kt ===
```

```
package com.echoling.app.domain.usecase
```

```
import com.echoling.app.domain.model.Sentence
import com.echoling.app.domain.repository.SentenceRepository
import kotlinx.coroutines.flow.Flow
import javax.inject.Inject
import javax.inject.Singleton
```

```
@Singleton
class GetCourseSentencesUseCase @Inject constructor(
    private val sentenceRepository: SentenceRepository
) {
    fun getByCourseId(courseId: String): Flow<List<Sentence>> =
        sentenceRepository.getSentencesByCourseId(courseId)

    suspend fun getByCourseIdSync(courseId: String): List<Sentence> =
        sentenceRepository.getSentencesByCourseIdSync(courseId)
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\GetCoursesUseCase.kt ===
```

```
package com.echoling.app.domain.usecase
```

```
import com.echoling.app.domain.model.Course
import com.echoling.app.domain.repository.CourseRepository
import kotlinx.coroutines.flow.Flow
import javax.inject.Inject
import javax.inject.Singleton
```

```
@Singleton
class GetCoursesUseCase @Inject constructor(
    private val courseRepository: CourseRepository
) {
    operator fun invoke(): Flow<List<Course>> = courseRepository.getAllCourses()
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\GetDictionaryCategoriesUseCase.kt ===
```

```
package com.echoling.app.domain.usecase
```

```
import com.echoling.app.domain.model.DictCategory
import com.echoling.app.domain.repository.DictionaryRepository
import javax.inject.Inject
import javax.inject.Singleton
```

```
/**
 * Returns the ordered list of bundled vocabulary categories (each with
 * its full entry list attached). Drives the category picker on the
 * "记单词" tab — the UI iterates the result to render one card per
 * category.
 *
 * Categories are returned in manifest-declaration order so the picker
 * shows 初中 → 高中 → CET-4 → CET-6 → TOEFL in the sequence the user
 * expects when scanning.
```



```
*/
@Singleton
class GetDictionaryCategoriesUseCase @Inject constructor(
    private val dictionaryRepository: DictionaryRepository,
) {
    suspend operator fun invoke(): List<DictCategory> = dictionaryRepository.categories()
}

// === app\src\main\java\com\echoling\app\domain\usecase\GetDictionaryWordsInCategoryUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.DictEntry
import com.echoling.app.domain.repository.DictionaryRepository
import javax.inject.Inject
import javax.inject.Singleton

/**
 * Returns the entries for one vocabulary category by its slug
 * (e.g. "junior", "senior", "cet4", "cet6", "toefl"). Drives the
 * per-category flashcard study screen — the iteration sees the
 * **category-specific** translation (not the cross-category merge),
 * so studying 高中 shows the high-school-specific gloss even when CET-6
 * has a different one for the same word.
 *
 * Returns an empty list when the slug is unknown; the study screen
 * should render an empty-state message in that case rather than
 * crashing.
 */
@Singleton
class GetDictionaryWordsInCategoryUseCase @Inject constructor(
    private val dictionaryRepository: DictionaryRepository,
) {
    suspend operator fun invoke(categoryId: String): List<DictEntry> =
        dictionaryRepository.wordsInCategory(categoryId)
}

// === app\src\main\java\com\echoling\app\domain\usecase\GetProgressUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.LearningProgress
import com.echoling.app.domain.repository.LearningProgressRepository
import kotlinx.coroutines.flow.Flow
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class GetProgressUseCase @Inject constructor(
    private val progressRepository: LearningProgressRepository
) {
    fun getAllProgress(): Flow<List<LearningProgress>> = progressRepository.getAllProgress()

    suspend fun getProgressByCourseId(courseId: String): LearningProgress? =
        progressRepository.getProgressByCourseId(courseId)
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\GetReciteProgressUseCase.kt ===
```

```
package com.echoling.app.domain.usecase

import com.echoling.app.data.local.db.entity.ReciteProgressEntity
import com.echoling.app.data.local.db.dao.ReciteProgressDao
import javax.inject.Inject
import javax.inject.Singleton

/**
 * Returns the persisted study progress for a single vocabulary
 * category (slug, e.g. "junior"). Returns `null` when the user has
 * never studied this category — callers should treat that as
 * "start from the first card" (`currentIndex = -1` and counts = 0).
 *
 * Used by [com.echoling.app.presentation.ui.screens.recite.CategoryStudyViewModel]
 * on `load(categoryId)` to seed its in-memory UI state so the user
 * resumes from where they left off after process death, app
 * cold-start, or tab switch.
 */
@Singleton
class GetReciteProgressUseCase @Inject constructor(
    private val dao: ReciteProgressDao,
) {
    suspend operator fun invoke(categoryId: String): ReciteProgressEntity? =
        dao.getByCategory(categoryId)
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\GetStatisticsUseCase.kt ===
```

```
package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.LearningProgress
import com.echoling.app.domain.model.Word
import com.echoling.app.domain.repository.LearningProgressRepository
import com.echoling.app.domain.repository.WordRepository
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.combine
import javax.inject.Inject
import javax.inject.Singleton

data class StatisticsData(
    val progressList: List<LearningProgress>,
    val words: List<Word>
)

@Singleton
class GetStatisticsUseCase @Inject constructor(
    private val progressRepository: LearningProgressRepository,
    private val wordRepository: WordRepository
) {
    fun getStatisticsFlow(): Flow<StatisticsData> = combine(
        progressRepository.getAllProgress(),
        wordRepository.getAllWords()
    ) { progressList, words ->
        StatisticsData(progressList, words)
    }
}
```

```
// === app\src\main\java\com\echoling\app\domain\usecase\GetWordsUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.Word
import com.echoling.app.domain.repository.WordRepository
import kotlinx.coroutines.flow.Flow
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class GetWordsUseCase @Inject constructor(
    private val wordRepository: WordRepository
) {
    fun getAllWords(): Flow<List<Word>> = wordRepository.getAllWords()

    fun getUnmasteredWords(): Flow<List<Word>> = wordRepository.getUnmasteredWords()
}

// === app\src\main\java\com\echoling\app\domain\usecase\ImportCourseUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.Course
import com.echoling.app.domain.repository.CourseRepository
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class ImportCourseUseCase @Inject constructor(
    private val courseRepository: CourseRepository
) {
    suspend operator fun invoke(course: Course) {
        courseRepository.insertCourse(course)
    }
}

// === app\src\main\java\com\echoling\app\domain\usecase\LookupWordUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.DictEntry
import com.echoling.app.domain.repository.DictionaryRepository
import javax.inject.Inject
import javax.inject.Singleton

/**
 * Local-first lookup against the bundled Gaokao 3500-word dictionary.
 *
 * Thin pass-through today, but kept as a use case (rather than calling
 * [DictionaryRepository] directly from `PracticeViewModel`) to:
 *
 * - match the rest of the VM's use-case-based dependency style, and
 * - keep a clean seam for future enhancements — suffix-stripping,
 *   fuzzy match, or frequency-based ranking would all live here.
 *
 * Returns `null` when the word is not in the bundled list; the caller
 * is expected to fall back to the network translation API in that case.
 */
```

```

@Singleton
class LookupWordUseCase @Inject constructor(
    private val dictionaryRepository: DictionaryRepository,
) {
    // TODO: add suffix-stripping / fuzzy match here when hit rate warrants it.
    suspend operator fun invoke(word: String): DictEntry? =
        dictionaryRepository.lookup(word)
}

// === app\src\main\java\com\echoling\app\domain\usecase\ObserveAllReciteProgressUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.data.local.db.dao.ReciteProgressDao
import com.echoling.app.data.local.db.entity.ReciteProgressEntity
import kotlinx.coroutines.flow.Flow
import javax.inject.Inject
import javax.inject.Singleton

/**
 * Streams every per-category progress row. Drives the "记单词" tab
 * category picker — the [com.echoling.app.presentation.ui.screens.recite.ReciteViewModel]
 * combines this with the bundled category list to render the
 * "已学 X / 1801 词" sub-label and the "上次学习于..." line on each
 * card. The flow re-emits whenever a study action persists a new
 * row, so the picker updates instantly as the user finishes a card
 * on a sub-page.
 */
@Singleton
class ObserveAllReciteProgressUseCase @Inject constructor(
    private val dao: ReciteProgressDao,
) {
    operator fun invoke(): Flow<List<ReciteProgressEntity>> = dao.observeAll()
}

// === app\src\main\java\com\echoling\app\domain\usecase\SaveProgressUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.LearningProgress
import com.echoling.app.domain.repository.LearningProgressRepository
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class SaveProgressUseCase @Inject constructor(
    private val progressRepository: LearningProgressRepository
) {
    suspend operator fun invoke(progress: LearningProgress) {
        progressRepository.saveProgress(progress)
    }

    suspend fun getProgressByCourseId(courseId: String): LearningProgress? =
        progressRepository.getProgressByCourseId(courseId)
}

// === app\src\main\java\com\echoling\app\domain\usecase\SaveReciteProgressUseCase.kt ===

```

```

package com.echoling.app.domain.usecase

import com.echoling.app.data.local.db.dao.ReciteProgressDao
import com.echoling.app.data.local.db.entity.ReciteProgressEntity
import javax.inject.Inject
import javax.inject.Singleton

/**
 * Upserts the per-category flashcard progress row. Called by
 * [com.echoling.app.presentation.ui.screens.recite.CategoryStudyViewModel]
 * after every `markKnown` / `markUnknown` / `saveCurrentToVocabulary`
 * and on `resetSession` (to clear the counters back to 0).
 *
 * Insert strategy is `REPLACE` (see [ReciteProgressDao.upsert]) so
 * callers don't need a separate "exists?" check — writing once per
 * action is enough.
 */
@Singleton
class SaveReciteProgressUseCase @Inject constructor(
    private val dao: ReciteProgressDao,
) {
    suspend operator fun invoke(entity: ReciteProgressEntity) {
        dao.upsert(entity)
    }
}

// === app\src\main\java\com\echoling\app\domain\usecase\SaveWordUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.Word
import com.echoling.app.domain.repository.WordRepository
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class SaveWordUseCase @Inject constructor(
    private val wordRepository: WordRepository
) {
    suspend operator fun invoke(word: Word) {
        val existingWord = wordRepository.getWord(word.word)
        if (existingWord == null) {
            wordRepository.insertWord(word)
        }
    }
}

// === app\src\main\java\com\echoling\app\domain\usecase\SyncSentencesUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.model.Sentence
import com.echoling.app.domain.repository.SentenceRepository
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class SyncSentencesUseCase @Inject constructor(

```

```

        private val sentenceRepository: SentenceRepository
    ) {
        suspend operator fun invoke(sentences: List<Sentence>) {
            sentenceRepository.syncSentences(sentences)
        }
    }
}

// === app\src\main\java\com\echoling\app\domain\usecase\UpdateSentenceCompletedUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.repository.SentenceRepository
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class UpdateSentenceCompletedUseCase @Inject constructor(
    private val sentenceRepository: SentenceRepository
) {
    suspend operator fun invoke(courseId: String, sentenceId: Int, isCompleted: Boolean) {
        sentenceRepository.updateCompletedStatus(courseId, sentenceId, isCompleted)
    }
}

// === app\src\main\java\com\echoling\app\domain\usecase\UpdateSentenceTestedUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.repository.SentenceRepository
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class UpdateSentenceTestedUseCase @Inject constructor(
    private val sentenceRepository: SentenceRepository
) {
    suspend operator fun invoke(courseId: String, sentenceId: Int, isTested: Boolean) {
        sentenceRepository.updateTestedStatus(courseId, sentenceId, isTested)
    }
}

// === app\src\main\java\com\echoling\app\domain\usecase\WarmupDictionaryUseCase.kt ===

package com.echoling.app.domain.usecase

import com.echoling.app.domain.repository.DictionaryRepository
import javax.inject.Inject
import javax.inject.Singleton

/**
 * Background-warms the dictionary's merged-lookup map. Used by
 * `ReciteViewModel` right after the picker categories finish loading
 * so the heavy 12 MB parse happens while the user is reading the
 * category cards, not while they're tapping one.
 *
 * Idempotent — see [DictionaryRepository.warmupAll].
 */

```

```

@Singleton
class WarmupDictionaryUseCase @Inject constructor(
    private val dictionaryRepository: DictionaryRepository,
) {
    suspend operator fun invoke() = dictionaryRepository.warmupAll()
}

// === app\src\main\java\com\echoling\app\EchoLingApplication.kt ===

package com.echoling.app

import android.app.Application
import android.util.Log
import dagger.hilt.android.HiltAndroidApp

@HiltAndroidApp
class EchoLingApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        // JNA dispatch library probe.
        //
        // Background: vosk-android:0.3.45 depends transitively on
        // jna:4.4.0, which has no Android .so files. We:
        // - exclude jna:4.4.0 from vosk-android (version mismatch
        //   with the .so we ship)
        // - declare jna:5.18.1 jar explicitly
        // - ship libjnidispatch.so (5.18.1) at lib/<abi>/ via
        //   :jna-bridge
        //
        // System.loadLibrary("jnidispatch") finds the bundled
        // libjnidispatch.so inside the APK and loads it. JNA's
        // Native.<clinit> later confirms the loaded lib's JNI
        // protocol version matches the bundled Java classes (both
        // 5.18.1 now) and proceeds.
        try {
            System.loadLibrary("jnidispatch")
            Log.i("JnaProbe", "loadLibrary('jnidispatch') OK")
        } catch (e: Throwable) {
            Log.e("JnaProbe", "loadLibrary('jnidispatch') FAILED", e)
        }
    }
}

```

```

// === app\src\main\java\com\echoling\app\player\AudioPlayer.kt ===

package com.echoling.app.player

import android.content.Context
import android.net.Uri
import androidx.media3.common.MediaItem
import androidx.media3.common.Player
import androidx.media3.exoplayer.ExoPlayer
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import javax.inject.Inject
import javax.inject.Singleton

```

```

@Singleton

```

```
class AudioPlayer @Inject constructor(
    @ApplicationContext private val context: Context
) {
    private var exoPlayer: ExoPlayer? = null

    private val _playbackState = MutableStateFlow(PlaybackState())
    val playbackState: StateFlow<PlaybackState> = _playbackState.asStateFlow()

    private val _currentSubtitle = MutableStateFlow<String?>(null)
    val currentSubtitle: StateFlow<String?> = _currentSubtitle.asStateFlow()

    private var subtitleProvider: ((Long) -> String?)? = null

    fun initialize() {
        if (exoPlayer == null) {
            exoPlayer = ExoPlayer.Builder(context).build().apply {
                addListener(object : Player.Listener {
                    override fun onIsPlayingChanged(isPlaying: Boolean) {
                        updateState { it.copy(isPlaying = isPlaying) }
                    }

                    override fun onPlaybackStateChanged(state: Int) {
                        updateState {
                            it.copy(
                                isBuffering = state == Player.STATE_BUFFERING,
                                isPlaying = exoPlayer?.isPlaying == true
                            )
                        }
                    }

                    override fun onPositionDiscontinuity(
                        oldPosition: Player.PositionInfo,
                        newPosition: Player.PositionInfo,
                        reason: Int
                    ) {
                        updateState { it.copy(currentPositionMs = newPosition.positionMs) }
                    }
                })
            }
        }
    }

    fun setMediaUri(uri: String) {
        exoPlayer?.apply {
            val mediaItem = MediaItem.fromUri(Uri.parse(uri))
            setMediaItem(mediaItem)
            prepare()
        }
    }

    fun setSubtitleProvider(provider: (Long) -> String?) {
        subtitleProvider = provider
    }

    fun play() {
        exoPlayer?.play()
    }

    fun pause() {
        exoPlayer?.pause()
    }

    fun seekTo(positionMs: Long) {
```



```

        exoPlayer?. seekTo(positionMs)
        updateState { it.copy(currentPositionMs = positionMs) }
    }

    fun setPlaybackSpeed(speed: Float) {
        exoPlayer?. setPlaybackSpeed(speed)
        updateState { it.copy(playbackSpeed = speed) }
    }

    fun setLooping(looping: Boolean) {
        exoPlayer?. repeatMode = if (looping) Player.REPEAT_MODE_ONE else Player.REPEAT_MODE_OFF
        updateState { it.copy(isLooping = looping) }
    }

    fun updatePosition() {
        exoPlayer?.let { player ->
            val position = player.currentPosition
            val duration = player.duration.coerceAtLeast(0)
            updateState {
                it.copy(
                    currentPositionMs = position,
                    durationMs = duration
                )
            }
            subtitleProvider?. invoke(position)?.let { subtitle ->
                _currentSubtitle.value = subtitle
            }
        }
    }

    fun updatePositionFromExternal(position: Long, duration: Long, isPlaying: Boolean = false) {
        updateState {
            it.copy(
                currentPositionMs = position,
                durationMs = duration,
                isPlaying = isPlaying
            )
        }
        subtitleProvider?. invoke(position)?.let { subtitle ->
            _currentSubtitle.value = subtitle
        }
    }

    private var videoPlayer: ExoPlayer? = null

    fun setVideoPlayer(player: ExoPlayer?) {
        videoPlayer = player
    }

    fun release() {
        exoPlayer?. release()
        exoPlayer = null
    }

    fun getCurrentPosition(): Long = exoPlayer?. currentPosition ?: 0L

    fun getDuration(): Long = exoPlayer?. duration?. coerceAtLeast(0) ?: 0L

    private fun updateState(update: (PlaybackState) -> PlaybackState) {
        _playbackState.value = update(_playbackState.value)
    }
}

```

```
// === app\src\main\java\com\echoling\app\player\PlaybackState.kt ===

package com.echoling.app.player

data class PlaybackState(
    val isPlaying: Boolean = false,
    val currentPositionMs: Long = 0L,
    val durationMs: Long = 0L,
    val playbackSpeed: Float = 1.0f,
    val isLooping: Boolean = false,
    val isBuffering: Boolean = false,
    val error: String? = null
)

// === app\src\main\java\com\echoling\app\player\subtitle\AssParser.kt ===

package com.echoling.app.player.subtitle

import javax.inject.Inject

class AssParser @Inject constructor() : SubtitleParser {

    override fun parse(content: String): List<Subtitle> {
        val subtitles = mutableListOf<Subtitle>()
        var inEventsSection = false
        var formatLine: String? = null

        val lines = content.split("\\r?\\n".toRegex())

        for (line in lines) {
            val trimmed = line.trim()

            when {
                trimmed.equals("[Events]", ignoreCase = true) -> {
                    inEventsSection = true
                }
                trimmed.startsWith("[") && trimmed.endsWith("]") -> {
                    inEventsSection = false
                }
                inEventsSection && trimmed.startsWith("Format:", ignoreCase = true) -> {
                    formatLine = trimmed.substringAfter("Format:").trim()
                }
                inEventsSection && trimmed.startsWith("Dialogue:", ignoreCase = true) -> {
                    parseDialogueLine(trimmed, formatLine)?.let { subtitles.add(it) }
                }
            }
        }

        return subtitles.sortedBy { it.startTimeMs }
    }

    override fun findSubtitleAtTime(subtitles: List<Subtitle>, timeMs: Long): Subtitle? {
        return subtitles.find { it.startTimeMs <= timeMs && it.endTimeMs >= timeMs }
    }

    private fun parseDialogueLine(line: String, formatLine: String?): Subtitle? {
        val dialogueContent = line.substringAfter("Dialogue:").trim() ?: return null

        if (formatLine == null) return null
    }
}
```

```

val formatFields = formatLine.split(",").map { it.trim() }
val fieldMap = mutableMapOf<String, String>()

var currentField = 0
var inTag = false
var fieldBuilder = StringBuilder()

for (char in dialogueContent) {
    when {
        char == '=' && fieldBuilder.isEmpty() && !inTag -> {
            val fieldName = fieldBuilder.toString().trim()
            fieldBuilder.clear()
            inTag = true
            currentField++
        }
        char == ',' && !inTag -> {
            if (currentField < formatFields.size) {
                fieldMap[formatFields[currentField]] = fieldBuilder.toString().trim()
            }
            fieldBuilder.clear()
            currentField++
        }
        else -> {
            fieldBuilder.append(char)
            if (char == '}' && inTag) {
                inTag = false
            }
        }
    }
}

if (fieldBuilder.isNotEmpty() && currentField < formatFields.size) {
    fieldMap[formatFields[currentField]] = fieldBuilder.toString().trim()
}

val startTime = parseAssTime(fieldMap["Start"]?: return null)?: return null
val endTime = parseAssTime(fieldMap["End"]?: return null)?: return null
val text = fieldMap["Text"]?.let { stripAssTags(it) }?: return null

if (text.isEmpty()) return null

return Subtitle(
    index = 0,
    startTimeMs = startTime,
    endTimeMs = endTime,
    contentEn = text
)
}

private fun parseAssTime(timeStr: String): Long? {
    // Format: H:MM:SS.cc (centiseconds)
    val regex = Regex("(\\d+):(\\d{2}):(\\d{2})\\. (\\d{2})")
    val match = regex.find(timeStr.trim())?: return null

    val (hours, minutes, seconds, centis) = match.destructured
    return (hours.toLongOrNull()?: return null) * 3600000 +
        (minutes.toLongOrNull()?: return null) * 60000 +
        (seconds.toLongOrNull()?: return null) * 1000 +
        (centis.toLongOrNull()?: 0) * 10
}

private fun stripAssTags(text: String): String {

```

```

        // Remove ASS style tags like {\an8}, {\pos(x,y)}, {\b1}, etc.
        return text
            .replace(Regex("\\[\\^]*\\)", "")
            .replace("\\N", "\n")
            .replace("\\n", "\n")
            .replace("\\\\\\".toRegex(), "\\")
            .trim()
    }
}

// === app\src\main\java\com\echoling\app\player\subtitle\LrcParser.kt ===

package com.echoling.app.player.subtitle

import android.util.Log
import java.io.BufferedReader
import javax.inject.Inject

class LrcParser @Inject constructor() : SubtitleParser {

    companion object {
        private const val TAG = "LrcParser"
        private val TIME_TAG_REGEX = Regex("\\[(\\d{2}): (\\d{2})\\. (\\d{2,3})\\]")
        private val WORD_TIME_TAG_REGEX = Regex("<(\\d{2}): (\\d{2})\\. (\\d{2,3})>")
    }

    override fun parse(content: String): List<Subtitle> {
        val subtitles = mutableListOf<Subtitle>()
        val normalizedContent = content.replace("\\r\\n", "\n").replace("\\r", "\n")

        val lines = normalizedContent.split("\\n")
        Log.d(TAG, "Total lines found: ${lines.size}")

        val lyricsWithTime = mutableListOf<LyricLine>()

        for (line in lines) {
            val trimmed = line.trim()
            if (trimmed.isEmpty()) continue

            // Extract all time tags from the line
            val timeTags = TIME_TAG_REGEX.findAll(trimmed).toList()
            if (timeTags.isEmpty()) continue

            // Get the text content (remove time tags)
            val textContent = trimmed.replace(TIME_TAG_REGEX, "").trim()
            if (textContent.isEmpty()) continue

            // Parse word-level timing if present
            val wordTimings = parseWordTimings(textContent)

            // Create a subtitle for each time tag (LRC can have multiple timestamps per line)
            for (timeTag in timeTags) {
                val (minutes, seconds, centis) = parseTimeComponents(timeTag)
                val startTimeMs = minutes.toLong() * 60000 + seconds.toLong() * 1000 + centis.toLong()

                val (contentEn, contentCn) = parseSubtitleContent(textContent)

                if (contentEn.isNotEmpty() || contentCn.isNotEmpty()) {
                    // For LRC, use the line index as subtitle index
                    val index = lyricsWithTime.size + 1
                    lyricsWithTime.add(LyricLine(index, startTimeMs, contentEn, contentCn, wordTimings))
                }
            }
        }
    }
}

```

```

    }
    }
}

// Sort by start time and assign proper indices
val sortedLyrics = lyricsWithTime.sortedBy { it.startTimeMs }
sortedLyrics.forEachIndexed { idx, lyric ->
    subtitles.add(Subtitle(idx + 1, lyric.startTimeMs, lyric.endTimeMs, lyric.contentEn, lyric.contentCn))
}

// Calculate end times based on next subtitle's start time
for (i in 0 until subtitles.size - 1) {
    val current = subtitles[i]
    val next = subtitles[i + 1]
    val updatedSubtitle = current.copy(endTimeMs = next.startTimeMs)
    subtitles[i] = updatedSubtitle
}

// Set end time for the last subtitle (default 5 seconds)
if (subtitles.isNotEmpty()) {
    val last = subtitles.last()
    subtitles[subtitles.size - 1] = last.copy(endTimeMs = last.startTimeMs + 5000)
}

Log.d(TAG, "Total LRC subtitles parsed: ${subtitles.size}")
return subtitles
}

fun parseStream(reader: BufferedReader): List<Subtitle> {
    val subtitles = mutableListOf<Subtitle>()
    val lyricsWithTime = mutableListOf<LyricLine>()
    var lineCount = 0

    try {
        var line: String? = reader.readLine()
        while (line != null) {
            lineCount++
            val trimmed = line.trim()
            if (trimmed.isNotEmpty()) {
                // Extract all time tags from the line
                val timeTags = TIME_TAG_REGEX.findAll(trimmed).toList()
                if (timeTags.isNotEmpty()) {
                    val textContent = trimmed.replace(TIME_TAG_REGEX, "").trim()
                    if (textContent.isNotEmpty()) {
                        val wordTimings = parseWordTimings(textContent)

                        for (timeTag in timeTags) {
                            val (minutes, seconds, centis) = parseTimeComponents(timeTag)
                            val startTimeMs = minutes.toLong() * 60000 + seconds.toLong() * 1000 + centis.toLong()

                            val (contentEn, contentCn) = parseSubtitleContent(textContent)

                            if (contentEn.isNotEmpty() || contentCn.isNotEmpty()) {
                                val index = lyricsWithTime.size + 1
                                lyricsWithTime.add(LyricLine(index, startTimeMs, contentEn, contentCn, wordTimings))
                            }
                        }
                    }
                }
            }
            line = reader.readLine()
        }
    } catch (e: Exception) {
        Log.e(TAG, "Error parsing LRC stream: ${e.message}")
    }

    return subtitles
}

```

```

        line = reader.readLine()

        // Safety limit
        if (lineCount > 100000) {
            Log.w(TAG, "Reached line limit, stopping parse")
            break
        }
    }
} catch (e: Exception) {
    Log.e(TAG, "Error reading stream: ${e.message}", e)
}

// Sort by start time and assign proper indices
val sortedLyrics = lyricsWithTime.sortedBy { it.startTimeMs }
sortedLyrics.forEachIndexed { idx, lyric ->
    subtitles.add(Subtitle(idx + 1, lyric.startTimeMs, lyric.endTimeMs, lyric.contentEn, lyric.contentCn))
}

// Calculate end times based on next subtitle's start time
for (i in 0 until subtitles.size - 1) {
    val current = subtitles[i]
    val next = subtitles[i + 1]
    subtitles[i] = current.copy(endTimeMs = next.startTimeMs)
}

// Set end time for the last subtitle (default 5 seconds)
if (subtitles.isNotEmpty()) {
    val last = subtitles.last()
    subtitles[subtitles.size - 1] = last.copy(endTimeMs = last.startTimeMs + 5000)
}

Log.d(TAG, "Stream parsed: ${subtitles.size} subtitles from $lineCount lines")
return subtitles
}

private fun parseWordTimings(text: String): List<WordTiming> {
    val timings = mutableList<WordTiming>()
    val matches = WORD_TIME_TAG_REGEX.findAll(text)

    for (match in matches) {
        val (minutes, seconds, centis) = parseTimeComponents(match)
        val timeMs = minutes.toLong() * 60000 + seconds.toLong() * 1000 + centis.toLong()
        // Extract the word after this timing tag
        val afterTag = text.substring(match.range.last + 1)
        val wordMatch = Regex("[^<\\[\\]]+").find(afterTag)
        if (wordMatch != null) {
            timings.add(WordTiming(timeMs, wordMatch.value.trim()))
        }
    }

    return timings
}

private fun parseTimeComponents(tag: MatchResult): Triple<Int, Int, Int> {
    val values = tag.destructured
    val parts = values.toList()
    val minutes = parts[0].toIntOrNull() ?: 0
    val seconds = parts[1].toIntOrNull() ?: 0
    val centis = if (parts[2].length == 3) {
        parts[2].toIntOrNull() ?: 0
    } else {
        (parts[2].toIntOrNull() ?: 0) * 10
    }
}

```

```

        return Triple(minutes, seconds, centis)
    }

    private fun parseSubtitleContent(text: String): Pair<String, String> {
        if (text.isEmpty()) return Pair("", "")

        // Remove word-level timing tags for content analysis
        val cleanText = text.replace(WORD_TIME_TAG_REGEX, "")

        val chineseRegex = Regex("[\\u4e00-\\u9fff]+")

        if (chineseRegex.containsMatchIn(cleanText)) {
            return splitBilingual(cleanText)
        }

        return Pair(cleanText.trim(), "")
    }

    private fun splitBilingual(content: String): Pair<String, String> {
        val chineseRegex = Regex("[\\u4e00-\\u9fff]+")
        val chineseMatches = chineseRegex.findAll(content).toList()

        if (chineseMatches.isEmpty()) {
            return Pair(content, "")
        }

        val firstChineseIndex = chineseMatches.first().range.first
        val englishPart = content.substring(0, firstChineseIndex).trim()
        val chinesePart = content.substring(firstChineseIndex).trim()

        return Pair(englishPart, chinesePart)
    }

    override fun findSubtitleAtTime(subtitles: List<Subtitle>, timeMs: Long): Subtitle? {
        return subtitles.find { it.startTimeMs <= timeMs && it.endTimeMs >= timeMs }
    }

    private data class LyricLine(
        val index: Int,
        val startTimeMs: Long,
        val contentEn: String,
        val contentCn: String,
        val wordTimings: List<WordTiming> = emptyList()
    ) {
        val endTimeMs: Long = 0L // Will be calculated later
    }

    private data class WordTiming(
        val timeMs: Long,
        val word: String
    )
}

// === app\\src\\main\\java\\com\\echoling\\app\\player\\subtitle\\SrtParser.kt ===

package com.echoling.app.player.subtitle

import android.util.Log
import javax.inject.Inject

class SrtParser @Inject constructor() : SubtitleParser {

```

```

companion object {
    private const val TAG = "SrtParser"
}

override fun parse(content: String): List<Subtitle> {
    val subtitles = mutableListOf<Subtitle>()

    // Normalize line endings
    val normalizedContent = content.replace("\r\n", "\n").replace("\r", "\n")

    // Split by empty lines to get subtitle blocks
    val blocks = normalizedContent.split(Regex("\n\n+"))

    Log.d(TAG, "Total blocks found: ${blocks.size}")

    for (block in blocks) {
        val trimmedBlock = block.trim()
        if (trimmedBlock.isEmpty()) continue

        val lines = trimmedBlock.split("\n")
        if (lines.size < 2) continue

        // First line should be the index number
        val indexLine = lines[0].trim()
        val index = indexLine.toIntOrNull()
        if (index == null) {
            Log.w(TAG, "Failed to parse index from: $indexLine")
            continue
        }

        // Second line should be the timestamp
        if (lines.size < 2) continue
        val timeLine = lines[1].trim()
        val timeParts = timeLine.split("-->")
        if (timeParts.size != 2) {
            Log.w(TAG, "Invalid timestamp line: $timeLine")
            continue
        }

        val startTime = parseTime(timeParts[0].trim())
        val endTime = parseTime(timeParts[1].trim())
        if (startTime == null || endTime == null) {
            Log.w(TAG, "Failed to parse times from: $timeLine")
            continue
        }

        // Rest is subtitle content
        val subtitleLines = lines.drop(2).map { it.trim() }.filter { it.isNotEmpty() }
        if (subtitleLines.isEmpty()) continue

        val (contentEn, contentCn) = parseSubtitleContent(subtitleLines)

        if (contentEn.isNotEmpty() || contentCn.isNotEmpty()) {
            subtitles.add(Subtitle(index, startTime, endTime, contentEn, contentCn))
            Log.d(TAG, "Parsed subtitle $index: $contentEn / $contentCn")
        }
    }

    Log.d(TAG, "Total subtitles parsed: ${subtitles.size}")
    return subtitles.sortedBy { it.startTimeMs }
}

private fun parseSubtitleContent(lines: List<String>): Pair<String, String> {

```



```

        if (lines.isEmpty()) return Pair("", "")

        // Join all lines - SRT subtitles can have multiple lines per entry
        val fullContent = lines.joinToString("\n").trim()

        if (fullContent.isEmpty()) return Pair("", "")

        // Chinese Unicode range
        val chineseRegex = Regex("[\\u4e00-\\u9fff]")

        if (chineseRegex.containsMatchIn(fullContent)) {
            // Has Chinese - try to split bilingual
            return splitBilingual(fullContent)
        }

        // English only
        return Pair(fullContent, "")
    }

    private fun splitBilingual(content: String): Pair<String, String> {
        val chineseRegex = Regex("[\\u4e00-\\u9fff]+")
        val chineseMatches = chineseRegex.findAll(content).toList()

        if (chineseMatches.isEmpty()) {
            return Pair(content, "")
        }

        val firstChineseIndex = chineseMatches.first().range.first

        val englishPart = content.substring(0, firstChineseIndex).trim()
        val chinesePart = content.substring(firstChineseIndex).trim()

        return Pair(englishPart, chinesePart)
    }

    override fun findSubtitleAtTime(subtitles: List<Subtitle>, timeMs: Long): Subtitle? {
        return subtitles.find { it.startTimeMs <= timeMs && it.endTimeMs >= timeMs }
    }

    private fun parseTime(timeStr: String): Long? {
        // Format: HH:MM:SS,mmm or HH:MM:SS.mmm
        val regex = Regex("(\\d{2}): (\\d{2}): (\\d{2})[.,] (\\d{3})")
        val match = regex.find(timeStr) ?: return null

        val (hours, minutes, seconds, millis) = match.destructured
        return (hours.toLongOrNull() ?: return null) * 3600000 +
            (minutes.toLongOrNull() ?: return null) * 60000 +
            (seconds.toLongOrNull() ?: return null) * 1000 +
            (millis.toLongOrNull() ?: 0).coerceIn(0, 999)
    }
}

// === app\\src\\main\\java\\com\\echoling\\app\\player\\subtitle\\Subtitle.kt ===

package com.echoling.app.player.subtitle

data class Subtitle(
    val index: Int,
    val startTimeMs: Long,
    val endTimeMs: Long,
    val contentEn: String,

```

```
        val contentCn: String = ""
    )

// === app\src\main\java\com\echoling\app\player\subtitle\SubtitleParser.kt ===

package com.echoling.app.player.subtitle

interface SubtitleParser {
    fun parse(content: String): List<Subtitle>
    fun findSubtitleAtTime(subtitles: List<Subtitle>, timeMs: Long): Subtitle?
}

// === app\src\main\java\com\echoling\app\player\subtitle\SubtitleParserFactory.kt ===

package com.echoling.app.player.subtitle

import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class SubtitleParserFactory @Inject constructor(
    private val srtParser: SrtParser,
    private val assParser: AssParser
) {
    fun getParser(fileName: String): SubtitleParser {
        return when {
            fileName.endsWith(".srt", ignoreCase = true) -> srtParser
            fileName.endsWith(".ass", ignoreCase = true) -> assParser
            fileName.endsWith(".ssa", ignoreCase = true) -> assParser
            else -> srtParser
        }
    }

    fun parseSubtitles(content: String, fileName: String): List<Subtitle> {
        val parser = getParser(fileName)
        return parser.parse(content)
    }
}

// === app\src\main\java\com\echoling\app\player\TtsManager.kt ===

package com.echoling.app.player

import android.content.Context
import android.media.AudioAttributes
import android.os.Handler
import android.os.Looper
import android.speech.tts.TextToSpeech
import android.speech.tts.UtteranceProgressListener
import android.util.Log
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import java.util.Locale
import javax.inject.Inject
import javax.inject.Singleton
```

```

/**
 * Application-scoped Text-to-Speech wrapper.
 *
 * Lives in `player/` next to [AudioPlayer] (the ExoPlayer wrapper) — same
 * architectural role: a side-effect service that the ViewModels inject
 * directly without a UseCase layer (CLAUDE.md § 5.3 — only Repository
 * calls need UseCases).
 *
 * Multi-engine fallback — different devices ship with different TTS
 * engines, and many devices (especially Chinese Xiaomi without Google
 * services) ship with NONE installed. We try a curated list of engine
 * package names in order, taking the first one that binds successfully.
 * If all fail, [isAvailable] stays false and the ViewModels surface a
 * "please install a TTS engine" snackbar instead of silently failing.
 *
 * Defensive — every operation on a TTS engine (shutdown, setLanguage,
 * setAudioAttributes, etc.) is wrapped in try/catch because third-party
 * engines (especially Chinese ones like iFlytek) often have quirky
 * implementations that throw on standard API calls. If any single
 * engine throws during init or config, we recover and try the next one
 * rather than crashing the whole app.
 *
 * Init is async (the [TextToSpeech.OnInitListener] callback fires on the
 * binder thread). We expose [isReady] as a StateFlow and queue any
 * speak() calls that arrive before init completes, draining the queue
 * on init success.
 *
 * Critical: `setAudioAttributes` is called first thing on API 21+.
 * Without it, TTS audio routes to STREAM_MUSIC with default attributes,
 * which on some devices (especially Chinese OEM ROMs) means the audio
 * plays through the ringer/notification stream and is muted by silent
 * mode, or doesn't reach the media volume control at all. With
 * USAGE_MEDIA + CONTENT_TYPE_SPEECH the audio follows the user's media
 * volume — same stream as a song.
 */
@Singleton
class TtsManager @Inject constructor(
    @ApplicationContext private val context: Context,
) {
    // Nullable backing field. Cannot be `private val tts = TextToSpeech(...) {
    //     tts.setLanguage(...) }` because the init lambda references `tts`
    // while Kotlin is still initializing it — the compiler (correctly)
    // refuses with "Variable 'tts' must be initialized". Defer assignment
    // to an init block so the property is fully set before any callback
    // can observe it.
    private var tts: TextToSpeech? = null

    // True once at least one engine has successfully bound. False while
    // we're trying engines / after all have failed. ViewModels read this
    // to decide whether to surface an "install a TTS engine" snackbar.
    private val _isAvailable = MutableStateFlow(false)
    val isAvailable: StateFlow<Boolean> = _isAvailable.asStateFlow()

    // True once the current engine is bound and ready to speak. False
    // while we're still trying engines. speak() gates on this.
    private val _isReady = MutableStateFlow(false)
    val isReady: StateFlow<Boolean> = _isReady.asStateFlow()

    // Pending speak() calls accumulated before any engine binds. Drained
    // on the first SUCCESS. Synchronized because init callbacks run on a
    // binder thread and speak() is called from the main thread. Capped
    // at [MAX_PENDING] to prevent unbounded growth if no engine ever
    // binds.

```

```

private val pending = mutableListOf<String>()
private val lock = Any()

// Main-thread handler for the per-engine watchdog and inter-engine
// spacing delay.
private val mainHandler = Handler(Looper.getMainLooper())

/**
 * Engine candidate list. Tried in order; first one whose
 * OnInitListener returns SUCCESS wins. `null` at the end asks the
 * system for its default engine.
 *
 * **CRITICAL: must be declared BEFORE [init]** — Kotlin
 * initializes properties in source order, and the `init` block
 * runs immediately after the primary constructor body (before
 * any property declared after the init block is assigned). If
 * `engineCandidates` is declared below `init`, it will still be
 * its default null value when [tryNextEngine] reads
 * `engineCandidates.size` → NullPointerException → app crash
 * before the engine loop even starts. The crash was observed on
 * the user's Xiaomi Mi 11 after installing iFlytek: the first
 * time TtsManager was injected (on tapping 高中英语词汇), the
 * `init { tryNextEngine(0) }` ran and threw on
 * `engineCandidates.size` — TtsManager was never usable.
 *
 * **List construction** — the static "known good" candidate list
 * is augmented at init time with any TTS engine the system has
 * actually installed and registered as a `TTS_SERVICE`
 * IntentService (queried via [PackageManager.queryIntentServices]).
 * This catches TTS engines whose package names we didn't
 * hardcode — notably 「讯飞语记」/iFlytek Yuji (`com.iflytek.yujizhushou`),
 * which is a note-taking app that bundles its own TTS engine
 * service but is NOT exposed under the SDK package
 * `com.iflytek.speechsuite`. Without the runtime query, our
 * hardcoded list misses every Chinese vendor's TTS that ships
 * under a "wrapper app" package name (note app, dictionary app,
 * keyboard app, ...).
 *
 * Order rationale:
 * 1. Google TTS — best English quality, ubiquitous on GMS devices
 * 2. Xiaomi TTS — pre-installed on global MIUI builds
 * 3. iFlytek — common on CN Xiaomi/Huawei
 * 4. Baidu — alternative CN option
 * 5. Android Pico — old AOSP fallback
 * 6. svov.pico — even older AOSP
 * 7. any TTS service the system knows about via PackageManager
 * (catches vendor-specific package names we didn't list)
 * 8. null — system default (last resort)
 */
private val engineCandidates: List<String?> = run {
    val known = listOf(
        "com.google.android.tts",
        "com.xiaomi.tts",
        "com.iflytek.speechsuite",
        "com.baidu.duersdk.tts",
        "com.android.tts",
        "com.svox.pico",
    )
    val discovered = try {
        context.packageManager
            .queryIntentServices(
                android.content.Intent(android.speech.tts.TextToSpeech.Engine.INTENT_ACTION_TTS_SERVICE),
                0,
            )
    }
}

```

```

    )
    .mapNotNull { it.serviceInfo?.packageName }
    .filter { it !in known }
} catch (e: Throwable) {
    Log.w(TAG, "queryIntentServices for TTS failed — using hardcoded list only", e)
    emptyList()
}
Log.i(TAG, "Discovered TTS engines via PackageManager: $discovered")
known + discovered + null
}

init {
    tryNextEngine(0)
}

/**
 * Attempt to bind to engineCandidates[index]. On success, configure
 * the engine. On failure (or watchdog timeout, or any exception),
 * try the next candidate. After all candidates exhausted, log +
 * flip isAvailable=false.
 *
 * Every operation is wrapped in try/catch because third-party TTS
 * engines (notably iFlytek on Xiaomi CN devices) have been observed
 * to throw on `shutdown()` when called before init completes, and
 * to throw on `setLanguage()` when the language data isn't fully
 * loaded. Without the defensive wrappers, a single bad engine would
 * crash the app.
 */
private fun tryNextEngine(index: Int) {
    if (index >= engineCandidates.size) {
        Log.w(TAG, "All ${engineCandidates.size} TTS engine candidates " +
            "failed to init — no TTS available on this device.")
        _isAvailable.value = false
        _isReady.value = false
        return
    }
    val engine = engineCandidates[index]
    Log.d(TAG, "Trying TTS engine [$index/${engineCandidates.size}]: $engine")

    // Safely shut down the previous instance. Some engines throw
    // if shutdown is called before init completes; catch + log.
    try {
        tts?.shutdown()
    } catch (e: Throwable) {
        Log.w(TAG, "shutdown() threw for previous engine — ignoring", e)
    }
    tts = null

    // Inter-engine spacing: 50ms delay so the previous shutdown
    // has time to dispatch its disconnect message before we try
    // to bind the next engine. Without this, some engines (notably
    // iFlytek) get into a confused state and crash on the next
    // bind attempt.
    mainHandler.postDelayed({
        tryBindEngine(index, engine)
    }, INTER_ENGINE_DELAY_MS)
}

/**
 * Actually create the TextToSpeech for [engine]. Separated from
 * [tryNextEngine] so we can apply an inter-engine delay via
 * [mainHandler.postDelayed].
 */

```

```

private fun tryBindEngine(index: Int, engine: String?) {
    try {
        tts = TextToSpeech(context, { status ->
            try {
                if (status == TextToSpeech.SUCCESS) {
                    Log.w(TAG, "Engine [$index] $engine init SUCCESS")
                    onEngineSuccess(engine, index)
                } else {
                    Log.w(TAG, "Engine [$index] $engine init FAILED, status=$status")
                    // removeCallbacksAndMessages(null) drops ALL
                    // pending messages on the handler — cancels
                    // the per-engine watchdog.
                    mainHandler.removeCallbacksAndMessages(null)
                    tryNextEngine(index + 1)
                }
            }
        } catch (e: Throwable) {
            Log.w(TAG, "OnInit callback threw for engine $engine — trying next", e)
            mainHandler.removeCallbacksAndMessages(null)
            tryNextEngine(index + 1)
        }
    }, engine)
} catch (e: Throwable) {
    // Some engines throw synchronously from the constructor
    // (e.g. on devices where the engine package is partially
    // installed but broken). Skip and try next.
    Log.w(TAG, "TextToSpeech() constructor threw for engine $engine — trying next", e)
    mainHandler.removeCallbacksAndMessages(null)
    tryNextEngine(index + 1)
    return
}

// Per-engine watchdog. If the callback doesn't fire within
// [ENGINE_BIND_TIMEOUT_MS], give up on this engine and try
// the next.
mainHandler.postDelayed({
    if (!isReady.value && tts != null) {
        Log.w(TAG, "Engine [$index] $engine did not respond within " +
            "${ENGINE_BIND_TIMEOUT_MS}ms — trying next")
        tryNextEngine(index + 1)
    }
}, ENGINE_BIND_TIMEOUT_MS)
}

/**
 * Engine bound successfully — finish configuration. Every config
 * call is wrapped in try/catch because some engines don't support
 * all standard APIs (e.g. some iFlytek builds throw on
 * setAudioAttributes).
 */
private fun onEngineSuccess(engine: String?, index: Int) {
    // Cancel the watchdog for this engine.
    mainHandler.removeCallbacksAndMessages(null)

    // CRITICAL — set audio attributes FIRST, before any
    // setLanguage / speak call.
    try {
        tts?.setAudioAttributes(
            AudioAttributes.Builder()
                .setContentType(AudioAttributes.CONTENT_TYPE_SPEECH)
                .setUsage(AudioAttributes.USAGE_MEDIA)
                .build()
        )
    } catch (e: Throwable) {

```

```

        Log.w(TAG, "setAudioAttributes threw for $engine — continuing", e)
    }

    try {
        val langResult = tts?.setLanguage(Locale.US)
        Log.w(TAG, "    setLanguage(Locale.US)=$langResult")
    } catch (e: Throwable) {
        Log.w(TAG, "setLanguage threw for $engine — continuing", e)
    }

    // Explicit defaults — some engines don't initialize these on
    // their own. Wrap each in case the engine doesn't support them.
    try { tts?.setSpeechRate(1.0f) } catch (e: Throwable) {
        Log.w(TAG, "setSpeechRate threw — continuing", e)
    }
    try { tts?.setPitch(1.0f) } catch (e: Throwable) {
        Log.w(TAG, "setPitch threw — continuing", e)
    }

    // UtteranceProgressListener — diagnostic only.
    try {
        tts?.setOnUtteranceProgressListener(object : UtteranceProgressListener() {
            override fun onStart(utteranceId: String?) {
                Log.d(TAG, "utterance onStart: $utteranceId")
            }
            override fun onDone(utteranceId: String?) {
                Log.d(TAG, "utterance onDone: $utteranceId")
            }
            @Deprecated("Deprecated in Java")
            override fun onError(utteranceId: String?) {
                Log.w(TAG, "utterance onError: $utteranceId")
            }
            override fun onError(utteranceId: String?, errorCode: Int) {
                Log.w(TAG, "utterance onError: $utteranceId, code=$errorCode")
            }
        })
    } catch (e: Throwable) {
        Log.w(TAG, "setOnUtteranceProgressListener threw — continuing", e)
    }

    _isAvailable.value = true
    _isReady.value = true
    drainPendingQueue()
}

/**
 * Speak [word] aloud. Safe to call before any engine binds — the
 * word is queued and played once the engine reports ready.
 * Re-tapping the speaker cancels any in-flight utterance
 * (QUEUE_FLUSH) rather than chaining.
 */
fun speak(word: String) {
    if (word.isBlank()) return
    if (_isReady.value) {
        try {
            // Flush mode: re-tapping cancels the prior utterance.
            val result = tts?.speak(word, TextToSpeech.QUEUE_FLUSH, null, utteranceId(word))
            Log.d(TAG, "speak('$word') result=$result")
        } catch (e: Throwable) {
            Log.w(TAG, "speak() threw for '$word'", e)
        }
    } else {
        synchronized(lock) {

```

```

        if (pending.size >= MAX_PENDING) {
            pending.removeAt(0)
        }
        pending.add(word)
    }
    Log.d(TAG, "speak('$word') queued (isReady=false), pending.size=${pending.size}")
}

private fun drainPendingQueue() {
    val drained: List<String>
    synchronized(lock) {
        drained = pending.toList()
        pending.clear()
    }
    drained.lastOrNull()?.let { last ->
        try {
            tts?.speak(last, TextToSpeech.QUEUE_FLUSH, null, utteranceId(last))
        } catch (e: Throwable) {
            Log.w(TAG, "drainPendingQueue speak() threw", e)
        }
    }
}

private fun utteranceId(word: String): String = "tts-${word.hashCode()}"

private companion object {
    const val TAG = "TtsManager"
    // Per-engine bind timeout.
    const val ENGINE_BIND_TIMEOUT_MS = 2_000L
    // Delay between consecutive engine attempts. Gives the
    // previous engine's shutdown message time to dispatch before
    // we try to bind the next one — some engines (iFlytek) get
    // confused with back-to-back shutdown+bind.
    const val INTER_ENGINE_DELAY_MS = 50L
    const val MAX_PENDING = 5
}

}

// === app\src\main\java\com\echoling\app\presentation\MainActivity.kt ===

package com.echoling.app.presentation

import android.graphics.Color
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.ui.Modifier
import com.echoling.app.R
import com.echoling.app.presentation.ui.navigation.MainScaffold
import com.echoling.app.presentation.ui.theme.EchoLingTheme
import dagger.hilt.android.AndroidEntryPoint

@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        // The launcher activity is declared with Theme.EchoLing.Splash in

```



```

// AndroidManifest so the system paints the splash image as the
// window background the moment the activity is created — this
// eliminates the pre-Compose white flash. Once onCreate starts
// we swap to the regular Compose theme so window decorations,
// status bar styling, etc. match the rest of the app.
setTheme(R.style.Theme_EchoLing)
super.onCreate(savedInstanceState)

enableEdgeToEdge()
// §12.32b: explicitly disable the navigation-bar contrast scrim
// (Android 10+). `enableEdgeToEdge()` already makes the nav bar
// transparent, but on Android 10+ the system can still enforce a
// translucent scrim on top of the app's content in the nav-bar
// zone to guarantee the gesture handle is readable. On a
// gesture-nav Xiaomi Mi 11 CN that scrim paints a near-white
// wash (253, 253, 255) over the bottom 16dp of our bar's
// surface, creating the "很浅的颜色" (very light color) the
// user reported at the bottom of the bar. Setting
// `isNavigationBarContrastEnforced = false` removes the scrim
// so the bar's surface color shows through cleanly behind the
// gesture handle.
if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.Q) {
    window.isNavigationBarContrastEnforced = false
}
// Belt-and-suspenders: also force the nav bar color to fully
// transparent. `enableEdgeToEdge()` already does this on
// API 29+, but some OEM skins (MIUI) honor
// `window.navigationBarColor` more strictly than the contrast-
// enforced flag.
window.navigationBarColor = Color.TRANSPARENT
setContent {
    EchoLingTheme {
        Surface(
            modifier = Modifier.fillMaxSize(),
            color = MaterialTheme.colorScheme.background,
        ) {
            MainScaffold()
        }
    }
}
}

// == app\src\main\java\com\echoling\app\presentation\ui\components\CompactConfirmDialog.kt ==

package com.echoling.app.presentation.ui.components

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton

```

```

import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.compose.ui.window.Dialog

/**
 * Compact replacement for [androidx.compose.material3.AlertDialog] used for
 * short confirmation prompts (delete, discard, etc.). The default M3
 * AlertDialog is wide (~280-560dp) with a 24sp `headlineSmall` title and
 * 24dp internal padding — visually too heavy for a "are you sure?"
 * affordance. This version is fixed at 240dp wide, with 16dp padding,
 * `titleSmall` (14sp) title and `bodySmall` (12sp) message.
 *
 * The confirm button is colored `colorScheme.error` so destructive
 * actions stay visually distinct. Both buttons use `labelMedium` and
 * tight content padding so the row stays narrow.
 *
 * Used by:
 * - [com.echoling.app.presentation.ui.screens.vocabulary.VocabularyScreen]
 *   (delete a saved word)
 * - [com.echoling.app.presentation.ui.screens.courses.CoursesScreen]
 *   (delete an imported course)
 */
@Composable
fun CompactConfirmDialog(
    title: String,
    message: String,
    confirmText: String,
    dismissText: String,
    onConfirm: () -> Unit,
    onDismiss: () -> Unit,
) {
    Dialog(onDismissRequest = onDismiss) {
        Surface(
            shape = RoundedCornerShape(12.dp),
            color = MaterialTheme.colorScheme.surface,
            tonalElevation = 6.dp,
            modifier = Modifier.width(240.dp),
        ) {
            Column(modifier = Modifier.padding(horizontal = 16.dp, vertical = 14.dp)) {
                Text(
                    text = title,
                    style = MaterialTheme.typography.titleSmall,
                    color = MaterialTheme.colorScheme.onSurface,
                )
                Spacer(modifier = Modifier.height(6.dp))
                Text(
                    text = message,
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                )
                Spacer(modifier = Modifier.height(10.dp))
                Row(
                    horizontalArrangement = Arrangement.End,
                    verticalAlignment = Alignment.CenterVertically,
                    modifier = Modifier.fillMaxWidth(),
                ) {
                    TextButton(
                        onClick = onDismiss,
                        contentPadding = PaddingValues(horizontal = 8.dp, vertical = 4.dp),
                    ) {
                        Text(

```

```

        text = dismissText,
        style = MaterialTheme.typography.labelMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
    )
}
Spacer(modifier = Modifier.width(4.dp))
TextButton(
    onClick = onConfirm,
    contentPadding = PaddingValues(horizontal = 8.dp, vertical = 4.dp),
) {
    Text(
        text = confirmText,
        style = MaterialTheme.typography.labelMedium,
        color = MaterialTheme.colorScheme.error,
    )
}
}
}
}
}

// === app\src\main\java\com\echoling\app\presentation\ui\components\ContinueLearningCard.kt ===

package com.echoling.app.presentation.ui.components

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.offset
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.outlined.PlayCircle
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FilledIconButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButtonDefaults
import androidx.compose.material3.LinearProgressIndicator
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp

/**
 * "Continue learning" hero card — the visual centerpiece of the Courses
 * tab. Sits at the top of the scroll, above [StatsSummaryCard], and shows
 * the course the user most recently started so they can resume in one tap.

```

```

*
* Visual recipe (see CLAUDE.md §12.16):
* - Solid violet-500 (#8B5CF6) background. Previously (§11.2) a deep
*   violet-600 → violet-700 linear gradient (felt too heavy); then
*   (§12.32c) a violet-400 → violet-500 linear gradient (felt "灰蒙蒙"
*   because the two shades were only ~10% apart in lightness and the
*   gradient read as a flat wash). 2026-07-04 settled on solid
*   violet-500 — one shade deeper than the §12.32c start, so it has
*   more presence than the near-flat gradient did, while staying
*   clearly less aggressive than the original §11.2 pair.
* - Two faint translucent white radial orbs (top-right + bottom-left)
*   for the "depth" feel that a flat color needs to read as a hero
*   rather than a sticker
* - 24dp rounded corners, 4dp tonal elevation, 20dp internal padding
* - "继续学习" eyebrow row (PlayCircle icon + label, white 85%)
* - Course title (titleSmall 14sp — same as [CourseListItem], white)
* - 56dp circular play button (white background, violet-400 icon) on
*   the right of the title row — the visual anchor of the hero
* - Progress bar with white fill + 25% white track, then "62% 完成" label
*
* Tapping the card or the play button calls [onClick]. Visual contrast is
* intentional: the lighter-purple gradient hero is the only saturated
* card on the page; [StatsSummaryCard] below stays in `primaryContainer`
* so the eye lands here first.
*/
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ContinueLearningCard(
    title: String,
    progress: Float,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
) {
    Card(
        onClick = onClick,
        modifier = modifier.fillMaxWidth(),
        shape = RoundedCornerShape(24.dp),
        colors = CardDefaults.cardColors(containerColor = Color.Transparent),
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
    ) {
        Box(
            modifier = Modifier
                .fillMaxWidth()
                // (2026-07-04) Background went through three states:
                // - original: deep violet-600 → violet-700 linear
                //   gradient (§11.2). User said it felt "too heavy".
                // - §12.32c: lightened to violet-400 → violet-500
                //   linear gradient. User now says it feels "灰蒙蒙"
                //   — the two shades are only ~10% apart in
                //   lightness so the gradient reads as a flat muddy
                //   wash instead of a hero.
                // - now: solid violet-500 (#8B5CF6) — mid-bright,
                //   one shade deeper than the gradient's start so
                //   it has more presence than the §12.32c palette.
                // The radial orb a few lines down (top-right
                // translucent white halo) already provides the
                // hero "depth" feel; a single solid color lets
                // that orb read as an actual highlight instead
                // of getting visually lost in a near-flat
                // gradient.
                .background(Color(0xFF8B5CF6)),
        ) {
            // Top-right translucent orb — gives the hero a sense of depth

```

```

Box(
  modifier = Modifier
    .align(Alignment.TopEnd)
    .size(160.dp)
    .background(
      brush = Brush.radialGradient(
        colors = listOf(
          Color.White.copy(alpha = 0.18f),
          Color.Transparent,
        ),
        radius = 260f,
      ),
    ),
)
// Bottom-left smaller orb — balances the composition
Box(
  modifier = Modifier
    .align(Alignment.BottomStart)
    .size(110.dp)
    .offset(x = (-20).dp, y = 30.dp)
    .background(
      brush = Brush.radialGradient(
        colors = listOf(
          Color.White.copy(alpha = 0.10f),
          Color.Transparent,
        ),
        radius = 200f,
      ),
    ),
)

Column(modifier = Modifier.padding(20.dp)) {
  Row(
    verticalAlignment = Alignment.CenterVertically,
    horizontalArrangement = Arrangement.spacedBy(6.dp),
  ) {
    Icon(
      imageVector = Icons.Outlined.PlayCircle,
      contentDescription = null,
      modifier = Modifier.size(16.dp),
      tint = Color.White.copy(alpha = 0.85f),
    )
    Text(
      text = "继续学习",
      style = MaterialTheme.typography.labelMedium,
      color = Color.White.copy(alpha = 0.85f),
    )
  }
  Spacer(modifier = Modifier.height(10.dp))

  Row(
    verticalAlignment = Alignment.CenterVertically,
    horizontalArrangement = Arrangement.spacedBy(12.dp),
  ) {
    Text(
      text = title,
      // Match the course-list item title (titleSmall
      // 14sp) so the two surfaces feel like the same
      // card system rather than two unrelated ones.
      // 2026-07-03 reduced titleMedium → titleSmall to
      // match the rest of the courses UI (the home
      // page already uses smaller fonts for density).
      style = MaterialTheme.typography.titleSmall,
    )
  }
}

```

```

        color = Color.White,
        maxLines = 2,
        overflow = TextOverflow.Ellipsis,
        modifier = Modifier.weight(1f),
    )
    FilledIconButton(
        onClick = onClick,
        modifier = Modifier.size(56.dp),
        shape = RoundedCornerShape(28.dp),
        colors = IconButtonDefaults.filledIconButtonColors(
            containerColor = Color.White,
            // § 12.32c: contentColor (the icon inside
            // the white circle) lightened to match the
            // new lighter-purple hero gradient.
            contentColor = Color(0xFFA78BFA),
        ),
    ) {
        Icon(
            imageVector = Icons.Outlined.PlayCircle,
            contentDescription = "Start",
            modifier = Modifier.size(30.dp),
            tint = Color(0xFFA78BFA),
        )
    }
}

Spacer(modifier = Modifier.height(14.dp))

LinearProgressIndicator(
    progress = progress.coerceIn(0f, 1f),
    modifier = Modifier
        .fillMaxWidth()
        .height(8.dp)
        .clip(RoundedCornerShape(4.dp)),
    color = Color.White,
    trackColor = Color.White.copy(alpha = 0.25f),
)
Spacer(modifier = Modifier.height(6.dp))
Text(
    text = "${(progress * 100).toInt()}% 完成",
    style = MaterialTheme.typography.bodySmall,
    color = Color.White.copy(alpha = 0.85f),
)
}
}
}

// === app\src\main\java\com\echoling\app\presentation\ui\components\CourseGroupItem.kt ===

package com.echoling.app.presentation.ui.components

import androidx.compose.animation.core.animateDpAsState
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.interaction.collectIsPressedAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.IntrinsicSize
import androidx.compose.foundation.layout.Row

```

```

import androidx.compose.foundation.layout.fillMaxHeight
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Folder
import androidx.compose.material.icons.filled.KeyboardArrowRight
import androidx.compose.material.ripple.rememberRipple
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.scale
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp

/**
 * Folder-style row on the Courses-tab home page. Represents a group of
 * lessons sharing a `courseName` and shows the group name + lesson
 * count. Tapping navigates to the category detail screen.
 *
 * Visual recipe mirrors [CourseListItem] (CLAUDE.md §12.16) — same
 * 16dp rounded card, same 4dp primary accent bar, same press animation
 * (elevation 1→8dp, scale 1.0→1.02). The accent color is always
 * `primary` here because `difficulty` is a per-course concept, not a
 * per-group one.
 */
@Composable
fun CourseGroupItem(
    courseName: String,
    count: Int,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
) {
    val interactionSource = remember { MutableInteractionSource() }
    val isPressed by interactionSource.collectIsPressedAsState()
    val animatedElevation by animateDpAsState(
        targetValue = if (isPressed) 8.dp else 1.dp,
        label = "groupCardElevation",
    )
    val animatedScale by animateFloatAsState(
        targetValue = if (isPressed) 1.02f else 1.0f,
        label = "groupCardScale",
    )

    Card(
        modifier = modifier
            .fillMaxWidth()
            .scale(animatedScale)
            .clip(RoundedCornerShape(16.dp))
            .clickable(

```

```

        interactionSource = interactionSource,
        indication = rememberRipple(),
        onClick = onClick,
    ),
    shape = RoundedCornerShape(16.dp),
    colors = CardDefaults.cardColors(
        containerColor = MaterialTheme.colorScheme.surface,
    ),
    elevation = CardDefaults.cardElevation(defaultElevation = animatedElevation),
) {
    // IntrinsicSize.Min lets the accent bar fillMaxHeight() to the
    // card's natural height without forcing a fixed dp value.
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .height(IntrinsicSize.Min),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Box(
            modifier = Modifier
                .width(4.dp)
                .fillMaxHeight()
                // §12.32c: accent bar lightened to match the
                // new lighter brand purple (#A78BFA) — same
                // shade used for the hero gradient and the
                // folder icon, so the courses area reads as
                // one consistent palette.
                .background(Color(0xFFA78BFA)),
        )

        Row(
            modifier = Modifier
                .weight(1f)
                .padding(horizontal = 16.dp, vertical = 14.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(12.dp),
        ) {
            Icon(
                imageVector = Icons.Filled.Folder,
                contentDescription = null,
                modifier = Modifier.size(28.dp),
                // §12.32c: folder icon tint lightened to
                // violet-400 (was the deep primary purple).
                tint = Color(0xFFA78BFA),
            )

            Text(
                text = courseName,
                // titleMedium (16sp) → titleSmall (14sp): keeps
                // parity with ContinueLearningCard, CourseListItem,
                // and CourseDetailScreen header so the courses area
                // reads as one consistent type system.
                style = MaterialTheme.typography.titleSmall,
                fontWeight = FontWeight.SemiBold,
                maxLines = 1,
                overflow = TextOverflow.Ellipsis,
                modifier = Modifier.weight(1f),
            )

            Text(
                text = "$count 课",
                style = MaterialTheme.typography.labelMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
                maxLines = 1,
            )
        }
    }
}

```



```

        Icon(
            imageVector = Icons.Filled.KeyboardArrowRight,
            contentDescription = null,
            modifier = Modifier.size(20.dp),
            tint = MaterialTheme.colorScheme.onSurfaceVariant,
        )
    }
}
}

// === app\src\main\java\com\echoling\app\presentation\ui\components\CourseListItem.kt ===

package com.echoling.app.presentation.ui.components

import androidx.compose.animation.core.animateDpAsState
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.interaction.collectIsPressedAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.IntrinsicSize
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxHeight
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.layout.widthIn
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.PlayArrow
import androidx.compose.material.icons.outlined.Headphones
import androidx.compose.material.icons.outlined.Mic
import androidx.compose.material.icons.outlined.VideoFile
import androidx.compose.material.ripple.rememberRipple
import androidx.compose.material3.AssistChip
import androidx.compose.material3.AssistChipDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.FilledIconButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.IconButtonDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.scale
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.style.TextOverflow

```

```

import androidx.compose.ui.unit.dp
import com.echoling.app.domain.model.Course

/**
 * Single row in the Courses tab's course list. Shows the course title,
 * description, difficulty chip, audio/video/subtitle indicators, and
 * duration. Has a delete affordance and a primary "start" button.
 *
 * Modern card recipe (CLAUDE.md §12.16):
 * - 4dp left accent bar, color picked by difficulty tier
 *   (A* → primary purple, B* → tertiary, C* → secondary, else → primary)
 * - Press animation: elevation 1dp → 8dp, scale 1.0 → 1.02, both eased
 *   with [animateDpAsState] / [animateFloatAsState] (≈150ms feel)
 * - Ripple bounded by the card's rounded clip
 * - Same content layout as before — accent bar + animation are additive
 */
@Composable
fun CourseListItem(
    course: Course,
    onClick: () -> Unit,
    onDeleteClick: () -> Unit,
    modifier: Modifier = Modifier,
) {
    val hasAudio = course.hasAudioContent()
    val hasVideo = course.hasVideoContent()
    val hasSubtitles = course.subtitleUri != null

    val interactionSource = remember { MutableInteractionSource() }
    val isPressed by interactionSource.collectIsPressedAsState()
    val animatedElevation by animateDpAsState(
        targetValue = if (isPressed) 8.dp else 1.dp,
        label = "courseCardElevation",
    )
    val animatedScale by animateFloatAsState(
        targetValue = if (isPressed) 1.02f else 1.0f,
        label = "courseCardScale",
    )

    val accentColor = accentColorFor(course.difficulty)

    Card(
        modifier = modifier
            .fillMaxWidth()
            .scale(animatedScale)
            .clip(RoundedCornerShape(16.dp))
            .clickable(
                interactionSource = interactionSource,
                indication = rememberRipple(),
                onClick = onClick,
            ),
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surface,
        ),
        elevation = CardDefaults.cardElevation(defaultElevation = animatedElevation),
    ) {
        // IntrinsicSize.Min lets the accent bar fillMaxHeight() to the
        // card's natural height without forcing a fixed dp value.
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .height(IntrinsicSize.Min),
            verticalAlignment = Alignment.CenterVertically,

```

```

    ) {
        Box(
            modifier = Modifier
                .width(4.dp)
                .fillMaxHeight()
                .background(accentColor),
        )

        Row(
            modifier = Modifier
                .weight(1f)
                .padding(horizontal = 16.dp, vertical = 8.dp),
            verticalAlignment = Alignment.CenterVertically,
        ) {
            Column(modifier = Modifier.weight(1f)) {
                Text(
                    text = course.title,
                    // titleMedium (16sp) → titleSmall (14sp): kept
                    // in lock-step with ContinueLearningCard and
                    // CourseGroupItem so the courses area type
                    // system stays consistent end-to-end.
                    style = MaterialTheme.typography.titleSmall,
                    maxLines = 1,
                    overflow = TextOverflow.Ellipsis,
                )
                Spacer(modifier = Modifier.height(1.dp))
                Text(
                    text = course.description,
                    style = MaterialTheme.typography.bodyMedium,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                    maxLines = 2,
                    overflow = TextOverflow.Ellipsis,
                )
                Spacer(modifier = Modifier.height(4.dp))
                Row(
                    horizontalArrangement = Arrangement.spacedBy(8.dp),
                    verticalAlignment = Alignment.CenterVertically,
                ) {
                    AssistChip(
                        onClick = {},
                        label = { Text(course.difficulty) },
                        colors = AssistChipDefaults.assistChipColors(
                            containerColor = MaterialTheme.colorScheme.secondaryContainer,
                        ),
                    )
                    if (hasAudio) {
                        Icon(
                            imageVector = Icons.Outlined.Headphones,
                            contentDescription = "Has audio",
                            modifier = Modifier.size(16.dp),
                            // § 12.32c: small capability icons
                            // (headphones / video / mic) lightened
                            // to violet-400 along with the rest of
                            // the brand purple elements on this
                            // screen.
                            tint = Color(0xFFA78BFA),
                        )
                    }
                    if (hasVideo) {
                        Icon(
                            imageVector = Icons.Outlined.VideoFile,
                            contentDescription = "Has video",
                            modifier = Modifier.size(16.dp),

```

```

        tint = Color(0xFFA78BFA),
    )
}
if (hasSubtitles) {
    Icon(
        imageVector = Icons.Outlined.Mic,
        contentDescription = "Has subtitles",
        modifier = Modifier.size(16.dp),
        tint = Color(0xFFA78BFA),
    )
}
Spacer(modifier = Modifier.weight(1f))
Text(
    text = formatDuration(course.durationMs),
    style = MaterialTheme.typography.bodySmall,
    color = MaterialTheme.colorScheme.onSurfaceVariant,
    maxLines = 1,
    overflow = TextOverflow.Ellipsis,
    modifier = Modifier.widthIn(min = 60.dp),
)
}
}
Spacer(modifier = Modifier.width(8.dp))
IconButton(
    onClick = onDeleteClick,
    modifier = Modifier.size(40.dp),
) {
    Icon(
        imageVector = Icons.Default.Delete,
        contentDescription = "Delete",
        tint = MaterialTheme.colorScheme.error,
    )
}
Spacer(modifier = Modifier.width(4.dp))
FilledIconButton(
    onClick = onClick,
    modifier = Modifier.size(48.dp),
    // (2026-07-04) Reverted the per-lesson "Start"
    // play button back to M3's default
    // `primary` (deep violet-600 #7C3AED) container
    // + `onPrimary` (white) content. The user
    // preferred the deeper brand purple over the
    // § 12.32c lightened violet-400 (#A78BFA) — the
    // latter blended with the rest of the screen
    // and made the CTA read as decorative rather
    // than actionable.
) {
    Icon(
        imageVector = Icons.Default.PlayArrow,
        contentDescription = "Start",
        modifier = Modifier.size(24.dp),
    )
}
}
}
}
}

/**
 * Pick an accent color for the difficulty tier. A* → lighter purple
 * (violet-400), B* → lighter purple (violet-500), C* → secondary
 * (violet-500). Unknown strings fall back to violet-400 so the bar
 * is always visible. The three difficulty tiers stay distinguishable

```

```

* through the three brand-purple shades.
*
* Marked @Composable because [MaterialTheme.colorScheme] is only readable
* inside the composition.
*/
@Composable
private fun accentColorFor(difficulty: String): Color = when {
    difficulty.startsWith("A", ignoreCase = true) ->
        Color(0xFFA78BFA)
    difficulty.startsWith("B", ignoreCase = true) ->
        Color(0xFF8B5CF6)
    difficulty.startsWith("C", ignoreCase = true) ->
        MaterialTheme.colorScheme.secondary
    else -> Color(0xFFA78BFA)
}

private fun formatDuration(ms: Long): String {
    if (ms == 0L) return "--:--"
    val totalSeconds = ms / 1000
    val minutes = totalSeconds / 60
    val seconds = totalSeconds % 60
    return "%d:%02d".format(minutes, seconds)
}

// === app\src\main\java\com\echoling\app\presentation\ui\components\PageHeader.kt ===

package com.echoling.app.presentation.ui.components

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.RowScope
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.heightIn
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

/**
 * Slim inline header that replaces [androidx.compose.material3.TopAppBar]
 * across the app. Designed for edge-to-edge layouts where content needs
 * to start right below the status bar.
 *
 * The header has a 48dp minimum height (slim vs M3 TopAppBar's 64dp)
 * and grows to fit its content — single-line titles stay 48dp, while
 * two-line brand bars (e.g. `每天进步一点` / `— Every day a little progress —`)
 * expand to ~60dp so the subtitle displays in full. The Row only adds
 * horizontal padding (4dp); vertical space is fully available to the
 * title content.
 *
 * It does NOT itself add [statusBarsPadding] — the caller is expected to
 * apply `.padding(padding)` on the Scaffold body, which already consumes

```

```

* the system bar insets (when the Scaffold has no topBar, the `padding.top`
* parameter is exactly the status bar height).
*
* Slots:
* - [onBack]: optional back button on the left (ArrowBack icon, 48dp touch)
* - [title]: any composable, takes the remaining space. Horizontal
* alignment is controlled by [titleAlignment] (default
* [Alignment.CenterHorizontally] — used by the brand-bar tabs Courses
* and Me). Sub-pages like Statistics, Import, Vocabulary, Api pass
* [Alignment.Start] to left-align the title.
* - [actions]: trailing slot (typically IconButton or Surface pill)
*
* Background: `MaterialTheme.colorScheme.surface` — matches the rest of
* the page and provides a visual boundary so the header doesn't blend
* into the body content.
*/
@Composable
fun PageHeader(
    modifier: Modifier = Modifier,
    title: @Composable () -> Unit = {},
    onBack: (() -> Unit)? = null,
    actions: @Composable RowScope. () -> Unit = {},
    titleAlignment: Alignment.Horizontal = Alignment.CenterHorizontally,
) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .background(MaterialTheme.colorScheme.surface)
            .heightIn(min = 48.dp)
            .padding(horizontal = 4.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        if (onBack != null) {
            IconButton(onClick = onBack) {
                Icon(
                    imageVector = Icons.Filled.ArrowBack,
                    contentDescription = "Back",
                )
            }
        } else {
            Spacer(modifier = Modifier.width(12.dp))
        }

        Box(
            modifier = Modifier
                .weight(1f)
                .padding(horizontal = 4.dp),
            contentAlignment = Alignment.CenterStart.let {
                // Map the requested horizontal alignment to a Box
                // contentAlignment. Center → centered, Start →
                // left-aligned, End → right-aligned.
                when (titleAlignment) {
                    Alignment.Start -> Alignment.CenterStart
                    Alignment.End -> Alignment.CenterEnd
                    else -> Alignment.Center
                }
            },
        ) {
            title()
        }

        Row(
            verticalAlignment = Alignment.CenterVertically,

```

```

        horizontalArrangement = Arrangement.spacedBy(0.dp),
    ) {
        actions()
    }
}

// === app\src\main\java\com\echoling\app\presentation\ui\components\StatsSummaryCard.kt ===

package com.echoling.app.presentation.ui.components

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.AccessTime
import androidx.compose.material.icons.filled.Translate
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.unit.dp

/**
 * Tappable summary card shown above the course list. Two stat cells (time
 * spent + words collected) plus a "view details →" affordance. Tapping the
 * card (or any stat) calls [onClick].
 */
@Composable
fun StatsSummaryCard(
    totalLearnTimeMs: Long,
    totalWordsCollected: Int,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
) {
    Card(
        modifier = modifier
            .fillMaxWidth()
            .clickable(onClick = onClick),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.primaryContainer,
        ),
        shape = RoundedCornerShape(20.dp),
    ) {
        Column(
            modifier = Modifier.padding(horizontal = 20.dp, vertical = 12.dp),
        ) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.spacedBy(12.dp),

```

```

    ) {
        StatCell(
            modifier = Modifier.weight(1f),
            icon = Icons.Default.AccessTime,
            label = "学习时长",
            value = formatTime(totalLearnTimeMs),
        )
        StatCell(
            modifier = Modifier.weight(1f),
            icon = Icons.Default.Translate,
            label = "单词",
            value = totalWordsCollected.toString(),
        )
    }
    Spacer(modifier = Modifier.height(6.dp))
    Text(
        text = "查看详情 →",
        style = MaterialTheme.typography.labelMedium,
        color = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.8f),
    )
}

@Composable
private fun StatCell(
    modifier: Modifier = Modifier,
    icon: ImageVector,
    label: String,
    value: String,
) {
    Column(
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        Icon(
            imageVector = icon,
            contentDescription = null,
            modifier = Modifier.size(22.dp),
            tint = MaterialTheme.colorScheme.primary,
        )
        Spacer(modifier = Modifier.height(4.dp))
        Text(
            text = value,
            // titleLarge (22sp) → titleMedium (16sp): shrink the stat
            // number so the stats row reads lighter — the card is
            // sitting right under the deep-purple Hero and the big
            // 22sp numerals were competing with the hero's 22sp course
            // title. Still 4sp bigger than the 12sp label below, so the
            // value>label hierarchy is preserved.
            style = MaterialTheme.typography.titleMedium,
            color = MaterialTheme.colorScheme.onPrimaryContainer,
        )
        Text(
            text = label,
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.8f),
        )
    }
}

private fun formatTime(ms: Long): String {
    if (ms <= 0L) return "0m"
}

```



```

        val totalSeconds = ms / 1000
        val hours = totalSeconds / 3600
        val minutes = (totalSeconds % 3600) / 60
        return if (hours > 0) "${hours}h ${minutes}m" else "${minutes}m"
    }

// === app\src\main\java\com\echoling\app\presentation\ui\navigation\AppBottomBar.kt ===

package com.echoling.app.presentation.ui.navigation

import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.indication
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.WindowInsets
import androidx.compose.foundation.layout.asPaddingValues
import androidx.compose.foundation.layout.fillMaxHeight
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.navigationBars
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.ripple.rememberRipple
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.layout.Layout
import androidx.compose.ui.platform.LocalDensity
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp

/**
 * Custom compact bottom navigation bar. The M3 [androidx.compose.material3.NavigationBar]
 * hardcodes its item height at 80dp and its indicator pill at 64×32dp, so there
 * is no public API to shrink the pill's vertical extent. This composable
 * reproduces the same visual contract (four equal-weight tabs, pill behind
 * the selected icon, primaryContainer highlight, ripple feedback) while
 * letting us choose a smaller bar height and a thinner pill.
 *
 * Sizing: bar is 60dp tall (vs M3's 80dp), the pill is 56×28dp with a 14dp
 * corner radius (vs M3's 64×32 with 16dp). Combined with the smaller icon
 * and label in [BottomTabItem] this gives the bar a noticeably lighter
 * footprint, which is the right call for a four-tab app on a phone — the
 * M3 default is designed for five-tab layouts and looks oversized here.
 *
 * Hit-target contract: the entire 60dp row per tab is clickable (so the
 * 48dp minimum tap target is satisfied by the row's full width × 60dp
 * height), but the pill is purely visual and does not enlarge the hit area.
 *
 * The bar handles the system navigation-bar insets via a separate

```

```

* bottom [Spacer] (see § 12.30 in CLAUDE.md) so the surface background
* extends behind the gesture bar on edge-to-edge devices, but the
* clickable row stays above it (gesture-bar swipe still works).
*/
@Composable
fun AppBarBottomBar(
    currentRoute: String?,
    onNavigate: (TopLevelDestination) -> Unit,
    modifier: Modifier = Modifier,
) {
    // § 12.30c: Read the navigation-bar bottom inset. Now that the bar
    // is rendered as a SIBLING of the Scaffold (not inside its
    // `bottomBar` slot — see MainScaffold.kt), `LocalWindowInsets` is
    // NOT overridden by the outer Scaffold's
    // `contentWindowInsets = WindowInsets(0, 0, 0, 0)`, so
    // `WindowInsets.navigationBars` correctly reports the real 16dp
    // nav-bar inset on this Xiaomi Mi 11 CN (1080x2400 at 2.625x
    // density; nav bar = 42px = 16dp).
    //
    // Using `WindowInsets.navigationBars.asPaddingValues()` (instead
    // of the previous `ViewCompat.getRootWindowInsets()` call) means
    // we get the same inset value Compose uses everywhere else, AND
    // we don't need to remember-capture the view.
    val navBarPadding = WindowInsets.navigationBars.asPaddingValues()
    val navBarBottom: Dp = navBarPadding.calculateBottomPadding()

    // § 12.30c: Use a custom `Layout` to FORCE the bar's height to
    // 60dp + navBarBottom. Reason: the M3 Scaffold's `bottomBar` slot
    // subcompose-measures the bar, and both `Box.height(76.dp)` and
    // `Box.requiredHeight(76.dp)` AND `Surface.height(76.dp)` are
    // silently overridden back to the bar's content intrinsic height
    // (60dp = the Row). A custom `Layout` that explicitly returns
    // `layout(width, 76.dp.toPx())` bypasses whatever clamping the
    // Scaffold does and forces the bar to 76dp.
    //
    // The `background(surface)` paints the full area; the clickable
    // Row sits at the top (60dp) and the bottom 16dp is the nav-bar
    // zone, filled with surface color (the system gesture bar draws
    // on top of this zone, but with the surface color behind it
    // there's no white gap).
    //
    // The bar is rendered as a sibling of the Scaffold (placed with
    // `Modifier.align(Alignment.BottomCenter)` in MainScaffold), NOT
    // as a child of Scaffold's `bottomBar` slot. The `modifier`
    // parameter carries that alignment into the bar.
    val density = LocalDensity.current
    val totalHeightDp = 60.dp + navBarBottom
    Layout(
        modifier = modifier
            .fillMaxWidth()
            .background(MaterialTheme.colorScheme.surface),
        content = {
            // The clickable Row lives at the top of the bar (60dp).
            // Anything below it (navBarBottom = 16dp) is filled by the
            // Layout's own `background` because the Layout paints its
            // background across its full measured size. Gesture swipes
            // from the bottom of the screen pass through this
            // non-clickable zone to the system.
            Column(
                modifier = Modifier.fillMaxWidth(),
            ) {
                Row(
                    modifier = Modifier

```

```

        .fillMaxWidth()
        // § 12.30b: raised from 48dp to 60dp — user reported the
        // bar's surface area felt too thin between the tab row
        // and the screen bottom (after § 12.30's Spacer fix made
        // the surface extend to the screen edge, the bar had
        // zero visual weight between the row and the gesture
        // bar). 60dp gives the bar a more substantial presence
        // while still under M3 NavigationBar's 80dp default.
        // Hit-target contract: 60dp row per tab, clickable.
        .height(60.dp),
    ) {
        TopLevelDestination.entries.forEach { dest ->
            val selected = dest.route == currentRoute
            BottomTabItem(
                destination = dest,
                selected = selected,
                onClick = { onNavigate(dest) },
                modifier = Modifier
                    .weight(1f)
                    .fillMaxHeight(),
            )
        }
    }
}

) { measurables, constraints ->
    val heightPx = with(density) { totalHeightDp.roundToPx() }
    val placeable = measurables[0].measure(
        constraints.copy(minWidth = 0, maxWidth = constraints.maxWidth, minHeight = 0, maxHeight = heightPx)
    )
    layout(constraints.maxWidth, heightPx) {
        placeable.placeRelative(0, 0)
    }
}
}

/**
 * One tab cell. Layout (top to bottom):
 * - 28dp pill area — shows the primaryContainer background when selected,
 *   transparent otherwise (so the row's surface bleeds through).
 * - 2dp spacer.
 * - 10sp label.
 *
 * The pill is centered horizontally inside the 28dp slot with 8dp side
 * padding, giving it 56dp width (cell width minus 16dp on a 360dp-wide
 * phone = 90dp-16dp = 74dp, but we cap it via the pill's explicit 56dp
 * width so it looks the same on tablets).
 *
 * § 12.30b: The outer [Row] is now 60dp (raised from 48dp) to give the
 * bar more visual weight after the surface-extension fix made the bar
 * paint to the screen edge. Content total height (28dp pill + 2dp gap +
 * ~14dp label) is ~44dp, so a 60dp Row leaves 16dp of dead space.
 * With `Arrangement.Bottom` (preserved from § 12.29) that 16dp sits at
 * the TOP of the cell (between the cell's top edge and the pill), and
 * the label hugs the BOTTOM of the cell — which is the gesture-bar
 * edge. This makes the bar look "grounded" against the screen bottom
 * rather than having the labels float mid-cell.
 */
@Composable
private fun BottomTabItem(
    destination: TopLevelDestination,
    selected: Boolean,
    onClick: () -> Unit,

```

```

        modifier: Modifier = Modifier,
    ) {
        val interactionSource = remember { MutableInteractionSource() }
        val label = stringResource(destination.labelRes)
        val iconColor =
            if (selected) MaterialTheme.colorScheme.primary
            else MaterialTheme.colorScheme.onSurfaceVariant
        val textColor = iconColor
        val pillColor = MaterialTheme.colorScheme.primaryContainer

        Box(
            modifier = modifier
                .clickable(
                    interactionSource = interactionSource,
                    indication = null, // ripple drawn manually below for finer control
                ) { onClick() }
                .indication(
                    interactionSource = interactionSource,
                    indication = rememberRipple(bounded = false, radius = 48.dp),
                ),
            contentAlignment = Alignment.Center,
        ) {
            Column(
                horizontalAlignment = Alignment.CenterHorizontally,
                verticalArrangement = Arrangement.Bottom,
            ) {
                // Pill slot: 28dp tall, centers a 56×28dp rounded rect.
                Box(
                    modifier = Modifier
                        .size(width = 56.dp, height = 28.dp)
                        .clip(RoundedCornerShape(14.dp))
                        .background(if (selected) pillColor else androidx.compose.ui.graphics.Color.Transparent)
                        .padding(horizontal = 14.dp),
                    contentAlignment = Alignment.Center,
                ) {
                    Icon(
                        imageVector = if (selected) destination.selectedIcon else destination.unselectedIcon,
                        contentDescription = label,
                        tint = iconColor,
                        modifier = Modifier.size(22.dp),
                    )
                }
                // 2dp breathing room between pill and label.
                Box(modifier = Modifier.size(height = 2.dp, width = 1.dp))
                Text(
                    text = label,
                    style = MaterialTheme.typography.labelSmall,
                    color = textColor,
                    maxLines = 1,
                    overflow = TextOverflow.Ellipsis,
                )
            }
        }
    }
}

```

```
// === app\src\main\java\com\echoling\app\presentation\ui\navigation\MainScaffold.kt ===
```

```
package com.echoling.app.presentation.ui.navigation
```

```
import androidx.compose.foundation.ExperimentalFoundationApi
import androidx.compose.foundation.layout.Box
```

```

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.WindowInsets
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.pager.rememberPagerState
import androidx.compose.material3.Scaffold
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.navigation.NavHostController
import androidx.navigation.compose.currentBackStackEntryAsState
import androidx.navigation.compose.rememberNavController
import com.echoling.app.presentation.ui.screens.splash.SplashScreen
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch

/** How long the cold-start splash is held before the app shell takes over. */
private const val SPLASH_DURATION_MS = 1_000L

/**
 * Top-level app shell. Holds three independent state machines and wires
 * them together so that the user gets a unified, gesture-driven tab
 * experience:
 *
 * 1. [rememberPagerState] — the active tab. The Pager natively
 *    animates page changes in the correct direction (forward tab
 *    slides in from the right, backward tab slides in from the left)
 *    and supports swipe gestures. The state lives here so it
 *    survives a sub-page push/pop (the Pager is rendered behind the
 *    sub-page NavHost and is not recreated).
 * 2. [rememberNavController] — the sub-page back stack. Used only
 *    for screens that are NOT one of the 4 bottom-bar tabs
 *    (Practice, CourseDetail, Import, Statistics, Instructions,
 *    CategoryStudy). The 4 tab routes are NOT in this NavHost.
 * 3. [AppBottomBar] — rendered as a sibling of the Scaffold in a
 *    `Column`, not via the Scaffold's `bottomBar` slot. See
 *    "Why Column + sibling bar" below.
 *
 * # Layout
 *
 * Column (fillMaxSize)
 *   └─ Scaffold (weight=1, no bottomBar) ← content area
 *     └─ Box (padding = innerPadding)
 *       └─ TabPagerHost ← 4 pages, behind everything
 *       └─ SubPageNavGraph ← overlaid when isOnSubPage
 *     └─ AppBottomBar (fixed height, only when !onSubPage)
 *
 * The sub-page NavHost is overlaid on top of the Pager, so pushing a
 * sub-page slides it over the Pager (iOS-style). The Pager itself
 * stays in place underneath; the user perceives a clean slide-in
 * for the sub-page even though the underlying Pager doesn't move.
 *
 * # Why Column + sibling bar (§ 12.32b)
 *
 * § 12.32a originally used `Box { Scaffold + AppBottomBar(align=BottomCenter) }`.

```

```

* That worked for the bar height (60dp + nav bar inset = 76dp), but had
* a side effect: the inner `Scaffold` (in CoursesScreen.kt) places its
* `floatingActionButton` (the "导入素材" FAB) at the bottom-end of its
* own content area, which is the full screen in the Box approach.
* The FAB ended up at  $y \approx \text{screenHeight} - 16\text{dp} - \text{FAB\_height}$ , which is
* inside the bar's 76dp area. Since the bar is drawn on top of
* the Scaffold in the Box, the bar's surface hid the FAB.
*
* Fix: use a `Column` instead of a `Box`. The Scaffold takes
* `weight(1f)` (remaining space after the bar), and the bar is a
* fixed-height child AFTER the Scaffold in the Column. The FAB
* inside the inner Scaffold is now positioned within the
* Scaffold's area, which is above the bar — the FAB is visible.
*
* Edge-to-edge still works: the Column fills the screen, the bar
* sits at the bottom of the Column (= bottom of the screen), and
* the bar's `background(MaterialTheme.colorScheme.surface)` paints
* the bottom 76dp. The system gesture handle is drawn on top of
* the bottom 16dp of the bar's surface (with `isNavigationBarContrastEnforced`
* = false` set in MainActivity so no scrim is applied).
*
* # Why not the Scaffold's `bottomBar` slot (§ 12.32a)
*
* We initially used `Scaffold(bottomBar = AppBottomBar(...))` and
* tried every reasonable way to make the bar measure to 76dp
* (60dp Row + 16dp nav-bar inset): `Box.height(76.dp)`,
* `Box.requiredHeight(76.dp)`, `Surface.height(76.dp)`, and a custom
* `Layout { measurables -> layout(w, 76.dp.toPx()) }` that
* explicitly returned 200px from its measure block. All of them
* measured correctly (logcat confirmed `layout(1080, 200)`) but
* the Scaffold still placed the bar at 158px (60dp). The M3 1.1.2
* Scaffold's `bottomBar` slot uses SubcomposeLayout with `looseConstraints`
* and reads `bottomBarPlaceables.maxByOrNull { it.height }?.height`
* — but empirically on this device the rendered height is clamped
* to the bar's content intrinsic size, ignoring the placeable's
* own measured height. See § 12.32a in CLAUDE.md for the full chain.
*
* # Why `contentWindowInsets = WindowInsets(0)`
*
* Every page-level screen wraps its content in another `Scaffold`
* (CoursesScreen, MeScreen, ImportScreen, etc.) and those inner
* Scaffolds default to `contentWindowInsets = WindowInsets.systemBars`.
* If THIS outer Scaffold also consumes systemBars, the status-bar
* height is added to the content's top via `padding(innerPadding)`
* AND then the inner Scaffold adds it AGAIN, producing `PageHeader`
* at `2 × status_bar_height` from the top of the window. Setting
* `contentWindowInsets = WindowInsets(0)` here means the outer's
* `innerPadding` is empty. The inner Scaffolds are then free to add
* the status-bar inset exactly once.
*/
@OptIn(ExperimentalFoundationApi::class)
@Composable
fun MainScaffold(
    navController: NavHostController = rememberNavController(),
) {
    // 2026-07-03: switch to `rememberSaveable` so the "splash already
    // shown" flag survives configuration changes (rotation, theme
    // change, font scale). Without this, every config change destroys
    // and recreates `MainScaffold`, re-initializing `showSplash = true`
    // and re-firing the 1-second `LaunchedEffect` — the user reported
    // the splash image re-appearing on every portrait↔landscape
    // rotation. With `rememberSaveable` the Boolean is written to the
    // activity's saved Bundle the moment the delay completes, so on

```

```

// recreation the restored value is `false` and the Compose splash
// is skipped. The manifest-level `Theme.EchoLing.Splash` window
// background is a separate (sub-100ms) flash and is also
// suppressed via the `savedInstanceState != null` check in
// `MainActivity.onCreate`.
var showSplash by rememberSaveable { mutableStateOf(true) }
LaunchedEffect(Unit) {
    delay(SPLASH_DURATION_MS)
    showSplash = false
}

if (showSplash) {
    SplashScreen(modifier = Modifier.fillMaxSize())
    return
}

// ——— State machines ———


---


val pagerState = rememberPagerState(pageCount = { 4 })
val backStackEntry by navController.currentBackStackEntryAsState()
val coroutineScope = rememberCoroutineScope()

// The NavHost's start destination is the transparent `tab_root`
// placeholder. When the user is on a tab (no sub-page pushed),
// backStackEntry is either null or at `tab_root`. When the user
// is on a real sub-page, backStackEntry is at a sub-page route.
val currentBackStackRoute = backStackEntry?.destination?.route
val isOnSubPage = currentBackStackRoute != null && currentBackStackRoute != TAB_ROOT
val showBottomBar = !isOnSubPage

// `currentRoute` is the source of truth for [AppBar] highlight.
// - On a tab: derived from `pagerState.currentPage` (the Pager
//   knows which tab is active regardless of whether the user
//   got there by tap or swipe).
// - On a sub-page: derived from the navController. The bottom
//   bar is hidden anyway in this case, but the value is kept
//   consistent for any other consumers.
val currentRoute: String? = if (isOnSubPage) {
    currentBackStackRoute
} else {
    TopLevelDestination.entries.getOrNull(pagerState.currentPage)?.route
}

// § 12.32b: use a Column instead of a Box. The Scaffold takes
// `weight(1f)` (remaining space after the bar), and the bar is a
// fixed-height child AFTER the Scaffold. This way the inner
// Scaffold's FAB (in CoursesScreen.kt) is positioned within the
// Scaffold's area, ABOVE the bar — the FAB is no longer hidden
// behind the bar.
Column(modifier = Modifier.fillMaxSize()) {
    Scaffold(
        modifier = Modifier.weight(1f),
        // No bottomBar — the AppBar is rendered as a
        // sibling in the outer Column (see below). § 12.32a
        // explained why the bottomBar slot clamps the bar to
        // 60dp; the Column approach is the actual fix.
        contentWindowInsets = WindowInsets(0, 0, 0, 0),
    ) { innerPadding ->
        Box(
            modifier = Modifier
                .fillMaxSize()
                .padding(innerPadding),
        ) {

```

```

// Pager (background). `userScrollEnabled = false` whenever a
// sub-page is on top so a swipe on the sub-page never
// accidentally fires a tab swap.
TabPagerHost(
    pagerState = pagerState,
    navController = navController,
    userScrollEnabled = !isOnSubPage,
)

// Sub-page NavHost (overlay). ALWAYS mounted so the
// navController's graph is set the moment the user first
// taps a sub-page link from a tab. If we only mount on
// isOnSubPage, the first navController.navigate(...) from
// inside a tab screen (e.g. tap a course in CoursesScreen)
// would fire before SubPageNavGraph has run setGraph(),
// and NavController would throw
// `IllegalArgumentException: Navigation graph has not been
// set for NavController`. The visible "transparent
// placeholder" path stays identical — the Pager shows
// through the tab_root Box — but the NavController is
// always warm.
SubPageNavGraph(
    navController = navController,
    modifier = Modifier.fillMaxSize(),
)
}
}

// §12.32b: AppBar as a sibling AFTER the Scaffold in the
// outer Column. The bar takes its natural measured height
// (60dp + navBarBottom = 76dp on this device). The bar's
// surface color extends to the screen bottom (no white gap),
// and the inner Scaffold's FAB is no longer hidden behind it.
if (showBottomBar && currentRoute != null) {
    AppBar(
        currentRoute = currentRoute,
        onNavigate = { dest ->
            // Tap a tab → drive the Pager. Do NOT navigate
            // in the NavController — the Pager IS the tab
            // navigator now. `animateScrollToPage` slides
            // the new page in from the right (target >
            // current) or from the left (target < current).
            val targetIndex = TopLevelDestination.entries
                .indexOf(dest)
                .coerceAtLeast(0)
            if (targetIndex != pagerState.currentPage) {
                coroutineScope.launch {
                    pagerState.animateScrollToPage(targetIndex)
                }
            }
        },
    )
}
}
}

// === app\src\main\java\com\echoling\app\presentation\ui\navigation\NavGraph.kt ===

package com.echoling.app.presentation.ui.navigation

import androidx.compose.animation.core.Tween

```



```

import androidx.compose.animation.slideInHorizontally
import androidx.compose.animation.slideOutHorizontally
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.navigation.NavHostController
import androidx.navigation.NavType
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.navArgument
import com.echoling.app.presentation.ui.screens.course.CourseDetailScreen
import com.echoling.app.presentation.ui.screens.import.ImportScreen
import com.echoling.app.presentation.ui.screens.instructions.InstructionsScreen
import com.echoling.app.presentation.ui.screens.permissions.PermissionsScreen
import com.echoling.app.presentation.ui.screens.practice.PracticeScreen
import com.echoling.app.presentation.ui.screens.recite.CategoryStudyScreen
import com.echoling.app.presentation.ui.screens.statistics.StatisticsScreen

```

```

/**
 * Internal start-destination route for the sub-page NavHost. The
 * composable for this route is a transparent [Box] — when the user is
 * on a tab (no sub-page pushed), the NavHost is parked here and the
 * [TabPagerHost] behind it is fully visible.
 *
 * When a sub-page is pushed, the NavHost transitions:
 * - **Forward (push)**: `tab_root` slides out to the left
 *   (`slideExitLeft`), the new sub-page slides in from the right
 *   (`slideEnterRight`). The Pager underneath is gradually covered.
 * - **Backward (pop)**: sub-page slides out to the right
 *   (`slideExitRight`), `tab_root` slides in from the left
 *   (`slideEnterLeft`). The Pager underneath is gradually revealed.
 *
 * The Pager itself does not move during a sub-page push/pop, only the
 * NavHost layer above it. This is the standard iOS-style overlay
 * pattern.
 */

```

```
private const val TAB_ROOT_ROUTE = "tab_root"
```

```

/** The dummy route used as the sub-page NavHost start destination. */
val TAB_ROOT: String = TAB_ROOT_ROUTE

```

```

// (2026-07-04) Made public instead of private so PracticeScreen can
// reference the same value to defer its heavy first-frame work past the
// slide-in animation window (see PracticeScreen.kt:LaunchedEffect).
// Without this shared constant, the screen-level delay would have to
// hard-code 350 and silently drift if the slide ever changes.
const val SUB_PAGE_NAV_ANIM_MS: Int = 350

```

```

private val slideEnterRight = slideInHorizontally(
    initialOffsetX = { fullWidth -> fullWidth },
    animationSpec = tween(SUB_PAGE_NAV_ANIM_MS)
)
private val slideExitLeft = slideOutHorizontally(
    targetOffsetX = { fullWidth -> -fullWidth },
    animationSpec = tween(SUB_PAGE_NAV_ANIM_MS)
)
private val slideEnterLeft = slideInHorizontally(
    initialOffsetX = { fullWidth -> -fullWidth },
    animationSpec = tween(SUB_PAGE_NAV_ANIM_MS)
)
private val slideExitRight = slideOutHorizontally(
    targetOffsetX = { fullWidth -> fullWidth },

```

```

        animationSpec = tween(SUB_PAGE_NAV_ANIM_MS)
    )

/**
 * NavHost for sub-pages only. The 4 main tabs are hosted in a
 * separate [TabPageHost] (HorizontalPager) in [MainScaffold].
 *
 * The 4 tab routes (Courses / Vocabulary / Recite / Me) used to live
 * here too (in the original NavGraph), but were extracted to the
 * Pager in 2026-06-28 so the tab switch could be direction-aware
 * (forward tabs slide in from the right; backward tabs slide in from
 * the left) and gesture-driven (swipe left/right to change tab).
 *
 * The sub-page transitions remain the standard slide-left / slide-right
 * pattern. Sub-pages are full-screen, so the bottom bar is hidden
 * while one is on top.
 */
@Composable
fun SubPageNavGraph(
    navController: NavHostController,
    modifier: Modifier = Modifier,
) {
    NavHost(
        navController = navController,
        startDestination = TAB_ROOT_ROUTE,
        enterTransition = { slideEnterRight },
        exitTransition = { slideExitLeft },
        popEnterTransition = { slideEnterLeft },
        popExitTransition = { slideExitRight },
        modifier = modifier,
    ) {
        // Transparent placeholder. The Pager is visible underneath.
        composable(TAB_ROOT_ROUTE) {
            Box(modifier = Modifier.fillMaxSize())
        }

        composable(
            route = Screen.Import.route,
            arguments = listOf(
                navArgument(Screen.Import.ARG_COURSE_NAME) {
                    type = NavType.StringType
                    nullable = true
                    defaultValue = null
                },
            ),
        ) { backStackEntry ->
            val raw = backStackEntry.arguments?.getString(Screen.Import.ARG_COURSE_NAME)
            // The query arg was URL-encoded by `Screen.Import.createRoute`
            // when navigating from the category-detail FAB, so decode it
            // back to the original (Chinese) name. The home page passes
            // null and we leave it null.
            val prefillCourseName = raw?.takeIf { it.isNotEmpty() }?.decodeFromRoute()
            ImportScreen(
                onNavigateBack = { navController.popBackStack() },
                onImportComplete = { navController.popBackStack() },
                prefillCourseName = prefillCourseName,
            )
        }
    }

    composable(
        route = Screen.CourseDetail.route,
        arguments = listOf(
            navArgument("courseName") { type = NavType.StringType },

```

```

    ),
    ) { backStackEntry ->
        val rawCourseName = backStackEntry.arguments?.getString("courseName")
        ?: return@composable
        val courseName = rawCourseName.decodeFromRoute()
        CourseDetailScreen(
            courseName = courseName,
            onNavigateBack = { navController.popBackStack() },
            onNavigateToPractice = { courseId ->
                navController.navigate(Screen.Practice.createRoute(courseId))
            },
            // § 12.19: category-detail has its own Import FAB; we
            // pass the current group name so the form pre-fills it.
            onNavigateToImport = {
                navController.navigate(Screen.Import.createRoute(courseName))
            },
        )
    }

    composable(
        route = Screen.Practice.route,
        arguments = listOf(
            navArgument("courseId") { type = NavType.StringType },
        ),
    ) { backStackEntry ->
        val courseId = backStackEntry.arguments?.getString("courseId") ?: return@composable
        PracticeScreen(
            courseId = courseId,
            onNavigateBack = { navController.popBackStack() },
        )
    }

    composable(Screen.Statistics.route) {
        StatisticsScreen(
            onNavigateBack = { navController.popBackStack() },
        )
    }

    // § 12.21: in-app "使用说明" page.
    composable(Screen.Instructions.route) {
        InstructionsScreen(
            onNavigateBack = { navController.popBackStack() },
        )
    }

    // (2026-07-04) in-app "权限使用说明" page. Required by 国内应用商店
    // review. Reached from MeScreen's "权限使用说明" entry.
    composable(Screen.Permissions.route) {
        PermissionsScreen(
            onNavigateBack = { navController.popBackStack() },
        )
    }

    // Per-category flashcard study sub-page reached from the
    // "记单词" tab. The `categoryId` route arg is the slug from
    // vocab_manifest.json (e.g. "junior", "senior", "cet4", "cet6",
    // "toefl") — URL-encoded in [Screen.CategoryStudy.createRoute]
    // and decoded here so Chinese-only slugs (none today, but the
    // helper exists) survive the round trip.
    composable(
        route = Screen.CategoryStudy.route,
        arguments = listOf(
            navArgument(Screen.CategoryStudy.ARG_CATEGORY_ID) {

```

```

        type = NavType.StringType
    },
),
) { backStackEntry ->
    val raw = backStackEntry.arguments
        ?.getString(Screen.CategoryStudy.ARG_CATEGORY_ID)
        ?: return@composable
    val categoryId = raw.decodeFromRoute()
    CategoryStudyScreen(
        categoryId = categoryId,
        onNavigateBack = { navController.popBackStack() },
    )
}
}
}

// === app\src\main\java\com\echoling\app\presentation\ui\navigation\Screen.kt ===

package com.echoling.app.presentation.ui.navigation

sealed class Screen(val route: String) {
    // Tab roots (bottom bar visible)
    data object Courses : Screen("courses")
    data object Vocabulary : Screen("vocabulary")
    // Renamed from "Learning" / "学习" to "Recite" / "记单词" — the tab
    // is now a category picker (初中 / 高中 / CET-4 / CET-6 / 托福) and
    // tapping a card navigates to the per-category flashcard study
    // screen at [CategoryStudy]. Keeping the internal route slug
    // "recite" (not "learning") to match the new naming.
    data object Recite : Screen("recite")
    data object Me : Screen("me")

    // Sub-pages (bottom bar hidden)
    // The route parameter is `courseName` (the parent group) — the
    // screen shows the lessons in that group; tapping a lesson goes
    // directly to Practice (§ 12.19). URL-encode the name so Chinese
    // characters, spaces, and slashes survive the round trip.
    data object CourseDetail : Screen("course_detail/{courseName}") {
        fun createRoute(courseName: String): String =
            "course_detail/${courseName.encodeForRoute()}"
    }
    data object Practice : Screen("practice/{courseId}") {
        fun createRoute(courseId: String) = "practice/${courseId}"
    }
    data object Statistics : Screen("statistics")
    // § 12.19: the import screen accepts an optional `courseName` query
    // param. When the user is on a category-detail sub-page and taps
    // the FAB, the parent group name is pre-filled into the form so
    // they only need to type the per-lesson title. The home page's FAB
    // calls the no-arg overload.
    data object Import : Screen("import?courseName={courseName}") {
        const val ARG_COURSE_NAME = "courseName"
        fun createRoute(courseName: String? = null): String =
            if (courseName?.isNullOrBlank()) "import"
            else "import?courseName=${courseName.encodeForRoute()}"
    }
    // § 12.21: in-app "使用说明" page. Reached from the home page's
    // top-right Help IconButton.
    data object Instructions : Screen("instructions")

    // (2026-07-04) in-app "权限使用说明" page. Reached from the "我的" tab.

```

```

// Required by the 国内应用商店 review (华为 / 小米 / OPPO / vivo /
// 应用宝) which ask for a built-in privacy / sensitive-permission
// disclosure page in addition to whatever is in the store listing.
// Only RECORD_AUDIO is currently declared, so the page covers that
// one permission + a short confirmation that INTERNET is not used.
data object Permissions : Screen("permissions")

// Per-category flashcard study sub-page. Tapping a category card
// on the "记单词" tab navigates here with the category slug (e.g.
// "junior", "senior", "cet4"). The screen header renders the
// category's display name (resolved from the manifest), and the
// flashcard iteration uses the per-category entries list — so
// studying CET-4 shows the CET-4-specific translation rather than
// the cross-category merged lookup.
data object CategoryStudy : Screen("recite/{categoryId}") {
    const val ARG_CATEGORY_ID = "categoryId"
    fun createRoute(categoryId: String): String =
        "recite/${categoryId.encodeForRoute()}"
}

private fun String.encodeForRoute(): String = java.net.URLEncoder.encode(this, "UTF-8")

fun String.decodeFromRoute(): String = java.net.URLDecoder.decode(this, "UTF-8")

// === app\src\main\java\com\echoling\app\presentation\ui\navigation\TabPagerHost.kt ===

package com.echoling.app.presentation.ui.navigation

import androidx.compose.foundation.ExperimentalFoundationApi
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.pager.HorizontalPager
import androidx.compose.foundation.pager.PagerState
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.navigation.NavHostController
import com.echoling.app.presentation.ui.screens.courses.CoursesScreen
import com.echoling.app.presentation.ui.screens.me.MeScreen
import com.echoling.app.presentation.ui.screens.recite.ReciteScreen
import com.echoling.app.presentation.ui.screens.vocabulary.VocabularyScreen

/**
 * Hosts the 4 main tabs in a [HorizontalPager] so that:
 * 1. Tap a bottom-bar tab — [MainScaffold] calls
 *    `pagerState.animateScrollToPage(targetIndex)`. The Pager
 *    natively slides the target page in from the right when
 *    `target > current` (forward, e.g. 首页→单词本) or from the
 *    left when `target < current` (backward, e.g. 我的→记单词).
 * 2. Swipe the screen — the Pager handles it natively with the
 *    same right-to-left / left-to-right slide semantics. A left
 *    swipe (finger drags right-to-left) advances to the next tab;
 *    a right swipe (finger drags left-to-right) goes back to the
 *    previous tab.
 *
 * The Pager is the source of truth for the active tab. [MainScaffold]
 * reads `pagerState.currentPage` to drive the [AppBar] highlight
 * (`currentRoute` derivation), so the bottom-bar pill stays in sync
 * with both tap-driven and swipe-driven page changes.
 *
 * Layout-wise the Pager sits behind the sub-page NavHost (rendered
 * as a sibling in [MainScaffold]'s Box). Sub-pages can therefore

```

```

* slide on top of it without disturbing the tab-switching animation.
* The Pager is `userScrollEnabled = false` whenever a sub-page is on
* top, so a swipe on a sub-page never accidentally fires a tab swap.
*/
@OptIn(ExperimentalFoundationApi::class)
@Composable
fun TabPagerHost(
    pagerState: PagerState,
    navController: NavHostController,
    userScrollEnabled: Boolean,
    modifier: Modifier = Modifier,
) {
    HorizontalPager(
        state = pagerState,
        modifier = modifier.fillMaxSize(),
        userScrollEnabled = userScrollEnabled,
    ) { page ->
        // The page index is the tab's position in [TopLevelDestination]
        // declaration order: 0=Courses, 1=Vocabulary, 2=Recite, 3=Me.
        when (page) {
            0 -> CoursesScreen(
                // §12.21: home page's top-right IconButton is a
                // Help / "使用说明" button. Statistics is still
                // reachable via the tappable StatsSummaryCard.
                onNavigateToInstructions = {
                    navController.navigate(Screen.Instructions.route)
                },
                onNavigateToStatistics = {
                    navController.navigate(Screen.Statistics.route)
                },
                onNavigateToPractice = { courseId ->
                    navController.navigate(Screen.Practice.createRoute(courseId))
                },
                onNavigateToCategory = { courseName ->
                    // §12.19: the home page now lists *groups* of
                    // courses. Tapping a group navigates to the
                    // category-detail screen with the group name.
                    navController.navigate(Screen.CourseDetail.createRoute(courseName))
                },
                onNavigateToImport = {
                    navController.navigate(Screen.Import.route)
                },
            )
            1 -> VocabularyScreen(onNavigateBack = null)
            2 -> ReciteScreen(
                onNavigateToCategory = { categoryId ->
                    navController.navigate(Screen.CategoryStudy.createRoute(categoryId))
                },
            )
            3 -> MeScreen(
                // (2026-07-04) wires the "权限使用说明" entry on MeScreen
                // to the new sub-page route. Reaching PermissionsScreen
                // sets `currentBackStackRoute = "permissions"` in
                // MainScaffold, which already hides the bottom bar
                // because of the `isOnSubPage` check.
                onNavigateToPermissions = {
                    navController.navigate(Screen.Permissions.route)
                },
            )
        }
    }
}

```

```
// === app\src\main\java\com\echoling\app\presentation\ui\navigation\TopLevelDestination.kt ===
```

```
package com.echoling.app.presentation.ui.navigation

import androidx.annotation.StringRes
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.MenuBook
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.Style
import androidx.compose.material.icons.filled.Translate
import androidx.compose.material.icons.outlined.MenuBook
import androidx.compose.material.icons.outlined.Person
import androidx.compose.material.icons.outlined.Style
import androidx.compose.material.icons.outlined.Translate
import androidx.compose.ui.graphics.vector.ImageVector
import com.echoling.app.R

/**
 * One row in the bottom navigation bar. Each entry pairs a route (which must
 * exist in [Screen]) with a label, an icon set, and the set of all routes
 * that "are this tab" — used by [MainScaffold] to decide when to render the
 * bar.
 */
enum class TopLevelDestination(
    val route: String,
    @StringRes val labelRes: Int,
    val selectedIcon: ImageVector,
    val unselectedIcon: ImageVector,
) {
    Courses(
        route = Screen.Courses.route,
        labelRes = R.string.tab_courses,
        selectedIcon = Icons.Filled.MenuBook,
        unselectedIcon = Icons.Outlined.MenuBook,
    ),
    Vocabulary(
        route = Screen.Vocabulary.route,
        labelRes = R.string.tab_vocabulary,
        selectedIcon = Icons.Filled.Translate,
        unselectedIcon = Icons.Outlined.Translate,
    ),
    Recite(
        route = Screen.Recite.route,
        labelRes = R.string.tab_recite,
        selectedIcon = Icons.Filled.Style,
        unselectedIcon = Icons.Outlined.Style,
    ),
    Me(
        route = Screen.Me.route,
        labelRes = R.string.tab_me,
        selectedIcon = Icons.Filled.Person,
        unselectedIcon = Icons.Outlined.Person,
    );

    companion object {
        /** All tab routes — used by [MainScaffold] for bar visibility. */
    }
}
```

```

        val tabRoutes: Set<String> = entries.map { it.route }.toSet()

        /** Lookup by current route, or null if route is not a tab. */
        fun fromRoute(route: String?): TopLevelDestination? =
            entries.firstOrNull { it.route == route }
    }
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\course\CourseDetailScreen.kt ===

package com.echoling.app.presentation.ui.screens.course

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExtendedFloatingActionButton
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import androidx.hilt.navigation.compose.hiltViewModel
import com.echoling.app.presentation.ui.components.CompactConfirmDialog
import com.echoling.app.presentation.ui.components.CourseListItem
import com.echoling.app.presentation.ui.components.PageHeader
import com.echoling.app.presentation.viewmodel.CourseDetailViewModel

/**
 * Category-detail screen — repurposed from the old "Start Learning"
 * single-course detail page in §12.19. The route parameter is now the
 * `courseName` group (the user-facing folder name), and the body shows
 * the list of lessons in that group via [CourseListItem]. Tapping a
 * lesson goes directly to Practice** (no second detail page in
 * between). Long-tap / delete-icon removes the lesson; when the last
 * lesson in a group is deleted, the empty-state hint tells the user
 * the group will disappear on back.
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CourseDetailScreen(
    courseName: String,
    onNavigateBack: () -> Unit,
    onNavigateToPractice: (courseId: String) -> Unit,
) {
    /**
     * Navigates to the Import screen with the current group name

```



```

    * pre-filled into the form (see § 12.19). The user only needs to
    * type the per-lesson title.
    */
    onNavigateToImport: () -> Unit,
    viewModel: CourseDetailViewModel = hiltViewModel()
) {
    val uiState by viewModel.uiState.collectAsState()
    var courseToDelete by remember { mutableStateOf<String?>(null) }

    LaunchedEffect(courseName) {
        viewModel.loadGroup(courseName)
    }

    Scaffold(
        // No topBar — back button + group-name title in PageHeader.
        floatingActionButton = {
            ExtendedFloatingActionButton(
                onClick = onNavigateToImport,
                icon = { Icon(Icons.Default.Add, contentDescription = null) },
                text = { Text("导入素材") },
                containerColor = MaterialTheme.colorScheme.primaryContainer,
                contentColor = MaterialTheme.colorScheme.onPrimaryContainer,
            )
        },
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
        ) {
            PageHeader(
                onBack = onNavigateBack,
                title = {
                    Text(
                        text = uiState.courseName.ifBlank { "练习" },
                        // titleMedium (16sp) → titleSmall (14sp): keeps
                        // parity with ContinueLearningCard / CourseGroupItem
                        // / CourseListItem so the courses area uses a
                        // consistent title size top-to-bottom.
                        style = MaterialTheme.typography.titleSmall,
                        fontWeight = FontWeight.SemiBold,
                        maxLines = 1,
                        overflow = TextOverflow.Ellipsis,
                    )
                },
            )
            Box(modifier = Modifier.fillMaxSize()) {
                when {
                    uiState.isLoading -> {
                        CircularProgressIndicator(
                            modifier = Modifier.align(Alignment.Center)
                        )
                    }
                    uiState.error != null -> {
                        Text(
                            text = uiState.error ?: "Unknown error",
                            modifier = Modifier.align(Alignment.Center),
                            color = MaterialTheme.colorScheme.error
                        )
                    }
                }
                uiState.courses.isEmpty() -> {
                    // Group exists but has no lessons (e.g. the
                    // user just deleted the last one). Show a soft

```

```

        // hint instead of an error — the home page
        // will drop this group on the next emission.
        Text(
            text = "暂无练习",
            style = MaterialTheme.typography.titleSmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
            modifier = Modifier.align(Alignment.Center),
        )
    }
    else -> {
        LazyColumn(
            modifier = Modifier.fillMaxSize(),
            contentPadding = PaddingValues(horizontal = 16.dp, vertical = 12.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp),
        ) {
            items(uiState.courses, key = { it.courseId }) { course ->
                CourseListItem(
                    course = course,
                    onClick = { onNavigateToPractice(course.courseId) },
                    onDeleteClick = { courseToDelete = course.courseId },
                )
            }
            // Reserve space at the bottom so the last
            // CourseListItem doesn't sit underneath the
            // Extended FAB.
            item("fab-clear") {
                Spacer(modifier = Modifier.height(80.dp))
            }
        }
    }
}

courseToDelete?.let { id ->
    CompactConfirmDialog(
        title = "删除练习",
        message = "确定要删除这个练习吗？",
        confirmText = "删除",
        dismissText = "取消",
        onConfirm = {
            viewModel.deleteCourse(id)
            courseToDelete = null
        },
        onDismiss = { courseToDelete = null },
    )
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\courses\CoursesScreen.kt ===

package com.echoling.app.presentation.ui.screens.courses

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.WindowInsets
import androidx.compose.foundation.layout.fillMaxSize

```

```

import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBars
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.HelpOutline
import androidx.compose.material.icons.outlined.Headphones
import androidx.compose.material3.Button
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.ExtendedFloatingActionButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.hilt.navigation.compose.hiltViewModel
import com.echoling.app.R
import com.echoling.app.domain.model.CourseGroup
import com.echoling.app.domain.model.effectiveCourseName
import com.echoling.app.presentation.ui.components.ContinueLearningCard
import com.echoling.app.presentation.ui.components.CourseGroupItem
import com.echoling.app.presentation.ui.components.PageHeader
import com.echoling.app.presentation.ui.components.StatsSummaryCard
import com.echoling.app.presentation.viewmodel.CoursesViewModel

```

```

/**
 * Root of the Courses tab. Single LazyColumn that scrolls:
 * 1. StatsSummaryCard (study time + words, tappable → statistics)
 * 2. ContinueLearningCard (only if there's a course in progress)
 * 3. Section header
 * 4. List of [CourseGroupItem] — each one is a "folder" of lessons
 *    sharing a courseName; tapping it opens the category detail
 *    page (§ 12.19) which lists the individual lessons.
 *
 * An Extended FAB anchored bottom-end opens the Import flow. Delete
 * moved off this screen — it now lives on the inner CourseListItem
 * inside the category detail.
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CoursesScreen(
    onNavigateToInstructions: () -> Unit,
    onNavigateToStatistics: () -> Unit,
    onNavigateToPractice: (courseId: String) -> Unit,
    onNavigateToCategory: (courseName: String) -> Unit,
    onNavigateToImport: () -> Unit,
    viewModel: CoursesViewModel = hiltViewModel(),
) {
    val uiState by viewModel.uiState.collectAsState()

```

```

Scaffold(
    // §12.30: only consume the top status-bar inset, not the bottom
    // navigation-bar inset. The outer MainScaffold already accounts
    // for the bottom tab bar (it lives in MainScaffold's bottomBar
    // slot and gets folded into innerPadding), so if THIS inner
    // Scaffold ALSO subtracts the nav bar inset, the LazyColumn
    // body ends 24dp shorter than the Pager that hosts it — leaving
    // a 24dp page-colored strip between the tab bar and the last
    // list item. See CLAUDE.md §12.30 for the full chain.
    contentWindowInsets = WindowInsets.statusBars,
    // No topBar — content starts right below the status bar via
    // Scaffold's `padding` parameter (which still includes the status
    // bar inset). The PageHeader at the top of the body holds the
    // brand title (see §12.18).
    floatingActionButton = {
        ExtendedFloatingActionButton(
            onClick = onNavigateToImport,
            icon = { Icon(Icons.Default.Add, contentDescription = null) },
            text = { Text("导入素材") },
            containerColor = MaterialTheme.colorScheme.primaryContainer,
            contentColor = MaterialTheme.colorScheme.onPrimaryContainer,
        )
    },
) { padding ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(padding),
    ) {
        PageHeader(
            title = {
                Column(horizontalAlignment = Alignment.CenterHorizontally) {
                    Text(
                        text = stringResource(R.string.courses_title),
                        style = androidx.compose.ui.text.TextStyle(
                            fontSize = 18.sp,
                            fontWeight = FontWeight.Bold,
                            letterSpacing = 3.sp,
                        ),
                        maxLines = 1,
                    )
                    Text(
                        text = stringResource(R.string.courses_subtitle),
                        style = androidx.compose.ui.text.TextStyle(
                            fontSize = 11.sp,
                            fontStyle = FontStyle.Italic,
                            fontWeight = FontWeight.Medium,
                            letterSpacing = 1.sp,
                        ),
                        color = MaterialTheme.colorScheme.onSurfaceVariant,
                        maxLines = 1,
                    )
                }
            },
            actions = {
                // §12.21: home page's stats IconButton has been
                // replaced with a Help / "使用说明" IconButton that
                // opens the new InstructionsScreen. The user can
                // still reach the statistics page via the
                // tappable StatsSummaryCard on the home body.
                IconButton(onClick = onNavigateToInstructions) {
                    Icon(Icons.Default.HelpOutline, contentDescription = "使用说明")
                }
            }
        )
    }
}

```

```

    }
},
)

Box(modifier = Modifier.fillMaxSize()) {
    when {
        // Only show the empty state once we KNOW there's no
        // data. Before the first Room emission, `groups` is
        // the default empty list — flashing "暂无练习" for
        // that brief window looks like the title + empty
        // state appear first and the real list arrives a
        // moment later, which is exactly what we want to
        // avoid.
        uiState.hasLoadedOnce && uiState.groups.isEmpty() -> {
            EmptyCourses(onNavigateToImport = onNavigateToImport)
        }
        else -> {
            CoursesList(
                totalLearnTimeMs = uiState.totalLearnTimeMs,
                totalWordsCollected = uiState.totalWordsCollected,
                // § 12.21: prefix the lesson title with the
                // parent group name so the user knows
                // "which course is this lesson from?" at
                // a glance — e.g. "新概念英语第一册 •
                // Lesson 1" instead of just "Lesson 1".
                continueTitle = uiState.continueLearning?.let { item ->
                    val group = item.course.effectiveCourseName
                    val lesson = item.course.title
                    if (group.isNotBlank() && group != lesson) "$group • $lesson"
                    else lesson
                },
                continueProgress = uiState.continueLearning?.progress?.finishRate ?: 0f,
                onContinueClick = {
                    uiState.continueLearning?.let {
                        onNavigateToPractice(it.course.courseId)
                    }
                },
                onStatsClick = onNavigateToStatistics,
                groups = uiState.groups,
                onGroupClick = onNavigateToCategory,
            )
        }
    }
}

}

}

}

}

}

@Composable
private fun CoursesList(
    totalLearnTimeMs: Long,
    totalWordsCollected: Int,
    continueTitle: String?,
    continueProgress: Float,
    onContinueClick: () -> Unit,
    onStatsClick: () -> Unit,
    groups: List<CourseGroup>,
    onGroupClick: (courseName: String) -> Unit,
) {
    LazyColumn(
        modifier = Modifier.fillMaxSize(),
        contentPadding = PaddingValues(horizontal = 16.dp, vertical = 8.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp),

```

```

    ) {
        // Stats summary on top — gives the user an at-a-glance "you've
        // studied X for Y" before the "pick up where you left off" CTA.
        item("stats") {
            StatsSummaryCard(
                totalLearnTimeMs = totalLearnTimeMs,
                totalWordsCollected = totalWordsCollected,
                onClick = onStatsClick,
            )
        }
        // Continue-learning card below the stats — the deep-purple gradient
        // hero is the visual anchor and the primary CTA on the page.
        if (continueTitle != null) {
            item("continue") {
                ContinueLearningCard(
                    title = continueTitle,
                    progress = continueProgress,
                    onClick = onContinueClick,
                )
            }
        }
        item("section-header") {
            Spacer(modifier = Modifier.height(4. dp))
            Text(
                text = "我的练习",
                style = MaterialTheme.typography.titleSmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
            )
        }
        // Group list — each row is a folder of lessons sharing a
        // courseName. Tapping a row opens the category detail screen.
        items(groups, key = { it.courseName }) { group ->
            CourseGroupItem(
                courseName = group.courseName,
                count = group.count,
                onClick = { onGroupClick(group.courseName) },
            )
        }
        item("fab-clear") {
            Spacer(modifier = Modifier.height(80. dp))
        }
    }
}

```

@Composable

```

private fun EmptyCourses(onNavigateToImport: () -> Unit) {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center,
    ) {
        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            modifier = Modifier.padding(32. dp),
        ) {
            Icon(
                imageVector = Icons.Outlined.Headphones,
                contentDescription = null,
                modifier = Modifier.size(80. dp),
                tint = MaterialTheme.colorScheme.primary.copy(alpha = 0.6f),
            )
            Spacer(modifier = Modifier.height(24. dp))
            Text(
                text = "暂无练习",
            )
        }
    }
}

```

```

        style = MaterialTheme.typography.headlineSmall,
    )
    Spacer(modifier = Modifier.height(8.dp))
    Text(
        text = "导入素材开始学习",
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
    )
    Spacer(modifier = Modifier.height(24.dp))
    Button(
        onClick = onNavigateToImport,
        modifier = Modifier.fillMaxWidth(),
    ) {
        Icon(Icons.Default.Add, contentDescription = null)
        Spacer(modifier = Modifier.height(8.dp))
        Text("导入素材")
    }
}

}

}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\import\ImportScreen.kt ===

package com.echoling.app.presentation.ui.screens.import

import android.net.Uri
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.AudioFile
import androidx.compose.material.icons.filled.Description
import androidx.compose.material.icons.filled.Upload
import androidx.compose.material.icons.filled.VideoFile
import androidx.compose.material.icons.outlined.AudioFile
import androidx.compose.material.icons.outlined.Description
import androidx.compose.material.icons.outlined.VideoFile
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.hilt.navigation.compose.hiltViewModel
import com.echoling.app.presentation.ui.components.PageHeader
import com.echoling.app.presentation.viewmodel.ImportState
import com.echoling.app.presentation.viewmodel.ImportViewModel

// (2026-06-28) Difficulty values kept in English internally so:
// 1. Existing Room rows persist with the original English strings
//    (Beginner/Intermediate/Advanced) and display identically in
//    [CourseListItem]'s difficulty chip (line ~147: `Text(course.difficulty)`)
// 2. [CourseListItem]'s accent-bar tier detection
//    (`accentColorFor(difficulty).startsWith("A"/"B"/"C")`) keeps
//    matching existing rows
// 3. iOS-side Course.difficulty stays a freeform String — no
//    schema-level coupling either way

```

```

// The dropdown UI now shows the Chinese `displayName` and writes
// the English `value` to the DB.
private data class DifficultyOption(val value: String, val displayName: String)
private val difficultyOptions = listOf(
    DifficultyOption("Beginner", "初级"),
    DifficultyOption("Intermediate", "中级"),
    DifficultyOption("Advanced", "高级"),
)

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ImportScreen(
    onNavigateBack: () -> Unit,
    onImportComplete: () -> Unit,
    /**
     * When the user opens import from a category-detail sub-page
     * (§ 12.19) this is the parent group name — the form's "素材名字"
     * field is seeded with it so the user only has to type the per-
     * lesson title. Null when launched from the home page.
     */
    prefillCourseName: String? = null,
    viewModel: ImportViewModel = hiltViewModel()
) {
    var audioUri by remember { mutableStateOf<Uri?>(null) }
    var videoUri by remember { mutableStateOf<Uri?>(null) }
    var subtitleUri by remember { mutableStateOf<Uri?>(null) }
    var courseName by remember(prefillCourseName) {
        mutableStateOf(prefillCourseName.orEmpty())
    }
    var courseTitle by remember { mutableStateOf("") }
    var selectedDifficulty by remember { mutableStateOf("Intermediate") } // English value, displayed via displayNa
me lookup
    var dropdownExpanded by remember { mutableStateOf(false) }
    var isAudioPickerLoading by remember { mutableStateOf(false) }
    var isVideoPickerLoading by remember { mutableStateOf(false) }
    var isSubtitlePickerLoading by remember { mutableStateOf(false) }

    val importState by viewModel.importState.collectAsState()
    val errorMessage by viewModel.errorMessage.collectAsState()

    val audioPickerLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.OpenDocument()
    ) { uri ->
        audioUri = uri
        isAudioPickerLoading = false
    }

    val videoPickerLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.OpenDocument()
    ) { uri ->
        videoUri = uri
        isVideoPickerLoading = false
    }

    val subtitlePickerLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.OpenDocument()
    ) { uri ->
        subtitleUri = uri
        isSubtitlePickerLoading = false
    }

    LaunchedEffect(importState) {
        if (importState == ImportState.SUCCESS) {

```



```

        onImportComplete()
    }
}

Scaffold(
    // No topBar — back + title in PageHeader (§ 12.18)
) { padding ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(padding),
    ) {
        PageHeader(
            onBack = onNavigateBack,
            // § 12.21: left-align the sub-page title (brand-bar
            // tabs Courses / Me stay centered by default).
            titleAlignment = Alignment.Start,
            title = {
                Text(
                    text = "导入素材",
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.SemiBold,
                )
            },
        )
        // Scrollable form area
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .weight(1f)
                .verticalScroll(rememberScrollState())
                .padding(horizontal = 24.dp, vertical = 12.dp),
            verticalArrangement = Arrangement.spacedBy(20.dp)
        ) {
            // Header
            Column {
                // (2026-07-04) Shrunk further: § 12.21 took it
                // from headlineSmall 24sp → titleLarge 22sp. The
                // user said it still felt like "page inside a
                // page" compared with the rest of the form, so
                // dropped another notch to titleMedium 16sp —
                // reads as a section label rather than a page
                // heading. Description below stays bodyMedium
                // (14sp), keeping clear hierarchy headline →
                // helper text.
                Text(
                    text = "导入你的素材",
                    style = MaterialTheme.typography.titleMedium
                )
                Spacer(modifier = Modifier.height(4.dp))
                Text(
                    text = "添加音频、视频和字幕文件，创建学习素材",
                    style = MaterialTheme.typography.bodyMedium,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }

            // (2026-07-04) Removed the 8dp Spacer that used to live
            // here. Combined with Arrangement.spacedBy(20.dp) the
            // header→form gap was 28dp, which the user said read
            // as "detached" — the description and the first OTF
            // felt like separate groups. The Arrangement's 20dp is
            // already plenty for visual separation; dropping the

```

```

// extra Spacer pulls the form up and lets the
// description and 素材分组名 row read as one block.

// Course group name (parent folder, §12.19). Required:
// blank values fall back to the lesson title at the
// ViewModel layer, but the button stays disabled until
// the user fills *something* in (see button enabled=
// expression at the bottom of the file).
OutlinedTextField(
    value = courseName,
    onValueChange = { courseName = it },
    label = { Text("素材分组名 (父目录)") },
    placeholder = { Text("例如: 摩登家庭第一季") },
    modifier = Modifier.fillMaxWidth(),
    singleLine = true,
    enabled = importState != ImportState.IMPORTING,
    shape = RoundedCornerShape(12.dp)
)

// Course title (per-lesson name)
OutlinedTextField(
    value = courseTitle,
    onValueChange = { courseTitle = it },
    label = { Text("素材标题") },
    placeholder = { Text("例如: 第 1 课") },
    modifier = Modifier.fillMaxWidth(),
    singleLine = true,
    enabled = importState != ImportState.IMPORTING,
    shape = RoundedCornerShape(12.dp)
)

// Audio file selector
FileSelectorCard(
    title = "音频文件 (可选)",
    subtitle = audioUri?.lastPathSegment ?: "选择音频文件",
    icon = Icons.Outlined.AudioFile,
    isSelected = audioUri != null,
    isLoading = isAudioPickerLoading,
    onClick = {
        if (importState != ImportState.IMPORTING) {
            isAudioPickerLoading = true
            audioPickerLauncher.launch(arrayOf("audio/*"))
        }
    },
    enabled = importState != ImportState.IMPORTING
)

// Video file selector
FileSelectorCard(
    title = "视频文件 (可选)",
    subtitle = videoUri?.lastPathSegment ?: "选择视频文件",
    icon = Icons.Outlined.VideoFile,
    isSelected = videoUri != null,
    isLoading = isVideoPickerLoading,
    onClick = {
        if (importState != ImportState.IMPORTING) {
            isVideoPickerLoading = true
            videoPickerLauncher.launch(arrayOf("video/*"))
        }
    },
    enabled = importState != ImportState.IMPORTING
)

```

```

// Subtitle file selector
FileSelectorCard(
    title = "字幕文件 (可选)",
    subtitle = subtitleUri?.lastPathSegment ?: "选择字幕文件",
    icon = Icons.Outlined.Description,
    isSelected = subtitleUri != null,
    isLoading = isSubtitlePickerLoading,
    onClick = {
        if (importState != ImportState.IMPORTING) {
            isSubtitlePickerLoading = true
            subtitlePickerLauncher.launch(arrayOf("text/plain", "application/x-subrip", "*/.*"))
        }
    },
    enabled = importState != ImportState.IMPORTING
)

// Difficulty selector
Card(
    modifier = Modifier.fillMaxWidth(),
    colors = CardDefaults.cardColors(
        containerColor = MaterialTheme.colorScheme.surfaceVariant
    ),
    shape = RoundedCornerShape(16.dp)
) {
    ExposedDropdownMenuBox(
        expanded = dropdownExpanded,
        onExpandedChange = {
            if (importState != ImportState.IMPORTING) {
                dropdownExpanded = it
            }
        },
        modifier = Modifier.fillMaxWidth()
    ) {
        OutlinedTextField(
            // Display Chinese label, store English value.
            // Lookup via `firstOrNull { it.value == selectedDifficulty }`
            // — the option list is tiny (3 entries) so linear scan is fine.
            value = difficultyOptions
                .firstOrNull { it.value == selectedDifficulty }
                ?.displayName ?: selectedDifficulty,
            onValueChange = {},
            readOnly = true,
            label = { Text("难度") },
            trailingIcon = {
                ExposedDropdownMenuDefaults.TrailingIcon(expanded = dropdownExpanded)
            },
            modifier = Modifier
                .fillMaxWidth()
                .menuAnchor(),
            enabled = importState != ImportState.IMPORTING,
            shape = RoundedCornerShape(12.dp)
        )
        ExposedDropdownMenu(
            expanded = dropdownExpanded,
            onDismissRequest = { dropdownExpanded = false }
        ) {
            difficultyOptions.forEach { option ->
                DropdownMenuItem(
                    text = { Text(option.displayName) },
                    onClick = {
                        selectedDifficulty = option.value
                        dropdownExpanded = false
                    },
                ),
            }
        }
    }
}

```

```

        leadingIcon = if (option.value == selectedDifficulty) {
            {
                Icon(
                    Icons.Default.AudioFile,
                    contentDescription = null,
                    tint = MaterialTheme.colorScheme.primary
                )
            }
        } else null
    )
}
}
}

// Error message
if (importState == ImportState.ERROR) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.errorContainer
        ),
        shape = RoundedCornerShape(12.dp)
    ) {
        Row(
            modifier = Modifier.padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text(
                text = errorMessage ?: "导入失败",
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onErrorContainer,
                modifier = Modifier.weight(1f)
            )
        }
    }
}

// Pinned import button at the bottom
Box(
    modifier = Modifier
        .fillMaxWidth()
        .padding(horizontal = 24.dp, vertical = 16.dp)
) {
    Button(
        onClick = {
            val hasMedia = audioUri != null || videoUri != null
            if (hasMedia && courseTitle.isNotBlank() && courseName.isNotBlank()) {
                viewModel.importCourse(
                    courseName = courseName,
                    title = courseTitle,
                    difficulty = selectedDifficulty,
                    audioUri = audioUri,
                    videoUri = videoUri,
                    subtitleUri = subtitleUri
                )
            }
        },
        modifier = Modifier
            .fillMaxWidth()
            .height(56.dp),
        enabled = (audioUri != null || videoUri != null) &&

```



```

        Icon(
            imageVector = icon,
            contentDescription = null,
            modifier = Modifier.size(40.dp),
            tint = if (isSelected)
                MaterialTheme.colorScheme.primary
            else
                MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
    Spacer(modifier = Modifier.width(16.dp))
    Column(modifier = Modifier.weight(1f)) {
        Text(
            text = title,
            style = MaterialTheme.typography.titleMedium,
            color = if (enabled)
                MaterialTheme.colorScheme.onSurface
            else
                MaterialTheme.colorScheme.onSurfaceVariant
        )
        Text(
            text = if (isLoading) "正在打开文件选择器..." else subtitle,
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
            maxLines = 1
        )
    }
    if (isSelected) {
        Icon(
            imageVector = Icons.Default.AudioFile,
            contentDescription = "已选择",
            tint = MaterialTheme.colorScheme.primary,
            modifier = Modifier.size(20.dp)
        )
    }
}
}
}
}

```

```
// === app\src\main\java\com\echoling\app\presentation\ui\screens\instructions\InstructionsScreen.kt ===
```

```
package com.echoling.app.presentation.ui.screens.instructions
```

```

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier

```

```

import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.echoling.app.presentation.ui.components.PageHeader

/**
 * In-app 使用说明 page (§12.21). Reached from the home page's top-right
 * Help IconButton. Drafted as a one-page reference covering the main
 * flows of the app — navigation, course management, practice,
 * flashcards, vocabulary, statistics — so a first-time user can read
 * it once and stop guessing what each button does.
 *
 * The API config section (§12.21 v1) was removed after §12.26 deleted
 * the entire API config feature. Two new behaviors were added since:
 * gesture-driven tab navigation (§12.27) and local on-device
 * pronunciation scoring in the testing tab. Both are documented here.
 *
 * Layout: a single scrollable Column of [InstructionSection] cards.
 * Each section has a bold title and a short bulleted list of steps.
 * The body sits below a left-aligned PageHeader so the screen follows
 * the same visual language as the other sub-pages (see PageHeader
 * §12.18 / §12.21).
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun InstructionsScreen(
    onNavigateBack: () -> Unit,
) {
    Scaffold(
        // No topBar — PageHeader at the top of the body handles the
        // back + title.
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
        ) {
            PageHeader(
                onBack = onNavigateBack,
                titleAlignment = Alignment.Start,
                title = {
                    Text(
                        text = "使用说明",
                        style = MaterialTheme.typography.titleMedium,
                        fontWeight = FontWeight.SemiBold,
                    )
                },
            )
            Column(
                modifier = Modifier
                    .fillMaxSize()
                    .verticalScroll(rememberScrollState())
                    .padding(horizontal = 16.dp, vertical = 8.dp),
            ) {
                // Sections are visually separated by a thin
                // HorizontalDivider rather than spacing. The divider's
                // vertical padding gives the eye a breath between
                // sections without resorting to a tinted card background.
                val sections = listOf(
                    "一、主界面与导航" to listOf(
                        "底部 4 个 Tab: 首页 / 单词本 / 记单词 / 我的。",
                        "切换 Tab: 点底部按钮, 或在屏幕上左右滑动 (Pager 手势驱动, 左滑下一个, 右滑上一个)。",
                        "左滑 / 右滑时屏幕会带方向动画: 左滑新页从右滑入, 右滑新页从左滑入。",
                        "进入子页面 (跟读练习 / 素材详情 / 统计 / 导入 / 闪卡) 后底部 Tab 自动隐藏, 按返回键回到上一"
                    )
                )
            }
        }
    }
}

```

级。",

),

"二、素材管理" to listOf(

"点击首页右下角的「导入素材」按钮开始导入。",

"依次选择音频或视频文件（任选其一），再选择字幕文件（可选，支持 SRT / ASS / LRC 格式）。",

"填写「素材名字」（例如「摩登家庭第一季」）和「素材标题」（例如「第 1 课」），点击底部「Import Course」完成导入。",

"素材名字相同的素材会自动归为一组；首页只显示分组名称，不显示具体素材。",

"点击分组进入子页面，可以查看、删除该组下的素材，或点击右上角「导入素材」按钮继续导入同组素材。",

"最近学习的素材会出现在首页紫色渐变「继续学习」Hero 卡片中，点圆图播放按钮可直接进入跟读练习。",

。",

),

"三、跟读练习" to listOf(

"在素材子页面点击任意素材即可直接进入跟读练习，没有中间页。",

"泛听：完整播放音频或视频，字幕与播放进度同步。点击列表中的任意字幕可跳转到对应句子。",

"精听：单句播放，支持按句切换、循环播放、调整速度。长按录音按钮进行复述，可回放录音。",

"测试：从已学习的句子中随机抽取练习，遮盖英文单词，逐一点击揭示，全部揭示后显示中文翻译。",

"测试页的核心交互：按住麦克风复述当前句子，松开后本地 STT 把你的声音转换成文字；点击文字可以手动修改，满意后「提交」——系统把你的文字和原文逐词比对，匹配即通过。",

"不通过时会用颜色标出漏词 / 多词 / 错词的位置，可以「重录」再来一次或直接修改文字后再提交。"

,

"本地 STT 把你的声音转换成文字，无需联网、无需任何 API Key。",

"长按字幕中的任意英文单词，弹出翻译与「保存到生词本」对话框——查询走本地词典，无需联网。",

),

"四、记单词" to listOf(

"切换到底部「记单词」标签页，可看到 11 个内置词库（初中 / 高中 / CET-4 / CET-6 / TOEFL 等）。",

",

"点击词库卡片进入闪卡学习。卡片正面显示英文单词和相应的英文句子，点击卡片可 3D 翻转到背面查看音标、词性和翻译，再次点击翻回正面。",

"单词旁 / 音标旁的喇叭按钮可发音（TTS），支持离线英语引擎，引擎未装时会提示安装。",

"「认识」：标记当前单词已掌握，自动 +1 认识数并切换到下一张。",

"「不认识」：弹出「加入单词本」按钮，点击后将单词保存到生词本，方便日后复习。",

"「上一张」/「下一张」：纯导航按钮，不计入统计，方便回看或跳过。",

"顶部右上角的刷新按钮可一键清空当前词库的学习进度（当前位置、认识数、不认识数全部归零）。",

"所有学习进度自动保存，下次打开同一词库会从上次位置继续，无需从头开始。",

"词库卡片实时显示「已学 N / total 词 • 认识 K • 不认识 U」以及「上次学习于 X 分钟前」。",

),

"五、生词本" to listOf(

"在跟读练习中长按单词 → 弹出翻译对话框 → 点击「保存」即可加入生词本。",

"在「记单词」页面点击「不认识」→「加入单词本」也会保存到生词本。",

"切换到底部「生词」标签页，可查看、搜索、删除所有保存过的单词。",

"每行单词右侧的喇叭按钮可一键发音。",

"点击单词可以切换「已掌握」状态；已掌握的单词会反映到首页统计的「已掌握」数字中。",

),

"六、学习统计" to listOf(

"点击首页上方的「学习时长」统计卡片可进入统计页面。",

"统计页面展示：累计学习时长、已开始学习的素材数、已收集与已掌握的单词数、连续打卡天数。",

"柱状图分别展示过去 7 天和过去 30 天的每日学习时长，方便对比短期与长期表现。",

"学习时长只在「音频或视频实际播放」时累计——暂停、切到测试题、翻看字幕列表时不会虚增。",

),

"七、其他" to listOf(

"所有学习进度、单词、统计在本地保存；删除 App 数据或清除存储会一并清空。",

"单词翻译、句子翻译、单词发音、复述测试音转文字均为本地离线功能，无需联网、无需任何 API Key。",

。",

"如遇问题或希望反馈，可进入「我的」标签页查看联系邮箱。",

),

)

sections.forEachIndexed { index, (title, items) ->

InstructionSection(title = title, items = items)

if (index < sections.lastIndex) {

// Thin ldp rule between sections. Drawn as a

// Box + background rather than Material 3's


```

        // HorizontalDivider because the latter is a
        // 1.2.0 token and this project pins
        // material3 to 1.1.2 (see StatisticsScreen
        // § 12.21/ § 12.32b notes on the same pin).
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .padding(vertical = 12.dp)
                .height(1.dp)
                .background(
                    MaterialTheme.colorScheme.outlineVariant.copy(alpha = 0.5f)
                )
        )
    }
}

Spacer(modifier = Modifier.height(16.dp))
}
}
}

@Composable
private fun InstructionSection(
    title: String,
    items: List<String>,
) {
    // No background fill — sections are visually separated by the
    // HorizontalDivider placed between them in the parent Column, so
    // each section reads as "a block of text with a thin rule under
    // it" rather than "a tinted card". titleSmall + primary keeps the
    // section anchor visually distinct from the body bullets below.
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 12.dp),
        verticalArrangement = Arrangement.spacedBy(6.dp),
    ) {
        Text(
            text = title,
            style = MaterialTheme.typography.titleSmall,
            fontWeight = FontWeight.SemiBold,
            color = MaterialTheme.colorScheme.primary,
        )
        items.forEach { line ->
            Text(
                text = "□ $line",
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
            )
        }
    }
}

```

```
// === app\src\main\java\com\echoling\app\presentation\ui\screens\me\MeScreen.kt ===
```

```
package com.echoling.app.presentation.ui.screens.me
```

```
import android.content.Intent
import android.content.pm.PackageManager
import android.net.Uri
```

```

import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.WindowInsets
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBars
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.Info
import androidx.compose.material.icons.filled.Mic
import androidx.compose.material.icons.filled.School
import androidx.compose.material.icons.filled.Star
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.echoling.app.R
import com.echoling.app.presentation.ui.components.PageHeader

/**
 * "Me" tab — app info, contact, and version. Body ported from the previous
 * AboutScreen with the back arrow removed and the version read from
 * PackageManager instead of a hardcoded string.
 *
 * Layout: the top of the page uses a Material 3 [CenterAlignedTopAppBar]
 * with a two-line [Column] title that mirrors the [CoursesScreen] header
 * exactly — same fontSize / fontWeight / letterSpacing / fontStyle for
 * both the main title (听言英语) and the English subtitle. This keeps
 * the four-tab app visually consistent: switching from "首页" to "我的"
 * should feel like the same brand bar, just with a different title.
 *
 * The [TopAppBar] natively handles the status-bar inset (no
 * [statusBarsPadding] needed on the body) and is always visible — it
 * does not scroll with the page, which was the original §12.11

```

```

* requirement ("听言英语 始终可见").
*/
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MeScreen(
    // (2026-07-04) New — wires the "权限使用说明" entry card below the
    // ContactCard to a sub-page navigation. TabPagerHost owns the
    // NavController and injects the lambda. Defaulted to a no-op so
    // future preview surfaces (composable previews, tests) still
    // compile without a NavController.
    onNavigateToPermissions: () -> Unit = {},
) {
    val context = LocalContext.current
    val version = remember {
        resolveVersion(context.packageManager, context.packageName)
    }

    Scaffold(
        // § 12.30: only consume the top status-bar inset, not the bottom
        // navigation-bar inset. The outer MainScaffold already accounts
        // for the bottom tab bar, so a second nav-bar subtraction here
        // would leave a 24dp page-colored strip between the tab bar
        // and the last list item. See CLAUDE.md § 12.30.
        contentWindowInsets = WindowInsets.statusBars,
        // No topBar — the brand title sits in the PageHeader at the top of
        // the body, right below the status bar (§ 12.18). Same two-line
        // layout as CoursesScreen's header.
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
            horizontalAlignment = Alignment.CenterHorizontally,
        ) {
            PageHeader(
                title = {
                    Column(horizontalAlignment = Alignment.CenterHorizontally) {
                        Text(
                            text = stringResource(R.string.me_title),
                            style = TextStyle(
                                fontSize = 18.sp,
                                fontWeight = FontWeight.Bold,
                                letterSpacing = 3.sp,
                            ),
                        ),
                        maxLines = 1,
                    )
                    Text(
                        text = stringResource(R.string.me_subtitle),
                        style = TextStyle(
                            fontSize = 11.sp,
                            fontStyle = FontStyle.Italic,
                            fontWeight = FontWeight.Medium,
                            letterSpacing = 1.sp,
                        ),
                    ),
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                    maxLines = 1,
                )
            },
        )
        Column(
            modifier = Modifier
                .fillMaxSize()

```

```

        .verticalScroll(rememberScrollState())
        .padding(horizontal = 24.dp, vertical = 4.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        Image(
            // Use a dedicated copy of the app icon (ic_me_page_icon.png)
            // instead of referencing R.drawable.ic_app_icon directly.
            // Sharing the launcher asset between the system launcher and
            // an in-page tile forces Android to scale the same bitmap to
            // wildly different target sizes, which can crop or distort
            // the rounded corners that the asset was designed around.
            // A separate copy keeps each drawable's density bucket
            // independent. `Image` (unlike `Icon`) does not apply a
            // default tint, so we don't need to set anything to keep
            // the bitmap's original colors.
            painter = painterResource(id = R.drawable.ic_me_page_icon),
            contentDescription = null,
            modifier = Modifier
                .size(80.dp)
                .clip(RoundedCornerShape(16.dp)),
        )

        Spacer(modifier = Modifier.height(12.dp))

        Text(
            text = stringResource(R.string.app_name),
            style = MaterialTheme.typography.titleMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
        )

        Spacer(modifier = Modifier.height(8.dp))

        Surface(
            shape = RoundedCornerShape(20.dp),
            color = MaterialTheme.colorScheme.primaryContainer,
        ) {
            Text(
                text = "Version $version",
                style = MaterialTheme.typography.labelMedium,
                color = MaterialTheme.colorScheme.onPrimaryContainer,
                modifier = Modifier.padding(horizontal = 16.dp, vertical = 6.dp),
            )
        }

        Spacer(modifier = Modifier.height(32.dp))

        // Sections below are flat (no Card background) and
        // separated by a thin 1dp divider drawn between them.
        // Same flat treatment as InstructionsScreen §12.21 — see
        // InfoCard comment for the rationale.
        //
        // (2026-07-07) ContactCard reordered to the bottom so the
        // disclosure-style "Permission Usage" sits closer to the
        // user-facing product features above, and the call-to-
        // action "Contact Us" lives at the end of the page where
        // users reach it after they've read about the app.
        val sections = listOf<@Composable () -> Unit>(
            {
                InfoCard(
                    icon = Icons.Default.Info,
                    title = "App Introduction",
                    body = "听言英语是一款专注于英语听力与口语练习的应用。" +
                        "通过跟读训练、智能断句、逐句播放等功能，帮助用户提升英语听说能力。",
                )
            }
        )
    }

```

```

    )
},
{
    InfoCard(
        icon = Icons.Default.Star,
        title = "Features",
        items = listOf(
            "跟读练习 - 智能录音对比",
            "逐句播放 - 精准控制进度",
            "英文字幕 - 纯英文显示",
            "断点续学 - 自动记忆进度",
            "本地词典 - 长按取词离线查",
            "单词收藏 - 一键保存复习",
            "声音识别 - 自动识别复述的句子",
        ),
    ),
},
{
    // §12.24: flashcard feature documentation. Dedicated
    // InfoCard (not merged into the Features list above)
    // so the two feature groups stay visually separate —
    // the icons differ (School vs Star) and users can
    // scan them as "Practice features" → "Vocabulary
    // features".
    InfoCard(
        icon = Icons.Default.School,
        title = "记单词 • Flashcards",
        items = listOf(
            "十一大词库 - 小学 / 初中 / 高中 / CET-4 / CET-6 / CET-8 / 考研 / 托福 / GRE / IELTS / B
EC",
            "方形卡片 - 点击 3D 翻牌查看翻译",
            "进度记忆 - 自动保存当前位置和已学数",
            "加入单词本 - 不认识的词一键收藏复习",
            "进度可视化 - 首页查看每词库已学进度",
        ),
    ),
},
{
    // (2026-07-04) Permissions disclosure card. 国内
    // 应用商店 review requires an in-app, user-
    // reachable page that lists each sensitive
    // permission and explains why the app needs it +
    // where the data goes. The card below jumps to
    // PermissionsScreen which covers RECORD_AUDIO (the
    // only current sensitive permission) and asserts
    // that INTERNET is not used at all.
    PermissionsCard(onClick = onNavigateToPermissions)
},
{
    ContactCard(context = context)
},
)
sections.forEachIndexed { index, section ->
    section()
    if (index < sections.lastIndex) {
        // Thin 1dp rule between sections. Box + background
        // instead of M3 HorizontalDivider because the
        // latter is a 1.2.0 token and this project pins
        // material3 to 1.1.2 (see InstructionsScreen
        // §12.21/ §12.32b notes on the same pin).
        Box(
            modifier = Modifier
                .fillMaxWidth()

```

```

                .padding(vertical = 12.dp)
                .height(1.dp)
                .background(
                    MaterialTheme.colorScheme.outlineVariant.copy(alpha = 0.5f)
                )
            )
        }
    }

    Spacer(modifier = Modifier.height(32.dp))

    Text(
        text = "☐ 2026 听言英语\nAll rights reserved.",
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
        textAlign = TextAlign.Center,
    )
}

}

}

@Composable
private fun InfoCard(
    icon: androidx.compose.ui.graphics.vector.ImageVector,
    title: String,
    body: String? = null,
    items: List<String> = emptyList(),
) {
    // No background fill — sections on this page are separated by a
    // thin 1dp divider drawn between them in the parent Column, so
    // each card reads as "icon + title + body" rather than "a tinted
    // box". Same flat treatment as InstructionsScreen §12.21.
    Column(modifier = Modifier.fillMaxWidth()) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
        ) {
            Icon(
                imageVector = icon,
                contentDescription = null,
                tint = MaterialTheme.colorScheme.primary,
            )
            Spacer(modifier = Modifier.padding(horizontal = 8.dp))
            Text(
                text = title,
                style = MaterialTheme.typography.titleMedium,
            )
        }
        if (body != null) {
            Spacer(modifier = Modifier.height(12.dp))
            Text(
                text = body,
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
            )
        }
        if (items.isNotEmpty()) {
            Spacer(modifier = Modifier.height(12.dp))
            items.forEach { item ->
                FeatureItem(text = item)
            }
        }
    }
}

```

```

}

@Composable
private fun FeatureItem(text: String) {
    androidx.compose.foundation.layout.Row(
        modifier = Modifier.padding(vertical = 4.dp),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Text(
            text = "□",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.primary,
        )
        Spacer(modifier = Modifier.padding(horizontal = 8.dp))
        Text(
            text = text,
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
        )
    }
}

@Composable
private fun PermissionsCard(onClick: () -> Unit) {
    // Flat layout (no Card background) to match the rest of the page;
    // see InfoCard comment.
    Column(modifier = Modifier.fillMaxWidth()) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
        ) {
            Icon(
                imageVector = Icons.Default.Mic,
                contentDescription = null,
                tint = MaterialTheme.colorScheme.primary,
            )
            Spacer(modifier = Modifier.padding(horizontal = 8.dp))
            Text(
                text = "Permission Usage",
                style = MaterialTheme.typography.titleMedium,
            )
        }
        Spacer(modifier = Modifier.height(12.dp))
        Text(
            text = "查看应用所使用的全部敏感权限、收集的数据与隐私承诺。",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
        )
        Spacer(modifier = Modifier.height(12.dp))
        // TextButton (not OutlinedButton) so the affordance is a
        // "查看 →" link, matching the lower visual weight of a
        // disclosure-page entry rather than a primary action like
        // "Contact Us" below.
        TextButton(onClick = onClick) {
            Text("查看权限使用说明 →")
        }
    }
}

@Composable
private fun ContactCard(context: android.content.Context) {
    // Flat layout (no Card background) to match the rest of the page;
    // see InfoCard comment.
    Column(modifier = Modifier.fillMaxWidth()) {

```

```

        Row(
            verticalAlignment = Alignment.CenterVertically,
        ) {
            Icon(
                imageVector = Icons.Default.Email,
                contentDescription = null,
                tint = MaterialTheme.colorScheme.primary,
            )
            Spacer(modifier = Modifier.padding(horizontal = 8.dp))
            Text(
                text = "Contact Us",
                style = MaterialTheme.typography.titleMedium,
            )
        }
        Spacer(modifier = Modifier.height(12.dp))
        Text(
            text = "If you have any questions or suggestions, please contact us:",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
        )
        Spacer(modifier = Modifier.height(12.dp))
        OutlinedButton(
            onClick = {
                val intent = Intent(Intent.ACTION_SENDTO).apply {
                    data = Uri.parse("mailto:ryming213@sina.com")
                    putExtra(Intent.EXTRA_SUBJECT, "Feedback for 听言英语")
                }
                context.startActivity(intent)
            },
            modifier = Modifier.fillMaxWidth(),
            shape = RoundedCornerShape(12.dp),
        ) {
            Icon(Icons.Default.Email, contentDescription = null)
            Spacer(modifier = Modifier.padding(horizontal = 8.dp))
            Text("ryming213@sina.com")
        }
    }
}

```

```

private fun resolveVersion(pm: PackageManager, packageName: String): String {
    return try {
        @Suppress("DEPRECATION")
        val info = pm.getPackageInfo(packageName, 0)
        // 用户 2026-07-08 反馈: Me 页只显示 versionName (如 "1.0"),
        // 不再拼接 versionCode (如 "(1)")。versionCode 仍由 build.gradle.kts
        // 管理, 用于商店上传递增; UI 上让版本号更干净。
        info.versionName ?: "?"
    } catch (e: Exception) {
        "?"
    }
}

```

```
// === app\src\main\java\com\echoling\app\presentation\ui\screens\permissions\PermissionsScreen.kt ===
```

```
package com.echoling.app.presentation.ui.screens.permissions
```

```

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer

```



```

import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.CheckCircle
import androidx.compose.material.icons.filled.Mic
import androidx.compose.material.icons.filled.PrivacyTip
import androidx.compose.material.icons.filled.Storage
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.echoling.app.presentation.ui.components.PageHeader

/**
 * 权限使用说明 page — required by the 国内应用商店 review (华为 / 小米 / OPPO /
 * vivo / 应用宝) so the user has a clear, in-app explanation of why each
 * sensitive permission is requested, what data is collected, and where that
 * data goes (or doesn't go).
 *
 * As of 2026-07-04 the only requested sensitive permission is
 * `android.permission.RECORD_AUDIO`. INTERNET and ACCESS_NETWORK_STATE were
 * removed when the Vosk STT model was bundled into the APK and the
 * `alphacephei.com` download fallback in `ModelManager` was deleted — see
 * the comment block in AndroidManifest.xml for the full reasoning.
 *
 * Layout: a [PageHeader] at the top + a vertically-scrolling Column of
 * [PermissionCard] / [InfoCard] sections. Each card has a tinted icon and
 * a short body. The page sits in the same visual family as the in-app
 * InstructionsScreen (使用说明) since both are explanatory sub-pages, but
 * this page is shorter — only sensitive permissions + privacy disclosure.
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun PermissionsScreen(
    onNavigateBack: () -> Unit,
) {
    Scaffold {
        padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
        ) {
            PageHeader(
                onBack = onNavigateBack,
                titleAlignment = Alignment.Start,
                title = {
                    Text(
                        text = "权限使用说明",
                        style = MaterialTheme.typography.titleMedium,
                        fontWeight = FontWeight.SemiBold,
                    )
                }
            )
        }
    }
}

```

```

    },
  ),
  Column(
    modifier = Modifier
      .fillMaxSize()
      .verticalScroll(rememberScrollState())
      .padding(horizontal = 16.dp, vertical = 8.dp),
  ) {
    // Sections below are flat (no Card background) and
    // separated by a thin 1dp divider drawn between them.
    // Same flat treatment as InstructionsScreen §12.21 and
    // MeScreen §12 — see the flat-divider-section-style
    // memory note.
    val sections = listOf<@Composable () -> Unit>()
    {
      // Intro — the page exists because the Chinese
      // app stores (not Google Play) require a
      // published privacy disclosure inside the
      // binary as well as in the store listing.
      InfoCard(
        icon = Icons.Default.PrivacyTip,
        title = "隐私声明",
        body = "听言英语承诺：所有个人数据均在您的设备本地处理与保存，" +
          "不会上传至任何服务器，不会用于广告投放，不会与第三方共享。\\n\\n" +
          "下方列出本应用所请求的全部敏感权限，以及每项权限的使用目的与数据流向。",
      )
    },
    {
      // The one permission the app actually uses.
      PermissionCard(
        icon = Icons.Default.Mic,
        title = "麦克风 (RECORD_AUDIO)",
        status = "可选 • 用户主动触发时申请",
        purpose = "用于跟读练习的录音与本地语音识别。" +
          "仅在跟读测试页面（按住麦克风按钮）时被调用，松开按钮即停止录音。",
        dataCollected = "用户主动录制的音频，存储在 App 私有目录 (filesDir) 下的 .wav 文件。",
        dataUsage = "本地处理：Vosk 离线语音识别引擎将音频转为文字，用于与原字幕逐词比对显示跟读
          匹配结果。" +
          "同时供用户回放自己刚才的录音。",
        privacy = "音频文件不会上传至任何服务器，App 退出后随系统清理而删除；" +
          "应用本身不会保留历史录音。",
      )
    },
    {
      InfoCard(
        icon = Icons.Default.Storage,
        title = "数据存储",
        body = "应用保存的所有数据（学习进度、收藏的单词、本地词典、跟读对比）均存储在您设备的 " +
          "App 私有目录 (/data/data/com.echoling.app/) 下，" +
          "仅本应用可访问，卸载 App 时一并清除。",
      )
    },
    {
      InfoCard(
        icon = Icons.Default.CheckCircle,
        title = "网络权限说明",
        body = "本应用已移除 INTERNET 与 ACCESS_NETWORK_STATE 权限，" +
          "完全不联网：所有功能（包括语音识别、词典查询、跟读对比、TTS 发音）均在设备本地
          完成。",
      )
    }
  )
}

```

```

        sections.forEachIndexed { index, section ->
            section()
            if (index < sections.lastIndex) {
                // Thin 1dp rule between sections. Box +
                // background instead of M3 HorizontalDivider
                // because the latter is a 1.2.0 token and this
                // project pins material3 to 1.1.2 (see
                // StatisticsScreen § 12.21/ § 12.32b notes on the
                // same pin).
                Box(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(vertical = 12.dp)
                        .height(1.dp)
                        .background(
                            MaterialTheme.colorScheme.outlineVariant.copy(alpha = 0.5f)
                        )
                )
            }
        }

        Spacer(modifier = Modifier.height(16.dp))
    }
}

@Composable
private fun InfoCard(
    icon: ImageVector,
    title: String,
    body: String,
) {
    // No background fill — sections on this page are separated by a
    // thin 1dp divider drawn between them in the parent Column, so
    // each card reads as "icon + title + body" rather than "a tinted
    // box". Same flat treatment as InstructionsScreen § 12.21 and
    // MeScreen § 12 — see the flat-divider-section-style memory note.
    Column(modifier = Modifier.fillMaxWidth()) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            Icon(
                imageVector = icon,
                contentDescription = null,
                tint = MaterialTheme.colorScheme.primary,
            )
            Spacer(modifier = Modifier.padding(horizontal = 8.dp))
            Text(
                text = title,
                style = MaterialTheme.typography.titleMedium,
            )
        }
        Spacer(modifier = Modifier.height(12.dp))
        Text(
            text = body,
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
        )
    }
}

@Composable
private fun PermissionCard(
    icon: ImageVector,

```

```

        title: String,
        status: String,
        purpose: String,
        dataCollected: String,
        dataUsage: String,
        privacy: String,
    ) {
        // Flat layout (no Card background) to match the rest of the page;
        // see InfoCard comment.
        Column(modifier = Modifier.fillMaxWidth()) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(
                    imageVector = icon,
                    contentDescription = null,
                    tint = MaterialTheme.colorScheme.primary,
                )
                Spacer(modifier = Modifier.padding(horizontal = 8.dp))
                Text(
                    text = title,
                    style = MaterialTheme.typography.titleMedium,
                )
            }
            // Status pill — muted tone. "可选 • 用户主动触发时申请"
            // signals the runtime consent pattern so the user knows
            // they're not being silently recorded.
            Spacer(modifier = Modifier.height(6.dp))
            Text(
                text = status,
                style = MaterialTheme.typography.labelSmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.75f),
            )

            PermissionField(label = "使用目的", body = purpose)
            PermissionField(label = "收集的数据", body = dataCollected)
            PermissionField(label = "数据用途", body = dataUsage)
            PermissionField(label = "隐私承诺", body = privacy)
        }
    }
}

```

```

@Composable
private fun PermissionField(label: String, body: String) {
    Spacer(modifier = Modifier.height(12.dp))
    Text(
        text = label,
        style = MaterialTheme.typography.titleSmall,
        fontWeight = FontWeight.SemiBold,
        color = MaterialTheme.colorScheme.primary,
    )
    Spacer(modifier = Modifier.height(4.dp))
    Text(
        text = body,
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
    )
}

```

```
// === app\src\main\java\com\echoling\app\presentation\ui\screens\practice\components\RecordingOverlay.kt ===
```

```
package com.echoling.app.presentation.ui.screens.practice.components
```

```
import androidx.compose.animation.core.LinearEasing
```

```

import androidx.compose.animation.core.RepeatMode
import androidx.compose.animation.core.animateFloat
import androidx.compose.animation.core.infiniteRepeatable
import androidx.compose.animation.core.rememberInfiniteTransition
import androidx.compose.animation.core.tween
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.graphics.drawscope.clipRect
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp

/**
 * 录音中提示 —— 只渲染「红色圆圈 + 涟漪」（复用 [RedRecordCircle]），
 * 不再带蓝色卡片底色、“请复述你听到的句子”标题或底部计时器
 * （用户 2026-07-08 第二轮调整：去掉一切外层包裹，只保留圆圈本身）。
 *
 * 历史：
 * - flf40b5: Card(errorContainer) + PulsingRedDot + 5 根 AmplitudeBars + 计时/取消按钮
 * - 今天会话开头（字节级状态没了）：青色圆盘 + 左右涟漪
 * - 第一轮重建（卡片版）：蓝色渐变卡片 + 请复述 + 红圈 + 涟漪 + 计时
 * - 第二轮：去掉卡片，只渲染红圈 + 填色圆盘式涟漪（用户报“只有1层”）
 * - 当前：去掉卡片，红圈 + 3 道 **Stroke 描边** 涟漪（拉开 scale 1.0×→2.4×，
 *   alpha 用慢淡出（保留 40% 残影）保证三层同时可见）
 *
 * API 简化：移除了 [elapsedMs] / [amplitudeBars] 入参 —— 当前视觉不需要。
 * 调用方 [TestingPage.kt] 也会同步简化调用点。
 */
@Composable
fun RecordingOverlay(
    modifier: Modifier = Modifier,
) {
    Box(
        modifier = modifier
            .fillMaxWidth()
            .padding(horizontal = 24.dp, vertical = 12.dp),
        contentAlignment = Alignment.Center,
    ) {
        RedRecordCircle()
    }
}

/**
 * 「红色圆圈 + 3 道向四周扩散的描边涟漪」独立 Composable。
 *
 * 复用场景：
 * 1. [RecordingOverlay]（测试页录音中提示）—— 现在只剩红圈 + 涟漪
 * 2. [SpeakingPage] 的单句录音指示器（不带蓝色卡片）
 */

```

```

* 视觉规格 (2026-07-08 第四轮调整——真正的涟漪感):
* - 中心红色圆直径 = `sizeDp * 0.72` (红圈大一点, 让"正在录音"装得下)
* - 3 道 Stroke 描边涟漪 (不是填色圆盘):
*   * 只显示**左右两边的"翼** (用 clipRect 把上下裁掉 + 画完整圆) ——
*     上下不显示, 避免挡住红圈上下的视线
*   * scale 从中心圆外缘的 1.0× 扩到 1.6× (之前 2.4× 拉太大)
*   * stroke width 3dp, 向外扩时略变细 (× 0.7) 但永远可见
*   * alpha 用慢淡出 baseAlpha × (1 - 0.6 × progress): 最后到 baseAlpha × 0.4
*     不再像旧版的 (1 - progress) 那样淡到 0; 这样 3 个 ring 永远同时可见
* - phase 偏移 0 / 0.33 / 0.66 (一个 ripple 周期 = 2000ms)
* - baseAlpha 三档 0.55 / 0.40 / 0.28 —— 让内圈更亮、外圈更淡, 层次感清晰
* - Canvas 尺寸 = sizeDp × 1.6 给最大 ring 留余量
* - 文本用 labelLarge (14sp Medium), 比 titleMedium 略小, 与紧凑的圆圈比例更协调
*/
@Composable
fun RedRecordCircle(
    modifier: Modifier = Modifier,
    sizeDp: Dp = 140.dp,
) {
    val transition = rememberInfiniteTransition(label = "ripple")
    val globalProgress by transition.animateFloat(
        initialValue = 0f,
        targetValue = 1f,
        animationSpec = infiniteRepeatable(
            animation = tween(durationMillis = 2000, easing = LinearEasing),
            repeatMode = RepeatMode.Restart,
        ),
        label = "ripple-progress",
    )

    // 中心红圆直径
    val centerDiscSize = sizeDp * 0.72f
    // 画布直径 —— 给最大 ring (scale 1.6×) 留余量 + stroke 空间
    val canvasSize = sizeDp * 1.6f
    val rippleColor = Color(0xFFE53935)
    val ringStrokeWidth = 3.dp

    // 3 道涟漪的相位偏移 + baseAlpha
    // 用 list 而不是 3 个 Composable —— Compose 重组粒度更细, 性能更好
    val rings = listOf(
        0.0f to 0.55f,
        0.33f to 0.40f,
        0.66f to 0.28f,
    )

    Box(
        modifier = modifier.size(canvasSize),
        contentAlignment = Alignment.Center,
    ) {
        // 3 道描边涟漪: 每道从中心圆外缘扩散到画布边缘, 描边始终可见
        // 用 clipRect 把画布裁成水平带 (只保留中心圆上下半径范围内),
        // 画完整圆但只显示左右两边的"翼", 上下被裁掉
        Canvas(modifier = Modifier.size(canvasSize)) {
            val cx = size.width / 2f
            val cy = size.height / 2f
            val minRadiusPx = (centerDiscSize / 2f).toPx()
            val maxRadiusPx = (canvasSize / 2f).toPx() - ringStrokeWidth.toPx()
            val strokeBasePx = ringStrokeWidth.toPx()

            // 水平带: 高度 = 中心圆直径 + 一点 stroke 余量; 上下各裁掉
            val bandHalfHeight = (centerDiscSize / 2f).toPx() + ringStrokeWidth.toPx() * 2f
            val bandTop = cy - bandHalfHeight
            val bandBottom = cy + bandHalfHeight

```

```

// clipRect 是 DrawScope 的扩展, 在 lambda 内的 drawCircle/drawArc 都会被裁切
clipRect(
    left = 0f,
    top = bandTop,
    right = size.width,
    bottom = bandBottom,
) {
    rings.forEach { (offset, baseAlpha) ->
        val progress = (globalProgress + offset) % 1f
        // 半径从中心圆外缘 → 画布边缘 (线性扩张)
        val radiusPx = minRadiusPx + progress * (maxRadiusPx - minRadiusPx)
        // alpha 慢淡出: 1 - 0.6 × progress, 最后保留 40% 可见
        val alpha = (baseAlpha * (1f - progress * 0.6f)).coerceIn(0f, 1f)
        // stroke width 略变细但永远清晰
        val strokeWidthPx = strokeBasePx * (1f - progress * 0.3f)

        drawCircle(
            color = rippleColor.copy(alpha = alpha),
            radius = radiusPx,
            center = Offset(cx, cy),
            style = Stroke(width = strokeWidthPx),
        )
    }
}

// 中心红色圆 + "正在录音"
// 直径 = sizeDp * 0.72 (从 sizeDp/2 放大): 140dp * 0.72 = 100.8dp
Box(
    modifier = Modifier
        .size(centerDiscSize)
        .clip(CircleShape)
        .background(rippleColor),
    contentAlignment = Alignment.Center,
) {
    Text(
        text = "正在录音",
        style = MaterialTheme.typography.labelLarge,
        color = Color.White,
    )
}
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\practice\components\TestResultCard.kt ===

@file:OptIn(ExperimentalLayoutApi::class)

package com.echoling.app.presentation.ui.screens.practice.components

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowForward
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import com.echoling.app.presentation.viewmodel.SttTestState

```

```

/**
 * Result card shown after the user submits their transcription.
 * Passed: green card with "下一题" button.
 * Failed: red card with word-by-word comparison chips and "重录" button.
 */
@Composable
fun TestResultCard(
    state: SttTestState,
    originalEn: String,
    onNextItem: () -> Unit,
    onRerecord: () -> Unit,
    modifier: Modifier = Modifier,
) {
    val (containerColor, contentColor, title) = when (state) {
        is SttTestState.Passed -> Triple(
            Color(0xFFE8F5E9),
            Color(0xFF1B5E20),
            "□ 通过！"
        )
        is SttTestState.Failed -> Triple(
            Color(0xFFFFEBEE),
            Color(0xFFB71C1C),
            "□ 不通过"
        )
        else -> return
    }
    Card(
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(containerColor = containerColor),
        modifier = modifier
    ) {
        Column(Modifier.padding(16.dp)) {
            Text(title, style = MaterialTheme.typography.titleLarge, color = contentColor)
            Spacer(Modifier.height(8.dp))
            Text(
                "原句: ",
                style = MaterialTheme.typography.labelMedium,
                color = contentColor
            )
            Text(originalEn, style = MaterialTheme.typography.bodyLarge)
            Spacer(Modifier.height(12.dp))
            if (state is SttTestState.Failed) {
                Text(
                    "你说: ",
                    style = MaterialTheme.typography.labelMedium,
                    color = contentColor
                )
                // (2026-06-28) Pass the per-word match flags
                // through. The old call site only sent origWords /
                // transWords, which forced WordChipsRow to compare
                // position-wise and gave the user false-positives
                // like "first word wrong → all subsequent words
                // wrong". Now each chip colors itself from its
                // own flag.
                WordChipsRow(
                    origWords = state.origWords,
                    transWords = state.transWords,
                    origMatched = state.origMatched,
                    transMatched = state.transMatched
                )
                Spacer(Modifier.height(8.dp))
                Text(

```



```

*/
@Composable
private fun WordChipsRow(
    origWords: List<String>,
    transWords: List<String>,
    origMatched: BooleanArray,
    transMatched: BooleanArray
) {
    Column(modifier = Modifier.fillMaxWidth()) {
        // Row 1: original sentence. Green if matched, red if not.
        if (origWords.isNotEmpty()) {
            FlowRow(
                horizontalArrangement = Arrangement.spacedBy(4.dp),
                modifier = Modifier.fillMaxWidth()
            ) {
                origWords.forEachIndexed { i, word ->
                    val matched = origMatched.getOrNull(i) == true
                    WordChip(
                        text = word,
                        bg = if (matched) Color(0xFFC8E6C9) else Color(0xFFFFCDD2)
                    )
                }
            }
        }
        // Row 2: user's transcription. Green if matched, yellow
        // if "extra" (not in the original at all). We deliberately
        // do NOT use red here — the user said an extra word, not
        // a wrong one, so the visual cue should be milder than
        // "you got this wrong".
        if (transWords.isNotEmpty()) {
            Spacer(Modifier.height(6.dp))
            FlowRow(
                horizontalArrangement = Arrangement.spacedBy(4.dp),
                modifier = Modifier.fillMaxWidth()
            ) {
                transWords.forEachIndexed { j, word ->
                    val matched = transMatched.getOrNull(j) == true
                    WordChip(
                        text = word,
                        bg = if (matched) Color(0xFFC8E6C9) else Color(0xFFFFF9C4)
                    )
                }
            }
        }
    }
}

@Composable
private fun WordChip(text: String, bg: Color) {
    Surface(
        shape = RoundedCornerShape(8.dp),
        color = bg
    ) {
        Text(
            text = text,
            modifier = Modifier.padding(horizontal = 8.dp, vertical = 4.dp),
            style = MaterialTheme.typography.bodyMedium
        )
    }
}

private fun failureReasonText(reason: String): String = when (reason) {
    "empty_transcription" -> "未识别到语音，请重录或手动输入"
}

```

```

        "missing_word" -> "缺少单词，请检查是否漏说了"
        "extra_word" -> "多出了单词，请检查是否说多了"
        "wrong_word" -> "单词不匹配，红色标注的为错误单词"
        "no_test_item" -> "当前没有测试句子"
        else -> reason
    }
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\practice\components\TranscriptionEditor.kt ===

package com.echoling.app.presentation.ui.screens.practice.components

import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

/**
 * Post-recording panel shown after the user releases the mic.
 *
 * Recognized text is displayed immediately on entry — no "reveal"
 * step. The displayed text is clickable to enter edit mode** for
 * manual correction; submitting happens via the bottom-right
 * 提交 button.
 *
 * (2026-06-28) UI flow was simplified per user feedback:
 * - Old: click text → submit immediately; click 手动修改 →
 *   enter edit mode; then click 提交 to submit.
 * - New: click text → enter edit mode (OutlinedTextField);
 *   type / leave as-is; click 提交 to submit. The 手动修改
 *   button was redundant with the text-click affordance and
 *   has been removed.
 *
 * The 提交 button stays because edit mode needs a way to commit
 * the final text. Re-record via 重录 on the left.
 *
 * @param initialText the STT-recognized text (may be blank)
 * @param onTextChange kept for API compatibility — not used internally
 * @param onSubmit called with the final text to submit for matching
 * @param onRerecord called when the user taps 重录
 */
@Composable
fun TranscriptionEditor(
    initialText: String,
    onTextChange: (String) -> Unit,
    onSubmit: (String) -> Unit,
    onRerecord: () -> Unit,
    modifier: Modifier = Modifier,
) {
    var isEditing by remember { mutableStateOf(false) }
    var editedText by remember { mutableStateOf(initialText) }

    Card(
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surface.copy(alpha = 0.92f)
        ),
        modifier = modifier
    )

```

```

    ) {
        Column(Modifier.padding(16.dp)) {
            Text(
                "录音完成",
                style = MaterialTheme.typography.titleMedium
            )
            Spacer(Modifier.height(12.dp))

            if (isEditing) {
                // Editing mode: editable text field. User types /
                // pastes / leaves as-is, then taps 提交 below.
                OutlinedTextField(
                    value = editedText,
                    onChange = { editedText = it },
                    placeholder = { Text("输入你复述的句子") },
                    modifier = Modifier
                        .fillMaxWidth()
                        .heightIn(min = 96.dp),
                    minLines = 3
                )
            } else {
                // Showing mode (default): recognized text inside a
                // clickable Surface. Clicking the text now enters
                // edit mode (not submits directly). The hint text
                // was reworded accordingly.
                val display = initialText.ifBlank { " (未识别到语音)" }
                Surface(
                    shape = RoundedCornerShape(8.dp),
                    color = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.6f),
                    modifier = Modifier
                        .fillMaxWidth()
                        .clickable {
                            // Seed editedText with the current
                            // recognized text so the OutlinedTextField
                            // opens with the STT result pre-filled.
                            editedText = initialText
                            isEditing = true
                        }
                ) {
                    Text(
                        display,
                        style = MaterialTheme.typography.bodyLarge,
                        modifier = Modifier.padding(12.dp)
                    )
                }
                Spacer(Modifier.height(8.dp))
                Text(
                    "点击文字修改",
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }
        }

        Spacer(Modifier.height(12.dp))

        // Bottom row: 重录 on the left, 提交 on the right. The
        // 手动修改 button was removed (text click now enters
        // edit mode directly). 提交 is the only way to commit.
        Row(
            modifier = Modifier.fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically,
        ) {
            OutlinedButton(onClick = onRerecord) {

```

```

        Text("重录")
    }
    Spacer(modifier = Modifier.weight(1f))
    Button(
        onClick = {
            onSubmit(if (isEditing) editedText else initialText)
        },
        enabled = (if (isEditing) editedText else initialText).isNotBlank(),
    ) {
        Text("提交")
    }
    }
}
}
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\practice\ListeningPage.kt ===

package com.echoling.app.presentation.ui.screens.practice

import androidx.compose.foundation.background
import androidx.compose.foundation.gestures.detectTapGestures
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.itemsIndexed
import androidx.compose.foundation.lazy.rememberLazyListState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import com.echoling.app.player.subtitle.Subtitle
import com.echoling.app.presentation.viewmodel.PracticeViewModel

@Composable
fun ListeningPage(
    viewModel: PracticeViewModel,
    modifier: Modifier = Modifier
) {
    val playbackState by viewModel.playbackState.collectAsState()
    val subtitles by viewModel.subtitles.collectAsState()
    val currentSubtitleIndex by viewModel.currentSubtitleIndex.collectAsState()
    val isVideoMode by viewModel.isVideoMode.collectAsState()
    val videoPlayerState by viewModel.videoPlayerState.collectAsState()

    var showSubtitles by remember { mutableStateOf(false) }
    // Track revealed words per subtitle index: Map<subtitleIndex, Set<wordIndex>>
    var revealedWordsMap by remember { mutableStateOf<Map<Int, Set<Int>>>>(emptyMap()) }
    // Long-press translation dialog state. `null` = dialog closed.
    var wordToTranslate by remember { mutableStateOf<String?>(null) }
    val translationState by viewModel.wordTranslation.collectAsState()
    val lazyListState = rememberLazyListState()

    // Auto-scroll to current subtitle

```

```

    LaunchedEffect(currentSubtitleIndex) {
        if (currentSubtitleIndex >= 0 && currentSubtitleIndex < subtitles.size) {
            lazyListState.animateScrollToItem(currentSubtitleIndex)
        }
    }

    Column(modifier = modifier.fillMaxSize()) {
        // Video Player (if available)
        if (isVideoMode && videoPlayerState != null) {
            VideoPlayerSection(
                exoPlayer = videoPlayerState!!,
                modifier = Modifier
                    .fillMaxWidth()
                    .aspectRatio(16f / 9f)
            )
        }

        // Progress bar with play button on the left
        ListeningProgressBar(
            currentPosition = playbackState.currentPositionMs,
            duration = playbackState.durationMs,
            isPlaying = playbackState.isPlaying,
            onPlayPause = {
                if (playbackState.isPlaying) viewModel.pause() else viewModel.play()
            },
            onSeek = { viewModel.seekTo(it) },
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 16.dp, vertical = 4.dp)
        )

        // Subtitle list
        LazyColumn(
            state = lazyListState,
            modifier = Modifier
                .fillMaxWidth()
                .weight(1f)
                .padding(horizontal = 16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            itemsIndexed(subtitles) { index, subtitle ->
                val revealedWords = revealedWordsMap[index] ?: emptySet()
                ListeningSubtitleListItem(
                    subtitle = subtitle,
                    isActive = index == currentSubtitleIndex,
                    onClick = { viewModel.playSubtitleOnce(subtitle) },
                    listIndex = index + 1,
                    showSubtitles = showSubtitles,
                    revealedWords = revealedWords,
                    onWordLongPress = { wordIndex ->
                        // Reveal this specific word (only used in hide-mode)
                        revealedWordsMap = revealedWordsMap.toMutableMap().apply {
                            val currentSet = this[index] ?: emptySet()
                            this[index] = currentSet + wordIndex
                        }
                    },
                    onWordTranslate = { word ->
                        wordToTranslate = word
                        viewModel.requestWordTranslation(word)
                    },
                )
            }
            item { Spacer(Modifier.height(16.dp)) }
        }
    }

```

```

    }

    // Bottom control buttons: subtitle toggle, play/pause, speed
    ListeningBottomControls(
        showSubtitles = showSubtitles,
        playbackSpeed = playbackState.playbackSpeed,
        isPlaying = playbackState.isPlaying,
        onToggleSubtitles = { showSubtitles = !showSubtitles },
        onPlayPause = {
            if (playbackState.isPlaying) viewModel.pause() else viewModel.play()
        },
        onSpeedChange = { viewModel.setPlaybackSpeed(it) }
    )
}

// Long-press → translate dialog
val pendingWord = wordToTranslate
if (pendingWord != null) {
    WordSaveDialog(
        initialWord = pendingWord,
        translationState = translationState,
        onDismiss = {
            wordToTranslate = null
            viewModel.clearWordTranslation()
        },
        onRetranslate = { newWord ->
            // Re-runs the local dictionary lookup on the
            // (possibly user-edited) word. The dialog will update
            // with the new translation when state emits Loaded.
            viewModel.requestWordTranslation(newWord)
        },
        onSave = { word, translation, phonetic, pos ->
            viewModel.saveWord(word, translation, "", phonetic, pos)
            wordToTranslate = null
            viewModel.clearWordTranslation()
        },
    )
}
}

@Composable
private fun ListeningProgressBar(
    currentPosition: Long,
    duration: Long,
    isPlaying: Boolean,
    onPlayPause: () -> Unit,
    onSeek: (Long) -> Unit,
    modifier: Modifier = Modifier
) {
    var sliderPosition by remember(currentPosition) { mutableFloatStateOf(currentPosition.toFloat()) }
    var isDragging by remember { mutableStateOf(false) }

    Row(
        modifier = modifier,
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        // Play/Pause button on the left
        IconButton(
            onClick = onPlayPause,
            modifier = Modifier.size(40.dp)
        ) {
            Surface(

```

```

        shape = CircleShape,
        color = MaterialTheme.colorScheme.primary
    ) {
        Icon(
            if (isPlaying) Icons.Default.Pause else Icons.Default.PlayArrow,
            if (isPlaying) "暂停" else "播放",
            modifier = Modifier.size(24.dp),
            tint = MaterialTheme.colorScheme.onPrimary
        )
    }
}

// Slider progress bar
Column(modifier = Modifier.weight(1f)) {
    Slider(
        value = if (duration > 0) sliderPosition else 0f,
        onValueChange = { newValue ->
            isDragging = true
            sliderPosition = newValue
        },
        onValueChangeFinished = {
            isDragging = false
            onSeek(sliderPosition.toLong())
        },
        valueRange = 0f..duration.toFloat().coerceAtLeast(1f),
        colors = SliderDefaults.colors(
            thumbColor = MaterialTheme.colorScheme.primary,
            activeTrackColor = MaterialTheme.colorScheme.primary,
            inactiveTrackColor = MaterialTheme.colorScheme.surfaceVariant
        ),
        modifier = Modifier.fillMaxWidth(),
        enabled = duration > 0
    )

    LaunchedEffect(currentPosition) {
        if (!isDragging) {
            sliderPosition = currentPosition.toFloat()
        }
    }
}
}

}

@Composable
private fun ListeningBottomControls(
    showSubtitles: Boolean,
    playbackSpeed: Float,
    isPlaying: Boolean,
    onToggleSubtitles: () -> Unit,
    onPlayPause: () -> Unit,
    onSpeedChange: (Float) -> Unit
) {
    val speedOptions = listOf(0.5f, 0.75f, 1.0f, 1.25f, 1.5f, 2.0f)
    var showSpeedMenu by remember { mutableStateOf(false) }

    // Pill-shaped background frame that visually separates the action bar
    // from the subtitle list above it. Slight tonal elevation + surfaceVariant
    // tint give it depth without competing with the colored circle buttons.
    Surface(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp),
        shape = RoundedCornerShape(24.dp),

```



```

        color = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f),
        tonalElevation = 2.dp,
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(vertical = 12.dp),
            horizontalArrangement = Arrangement.SpaceEvenly,
            verticalAlignment = Alignment.CenterVertically
        ) {
            // Subtitle toggle
            IconButton(onClick = onToggleSubtitles) {
                Surface(
                    shape = CircleShape,
                    color = if (showSubtitles)
                        MaterialTheme.colorScheme.primaryContainer
                    else
                        MaterialTheme.colorScheme.surfaceVariant
                ) {
                    Icon(
                        if (showSubtitles) Icons.Default.Visibility else Icons.Default.VisibilityOff,
                        "字幕",
                        modifier = Modifier.size(24.dp),
                        tint = if (showSubtitles)
                            MaterialTheme.colorScheme.onPrimaryContainer
                        else
                            MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            }
        }
    }

    // Play/Pause
    IconButton(onClick = onPlayPause) {
        Surface(shape = CircleShape, color = MaterialTheme.colorScheme.primary) {
            Icon(
                if (isPlaying) Icons.Default.Pause else Icons.Default.PlayArrow,
                if (isPlaying) "暂停" else "播放",
                modifier = Modifier.size(28.dp),
                tint = MaterialTheme.colorScheme.onPrimary
            )
        }
    }

    // Speed
    Box {
        TextButton(
            onClick = { showSpeedMenu = true },
            modifier = Modifier.height(32.dp),
            contentPadding = PaddingValues(horizontal = 4.dp)
        ) {
            Text(
                text = "${playbackSpeed}x",
                style = MaterialTheme.typography.labelMedium
            )
        }
        DropdownMenu(
            expanded = showSpeedMenu,
            onDismissRequest = { showSpeedMenu = false }
        ) {
            speedOptions.forEach { speed ->
                DropdownMenuItem(
                    text = { Text("${speed}x") },
                    onClick = {

```

```
        onSpeedChange(speed)
        showSpeedMenu = false
    },
    leadingIcon = if (speed == playbackSpeed) {
        { Icon(Icons.Default.Check, contentDescription = null) }
    } else null
    )
}
}
}
}
```

```
@Composable
private fun ListeningSubtitleListItem(
    subtitle: Subtitle,
    isActive: Boolean,
    onClick: () -> Unit,
    listIndex: Int,
    showSubtitles: Boolean,
    revealedWords: Set<Int>,
    onWordLongPress: (Int) -> Unit,
    onWordTranslate: (String) -> Unit,
) {
    val words = subtitle.contentEn.split(Regex("\\s+")).filter { it.isNotEmpty() }
    val cnText = subtitle.contentCn
```

```

Surface(
    modifier = Modifier
        .fillMaxWidth()
        // §12.34: use pointerInput + detectTapGestures (NOT
        // Modifier.clickable) so the parent only handles taps and
        // never long-press. This is what lets the child words'
        // combinedClickable long-press handlers fire in the
        // showSubtitles branch. The tap handler is what fires for
        // taps that land on whitespace / Chinese / the index
        // number / the active indicator — taps on word Boxes are
        // handled by the words themselves (see below).
        .pointerInput(onClick) {
            detectTapGestures(onTap = { onClick() })
        },
    shape = RoundedCornerShape(8.dp),
    color = if (isActive)
        MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.5f)
    else
        MaterialTheme.colorScheme.surface,
    tonalElevation = if (isActive) 2.dp else 0.dp
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(12.dp),
        verticalAlignment = Alignment.Top
    ) {
        // Active indicator
        if (isActive) {
            Box(
                modifier = Modifier
                    .width(3.dp)
                    .height(32.dp)
                    .background(
                        MaterialTheme.colorScheme.primary,

```

```
RoundedCornerShape(2.dp)
    )
    Spacer(Modifier.width(8.dp))
}

Text(
    text = "$listIndex.",
    style = MaterialTheme.typography.labelSmall,
    color = if (isActive)
        MaterialTheme.colorScheme.primary
    else
        MaterialTheme.colorScheme.onSurfaceVariant
)

Spacer(Modifier.width(8.dp))

if (showSubtitles) {
    // Visible text — long-press any word to translate it,
    // tap any word to play this sentence.
    // § 12.35: always renders ENGLISH only (no Chinese /
    // bilingual branches — the page only displays English).
    ListeningVisibleSubtitleText(
        subtitle = subtitle,
        onWordLongPress = onWordTranslate,
        onSentenceClick = onClick,
        modifier = Modifier.weight(1f),
    )
} else {
    // Show hidden words as covered blocks with long-press to reveal
    Column(modifier = Modifier.weight(1f)) {
        // English words with wrapping — tap a block to play
        // this sentence, long-press to reveal the word.
        ListeningHiddenWordsFlowRow(
            words = words,
            revealedWords = revealedWords,
            onWordLongPress = onWordLongPress,
            onSentenceClick = onClick,
            modifier = Modifier.fillMaxWidth()
        )

        // Chinese translation (always visible when subtitles hidden)
        if (cnText.isNotEmpty()) {
            Spacer(Modifier.height(4.dp))
            Text(
                text = cnText,
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
    }
}

}

}
```

/**

- * Renders the subtitle as inline Text composables that wrap naturally.
- * § 12.35: the practice page only displays English now (the subtitle-mode pill is gone), so this function is ENGLISH-only
- * — no bilingual / Chinese branches.

*

* English words are tappable (play this sentence) and long-pressable

```

* (translate it). Whitespace between tokens is preserved via explicit
* [Text](" ").
*/
@OptIn(androidx.compose.foundation.layout.ExperimentalLayoutApi::class)
@Composable
private fun ListeningVisibleSubtitleText(
    subtitle: Subtitle,
    onWordLongPress: (String) -> Unit,
    onSentenceClick: () -> Unit,
    modifier: Modifier = Modifier,
) {
    val enText = subtitle.contentEn
    // Tokenize into "word" units: each word is a run of [A-Za-z']
    // characters, possibly with leading/trailing punctuation attached
    // (so "Hello," stays one visual unit). Whitespace between words
    // becomes its own token. Example: "Hello, world." →
    // ["Hello,", " ", "world", ".", " "].
    val tokens = remember(enText) {
        enText.split(Regex("(?<=\\s)|(?=\\s)"))
            .filter { it.isNotEmpty() }
    }

    val showEn = enText.isNotEmpty()

    val style = MaterialTheme.typography.bodyMedium
    val color = MaterialTheme.colorScheme.onSurface

    androidx.compose.foundation.layout.FlowRow(
        modifier = modifier,
        horizontalArrangement = Arrangement.Start,
        verticalArrangement = Arrangement.Top,
    ) {
        if (showEn) {
            for (token in tokens) {
                if (token.isBlank()) {
                    // Preserve the original whitespace (could be multi-space).
                    Text(text = token, style = style, color = color)
                } else {
                    val clean = token.replace(Regex("[^\\w']"), "")
                    if (clean.isEmpty()) {
                        // Punctuation only — no word to translate.
                        Text(text = token, style = style, color = color)
                    } else {
                        // §12.34: each word is a tap+long-press
                        // Box. Tap plays the whole sentence (the
                        // parent's onClick); long-press translates
                        // this word. Using pointerInput +
                        // detectTapGestures (NOT combinedClickable)
                        // so the press does NOT show the M3 ripple —
                        // words are inline text, not buttons, and the
                        // ripple on a press in the middle of a flowing
                        // sentence reads as visual noise. The hit
                        // target is unchanged; only the visual
                        // feedback is suppressed. Whitespace /
                        // Chinese / index / active indicator taps
                        // still fall through to the parent Surface's
                        // detectTapGestures handler and play the
                        // sentence the same way.
                        Box(
                            modifier = modifier
                                .clip(RoundedCornerShape(4.dp))
                                .pointerInput(onSentenceClick, clean) {
                                    detectTapGestures(

```

```

        onTap = { onSentenceClick() },
        onLongPress = { onWordLongPress(clean) },
    )
    }
    .padding(horizontal = 2.dp, vertical = 2.dp),
    contentAlignment = Alignment.Center,
) {
    Text(text = token, style = style, color = color)
}
}
}
}
}

@Composable
private fun ListeningHiddenWordsFlowRow(
    words: List<String>,
    revealedWords: Set<Int>,
    onWordLongPress: (Int) -> Unit,
    onSentenceClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    // Pre-calculate which words fit on each line
    val lines = remember(words) {
        val result = mutableListOf<List<Pair<Int, String>>>()
        var currentLine = mutableListOf<Pair<Int, String>>()
        var currentLineCharCount = 0
        val maxCharsPerLine = 25

        words.forEachIndexed { index, word ->
            val cleanWord = word.replace(Regex("[^\\w']"), "")
            if (cleanWord.isNotEmpty()) {
                if (currentLineCharCount + cleanWord.length > maxCharsPerLine && currentLine.isNotEmpty()) {
                    result.add(currentLine.toList())
                    currentLine = mutableListOf()
                    currentLineCharCount = 0
                }
                currentLine.add(index to cleanWord)
                currentLineCharCount += cleanWord.length + 1
            }
        }
        if (currentLine.isNotEmpty()) {
            result.add(currentLine.toList())
        }
        result
    }

    Column(
        modifier = modifier,
        horizontalAlignment = Alignment.Start
    ) {
        lines.forEach { lineWords ->
            Row(
                horizontalArrangement = Arrangement.Start,
                verticalAlignment = Alignment.CenterVertically,
                modifier = Modifier.fillMaxWidth()
            ) {
                lineWords.forEach { (index, cleanWord) ->
                    val isRevealed = revealedWords.contains(index)

                    HiddenWordBlock(

```

```

        word = cleanWord,
        isRevealed = isRevealed,
        onClick = onSentenceClick,
        onLongPress = { onWordLongPress(index) }
    )
    Spacer(modifier = Modifier.width(4. dp))
}
}
}
}
}

@Composable
private fun HiddenWordBlock(
    word: String,
    isRevealed: Boolean,
    onClick: () -> Unit,
    onLongPress: () -> Unit
) {
    if (isRevealed) {
        // § 12.34: even when revealed, tap on a word plays the
        // sentence. § 12.34a: pointerInput + detectTapGestures
        // (NOT combinedClickable) so the press does NOT show the
        // M3 ripple on the word block.
        Box(
            modifier = Modifier
                .clip(RoundedCornerShape(4. dp))
                .pointerInput(onClick, onLongPress) {
                    detectTapGestures(
                        onTap = { onClick() },
                        onLongPress = { onLongPress() },
                    )
                }
            .padding(horizontal = 6. dp, vertical = 2. dp),
            contentAlignment = Alignment.Center,
        ) {
            Text(
                text = word,
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurface
            )
        }
    } else {
        // § 12.34a: pointerInput + detectTapGestures — no M3 ripple
        // when the user taps a hidden word block.
        Box(
            modifier = Modifier
                .clip(RoundedCornerShape(4. dp))
                .background(Color(0xFFE0E0E0))
                .pointerInput(onClick, onLongPress) {
                    detectTapGestures(
                        onTap = { onClick() },
                        onLongPress = { onLongPress() },
                    )
                }
            .padding(horizontal = 6. dp, vertical = 2. dp),
            contentAlignment = Alignment.Center
        ) {
            Text(
                text = word,
                style = MaterialTheme.typography.bodyMedium,
                color = Color.Transparent
            )
        }
    }
}

```

```

    )
    }
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\practice\PracticeScreen.kt ===

package com.echoling.app.presentation.ui.screens.practice

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.viewinterop.AndroidView
import androidx.hilt.navigation.compose.hiltViewModel
import androidx.media3.ui.PlayerView
import androidx.media3.exoplayer.ExoPlayer
import com.echoling.app.presentation.ui.navigation.SUB_PAGE_NAV_ANIM_MS
import com.echoling.app.presentation.viewmodel.PracticeViewModel
import com.echoling.app.presentation.viewmodel.WordTranslationState
import kotlinx.coroutines.delay

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun PracticeScreen(
    courseId: String,
    onNavigateBack: () -> Unit,
    viewModel: PracticeViewModel = hiltViewModel()
) {
    val currentPage by viewModel.currentPage.collectAsState()

    // Initialize on first composition
    LaunchedEffect(courseId) {
        // (2026-07-04) Defer ExoPlayer init + course load past the
        // slide-in animation window. The user's reported "轻微卡顿"
        // jumping from the ContinueLearningCard to Practice was a
        // race between the slide animation's per-frame work and the
        // ExoPlayer.Builder().build() / setMediaItem / prepare
        // pipeline — both run on Main and both want the same
        // choreographer slot during the 350ms slide. The same code
        // path is reachable via CourseDetail → Practice, but the user
        // doesn't perceive the jank there because the tap-to-row
        // motion masks the race; the direct-jump path has nothing to
        // hide it. Letting the slide complete first means the user
        // sees the page "settle in" cleanly, then content populates a
        // frame or two later — visually equivalent to a slow network
        // page, much smoother than a janky slide.
        //
        // The PracticeScreen renders fine in its initial state for
        // ~350ms: black-player-area placeholder + empty subtitle list
        // + ListeningPage's empty controls. None of those are bound
        // to the ExoPlayer instance before [loadVideo] runs, so the
    }
}

```

```

        // delay is safe.
        //
        // Sharing SUB_PAGE_NAV_ANIM_MS with [SubPageNavGraph]'s
        // tween() spec means changing the slide duration is a
        // single-site edit; if the slide ever shrinks below ~250ms
        // this delay should be reconsidered.
        delay(SUB_PAGE_NAV_ANIM_MS.toLong())
        viewModel.initializePlayer()
        viewModel.loadCourse(courseId)
        viewModel.loadSentenceStates(courseId)
    }

    // Reload sentence states when switching to SpeakingPage
    LaunchedEffect(currentPage) {
        if (currentPage == PracticeViewModel.PracticePage.SPEAKING) {
            viewModel.loadSentenceStates(courseId)
        }
    }

    Scaffold(
        // No topBar — back / title / subtitle pill live in a single slim
        // Row right below the status bar (§12.18). This keeps the video
        // player edge-to-edge while still exposing the back action and
        // the subtitle mode toggle right next to the TabRow.
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
        ) {
            // Compact back-button + title row
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .height(48.dp)
                    .background(MaterialTheme.colorScheme.surface),
                verticalAlignment = Alignment.CenterVertically,
            ) {
                IconButton(onClick = onNavigateBack) {
                    Icon(
                        imageVector = Icons.Filled.ArrowBack,
                        contentDescription = "返回",
                    )
                }
                Text(
                    text = "跟读练习",
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.SemiBold,
                    modifier = Modifier.weight(1f),
                    // §12.21: left-align the title — the back button
                    // visually anchors the row's left edge, so the
                    // title belongs on the left rather than floating
                    // centered. §12.35: the subtitle-mode pill that
                    // used to live on the right edge is gone (the
                    // page only shows English), so the back button is
                    // now the sole horizontal anchor.
                    textAlign = androidx.compose.ui.text.style.TextAlign.Start,
                )
            }
        }

        // Tab Row for page switching
        TabRow(
            selectedTabIndex = currentPage.ordinal,

```



```

        containerColor = MaterialTheme.colorScheme.surface,
        contentColor = MaterialTheme.colorScheme.primary
    ) {
        PracticeViewModel.PracticePage.entries.forEach { page ->
            Tab(
                selected = currentPage == page,
                onClick = {
                    if (currentPage != page) {
                        viewModel.pause()
                    }
                    viewModel.setCurrentPage(page)
                },
                text = {
                    Row(
                        verticalAlignment = Alignment.CenterVertically,
                        horizontalArrangement = Arrangement.spacedBy(4.dp)
                    ) {
                        Icon(
                            imageVector = when (page) {
                                PracticeViewModel.PracticePage.LISTENING -> Icons.Default.Headphones
                                PracticeViewModel.PracticePage.SPEAKING -> Icons.Default.RecordVoiceOver
                                PracticeViewModel.PracticePage.TESTING -> Icons.Default.Quiz
                            },
                            contentDescription = null,
                            modifier = Modifier.size(18.dp)
                        )
                        Text(
                            text = when (page) {
                                PracticeViewModel.PracticePage.LISTENING -> "泛听"
                                PracticeViewModel.PracticePage.SPEAKING -> "精听"
                                PracticeViewModel.PracticePage.TESTING -> "测试"
                            }
                        )
                    }
                },
                selectedContentColor = MaterialTheme.colorScheme.primary,
                unselectedContentColor = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
    }

    // Page content
    Box(modifier = Modifier.weight(1f)) {
        when (currentPage) {
            PracticeViewModel.PracticePage.LISTENING -> ListeningPage(
                viewModel = viewModel,
                modifier = Modifier.fillMaxSize()
            )
            PracticeViewModel.PracticePage.SPEAKING -> SpeakingPage(
                viewModel = viewModel,
                modifier = Modifier.fillMaxSize()
            )
            PracticeViewModel.PracticePage.TESTING -> TestingPage(
                viewModel = viewModel,
                modifier = Modifier.fillMaxSize()
            )
        }
    }

    // Shared Playback Controls Bar (hidden for all pages - each page has its own controls)
    // PlaybackControlsBar is now only used within individual pages if needed
}

```

```

}

@Composable
private fun PlaybackControlsBar(
    isPlaying: Boolean,
    isLooping: Boolean,
    playbackSpeed: Float,
    currentPosition: Long,
    duration: Long,
    onPlayPause: () -> Unit,
    onSeekBackward: () -> Unit,
    onSeekForward: () -> Unit,
    onToggleLoop: () -> Unit,
    onSpeedChange: (Float) -> Unit,
    onSeek: (Long) -> Unit
) {
    val speedOptions = listOf(0.5f, 0.75f, 1.0f, 1.25f, 1.5f, 2.0f)
    var showSpeedMenu by remember { mutableStateOf(false) }

    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 8.dp, vertical = 4.dp)
    ) {
        // Progress bar
        var sliderPosition by remember(currentPosition) { mutableFloatStateOf(currentPosition.toFloat()) }
        var isDragging by remember { mutableStateOf(false) }

        Slider(
            value = if (duration > 0) sliderPosition else 0f,
            onValueChange = { newValue ->
                isDragging = true
                sliderPosition = newValue
            },
            onValueChangeFinished = {
                isDragging = false
                onSeek(sliderPosition.toLong())
            },
            valueRange = 0f..duration.toFloat().coerceAtLeast(1f),
            colors = SliderDefaults.colors(
                thumbColor = MaterialTheme.colorScheme.primary,
                activeTrackColor = MaterialTheme.colorScheme.primary,
                inactiveTrackColor = MaterialTheme.colorScheme.surfaceVariant
            ),
            modifier = Modifier.fillMaxWidth(),
            enabled = duration > 0
        )

        LaunchedEffect(currentPosition) {
            if (!isDragging) {
                sliderPosition = currentPosition.toFloat()
            }
        }

        // Time and controls row
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text(
                text = formatTime(currentPosition),
                style = MaterialTheme.typography.labelSmall,

```

```

        color = MaterialTheme.colorScheme.onSurfaceVariant
    )

    Row(
        horizontalArrangement = Arrangement.spacedBy(2.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        // Loop
        IconButton(
            onClick = onToggleLoop,
            modifier = Modifier.size(32.dp)
        ) {
            Icon(
                imageVector = Icons.Default.Repeat,
                contentDescription = "循环",
                tint = if (isLooping)
                    MaterialTheme.colorScheme.primary
                else
                    MaterialTheme.colorScheme.onSurfaceVariant,
                modifier = Modifier.size(16.dp)
            )
        }

        // Seek backward
        IconButton(
            onClick = onSeekBackward,
            modifier = Modifier.size(32.dp)
        ) {
            Icon(
                imageVector = Icons.Default.Replay10,
                contentDescription = "后退10秒",
                modifier = Modifier.size(18.dp)
            )
        }

        // Play/Pause
        IconButton(
            onClick = onPlayPause,
            modifier = Modifier.size(36.dp)
        ) {
            Surface(
                shape = androidx.compose.foundation.shape.CircleShape,
                color = MaterialTheme.colorScheme.primary
            ) {
                Icon(
                    imageVector = if (isPlaying) Icons.Default.Pause else Icons.Default.PlayArrow,
                    contentDescription = if (isPlaying) "暂停" else "播放",
                    modifier = Modifier.size(18.dp),
                    tint = MaterialTheme.colorScheme.onPrimary
                )
            }
        }

        // Seek forward
        IconButton(
            onClick = onSeekForward,
            modifier = Modifier.size(32.dp)
        ) {
            Icon(
                imageVector = Icons.Default.Forward10,
                contentDescription = "前进10秒",
                modifier = Modifier.size(18.dp)
            )
        }
    }

```

```

    }

    // Speed
    Box {
        TextButton(
            onClick = { showSpeedMenu = true },
            modifier = Modifier.height(32.dp),
            contentPadding = PaddingValues(horizontal = 4.dp)
        ) {
            Text(
                text = "${playbackSpeed}x",
                style = MaterialTheme.typography.labelSmall,
                fontWeight = FontWeight.Medium
            )
        }
        DropdownMenu(
            expanded = showSpeedMenu,
            onDismissRequest = { showSpeedMenu = false }
        ) {
            speedOptions.forEach { speed ->
                DropdownMenuItem(
                    text = { Text("${speed}x") },
                    onClick = {
                        onSpeedChange(speed)
                        showSpeedMenu = false
                    },
                    leadingIcon = if (speed == playbackSpeed) {
                        { Icon(Icons.Default.Check, contentDescription = null) }
                    } else null
                )
            }
        }
    }

    Text(
        text = formatTime(duration),
        style = MaterialTheme.typography.labelSmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
}

private fun formatTime(ms: Long): String {
    val totalSeconds = ms / 1000
    val minutes = totalSeconds / 60
    val seconds = totalSeconds % 60
    return "%02d:%02d".format(minutes, seconds)
}

@Composable
internal fun VideoPlayerSection(
    exoPlayer: ExoPlayer,
    modifier: Modifier = Modifier
) {
    Box(
        modifier = modifier
        .background(Color.Black)
    ) {
        AndroidView(
            factory = { context ->
                PlayerView(context).apply {

```

```

        player = exoPlayer
        useController = false
    }
},
modifier = Modifier.fillMaxSize()
)
}
}

@Composable
internal fun WordSaveDialog(
    initialWord: String,
    translationState: WordTranslationState,
    onDismiss: () -> Unit,
    onSave: (word: String, translation: String, phonetic: String, pos: String) -> Unit,
    onRetranslate: (word: String) -> Unit,
) {
    // Word is editable so the user can correct inflected forms (plurals,
    // past tense, future "will X", etc.) into the dictionary's base
    // form and re-look up. The `remember(initialWord)` key resets the
    // field when a different long-press opens a fresh dialog.
    var currentWord by remember(initialWord) { mutableStateOf(initialWord) }
    // Local editable translation. We seed it from the Loaded result
    // when it arrives, but the user can still tweak it before saving.
    var translation by remember(initialWord) { mutableStateOf("") }
    LaunchedEffect(translationState) {
        if (translationState is WordTranslationState.Loaded) {
            translation = translationState.translation
        }
    }

    val isLoading = translationState is WordTranslationState.Loading
    val errorMessage = (translationState as? WordTranslationState.Failed)?.message
    // Phonetic + POS come from the local-dictionary hit. Practice
    // flow is local-only; phonetic/pos are non-null on `Loaded`.
    val loaded = translationState as? WordTranslationState.Loaded
    val phonetic = loaded?.phonetic.orEmpty()
    val pos = loaded?.pos.orEmpty()
    val canSave = !isLoading &&
        currentWord.isNotBlank() &&
        translation.isNotBlank()

    AlertDialog(
        onDismissRequest = onDismiss,
        // (2026-07-04) Background switched from default M3
        // `surface` (#FFFBFE — near white) to a clearly-visible
        // light gray (#E0E0E0) per the user's "浅灰色" request.
        //
        // Earlier attempt used `surfaceVariant` (#E7E0EC) but the
        // user reported "跟改之前没有差别" — the two values are
        // only ~6 lightness steps apart and visually
        // indistinguishable. Tried `surfaceContainerHigh` next but
        // that's a M3 1.2.0 token; this project pins material3 to
        // 1.1.2 (verified via `./gradlew :app:dependencies`), so
        // it doesn't exist in the classpath.
        //
        // Hardcoded #E0E0E0 is the simplest fix: clearly readable
        // as "light gray", and the rest of the dialog (violet-400
        // confirm button, dark text on the fields) keeps enough
        // brand purple that it doesn't feel like a foreign screen.
        // Trade-off: dark mode would still render #E0E0E0 (a
        // glaringly bright dialog on a near-black page); the app
        // doesn't appear to ship a dark mode today, so we

```

```

// accept that and revisit if/when dark mode lands.
containerColor = Color(0xFFE0E0E0),
title = {
    Text(
        text = "翻译 & 保存",
        // (2026-07-04) 22sp (M3 titleLarge default) → 20sp
        // (earlier this turn) → 18sp now. The user said the
        // title still reads too heavy for the dialog's
        // light-gray surface; 18sp sits between M3
        // titleLarge and titleMedium and feels closer to a
        // dialog-label than a screen-heading.
        style = MaterialTheme.typography.titleLarge.copy(fontSize = 18.sp)
    )
},
text = {
    Column {
        // Editable word field + "翻译" button. The button
        // re-runs the local dictionary lookup on whatever the
        // user has typed — so they can change "went" → "go",
        // "children" → "child", "will become" → "become", etc.
        Row(
            verticalAlignment = Alignment.CenterVertically,
        ) {
            OutlinedTextField(
                value = currentWord,
                onValueChange = { currentWord = it },
                label = { Text("单词") },
                singleLine = true,
                enabled = !isLoading,
                shape = RoundedCornerShape(12.dp),
                modifier = Modifier.weight(1f),
            )
            Spacer(modifier = Modifier.width(8.dp))
            TextButton(
                onClick = { onRetranslate(currentWord.trim()) },
                enabled = currentWord.isNotBlank() && !isLoading,
            ) {
                Text("翻译")
            }
        }
    }
    if (phonetic.isNotBlank()) {
        Text(
            text = phonetic,
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
            modifier = Modifier.padding(top = 8.dp, start = 4.dp),
        )
    }
    Spacer(modifier = Modifier.height(16.dp))
    OutlinedTextField(
        value = translation,
        onValueChange = { translation = it },
        label = { Text(if (pos.isNotBlank()) "$pos 翻译" else "翻译") },
        placeholder = {
            Text(
                if (isLoading) "正在查询本地词典..."
                else "可编辑后再保存"
            )
        },
        modifier = Modifier.fillMaxWidth(),
        singleLine = true,
        shape = RoundedCornerShape(12.dp),
        enabled = !isLoading,
    )
}

```

```

        trailingIcon = {
            if (isLoading) {
                CircularProgressIndicator(
                    modifier = Modifier.size(20.dp),
                    strokeWidth = 2.dp,
                )
            }
        },
    ),
    if (errorMessage != null) {
        Spacer(modifier = Modifier.height(8.dp))
        Text(
            text = errorMessage,
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.error,
        )
    }
},
confirmButton = {
    Button(
        onClick = { onSave(currentWord.trim(), translation, phonetic, pos) },
        enabled = canSave,
        shape = RoundedCornerShape(12.dp),
        // §12.32d: "保存" button uses the brand primary
        // #7C3AED (violet-600) for unambiguous purple
        // identification. White text keeps contrast on the
        // darker background. Previously used violet-400
        // (#A78BFA, §12.32c) — too light, user could not
        // recognize as "purple" — reverted to brand primary.
        colors = ButtonDefaults.buttonColors(
            containerColor = Color(0xFF7C3AED),
            contentColor = Color.White,
        ),
    ) {
        Text("保存")
    }
},
dismissButton = {
    TextButton(onClick = onDismiss) {
        Text("取消")
    }
},
shape = RoundedCornerShape(24.dp)
)
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\practice\SpeakingPage.kt ===

package com.echoling.app.presentation.ui.screens.practice

import android.Manifest
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.ExperimentalFoundationApi
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.combinedClickable
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.interaction.collectIsPressedAsState
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape

```

```

import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import com.echoling.app.player.subtitle.Subtitle
import com.echoling.app.presentation.ui.screens.practice.components.RedRecordCircle
import com.echoling.app.presentation.viewmodel.PracticeViewModel
import com.echoling.app.presentation.viewmodel.SentenceState
import com.echoling.app.speech.RecordingState

```

```
@Composable
```

```

fun SpeakingPage(
    viewModel: PracticeViewModel,
    modifier: Modifier = Modifier
) {
    val playbackState by viewModel.playbackState.collectAsState()
    val subtitles by viewModel.subtitles.collectAsState()
    val currentSubtitleIndex by viewModel.currentSubtitleIndex.collectAsState()
    val recordingState by viewModel.recordingState.collectAsState()
    val isPlayingRecording by viewModel.isPlayingRecording.collectAsState()
    val sentenceStates by viewModel.sentenceStates.collectAsState()
    // (2026-06-28) Per-page recording path. Collects only the
    // speaking-page path; test-page recordings are invisible to
    // this page so the "我的录音" playback card won't show a
    // test-page recording here.
    val recordingPath by viewModel.speakingRecordingPath.collectAsState()
    val isVideoMode by viewModel.isVideoMode.collectAsState()
    val videoPlayerState by viewModel.videoPlayerState.collectAsState()

    var revealedWords by remember { mutableStateOf<Set<Int>>(emptySet()) }
    var showWordDialog by remember { mutableStateOf(false) }
    var selectedWord by remember { mutableStateOf("") }
    var hasRecordPermission by remember { mutableStateOf(false) }
    var showSentenceMenu by remember { mutableStateOf(false) }

    val translationState by viewModel.wordTranslation.collectAsState()
    val currentSubtitle = subtitles.getOrNull(currentSubtitleIndex)
    val exampleSentence = currentSubtitle?.contentEn.orEmpty()

    val permissionLauncher = rememberLauncherForActivityResult(
        contract = ActivityResultContracts.RequestPermission()
    ) { isGranted ->
        hasRecordPermission = isGranted
        if (isGranted) viewModel.startRecording()
    }

    val currentSub = subtitles.getOrNull(currentSubtitleIndex)
    val currentSentenceState = currentSub?.let { sentenceStates[it.index] }

    Column(modifier = modifier.fillMaxSize()) {
        // Video Player
        if (isVideoMode && videoPlayerState != null) {
            VideoPlayerSection(
                exoPlayer = videoPlayerState!!,
                modifier = Modifier
                    .fillMaxWidth()

```



```

        .aspectRatio(16f / 9f)
    )
}

// Sentence selector dropdown
Box(
    modifier = Modifier
        .fillMaxWidth()
        // (2026-07-04) Bottom padding dropped from 8dp → 0dp
        // so the [SpeakingSubtitleCard] below sits flush
        // against the OutlinedButton. Top padding kept at
        // 8dp to keep the dropdown away from the video
        // player frame above. Combined with the card's
        // own internal 16dp padding, the first row
        // ("句子 N" + completion check) now sits 16dp
        // below the button's bottom edge (was 24dp).
        .padding(start = 16.dp, top = 8.dp, end = 16.dp, bottom = 0.dp)
) {
    OutlinedButton(
        onClick = { showSentenceMenu = true },
        modifier = Modifier.fillMaxWidth()
    ) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text("${currentSubtitleIndex + 1} / ${subtitles.size}")
            Icon(Icons.Default.ArrowDropDown, null)
        }
    }
}

DropdownMenu(
    expanded = showSentenceMenu,
    onDismissRequest = { showSentenceMenu = false },
    // (2026-07-04) Removed the `widthIn(max = 300.dp)`
    // cap that fit the previous 5-column grid. With
    // 8 × 36dp cells + 7 × 4dp gaps = 316dp the row
    // already fits inside the full-width anchor (≈
    // screen-width minus 32dp Box padding) on any
    // modern phone. Forcing 300dp here would clip the
    // rightmost column on phones narrower than 350dp.
) {
    // (2026-07-04) Layout reshaped per user request:
    // - columns 5 → 8 (more sentence numbers per row)
    // - cell size 48dp → 36dp (so 8 cells × 36dp +
    //   7 × 4dp gaps = 316dp fits within the menu)
    // - completed style: previously "faded lavender
    //   secondaryContainer + green text + corner check
    //   badge"; now "solid green + white text, no
    //   badge". The solid green + white is the
    //   strongest readability signal on its own — the
    //   check badge cluttered the smaller 36dp cells
    //   and made completed numbers look noisier than
    //   the rest. Selected (current subtitle) styling
    //   unchanged: primaryContainer + onPrimaryContainer
    //   — keeping primary purple for selection so
    //   "completed" (green) and "selected"
    //   (purple) read as two independent states.
    val columns = 8
    val cellSize = 36.dp
    val rows = (subtitles.size + columns - 1) / columns
    for (row in 0 until rows) {
        Row(

```

```

        horizontalArrangement = Arrangement.spacedBy(4. dp),
        modifier = Modifier.padding(vertical = 4. dp)
    ) {
        for (col in 0 until columns) {
            val index = row * columns + col
            if (index < subtitles.size) {
                val state = sentenceStates[subtitles[index].index]
                val isCompleted = state?.isCompleted == true
                val isSelected = index == currentSubtitleIndex

                Surface(
                    modifier = Modifier
                        .size(cellSize)
                        .clickable {
                            viewModel.skipToSubtitle(index)
                            revealedWords = emptySet()
                            showSentenceMenu = false
                        },
                    shape = RoundedCornerShape(8. dp),
                    color = when {
                        isSelected -> MaterialTheme.colorScheme.primaryContainer
                        isCompleted -> Color(0xFF4CAF50)
                        else -> Color.Transparent
                    }
                ) {
                    Box(contentAlignment = Alignment.Center) {
                        Text(
                            text = "${index + 1}",
                            style = MaterialTheme.typography.bodyMedium,
                            color = when {
                                isSelected -> MaterialTheme.colorScheme.onPrimaryContainer
                                isCompleted -> Color.White
                                else -> MaterialTheme.colorScheme.onSurface
                            }
                        )
                    }
                }
            }
        }
    }
}

// Current sentence card
if (currentSub != null) {
    SpeakingSubtitleCard(
        subtitle = currentSub,
        displayIndex = currentSubtitleIndex,
        isCompleted = currentSentenceState?.isCompleted == true,
        revealedWords = revealedWords,
        onWordReveal = { wordIdx ->
            revealedWords = if (revealedWords.contains(wordIdx)) {
                revealedWords - wordIdx
            } else {
                revealedWords + wordIdx
            }
        },
        onWordLongClick = { word ->
            selectedWord = word
            showWordDialog = true
            viewModel.requestWordTranslation(word)
        },
    )
}

```

```

        onMarkCompleted = { completed ->
            viewModel.markSentenceCompleted(currentSub.index, completed)
        },
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp)
    )
}

Box(
    modifier = Modifier
        .weight(1f)
        .fillMaxWidth(),
    contentAlignment = Alignment.Center
) {
    if (recordingState == RecordingState.RECORDING) {
        RedRecordCircle()
    }
}

// Recording playback card (if recording exists)
recordingPath?.let {
    // (2026-06-28) Pass speakingRecordingPath.value so the
    // ViewModel can't accidentally play a test-page
    // recording here (each page's playback is bound to its
    // own path).
    val currentSpeakingPath = recordingPath
    RecordingPlaybackCard(
        isPlaying = isPlayingRecording,
        onPlayRecording = { viewModel.playRecording(currentSpeakingPath) },
        onStopPlaying = { viewModel.stopPlayingRecording() },
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp)
    )
}

// Bottom controls: Prev, Play, Record, Playback, Next
SpeakingControlBar(
    recordingState = recordingState,
    hasRecordPermission = hasRecordPermission,
    permissionLauncher = permissionLauncher,
    onSkipPrevious = {
        revealedWords = emptySet()
        viewModel.skipToPreviousSubtitle()
    },
    onSkipNext = {
        revealedWords = emptySet()
        viewModel.skipToNextSubtitle()
    },
    onPlayPause = {
        if (playbackState.isPlaying) {
            viewModel.pause()
        } else {
            currentSub?.let { viewModel.playSubtitleOnce(it) }
        }
    },
    viewModel = viewModel
)
}

if (showWordDialog) {
    WordSaveDialog(

```

```

        initialWord = selectedWord,
        translationState = translationState,
        onDismiss = {
            showWordDialog = false
            viewModel.clearWordTranslation()
        },
        onRetranslate = { newWord ->
            // Re-runs the local dictionary lookup on the
            // (possibly user-edited) word. The dialog will update
            // with the new translation when state emits Loaded.
            viewModel.requestWordTranslation(newWord)
        },
        onSave = { word, translation, phonetic, pos ->
            viewModel.saveWord(word, translation, exampleSentence, phonetic, pos)
            showWordDialog = false
            viewModel.clearWordTranslation()
        }
    )
}
}
}

@Composable
private fun SpeakingSubtitleCard(
    subtitle: Subtitle,
    displayIndex: Int,
    isCompleted: Boolean,
    revealedWords: Set<Int>,
    onWordReveal: (Int) -> Unit,
    onWordLongClick: (String) -> Unit,
    onMarkCompleted: (Boolean) -> Unit,
    modifier: Modifier = Modifier
) {
    val words = subtitle.contentEn.split(Regex("\\s+")).filter { it.isNotEmpty() }
    // Calculate max word length for uniform block width
    val maxWordLength = words.maxOfOrNull { it.replace(Regex("[^\\w']"), "").length } ?: 0

    Card(
        modifier = modifier,
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = if (isCompleted)
                MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.3f)
            else
                MaterialTheme.colorScheme.surface
        )
    ) {
        Column(
            modifier = Modifier.padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Text(
                        "句子 ${displayIndex + 1}",
                        style = MaterialTheme.typography.labelMedium,
                        color = MaterialTheme.colorScheme.primary
                    )
                    if (!isCompleted) {
                        Spacer(modifier = Modifier.width(8.dp))
                    }
                }
            }
        }
    }
}

```

```

        Surface(
            shape = RoundedCornerShape(4.dp),
            color = MaterialTheme.colorScheme.errorContainer.copy(alpha = 0.5f)
        ) {
            Text(
                "未完成",
                style = MaterialTheme.typography.labelSmall,
                color = MaterialTheme.colorScheme.onErrorContainer,
                modifier = Modifier.padding(horizontal = 6.dp, vertical = 2.dp)
            )
        }
    }

    // Completion toggle
    IconButton(onClick = { onMarkCompleted(!isCompleted) }) {
        Icon(
            imageVector = if (isCompleted) Icons.Default.CheckCircle else Icons.Default.RadioButtonUnchecked,
            contentDescription = if (isCompleted) "取消完成" else "标记完成",
            tint = if (isCompleted)
                MaterialTheme.colorScheme.primary
            else
                MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}

Spacer(Modifier.height(12.dp))

// Words row - wrap words into multiple lines
SpeakingWordsFlowRow(
    words = words,
    maxWordLength = maxWordLength,
    revealedWords = revealedWords,
    onWordClick = onWordReveal,
    onWordLongClick = onWordLongClick,
    modifier = Modifier.fillMaxWidth()
)

// Chinese translation
if (subtitle.contentCn.isNotEmpty()) {
    Spacer(modifier = Modifier.height(8.dp))
    Text(
        text = subtitle.contentCn,
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
}
}
}

@OptIn(ExperimentalFoundationApi::class)
@Composable
private fun SpeakingWordsFlowRow(
    words: List<String>,
    maxWordLength: Int,
    revealedWords: Set<Int>,
    onWordClick: (Int) -> Unit,
    onWordLongClick: (String) -> Unit,
    modifier: Modifier = Modifier
) {

```

```

// Pre-calculate which words fit on each line
val lines = remember(words, maxWordLength) {
    val result = mutableListOf<List<Pair<Int, String>>>()
    var currentLine = mutableListOf<Pair<Int, String>>()
    var currentLineCharCount = 0
    val maxCharsPerLine = 25 // Approximate characters per line

    words.forEachIndexed { index, word ->
        val cleanWord = word.replace(Regex("[^\\w']"), "")
        if (cleanWord.isNotEmpty()) {
            if (currentLineCharCount + cleanWord.length > maxCharsPerLine && currentLine.isNotEmpty()) {
                result.add(currentLine.toList())
                currentLine = mutableListOf()
                currentLineCharCount = 0
            }
            currentLine.add(index to cleanWord)
            currentLineCharCount += cleanWord.length + 1 // +1 for space
        }
    }
    if (currentLine.isNotEmpty()) {
        result.add(currentLine.toList())
    }
    result
}

Column(
    modifier = modifier,
    horizontalAlignment = Alignment.Start
) {
    lines.forEach { lineWords ->
        Row(
            horizontalArrangement = Arrangement.Start,
            verticalAlignment = Alignment.CenterVertically,
            modifier = Modifier.fillMaxWidth()
        ) {
            lineWords.forEach { (index, cleanWord) ->
                val isRevealed = revealedWords.contains(index)

                if (isRevealed) {
                    // Visible word — tap is a no-op, long-press opens
                    // the translation dialog. Padded Box for a more
                    // forgiving long-press hit target.
                    Box(
                        modifier = Modifier
                            .clip(RoundedCornerShape(4.dp))
                            .combinedClickable(
                                onClick = { },
                                onLongClick = { onWordLongClick(cleanWord) },
                            )
                        .padding(horizontal = 2.dp, vertical = 2.dp),
                        contentAlignment = Alignment.Center,
                    ) {
                        Text(
                            text = cleanWord,
                            style = MaterialTheme.typography.bodyMedium,
                            color = MaterialTheme.colorScheme.onSurface,
                        )
                    }
                } else {
                    // Hidden word block — invisible text determines natural size, avoiding descender clipping
                    Box(
                        modifier = Modifier
                            .clip(RoundedCornerShape(4.dp))

```

```

        .background(Color(0xFFE0E0E0))
        .clickable { onWordClick(index) }
        .padding(horizontal = 6.dp, vertical = 2.dp),
        contentAlignment = Alignment.Center
    ) {
        Text(
            text = cleanWord,
            style = MaterialTheme.typography.bodyMedium,
            color = Color.Transparent
        )
    }
    Spacer(modifier = Modifier.width(4.dp))
}
}
}
}
}

@Composable
private fun SpeakingControlBar(
    recordingState: RecordingState,
    hasRecordPermission: Boolean,
    permissionLauncher: androidx.activity.result.ActivityResultLauncher<String>,
    onSkipPrevious: () -> Unit,
    onSkipNext: () -> Unit,
    onPlayPause: () -> Unit,
    viewModel: PracticeViewModel
) {
    val interactionSource = remember { MutableInteractionSource() }
    val isPressed by interactionSource.collectIsPressedAsState()

    LaunchedEffect(isPressed) {
        if (isPressed) {
            if (hasRecordPermission) {
                viewModel.startRecording()
            } else {
                permissionLauncher.launch(Manifest.permission.RECORD_AUDIO)
            }
        } else {
            if (recordingState == RecordingState.RECORDING) {
                viewModel.stopRecording()
            }
        }
    }

    val isPlayingRecording by viewModel.isPlayingRecording.collectAsState()
    val playbackState by viewModel.playbackState.collectAsState()

    // Pill-shaped background frame matching ListeningPage's bottom bar style.
    // Gives the 5-button action bar a clear visual anchor at the bottom of
    // the practice screen without overpowering the recording button.
    Surface(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp),
        shape = RoundedCornerShape(24.dp),
        color = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f),
        tonalElevation = 2.dp,
    ) {
        Row(
            modifier = Modifier

```

```

        .fillMaxWidth()
        .padding(vertical = 12.dp),
horizontalArrangement = Arrangement.SpaceEvenly,
verticalAlignment = Alignment.CenterVertically
) {
    // Previous sentence
    IconButton(onClick = onSkipPrevious) {
        Surface(shape = CircleShape, color = MaterialTheme.colorScheme.secondaryContainer) {
            Icon(
                Icons.Default.SkipPrevious, "上一句",
                modifier = Modifier.size(26.dp),
                tint = MaterialTheme.colorScheme.onSecondaryContainer
            )
        }
    }

    // Play current sentence
    IconButton(onClick = onPlayPause) {
        Surface(shape = CircleShape, color = MaterialTheme.colorScheme.primary) {
            Icon(
                if (playbackState.isPlaying) Icons.Default.Pause else Icons.Default.PlayArrow,
                "播放",
                modifier = Modifier.size(28.dp),
                tint = MaterialTheme.colorScheme.onPrimary
            )
        }
    }

    // Recording button (press and hold)
    Box(
        modifier = Modifier
            .size(54.dp)
            .clickable(
                interactionSource = interactionSource,
                indication = null,
                onClick = { }
            )
    ) {
        Surface(
            shape = CircleShape,
            color = if (recordingState == RecordingState.RECORDING) Color.Red
                else MaterialTheme.colorScheme.secondaryContainer,
            modifier = Modifier.fillMaxSize()
        ) {
            Icon(
                if (recordingState == RecordingState.RECORDING) Icons.Default.Stop else Icons.Default.Mic,
                "录音",
                modifier = Modifier.size(26.dp),
                tint = if (recordingState == RecordingState.RECORDING) Color.White
                    else MaterialTheme.colorScheme.onSecondaryContainer
            )
        }
    }

    // Play recording
    // (2026-06-28) Reads the latest speakingRecordingPath
    // from the StateFlow at click time and passes it
    // explicitly to playRecording. The button is only
    // enabled when speakingRecordingPath != null, but
    // capturing the value at click time is more robust
    // than relying on a closure-captured `recordingPath`
    // variable (which could in theory be stale if the
    // StateFlow re-emitted between render and click).

```



```

        IconButton(
            onClick = {
                if (isPlayingRecording) {
                    viewModel.stopPlayingRecording()
                } else {
                    viewModel.playRecording(viewModel.speakingRecordingPath.value)
                }
            }
        ) {
            Surface(
                shape = CircleShape,
                color = if (isPlayingRecording) MaterialTheme.colorScheme.primary
                    else MaterialTheme.colorScheme.primaryContainer
            ) {
                Icon(
                    if (isPlayingRecording) Icons.Default.Stop else Icons.Default.PlayArrow,
                    if (isPlayingRecording) "停止" else "播放录音",
                    modifier = Modifier.size(26.dp),
                    tint = if (isPlayingRecording) Color.White
                        else MaterialTheme.colorScheme.onPrimaryContainer
                )
            }
        }
    }

    // Next sentence
    IconButton(onClick = onSkipNext) {
        Surface(shape = CircleShape, color = MaterialTheme.colorScheme.secondaryContainer) {
            Icon(
                Icons.Default.SkipNext, "下一句",
                modifier = Modifier.size(26.dp),
                tint = MaterialTheme.colorScheme.onSecondaryContainer
            )
        }
    }
}

@Composable
private fun RecordingPlaybackCard(
    isPlaying: Boolean,
    onPlayRecording: () -> Unit,
    onStopPlaying: () -> Unit,
    modifier: Modifier = Modifier
) {
    Card(
        modifier = modifier,
        shape = RoundedCornerShape(12.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.secondaryContainer.copy(alpha = 0.5f)
        )
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text(
                    "我的录音",
                    style = MaterialTheme.typography.labelMedium,

```



```

// user has only a speaking-page recording.
val recordingPath by viewModel.testRecordingPath.collectAsState()
val isPlayingRecording by viewModel.isPlayingRecording.collectAsState()
val modelDownloadState by viewModel.modelDownloadState.collectAsState()

// RECORD_AUDIO permission for the WavRecorder. The previous version
// of TestingPage skipped this and called viewModel.startStt() directly
// — which on a freshly-installed app (no prior SpeakingPage visit)
// would silently fail at the WavRecorder / MediaRecorder level. Mirrors
// SpeakingPage.kt:48-60.
var hasRecordPermission by remember { mutableStateOf(false) }
val permissionLauncher = rememberLauncherForActivityResult(
    contract = ActivityResultContracts.RequestPermission()
) { isGranted ->
    hasRecordPermission = isGranted
    if (isGranted) viewModel.startStt()
}

val currentTest = testState.testItems.getOrNull(testState.currentTestIndex)
val currentRealIndex = subtitles.indexOf(currentTest)
val currentSentenceState = currentRealIndex.takeIf { it >= 0 }?.let {
    sentenceStates[subtitles[it].index]
}

Column(modifier = modifier.fillMaxSize()) {
    // Test progress header
    TestingProgressHeader(
        testedCount = testState.testedCount,
        // (2026-06-28) Pass currentTestIndex so the X/Y text
        // reflects the current test position (1-indexed) and
        // visibly changes when the user taps 上一题/下一题.
        // The X is the *current* question number, not the count
        // of completed ones — see TestingProgressHeader body.
        currentTestIndex = testState.currentTestIndex,
        totalCount = if (testState.isActive) testState.testItems.size else subtitles.size,
        isActive = testState.isActive,
        onStartTest = { viewModel.startTestMode() },
        // (2026-06-28) "重新测试" button on the top-right. Wires
        // to viewModel.restartTest() which un-marks all test
        // items and re-initializes the test from X=1. Per user
        // spec: "用户也可以再进行新一轮测试, 所有需要增加一个
        // 按钮 重新测试/刷新图标, 放在测试页面的右上角".
        onRestartTest = { viewModel.restartTest() }
    )

    if (!testState.isActive || testState.testItems.isEmpty()) {
        // Empty state - prompt to start test
        TestingEmptyState(
            onStartTest = { viewModel.startTestMode() },
            modifier = Modifier.weight(1f)
        )
    } else {
        // Test item card
        currentTest?.let { sub ->
            TestingSubtitleCard(
                subtitle = sub,
                revealedWords = testState.revealedWords,
                isTested = currentSentenceState?.isTested == true,
                onWordReveal = { viewModel.revealTestWord(it) },
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(horizontal = 16.dp)
            )
        }
    }
}

```

```

    }

    // —— STT overlay: show based on sttTestState, anchored above
    // the control bar. Only show for the current test item.
    when (val s = sttTestState) {
        is SttTestState.Idle -> { /* nothing */ }
        is SttTestState.Listening -> {
            RecordingOverlay(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(horizontal = 16.dp, vertical = 8.dp)
            )
        }
        is SttTestState.Transcribing -> {
            // Vosk is processing the recorded WAV file. We can't
            // show the editor yet (the text isn't ready) and we
            // can't let the user press the mic again (would
            // cancel the in-flight transcribe). Show a small
            // "正在识别" card with a progress indicator. Also
            // surfaces a first-run "下载模型中" banner if the
            // model is being downloaded right now.
            TranscribingCard(
                downloadState = modelDownloadState,
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(horizontal = 16.dp, vertical = 8.dp)
            )
        }
        is SttTestState.Transcribed -> {
            TranscriptionEditor(
                initialText = s.text,
                onTextChange = { /* editor manages internally */ },
                onSubmit = { text -> viewModel.submitTranscription(text) },
                onRerecord = { viewModel.resetStt() },
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(horizontal = 16.dp, vertical = 8.dp)
            )
        }
        is SttTestState.Passed -> {
            TestResultCard(
                state = s,
                originalEn = currentTest?.contentEn.orEmpty(),
                // (2026-06-28) Removed the explicit
                // markSentenceTested() call here — the
                // ViewModel.submitTranscription() method
                // already auto-marks the sentence when the
                // STT result passes (see
                // PracticeViewModel.submitTranscription,
                // where it calls
                // `markSentenceTested(currentTest.index, true)`
                // before flipping _sttTestState to Passed).
                // Calling it again from the UI would be
                // redundant (markSentenceTested is idempotent
                // — not a bug, just dead code).
                //
                // (2026-07-03) Also removed the manual
                // "mark tested" checkmark button — 下一题
                // is always free, "tested" is just a display
                // flag surfaced on the sentence grid. The
                // checkmark was redundant with the auto-mark
                // path above and gave the user a way to mark
                // a sentence tested without actually testing
            )
        }
    }

```

```

        // it, which defeats the practice flow.
        onNextItem = {
            // (2026-07-05) Route through
            // advanceToNextTestItem so the result card
            // is cleared at the same time the index
            // advances — see PracticeViewModel for
            // why the explicit resetStt() used to be
            // required on this callsite only.
            viewModel.advanceToNextTestItem()
        },
        onRerecord = { viewModel.resetStt() },
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp)
    )
}
is SttTestState.Failed -> {
    TestResultCard(
        state = s,
        originalEn = s.original,
        onNextItem = {
            // (2026-07-05) Same fix as Passed's
            // onNextItem — clear STT state so the
            // Failed card disappears when the user
            // moves on. Previously the old explicit
            // nextTestItem() + resetStt() pair had
            // the same risk of being desynced from
            // the ControlBar's button.
            viewModel.advanceToNextTestItem()
        },
        onRerecord = { viewModel.resetStt() },
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp)
    )
}
}

Spacer(Modifier.weight(1f))

// Test control bar
TestingControlBar(
    currentIndex = testState.currentTestIndex,
    isSttListening = sttTestState is SttTestState.Listening,
    hasRecording = recordingPath != null,
    isPlayingRecording = isPlayingRecording,
    onPrevious = { viewModel.previousTestItem() },
    // (2026-07-03) Removed onMarkTested entirely. A sentence
    // is now marked as tested only by submitting a
    // transcription that passes the STT matcher — see
    // PracticeViewModel.submitTranscription which calls
    // markSentenceTested() on pass. The checkmark button is
    // gone from the control bar; the 5th slot is now a
    // plain 下一题 button that always advances.
    onNextItem = { viewModel.advanceToNextTestItem() },
    onPlayAudio = {
        currentTest?.let { viewModel.playSubtitleOnce(it) }
    },
    onPressMic = {
        // Gate on RECORD_AUDIO: without it, SpeechRecognizer
        // returns ERROR_INSUFFICIENT_PERMISSIONS (code 9) and
        // the result is empty ("未识别到语音"). Asking only on
        // press (not on every render) keeps the prompt

```

```

        // dismissible — same pattern as SpeakingPage.
        if (hasRecordPermission) viewModel.startStt()
        else permissionLauncher.launch(Manifest.permission.RECORD_AUDIO)
    },
    onReleaseMic = { viewModel.stopStt() },
    // (2026-06-28) Pass testRecordingPath.value
    // explicitly so the 回放 button can only ever
    // play test-page recordings. The button is
    // already disabled by hasRecording = (test page
    // path != null), so the click is impossible when
    // there's no test recording — this is the
    // belt-and-suspenders that the correct path
    // is used.
    onPlayRecording = { viewModel.playRecording(viewModel.testRecordingPath.value) },
)
    }
}
}
}

```

@Composable

```

private fun TestingProgressHeader(
    testedCount: Int,
    // (2026-06-28) Added currentTestIndex so the X/Y display can
    // show the current test position (1-indexed) — the user's
    // mental model of "X/Y" is "you are on question X of Y", which
    // must increment/decrement in lock-step with the 上一题/下一题
    // buttons. The old display used `testedCount` here, which is
    // a different semantic ("X out of Y are completed") and
    // doesn't change when the user navigates back via 上一题.
    currentTestIndex: Int,
    totalCount: Int,
    isTestActive: Boolean,
    onStartTest: () -> Unit,
    // (2026-06-28) Replaces the old placeholder `onResetTest`. Wired
    // to viewModel.restartTest() which un-marks all test items and
    // re-initializes the test from X=1, so the user can start a new
    // round of testing at any point while a test is active.
    onRestartTest: () -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp)
    ) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text(
                    "测试",
                    style = MaterialTheme.typography.titleMedium
                )
                Text(
                    // (2026-06-28) X is now the *current test
                    // position* (1-indexed: `currentTestIndex + 1`),
                    // not the tested count. The user's mental model
                    // of "X / Y" matches what the 上一题/下一题
                    // buttons do — X must visibly decrement when
                    // tapping 上一题 and increment when tapping
                    // 下一题. testedCount is a different concept
                    // (how many sentences in the pool have been

```

```

        // marked isTested=true) and only changes when
        // the user marks a sentence as tested; it can
        // drift up independently of navigation (e.g.
        // marking the same question twice from the
        // previous-page card).
        //
        // Clamped to totalCount in the (theoretical)
        // case where currentTestIndex exceeds the test
        // pool size after a re-init.
        if (totalCount > 0)
            "第 ${currentTestIndex + 1}.coerceAtMost(totalCount)} / $totalCount 题"
        else
            "点击开始测试",
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
}

// (2026-06-28) Top-right action: 开始测试 (when not
// active) or 重新测试 (when active). The two states
// occupy the same horizontal slot to keep the header's
// layout stable as the user toggles between "not
// started" and "in progress".
if (!isTestActive) {
    Button(
        onClick = onStartTest,
        shape = RoundedCornerShape(12.dp)
    ) {
        Icon(
            Icons.Default.PlayArrow,
            null,
            modifier = Modifier.size(18.dp)
        )
        Spacer(Modifier.width(4.dp))
        Text("开始测试")
    }
} else {
    // "重新测试" — outlined button with refresh icon.
    // Outlined (not filled) to keep it visually less
    // prominent than the original 开始测试 filled button
    // — a user mid-test shouldn't feel like the app is
    // pushing them to restart. The icon (Refresh) is
    // the standard "restart" affordance; the text label
    // makes the action unambiguous.
    OutlinedButton(
        onClick = onRestartTest,
        shape = RoundedCornerShape(12.dp)
    ) {
        Icon(
            Icons.Default.Refresh,
            null,
            modifier = Modifier.size(18.dp)
        )
        Spacer(Modifier.width(4.dp))
        Text("重新测试")
    }
}
}

// Progress bar
if (totalCount > 0) {
    Spacer(Modifier.height(12.dp))
    LinearProgressIndicator(

```

```

        progress = testedCount.toFloat() / totalCount.coerceAtLeast(1),
        modifier = Modifier
            .fillMaxWidth()
            .height(4.dp),
        color = MaterialTheme.colorScheme.primary,
        trackColor = MaterialTheme.colorScheme.surfaceVariant,
    )
}
}

@Composable
private fun TestingEmptyState(
    onStartTest: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxWidth(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Icon(
            Icons.Default.Quiz,
            contentDescription = null,
            modifier = Modifier.size(64.dp),
            tint = MaterialTheme.colorScheme.onSurfaceVariant
        )
        Spacer(Modifier.height(16.dp))
        Text(
            "点击开始测试检验学习效果",
            style = MaterialTheme.typography.bodyLarge,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
        Spacer(Modifier.height(24.dp))
        Button(
            onClick = onStartTest,
            shape = RoundedCornerShape(12.dp)
        ) {
            Icon(Icons.Default.PlayArrow, null)
            Spacer(Modifier.width(8.dp))
            Text("开始测试")
        }
    }
}

@Composable
private fun TestingSubtitleCard(
    subtitle: com.echoling.app.player.subtitle.Subtitle,
    revealedWords: Set<Int>,
    isTested: Boolean,
    onWordReveal: (Int) -> Unit,
    modifier: Modifier = Modifier
) {
    val words = subtitle.contentEn.split(Regex("\\s+")).filter { it.isNotEmpty() }

    Card(
        modifier = modifier,
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = if (isTested)
                MaterialTheme.colorScheme.secondaryContainer.copy(alpha = 0.3f)
            else
                MaterialTheme.colorScheme.surface
    )

```



```

    )
  ) {
    Column(
      modifier = Modifier.padding(16.dp),
      horizontalAlignment = Alignment.Start
    ) {
      Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
      ) {
        Text(
          "句子 ${subtitle.index + 1}",
          style = MaterialTheme.typography.labelMedium,
          color = MaterialTheme.colorScheme.primary
        )

        if (isTested) {
          Row(
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(4.dp)
          ) {
            Icon(
              Icons.Default.CheckCircle,
              null,
              modifier = Modifier.size(16.dp),
              tint = MaterialTheme.colorScheme.primary
            )
            Text(
              "已测试",
              style = MaterialTheme.typography.bodySmall,
              color = MaterialTheme.colorScheme.primary
            )
          }
        }
      }
    }

    Spacer(Modifier.height(12.dp))

    // Test mode: words displayed horizontally, left-aligned, wrap to new lines
    TestingWordsFlowRow(
      words = words,
      revealedWords = revealedWords,
      onWordClick = onWordReveal,
      modifier = Modifier.fillMaxWidth()
    )

    // Show Chinese translation hint when all revealed
    if (revealedWords.size == words.size && subtitle.contentCn.isNotEmpty()) {
      Spacer(Modifier.height(8.dp))
      Text(
        text = subtitle.contentCn,
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant
      )
    }
  }
}

@Composable
private fun TestingWordsFlowRow(
  words: List<String>,

```

```

        revealedWords: Set<Int>,
        onWordClick: (Int) -> Unit,
        modifier: Modifier = Modifier
    ) {
        // Pre-calculate which words fit on each line
        val lines = remember(words) {
            val result = mutableListOf<List<Pair<Int, String>>>()
            var currentLine = mutableListOf<Pair<Int, String>>()
            var currentLineCharCount = 0
            val maxCharsPerLine = 25 // Approximate characters per line

            words.forEachIndexed { index, word ->
                val cleanWord = word.replace(Regex("[^\\w']"), "")
                if (cleanWord.isNotEmpty()) {
                    if (currentLineCharCount + cleanWord.length > maxCharsPerLine && currentLine.isNotEmpty()) {
                        result.add(currentLine.toList())
                        currentLine = mutableListOf()
                        currentLineCharCount = 0
                    }
                    currentLine.add(index to cleanWord)
                    currentLineCharCount += cleanWord.length + 1 // +1 for space
                }
            }
            if (currentLine.isNotEmpty()) {
                result.add(currentLine.toList())
            }
            result
        }

        Column(
            modifier = modifier,
            horizontalAlignment = Alignment.Start
        ) {
            lines.forEach { lineWords ->
                Row(
                    horizontalArrangement = Arrangement.Start,
                    verticalAlignment = Alignment.CenterVertically,
                    modifier = Modifier.fillMaxWidth()
                ) {
                    lineWords.forEach { (index, cleanWord) ->
                        val isRevealed = revealedWords.contains(index)

                        if (isRevealed) {
                            Text(
                                text = cleanWord,
                                style = MaterialTheme.typography.bodyLarge,
                                color = MaterialTheme.colorScheme.onSurface,
                                modifier = Modifier.padding(vertical = 2. dp)
                            )
                        } else {
                            // Hidden word block with fixed height matching word length width
                            Box(
                                modifier = Modifier
                                    .height(28. dp)
                                    .clip(RoundedCornerShape(4. dp))
                                    .background(Color(0xFFE0E0E0))
                                    .clickable { onWordClick(index) }
                                    .padding(horizontal = 6. dp),
                                contentAlignment = Alignment.Center
                            ) {
                                Text(
                                    text = cleanWord,
                                    style = MaterialTheme.typography.bodyLarge,

```

```

        color = Color.Transparent
    )
    }
    }
    Spacer(modifier = Modifier.width(4.dp))
}
}
Spacer(modifier = Modifier.height(4.dp))
}
}

@Composable
private fun TestingControlBar(
    currentIndex: Int,
    isSttListening: Boolean,
    hasRecording: Boolean,
    isPlayingRecording: Boolean,
    onPrevious: () -> Unit,
    // (2026-07-03) Removed the old onMarkTested parameter. A sentence
    // is now marked tested only via submitTranscription() passing the
    // STT matcher — there is no manual "mark complete" path anymore,
    // per user request "勾 去掉". This bar's 5th slot is just 下一题
    // and always advances.
    onNextItem: () -> Unit,
    onPlayAudio: () -> Unit,
    onPressMic: () -> Unit,
    onReleaseMic: () -> Unit,
    onPlayRecording: () -> Unit,
) {
    // Pill-shaped background frame matching the other practice pages'
    // bottom bars. Houses 4 large action buttons (previous / play
    // audio / mic / play recording / 下一题) with consistent
    // visual depth.
    Surface(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp),
        shape = RoundedCornerShape(24.dp),
        color = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f),
        tonalElevation = 2.dp,
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(vertical = 12.dp),
            horizontalArrangement = Arrangement.SpaceEvenly,
            verticalAlignment = Alignment.CenterVertically
        ) {
            // 1. Previous
            IconButton(
                onClick = onPrevious,
                enabled = currentIndex > 0
            ) {
                Surface(
                    shape = CircleShape,
                    color = if (currentIndex > 0)
                        MaterialTheme.colorScheme.secondaryContainer
                    else
                        MaterialTheme.colorScheme.surfaceVariant,
                    modifier = Modifier.size(54.dp)
                ) {
                    Icon(

```

```

        Icons.Default.SkipPrevious, "上一题",
        modifier = Modifier.size(26.dp),
        tint = if (currentIndex > 0)
            MaterialTheme.colorScheme.onSecondaryContainer
        else
            MaterialTheme.colorScheme.onSurfaceVariant
    )
}
}

// 2. Play audio (the original sentence's TTS)
IconButton(onClick = onPlayAudio) {
    Surface(
        shape = CircleShape,
        color = MaterialTheme.colorScheme.secondaryContainer,
        modifier = Modifier.size(54.dp)
    ) {
        Icon(
            Icons.Default.VolumeUp, "播放音频",
            modifier = Modifier.size(26.dp),
            tint = MaterialTheme.colorScheme.onSecondaryContainer
        )
    }
}

// 3. Microphone (press and hold to record, release to stop).
// Restored to the original look per user spec:
// - no ripple rings (the in-overlay sound waves now carry
//   the active-feedback signal)
// - no scale-up animation
// - button is bigger (72dp) for a more confident tap target
// - color flips to red on press
MicButton(
    isListening = isSttListening,
    onPress = onPressMic,
    onRelease = onReleaseMic,
)

// 4. Play back the user's just-recorded audio. Disabled
// until the user has finished at least one recording in
// the current STT session; the icon swaps to Stop while
// audio is playing.
IconButton(
    onClick = onPlayRecording,
    enabled = hasRecording
) {
    Surface(
        shape = CircleShape,
        color = if (hasRecording)
            MaterialTheme.colorScheme.secondaryContainer
        else
            MaterialTheme.colorScheme.surfaceVariant,
        modifier = Modifier.size(54.dp)
    ) {
        Icon(
            if (isPlayingRecording) Icons.Default.Stop else Icons.Default.PlayArrow,
            if (isPlayingRecording) "停止回放" else "回放录音",
            modifier = Modifier.size(26.dp),
            tint = if (hasRecording)
                MaterialTheme.colorScheme.onSecondaryContainer
            else
                MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}

```

```

    }
}

// 5. Next — always advances; the mark-tested checkmark
// branch was removed per user spec (2026-07-03). A
// sentence is now marked tested solely by
// submitTranscription() passing the STT matcher.
IconButton(onClick = onNextItem) {
    Surface(
        shape = CircleShape,
        color = MaterialTheme.colorScheme.primary,
        modifier = Modifier.size(54.dp)
    ) {
        Icon(
            Icons.Default.SkipNext,
            "下一题",
            modifier = Modifier.size(26.dp),
            tint = MaterialTheme.colorScheme.onPrimary
        )
    }
}
}
}
}

/**
 * Press-and-hold mic button. While the user holds:
 * - button color flips from primaryContainer to error (red)
 * - NO ripple / scale animation (those have been moved into the
 *   [RecordingOverlay] which now carries the active-feedback signal
 *   on its own, so the bar around the mic stays calm and the
 *   overlay is the focal point)
 *
 * Sized at 72dp (up from the previous 54dp) per user request — the
 * button now reads as a more confident primary action.
 */
@Composable
private fun MicButton(
    isListening: Boolean,
    onPress: () -> Unit,
    onRelease: () -> Unit,
) {
    Box(
        modifier = Modifier
            .size(72.dp)
            .pointerInput(Unit) {
                detectTapGestures(
                    onPress = {
                        onPress()
                        tryAwaitRelease()
                        onRelease()
                    }
                )
            },
        contentAlignment = Alignment.Center,
    ) {
        Surface(
            shape = CircleShape,
            color = if (isListening) MaterialTheme.colorScheme.error
                else MaterialTheme.colorScheme.primaryContainer,
            modifier = Modifier.fillMaxSize()
        ) {
            Box(contentAlignment = Alignment.Center) {

```

```

        Icon(
            Icons.Default.Mic,
            contentDescription = if (isListening) "正在录音, 松开结束" else "按住录音",
            modifier = Modifier.size(34.dp),
            tint = if (isListening)
                MaterialTheme.colorScheme.onError
            else
                MaterialTheme.colorScheme.onPrimaryContainer
        )
    }
}

/**
 * "正在识别" card shown briefly between releasing the mic and the
 * editor appearing. Vosk runs on the recorded WAV file — usually
 * 100–500ms on a warm device, but can be many seconds on first use
 * while the ~40MB acoustic model is being downloaded.
 */
@Composable
private fun TranscribingCard(
    downloadState: com.echoling.app.speech.ModelManager.DownloadState,
    modifier: Modifier = Modifier,
) {
    androidx.compose.material3.Card(
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surface.copy(alpha = 0.92f)
        ),
        modifier = modifier
    ) {
        Column(
            Modifier.padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            androidx.compose.material3.CircularProgressIndicator(
                modifier = Modifier.size(40.dp),
                strokeWidth = 3.dp
            )
            Spacer(Modifier.height(12.dp))
            Text(
                "正在识别...",
                style = MaterialTheme.typography.titleMedium
            )
            Spacer(Modifier.height(4.dp))
            when (val d = downloadState) {
                is com.echoling.app.speech.ModelManager.DownloadState.Deploying -> {
                    // (2026-07-04) First-run: model is being copied
                    // out of APK assets into internal storage
                    // (~68 MB unpacked). Was a network download
                    // before the offline cutover; the user-facing
                    // copy is now "正在准备识别模型" so the message
                    // doesn't claim a network step that no longer
                    // exists.
                    val pct = (d.progress * 100).toInt()
                    Text(
                        "首次使用: 正在准备识别模型 ($pct%)",
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant
                    )
                }
            }
        }
    }
}

```

```

        )
        Spacer(Modifier.height(8.dp))
        androidx.compose.material3.LinearProgressIndicator(
            progress = d.progress,
            modifier = Modifier.fillMaxWidth()
        )
    }
    is com.echoling.app.speech.ModelManager.DownloadState.Failed -> {
        Text(
            "模型加载失败: ${d.message}",
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.error
        )
    }
    else -> {
        // NotStarted / Ready — model is fine, Vosk is just
        // processing the audio.
        Text(
            "请稍候",
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}
}
}
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\recite\CategoryStudyScreen.kt ===

package com.echoling.app.presentation.ui.screens.recite

import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.animation.core.tween
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.aspectRatio
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Check
import androidx.compose.material.icons.filled.Close
import androidx.compose.material.icons.filled.Refresh
import androidx.compose.material.icons.filled.SkipNext
import androidx.compose.material.icons.filled.SkipPrevious
import androidx.compose.material.icons.filled.VolumeUp
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.ExperimentalMaterial3Api

```

```

import androidx.compose.material3.FilledTonalButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Snackbar
import androidx.compose.material3.SnackbarDuration
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.hilt.navigation.compose.hiltViewModel
import com.echoling.app.domain.model.DictEntry
import com.echoling.app.presentation.ui.components.PageHeader

/**
 * Flashcard study screen for one vocabulary category. UI mirrors the
 * previous LearningScreen word-for-word — the only difference is that
 * the word list comes from
 * [com.echoling.app.domain.usecase.GetDictionaryWordsInCategoryUseCase]
 * for the [categoryId] supplied via the nav route, so the user studies
 * only the words belonging to the selected category (and sees the
 * category-specific translation, not the cross-category merge).
 *
 * The card uses an [AnimatedContent] crossfade + scale transition as
 * the "flip" effect — same UX as the original screen, kept identical
 * so existing users recognize the gesture pattern.
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CategoryStudyScreen(
    categoryId: String,
    onNavigateBack: () -> Unit,
    viewModel: CategoryStudyViewModel = hiltViewModel(),
) {
    LaunchedEffect(categoryId) {
        viewModel.load(categoryId)
    }
    val uiState by viewModel.uiState.collectAsState()
    val snackbarHostState = remember { SnackbarHostState() }

    LaunchedEffect(uiState.lastSaved) {
        val saved = uiState.lastSaved
        if (saved != null) {
            snackbarHostState.showSnackbar("已加入单词本: $saved")
            viewModel.consumeLastSaved()
        }
    }
}

```



```

// Surface TTS unavailability as a snackbar. Long-press / repeat
// tap behavior: each non-null message fires exactly once, then
// consumeTtsUnavailableMessage clears it so the next tap re-fires.
LaunchedEffect(uiState.ttsUnavailableMessage) {
    val msg = uiState.ttsUnavailableMessage
    if (msg != null) {
        snackbarHostState.showSnackbar(msg, duration = SnackbarDuration.Long)
        viewModel.consumeTtsUnavailableMessage()
    }
}

Scaffold(
    // No `snackbarHost` slot here — the snackbar is anchored
    // INSIDE StudyBody right above the 认识/不认识 row (see
    // StudyBody). Putting it in the default bottom slot would
    // cover the 上一张/下一张 row when it pops up; placing it
    // inline keeps the confirmation visible without occluding
    // the navigation buttons below.
) { padding ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(padding),
    ) {
        PageHeader(
            onBack = onNavigateBack,
            title = {
                Text(
                    text = uiState.categoryName.ifBlank { "记单词" },
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.SemiBold,
                )
            },
            actions = {
                IconButton(onClick = { viewModel.resetSession() }) {
                    Icon(
                        imageVector = Icons.Default.Refresh,
                        contentDescription = "重置进度",
                    )
                }
            },
        )

        when {
            uiState.isLoading -> {
                Box(
                    modifier = Modifier.fillMaxSize(),
                    contentAlignment = Alignment.Center,
                ) {
                    CircularProgressIndicator()
                }
            }
            uiState.words.isEmpty() -> {
                Box(
                    modifier = Modifier.fillMaxSize(),
                    contentAlignment = Alignment.Center,
                ) {
                    Text(
                        text = "该词库为空",
                        style = MaterialTheme.typography.bodyMedium,
                        color = MaterialTheme.colorScheme.onSurfaceVariant,
                    )
                }
            }
        }
    }
}

```

```

    }
  }
  else -> {
    StudyBody(
      uiState = uiState,
      snackbarHostState = snackbarHostState,
      onFlip = viewModel::flipCard,
      onKnown = viewModel::markKnown,
      onUnknown = viewModel::markUnknown,
      onSave = viewModel::saveCurrentToVocabulary,
      onPrev = viewModel::skipToPrevious,
      onNext = viewModel::skipToNext,
      onPronounce = viewModel::pronounceCurrent,
      // (2026-07-04) new — speaks the example sentence
      // for the speaker button on the flashcard front.
      onPronounceSentence = viewModel::pronounceCurrentSentence,
    )
  }
}
}
}

// (2026-07-04) "已加入单词本" confirmation surfaced as a snackbar
// — earlier today this was an AlertDialog, but the user found
// a blocking dialog disruptive mid-flip (the user has just
// tapped "加入单词本" and the card is about to advance; forcing
// a "确定" tap to dismiss is too much). Back to a transient
// snackbar, but the snackbar's `containerColor` is themed to
// `surfaceVariant` (light gray) via the custom `snackbar` slot
// on `SnackbarHost` below — same muted tone the dialog had, so
// the visual signal still reads as "different from a default
// info toast" without the blocking modal interaction.
//
// Lifecycle: snackbar appears, M3 auto-dismisses after
// `SnackbarDuration.Short` (~4s), then we call
// `consumeLastSaved()` to clear the state field. A second tap
// on "加入单词本" re-fires it from the new word.
}

```

@Composable

```

private fun StudyBody(
  uiState: CategoryStudyUiState,
  snackbarHostState: SnackbarHostState,
  onFlip: () -> Unit,
  onKnown: () -> Unit,
  onUnknown: () -> Unit,
  onSave: () -> Unit,
  onPrev: () -> Unit,
  onNext: () -> Unit,
  onPronounce: () -> Unit,
  // (2026-07-04) new — plays the example sentence via TTS.
  onPronounceSentence: () -> Unit = {},
) {
  val current = uiState.currentWord ?: return
  Column(
    modifier = Modifier
      .fillMaxSize()
      .padding(horizontal = 16.dp, vertical = 4.dp),
  ) {
    ProgressRow(
      currentNumber = uiState.currentIndex + 1,
      total = uiState.totalCount,
      known = uiState.knownCount,
    )
  }
}

```

```

        unknown = uiState.unknownCount,
    )

    // Spacer between the progress row and the card: tightened
    // from 20dp → 12dp so the controls below feel closer to the
    // card (the "buttons move up to hug the card" request).
    Spacer(modifier = Modifier.height(12.dp))

    // The flashcard is a fixed-aspect square, centered in the
    // remaining vertical space — smaller than the old `weight(1f)`
    // full-height card so the word feels like a "flashcard" rather
    // than a full-screen hero. `weight(1f)` on the outer Box keeps
    // the card vertically centered when there's spare height.
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .weight(1f),
        crossAxisAlignment = Alignment.Center,
    ) {
        FlashCard(
            entry = current,
            isFlipped = uiState.isFlipped,
            onClick = onFlip,
            onPronounce = onPronounce,
            // (2026-07-04) new — drives the speaker button on
            // the front face's example-sentence row.
            onPronounceSentence = onPronounceSentence,
            modifier = Modifier
                .padding(horizontal = 32.dp)
                .fillMaxWidth()
                .aspectRatio(1f),
        )
    }

    // Snackbar anchored above the answer buttons. Default M3
    // placement puts it at the very bottom of the Scaffold — there
    // it would overlap the 上一张/下一张 row when "已加入单词本"
    // pops up. Hosting it inline here keeps the confirmation in
    // the user's eye line (right above the button they just
    // tapped) and far from the navigation row at the bottom.
    //
    // (2026-07-04) Custom `snackbar` slot overrides M3's default
    // container color (which is `inverseSurface` — near-black) with
    // `surfaceVariant` (light purple-tinted gray) per the user's
    // "背景色改为浅灰色" request. The text color switches to
    // `onSurfaceVariant` to keep AA contrast on the lighter
    // background; M3's default `inverseOnSurface` would render the
    // text light on light and be unreadable.
    SnackbarHost(
        hostState = snackbarHostState,
        modifier = Modifier.padding(bottom = 4.dp),
    ) { data ->
        Snackbar(
            snackbarData = data,
            containerColor = MaterialTheme.colorScheme.surfaceVariant,
            contentColor = MaterialTheme.colorScheme.onSurfaceVariant,
        )
    }

    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.spacedBy(12.dp),
    ) {

```

```

        FilledTonalButton(
            onClick = onKnown,
            modifier = Modifier
                .weight(1f)
                .height(56.dp),
            shape = RoundedCornerShape(16.dp),
        ) {
            Icon(Icons.Default.Check, contentDescription = null)
            Spacer(modifier = Modifier.width(8.dp))
            Text("认识", fontSize = 16.sp)
        }
        Button(
            onClick = onUnknown,
            modifier = Modifier
                .weight(1f)
                .height(56.dp),
            shape = RoundedCornerShape(16.dp),
            colors = ButtonDefaults.buttonColors(
                containerColor = MaterialTheme.colorScheme.errorContainer,
                contentColor = MaterialTheme.colorScheme.onErrorContainer,
            ),
        ) {
            Icon(Icons.Default.Close, contentDescription = null)
            Spacer(modifier = Modifier.width(8.dp))
            Text("不认识", fontSize = 16.sp)
        }
    }

    if (uiState.showSaveButton) {
        // "加入单词本" sits tucked under the 认识/不认识 row — same
        // 8dp gap so it reads as part of the "answer" group
        // (visual pairing with the button that revealed it).
        Spacer(modifier = Modifier.height(8.dp))
        Button(
            onClick = onSave,
            modifier = Modifier
                .fillMaxWidth()
                .height(52.dp),
            shape = RoundedCornerShape(16.dp),
        ) {
            Text("加入单词本", fontSize = 16.sp)
        }
    }

    // Visual separator between the answer group (认识/不认识 +
    // 加入单词本) and the navigation group (上一张/下一张). The
    // 20dp blank line is wide enough to feel like a deliberate
    // divider between the two control groups.
    Spacer(modifier = Modifier.height(20.dp))
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically,
    ) {
        TextButton(onClick = onPrev) {
            Icon(Icons.Default.SkipPrevious, contentDescription = null)
            Spacer(modifier = Modifier.padding(end = 4.dp))
            Text("上一张")
        }
        TextButton(onClick = onNext) {
            Text("下一张")
            Spacer(modifier = Modifier.padding(start = 4.dp))
            Icon(Icons.Default.SkipNext, contentDescription = null)
        }
    }

```

```

    }
    }
}

@Composable
private fun ProgressRow(
    currentNumber: Int,
    total: Int,
    known: Int,
    unknown: Int,
) {
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically,
    ) {
        Text(
            text = "已学 $currentNumber / $total",
            style = MaterialTheme.typography.titleSmall,
            fontWeight = FontWeight.SemiBold,
        )
        Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            ScoreBadge(label = "认识", count = known, color = MaterialTheme.colorScheme.primary)
            ScoreBadge(label = "不认识", count = unknown, color = MaterialTheme.colorScheme.error)
        }
    }
}

@Composable
private fun ScoreBadge(label: String, count: Int, color: Color) {
    Surface(
        shape = RoundedCornerShape(12.dp),
        color = color.copy(alpha = 0.12f),
    ) {
        Text(
            text = "$label $count",
            style = MaterialTheme.typography.labelMedium,
            color = color,
            modifier = Modifier.padding(horizontal = 10.dp, vertical = 4.dp),
        )
    }
}

@Composable
private fun FlashCard(
    entry: DictEntry,
    isFlipped: Boolean,
    onClick: () -> Unit,
    onPronounce: () -> Unit,
    // (2026-07-04) new — passed through to CardFront for the
    // speaker button on the example-sentence row.
    onPronounceSentence: () -> Unit = {},
    modifier: Modifier = Modifier,
) {
    // Drive a continuous 0..180° rotation off [isFlipped]. We use
    // `graphicsLayer.rotationY` so the rotation is GPU-composited (no
    // layout pass) and `cameraDistance` is bumped up — Compose's
    // default camera is so close that anything beyond ~120° looks
    // like a fisheye; multiplying it by 8 keeps the perspective
    // natural at 180° .
    //
    // At exactly 90° the card is edge-on (invisible). We swap

```

```

// front□back content at that midpoint so the back face is never
// rendered mirror-reversed. `[front, back]` is laid out in the
// same Box stacked with the back pre-rotated 180° so its final
// position (after the outer rotation) reads correctly.
val rotation by animateFloatAsState(
    targetValue = if (isFlipped) 180f else 0f,
    animationSpec = tween(durationMillis = 420),
    label = "flashcard-rotation",
)
val showBack = rotation > 90f

Card(
    modifier = modifier
        .graphicsLayer {
            rotationY = rotation
            cameraDistance = 12f * density
        }
        .clickable(onClick = onClick),
    shape = RoundedCornerShape(24.dp),
    colors = CardDefaults.cardColors(
        containerColor = if (showBack)
            MaterialTheme.colorScheme.secondaryContainer
        else
            MaterialTheme.colorScheme.primaryContainer,
    ),
    elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
) {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center,
    ) {
        // Both faces are mounted in the same Box and inherit the
        // outer Card's `rotationY` (0° → 180°). To stop the front
        // face's text from showing through as a mirror image once
        // the card has flipped, we **counter-rotate** each face:
        //
        // - [CardFront] is the visible layer during [0°, 90°).
        //   When the outer card passes 90° we hide it via
        //   `alpha = 0` and counter-rotate it 180° so it ends
        //   up orientation-correct if it ever gets composited.
        // - [CardBack] is pre-rotated 180° (so its text reads
        //   correctly after the outer rotation) and only fades
        //   in once `rotation > 90°` (i.e. once the front is
        //   edge-on and no longer visible).
        //
        // Without the per-face `rotationY`, the front face's word
        // would still be compositing in mirrored form during the
        // second half of the flip — looking like the front's word
        // had "leaked" onto the back.
        CardFront(
            entry = entry,
            onPronounce = onPronounce,
            // (2026-07-04) new — wires the front's sentence
            // speaker button to the ViewModel's sentence TTS call.
            onPronounceSentence = onPronounceSentence,
            modifier = Modifier.graphicsLayer {
                rotationY = if (showBack) 180f else 0f
                alpha = if (showBack) 0f else 1f
            },
        )
        CardBack(
            entry = entry,
            onPronounce = onPronounce,

```

```

        modifier = Modifier.graphicsLayer {
            rotationY = 180f
            alpha = if (showBack) 1f else 0f
        },
    )
}
}

@Composable
private fun CardFront(
    entry: DictEntry,
    onPronounce: () -> Unit,
    // (2026-07-04) new — drives the speaker button on the
    // example-sentence row, sitting to the right of the English
    // sentence. Tapping it speaks the sentence aloud via TTS.
    onPronounceSentence: () -> Unit = {},
    modifier: Modifier = Modifier,
) {
    Box(modifier = modifier.fillMaxSize()) {
        // Main content (word + example sentences) centered. The
        // hint "点击卡片查看翻译" is rendered as a separate sibling
        // anchored to the bottom of the card — see below.
        //
        // (2026-07-05) Horizontal padding split out from the
        // uniform 20dp so the left-aligned example sentence below
        // can run closer to the card edges. The word + speaker row
        // is centered horizontally so it isn't visually affected
        // by the wider container — only the English sentence and
        // its TTS button (which use fillMaxWidth + Start / the
        // Column's CenterHorizontally) gain the extra room.
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(start = 12.dp, end = 12.dp, top = 20.dp, bottom = 20.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center,
        ) {
            // Word + speaker on the same Row. The user asked for the
            // speaker to sit immediately to the right of the word
            // (rather than as a floating top-right corner button) so
            // the pair reads as one unit: "word [□]". 44dp box /
            // 26dp icon is slightly larger than the back's 32/20
            // because the front has no other controls nearby — the
            // speaker is the front's only secondary action and
            // deserves to be a clear touch target.
            Row(
                horizontalArrangement = Arrangement.Center,
                verticalAlignment = Alignment.CenterVertically,
            ) {
                // (2026-07-05) headlineMedium (28sp) → headlineSmall (24sp) per
                // user request — one step down in the M3 headline scale.
                // The word stays the focal point of the front face but
                // no longer overwhelms the example sentence below it
                // (which is bodyMedium / 14sp). The Bold weight is
                // kept so the word still reads as "primary signal"
                // even at the smaller size.
                Text(
                    text = entry.word,
                    style = MaterialTheme.typography.headlineSmall,
                    fontWeight = FontWeight.Bold,
                    color = MaterialTheme.colorScheme.onPrimaryContainer,
                    maxLines = 2,
                )
            }
        }
    }
}

```

```

        overflow = TextOverflow.Ellipsis,
        textAlign = TextAlign.Center,
    )
    Spacer(modifier = Modifier.width(8.dp))
    IconButton(
        onClick = onPronounce,
        modifier = Modifier.size(44.dp),
    ) {
        Icon(
            imageVector = Icons.Default.VolumeUp,
            contentDescription = "朗读单词",
            tint = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.85f),
            modifier = Modifier.size(26.dp),
        )
    }
}

// (2026-07-04) English example sentence on the front.
// Layout split per user's request: the front now holds
// ONLY English content (word + EN example), the back
// holds ONLY Chinese / pronunciation content (phonetic +
// word translation + CN example translation). This keeps
// each face linguistically pure and prevents long
// translations from squeezing the front.
//
// (2026-07-05) Sentence layout reshaped: the text is
// left-aligned (fillMaxWidth + TextAlign.Start) so long
// sentences have a clean left edge as they wrap, and the
// TTS speaker button now sits BELOW the sentence,
// centered horizontally (the parent Column already has
// `horizontalAlignment = CenterHorizontally`, so the 32dp
// IconButton centers itself without an extra modifier).
// Previous row-with-button-on-right design had the speaker
// squeezed against the right edge on long sentences — the
// vertical stack reads more naturally and keeps the
// button at a predictable x-coordinate regardless of
// sentence length.
//
// [ English sentence text,   ]
// [ left-aligned, wrapping  ]
// [   □   ]
//
// Gated on `isNotBlank()` because the source JSON's
// coverage isn't 100% — short multi-word phrases often
// lack an example entirely. If the EN sentence is blank
// we skip both the text AND the speaker button so the
// card doesn't render a stranded button below an empty
// block.
if (entry.exampleSentenceEn.isNotBlank()) {
    Spacer(modifier = Modifier.height(16.dp))
    // Sentence — `fillMaxWidth` so multi-line wrap has a
    // consistent left edge, `TextAlign.Start` so the text
    // reads from the left rather than centering (which
    // made long sentences look ragged with the previous
    // design).
    Text(
        text = entry.exampleSentenceEn,
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.9f),
        modifier = Modifier.fillMaxWidth(),
        textAlign = TextAlign.Start,
    )
    // Speaker button — directly under the sentence,

```



```

        // horizontally centered via the parent Column's
        // `horizontalAlignment = CenterHorizontally`. The
        // 32dp box / 20dp icon match the back face's
        // phonetic/word speakers so the card has a
        // consistent touch-target vocabulary across faces.
        IconButton(
            onClick = onPronounceSentence,
            modifier = Modifier
                .padding(top = 4. dp)
                .size(32. dp),
        ) {
            Icon(
                imageVector = Icons.Default.VolumeUp,
                contentDescription = "朗读句子",
                tint = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.85f),
                modifier = Modifier.size(20. dp),
            )
        }
    }

    // (2026-07-04) Hint removed from the bottom of the
    // centered Column — see the bottom-anchored sibling below
    // for why.
}
// (2026-07-04) Hint is now a sibling of the centered Column,
// anchored to the card's bottom edge with `Alignment.BottomCenter`.
//
// The previous layout put it directly under the Chinese
// translation with only an 8dp gap, which made the two read
// as one block ("Chinese sentence [hint]") and confused users
// into thinking the hint was somehow about the Chinese text.
//
// Pinning the hint to the bottom of the card — separated from
// the centered content by the card's own vertical space —
// gives it a clear "this is a UI affordance, not part of the
// vocabulary" identity. The 16dp bottom padding keeps the
// text from kissing the card's rounded corner.
Text(
    text = "点击卡片查看翻译",
    style = MaterialTheme.typography.bodySmall,
    color = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.7f),
    modifier = Modifier
        .align(Alignment.BottomCenter)
        .padding(bottom = 16. dp),
)
}
}

@Composable
private fun CardBack(
    entry: DictEntry,
    onPronounce: () -> Unit,
    modifier: Modifier = Modifier,
) {
    // The back face intentionally omits the word itself — the user
    // already saw it on the front. Re-rendering it here would (a) feel
    // like the front's word "leaked" onto the back, and (b) eat
    // vertical space in the small square card. Layout becomes
    // [phonetic + □] → [pos chip] → [translation], top-down.
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(horizontal = 20. dp, vertical = 16. dp),
    )
}

```

```

        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center,
    ) {
        if (entry.phonetic.isNotBlank()) {
            // Phonetic + speaker on one row — natural pair, the user
            // reads the IPA and hears the word in one eye-movement.
            Row(verticalAlignment = Alignment.CenterVertically) {
                Text(
                    text = entry.phonetic,
                    style = MaterialTheme.typography.bodyMedium,
                    color = MaterialTheme.colorScheme.onSecondaryContainer.copy(alpha = 0.8f),
                    textAlign = TextAlign.Center,
                    maxLines = 1,
                )
                Spacer(modifier = Modifier.width(4.dp))
                IconButton(
                    onClick = onPronounce,
                    modifier = Modifier.size(32.dp),
                ) {
                    Icon(
                        imageVector = Icons.Default.VolumeUp,
                        contentDescription = "朗读单词",
                        tint = MaterialTheme.colorScheme.onSecondaryContainer.copy(alpha = 0.8f),
                        modifier = Modifier.size(20.dp),
                    )
                }
            }
        } else {
            // No phonetic — still show the speaker so the affordance
            // is consistent across cards. Without this fallback the
            // icon would pop in/out as the user flips through words,
            // which feels broken.
            IconButton(
                onClick = onPronounce,
                modifier = Modifier.size(32.dp),
            ) {
                Icon(
                    imageVector = Icons.Default.VolumeUp,
                    contentDescription = "朗读单词",
                    tint = MaterialTheme.colorScheme.onSecondaryContainer.copy(alpha = 0.8f),
                    modifier = Modifier.size(20.dp),
                )
            }
        }
        if (entry.pos.isNotBlank()) {
            Spacer(modifier = Modifier.height(6.dp))
            Surface(
                shape = RoundedCornerShape(8.dp),
                color = MaterialTheme.colorScheme.onSecondaryContainer.copy(alpha = 0.08f),
            ) {
                Text(
                    text = entry.pos,
                    style = MaterialTheme.typography.labelMedium,
                    color = MaterialTheme.colorScheme.onSecondaryContainer,
                    modifier = Modifier.padding(horizontal = 8.dp, vertical = 2.dp),
                )
            }
        }
        Spacer(modifier = Modifier.height(12.dp))
        // Translation reads as supporting context for the word, not
        // the primary signal — so on the back face's
        // secondaryContainer background we use the muted onSurfaceVariant
        // for the color and Normal for the weight. The M3 theme

```

```

// declares titleMedium as SemiBold (600) — see Type.kt:64-70 —
// so without the explicit fontWeight override the translation
// would render heavier than the phonetic above (bodyMedium,
// Normal/400) and the POS chip below, breaking visual
// consistency.
//
// (2026-07-05) Title size downshifted one step from
// titleMedium (16sp) → titleSmall (14sp) per user request.
// The translation is no longer the visual focal point of the
// back face — the multi-line examples and phonetic row above
// already carry enough weight, and the previous 16sp read as
// "loud" on dense cards. The Normal weight override is kept so
// the text stays visually subordinate to the POS chip's Medium
// weight, even though they're now the same sp size — the
// letter-spacing difference (titleSmall = 0.1sp vs
// POS-labelMedium = 0.5sp) is the cue that signals "supporting
// text, not a label".
Text(
    text = entry.translation,
    style = MaterialTheme.typography.titleSmall.copy(
        fontWeight = FontWeight.Normal,
    ),
    color = MaterialTheme.colorScheme.onSurfaceVariant,
    textAlign = TextAlign.Center,
)

// (2026-07-04) Example sentence translation (CN). Lives on
// the back face alongside the word translation, so the back
// is purely the "Chinese / explanation" face:
// [phonetic + □] → [pos chip] → [word translation]
// → [example sentence translation]
//
// The English example sentence (with its speaker button)
// sits on the FRONT face — splitting the languages per face
// (English only on front, Chinese only on back) prevents the
// long-translation overflow problem we had when both
// languages shared one face.
//
// No maxLines / ellipsis here: long Chinese translations
// are fine because the back face has more vertical room now
// that it no longer carries the English sentence pair, and
// an interrupted CN gloss is worse than a slightly tall
// card. The parent Column doesn't clipToBounds so any
// extreme overflow stays readable instead of being cut off.
if (entry.exampleSentenceCn.isNotBlank()) {
    Spacer(modifier = Modifier.height(10.dp))
    Text(
        text = entry.exampleSentenceCn,
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.8f),
        textAlign = TextAlign.Center,
    )
}
}
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\recite\CategoryStudyViewModel.kt ===

package com.echoling.app.presentation.ui.screens.recite

import androidx.lifecycle.SavedStateHandle

```

```

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.echoling.app.data.local.db.entity.ReciteProgressEntity
import com.echoling.app.domain.model.DictCategory
import com.echoling.app.domain.model.DictEntry
import com.echoling.app.domain.model.Word
import com.echoling.app.domain.usecase.GetDictionaryCategoriesUseCase
import com.echoling.app.domain.usecase.GetDictionaryWordsInCategoryUseCase
import com.echoling.app.domain.usecase.GetReciteProgressUseCase
import com.echoling.app.domain.usecase.SaveReciteProgressUseCase
import com.echoling.app.domain.usecase.SaveWordUseCase
import com.echoling.app.player.TtsManager
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import javax.inject.Inject

/**
 * UI state for the per-category flashcard study screen.
 *
 * - [categoryName] — name (e.g. "高中英语词汇") to render in the page
 *   header. Empty until the categories manifest has loaded.
 * - [words] / [currentIndex] / [isFlipped] / [knownCount] /
 *   [unknownCount] / [showSaveButton] / [lastSaved] — same shape as
 *   the original learning VM. See `LearningUiState` doc for
 *   per-field semantics.
 * - [isLoading] — true while the category entries are still being
 *   fetched from the bundled assets.
 * - [ttsUnavailableMessage] — non-null when the user just tapped the
 *   speaker but the device has no TTS engine installed (e.g. Xiaomi
 *   CN without Google services). The screen reads this and shows a
 *   snackbar instructing the user to install Google TTS / iFlytek.
 *   Cleared by [CategoryStudyViewModel.consumeTtsUnavailableMessage].
 */
data class CategoryStudyUiState(
    val categoryName: String = "",
    val words: List<DictEntry> = emptyList(),
    val currentIndex: Int = -1,
    val isFlipped: Boolean = false,
    val knownCount: Int = 0,
    val unknownCount: Int = 0,
    val showSaveButton: Boolean = false,
    val isLoading: Boolean = true,
    val lastSaved: String? = null,
    val ttsUnavailableMessage: String? = null,
) {
    val totalCount: Int get() = words.size
    val currentWord: DictEntry? get() = words.getOrNull(currentIndex)
}

/**
 * State holder for the per-category flashcard study screen.
 *
 * Reads the `categoryId` route argument via [SavedStateHandle] (Hilt
 * populates it from the nav entry) and lazily loads that category's
 * word list. Behavioural mirrors of the original LearningViewModel —
 * same `flipCard` / `markKnown` / `markUnknown` / `saveCurrentToVocabulary`
 * / `skipToNext` / `skipToPrevious` / `resetSession` semantics — so the
 * existing flashcard UX is unchanged.
 *
 * **Persistence (DB v5 / `recite_progress` table):**

```

```

* - On [load] we read the saved progress row for this category and
*   seed `currentIndex` / `knownCount` / `unknownCount` from it.
*   Without this, process death / app cold-start / tab switch would
*   drop the user back on card 0 every time.
* - After every `markKnown` / `markUnknown` / `saveCurrentToVocabulary`
*   / `skipToNext` / `skipToPrevious` / `resetSession` we write the
*   new state back via [SaveReciteProgressUseCase]. The `currentIndex`
*   we persist is the *destination* of the action (i.e. the card
*   the user is now looking at), not the one they just left, so
*   re-entering lands on the right card.
*
* When the user taps "加入单词本" the resulting [Word] is saved with
* the entry's full data (word, phonetic, pos, translation) so the
* vocabulary book renders the part-of-speech chip correctly.
*/
@HiltViewModel
class CategoryStudyViewModel @Inject constructor(
    savedStateHandle: SavedStateHandle,
    private val getDictionaryCategories: GetDictionaryCategoriesUseCase,
    private val getDictionaryWordsInCategory: GetDictionaryWordsInCategoryUseCase,
    private val saveWordUseCase: SaveWordUseCase,
    private val getReciteProgress: GetReciteProgressUseCase,
    private val saveReciteProgress: SaveReciteProgressUseCase,
    private val ttsManager: TtsManager,
) : ViewModel() {

    private val initialCategoryId: String =
        savedStateHandle.get<String>(ARG_CATEGORY_ID)?.decodeFromRoute().orEmpty()

    private val _uiState = MutableStateFlow(CategoryStudyUiState())
    val uiState: StateFlow<CategoryStudyUiState> = _uiState.asStateFlow()

    fun load(categoryId: String = initialCategoryId) {
        if (categoryId.isBlank()) return
        viewModelScope.launch {
            _uiState.value = _uiState.value.copy(isLoading = true)
            // Resolve the human-readable name from the categories list
            // (so the header can show "高中英语词汇" rather than "senior").
            // Fall back to the slug if the manifest can't be loaded —
            // unlikely but keeps the screen usable.
            val categories = runCatching { getDictionaryCategories() }.getOrDefault(emptyList())
            val name = categories.firstOrNull { it.id == categoryId }?.name ?: categoryId
            val words = getDictionaryWordsInCategory(categoryId)
            // Resume from the persisted progress row if any. Without
            // this the user lands on card 0 every time they re-open
            // the category. The `coerceIn` clamps the index into the
            // valid range so a stale row that points past the new
            // word-list size (e.g. asset got smaller in a future
            // update) doesn't leave the screen permanently stuck on
            // the empty-state branch.
            val saved = getReciteProgress(categoryId)
            val maxValid = (words.size - 1).coerceAtLeast(-1)
            val resumedIndex = saved?.currentIndex?.coerceIn(-1, maxValid) ?: -1
            _uiState.value = CategoryStudyUiState(
                categoryName = name,
                words = words,
                currentIndex = if (words.isEmpty()) -1 else resumedIndex.coerceAtLeast(0),
                isFlipped = false,
                knownCount = saved?.knownCount ?: 0,
                unknownCount = saved?.unknownCount ?: 0,
                showSaveButton = false,
                isLoading = false,
            )
        }
    }
}

```

```

    }
}

fun flipCard() {
    // isFlipped is purely UI — no DB write (we don't want to
    // persist transient per-card state on every tap).
    _uiState.value = _uiState.value.copy(isFlipped = !_uiState.value.isFlipped)
}

fun markKnown() {
    val s = _uiState.value
    if (s.currentWord == null) return
    val next = s.copy(
        knownCount = s.knownCount + 1,
        showSaveButton = false,
        isFlipped = false,
        currentIndex = (s.currentIndex + 1).mod(s.words.size.coerceAtLeast(1)),
    )
    _uiState.value = next
    persist(initialCategoryId, next)
}

fun markUnknown() {
    val s = _uiState.value
    if (s.currentWord == null) return
    val next = s.copy(
        unknownCount = s.unknownCount + 1,
        showSaveButton = true,
    )
    _uiState.value = next
    persist(initialCategoryId, next)
}

fun saveCurrentToVocabulary() {
    val entry = _uiState.value.currentWord ?: return
    viewModelScope.launch {
        saveWordUseCase(
            Word(
                word = entry.word,
                phonetic = entry.phonetic,
                pos = entry.pos,
                translation = entry.translation,
                exampleSentence = "",
                sourceCourseId = "",
                sourceSentenceId = 0,
                collectedAt = System.currentTimeMillis(),
                nextReviewTime = System.currentTimeMillis(),
            )
        )
        val s = _uiState.value
        val next = s.copy(
            showSaveButton = false,
            isFlipped = false,
            lastSaved = entry.word,
            currentIndex = (s.currentIndex + 1).mod(s.words.size.coerceAtLeast(1)),
        )
        _uiState.value = next
        persist(initialCategoryId, next)
    }
}

fun skipToNext() {
    val s = _uiState.value

```

```

        if (s.currentWord == null) return
        val next = s.copy(
            showSaveButton = false,
            isFlipped = false,
            currentIndex = (s.currentIndex + 1).mod(s.words.size.coerceAtLeast(1)),
        )
        _uiState.value = next
        persist(initialCategoryId, next)
    }

    fun skipToPrevious() {
        val s = _uiState.value
        if (s.currentWord == null) return
        val next = s.copy(
            showSaveButton = false,
            isFlipped = false,
            currentIndex = if (s.currentIndex <= 0) s.words.size - 1 else s.currentIndex - 1,
        )
        _uiState.value = next
        persist(initialCategoryId, next)
    }

    fun resetSession() {
        val s = _uiState.value
        val next = s.copy(
            currentIndex = if (s.words.isEmpty()) -1 else 0,
            knownCount = 0,
            unknownCount = 0,
            isFlipped = false,
            showSaveButton = false,
        )
        _uiState.value = next
        persist(initialCategoryId, next)
    }

    fun consumeLastSaved() {
        if (_uiState.value.lastSaved != null) {
            _uiState.value = _uiState.value.copy(lastSaved = null)
        }
    }

    /**
     * Speak the current word aloud via the system TTS engine. No-op
     * if there is no current word (empty state). Safe to call before
     * TTS init completes — [TtsManager] queues the call internally.
     *
     * If no TTS engine is installed on the device (e.g. CN Xiaomi
     * without Google services), sets [CategoryStudyUiState.ttsUnavailableMessage]
     * instead of speaking — the screen observes this and shows a
     * snackbar telling the user how to install a TTS engine.
     */
    fun pronounceCurrent() {
        if (!ttsManager.isAvailable.value) {
            _uiState.value = _uiState.value.copy(
                ttsUnavailableMessage = "设备未安装 TTS 引擎，请到应用商店安装「Google 文字转语音」或「讯飞语音」后  
再试"
            )
            return
        }
        _uiState.value.currentWord?.let { word ->
            ttsManager.speak(word.word)
        }
    }
}

```

```

/**
 * (2026-07-04) Speak the current example sentence aloud. Triggered
 * by the speaker button next to the EN sentence on the flashcard
 * front. No-op if there is no current word, or if the source JSON
 * didn't carry an English sentence for this entry (some short
 * multi-word phrases lack one entirely).
 *
 * The sentence is stripped of any `...` tags before being
 * handed to TTS — those tags are visual markers for the
 * flashcard UI ("make headword bold"), they have no audible
 * meaning, and Google's TTS engine reads the literal characters
 * ("less-than b greater-than younger less-than slash b greater-than")
 * which is unintelligible.
 *
 * Same TTS-unavailable fallback as [pronounceCurrent]: sets
 * [CategoryStudyUiState.ttsUnavailableMessage] instead of speaking.
 */
fun pronounceCurrentSentence() {
    val sentence = _uiState.value.currentWord?.exampleSentenceEn
        ?.takeIf { it.isNotBlank() }
        ?: return
    if (!ttsManager.isAvailable.value) {
        _uiState.value = _uiState.value.copy(
            ttsUnavailableMessage = "设备未安装 TTS 引擎，请到应用商店安装「Google 文字转语音」或「讯飞语音」后
再试"
        )
        return
    }
    ttsManager.speak(sentence.replace(Regex("<[>]+>"), ""))
}

/**
 * Clear [CategoryStudyUiState.ttsUnavailableMessage] after the
 * screen has shown the snackbar, so a subsequent tap re-fires it.
 */
fun consumeTtsUnavailableMessage() {
    if (_uiState.value.ttsUnavailableMessage != null) {
        _uiState.value = _uiState.value.copy(ttsUnavailableMessage = null)
    }
}

/**
 * Persist the current in-memory state for [categoryId] to the
 * `recite_progress` table. Called after every score / navigation
 * action so the user resumes exactly where they left off. Writes
 * are fire-and-forget on `viewModelScope` — the in-memory
 * `_uiState` is the source of truth for the running screen.
 */
private fun persist(categoryId: String, state: CategoryStudyUiState) {
    if (categoryId.isBlank()) return
    viewModelScope.launch {
        saveReciteProgress(
            ReciteProgressEntity(
                categoryId = categoryId,
                currentIndex = state.currentIndex,
                knownCount = state.knownCount,
                unknownCount = state.unknownCount,
                lastStudiedAt = System.currentTimeMillis(),
            )
        )
    }
}

```



```
companion object {
    /** Nav argument name; kept here so the VM and the nav graph stay in sync. */
    const val ARG_CATEGORY_ID = "categoryId"
}

/** Decoded from a URL-encoded nav route — mirrors `Screen.encodeForRoute`. */
private fun String.decodeFromRoute(): String =
    java.net.URLDecoder.decode(this, "UTF-8")

// === app\src\main\java\com\echoling\app\presentation\ui\screens\recite\ReciteScreen.kt ===

package com.echoling.app.presentation.ui.screens.recite

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.WindowInsets
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBars
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowForward
import androidx.compose.material.icons.filled.AutoStories
import androidx.compose.material.icons.filled.Check
import androidx.compose.material.icons.filled.MenuBook
import androidx.compose.material.icons.filled.Public
import androidx.compose.material.icons.filled.School
import androidx.compose.material.icons.filled.Star
import androidx.compose.material.icons.filled.Translate
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
```

```

import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.hilt.navigation.compose.hiltViewModel
import com.echoling.app.R
import com.echoling.app.data.local.db.entity.ReciteProgressEntity
import com.echoling.app.domain.model.DictCategory
import com.echoling.app.presentation.ui.components.PageHeader

/**
 * "记单词" tab landing — renders the five bundled vocabulary
 * categories as tappable cards. Each card shows the category name,
 * description, and entry count. Tapping a card navigates to the
 * per-category flashcard study screen.
 *
 * Cards are sorted in manifest-declaration order
 * (初中 → 高中 → CET-4 → CET-6 → TOEFL) so the picker shows the
 * natural difficulty progression. The icons follow the same sequence
 * (junior → senior → intermediate → advanced → academic) — picked to
 * hint at the level without literal mapping that could feel
 * preachy (e.g. we don't use a "hard" icon for TOEFL, just "global"
 * to signal "broader scope").
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ReciteScreen(
    onNavigateToCategory: (categoryId: String) -> Unit,
    viewModel: ReciteViewModel = hiltViewModel(),
) {
    val uiState by viewModel.uiState.collectAsState()

    Scaffold(
        // § 12.30: only consume the top status-bar inset, not the bottom
        // navigation-bar inset. The outer MainScaffold already accounts
        // for the bottom tab bar, so a second nav-bar subtraction here
        // would leave a 24dp page-colored strip between the tab bar
        // and the last list item. See CLAUDE.md § 12.30.
        contentWindowInsets = WindowInsets.statusBars,
        // No topBar — the "记单词" title sits in PageHeader below (§ 12.18)
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
        ) {
            PageHeader(
                title = {
                    // Two-line brand bar matching the home page (Courses)
                    // and Me tabs — Chinese title with 3sp letter-spacing
                    // so the three characters don't visually crowd, plus
                    // an italic English subtitle below in onSurfaceVariant.
                    Column(horizontalAlignment = Alignment.CenterHorizontally) {
                        Text(
                            text = stringResource(R.string.recite_title),
                            style = TextStyle(
                                fontSize = 18.sp,
                                fontWeight = FontWeight.Bold,
                                letterSpacing = 3.sp,
                            ),
                            maxLines = 1,
                        )
                        Text(
                            text = stringResource(R.string.recite_subtitle),
                            style = TextStyle(

```

```

        fontSize = 11.sp,
        fontStyle = FontStyle.Italic,
        fontWeight = FontWeight.Medium,
        letterSpacing = 1.sp,
    ),
    color = MaterialTheme.colorScheme.onSurfaceVariant,
    maxLines = 1,
)
}
},
)
when {
    uiState.isLoading -> {
        Box(
            modifier = Modifier.fillMaxSize(),
            contentAlignment = Alignment.Center,
        ) {
            CircularProgressIndicator()
        }
    }
    uiState.categories.isEmpty() -> {
        Box(
            modifier = Modifier.fillMaxSize(),
            contentAlignment = Alignment.Center,
        ) {
            Text(
                text = "未找到词库，请检查 app/src/main/assets/vocab_manifest.json",
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
                modifier = Modifier.padding(horizontal = 32.dp),
            )
        }
    }
}
else -> {
    LazyColumn(
        modifier = Modifier.fillMaxSize(),
        contentPadding = PaddingValues(horizontal = 16.dp, vertical = 8.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp),
    ) {
        items(uiState.categories, key = { it.id }) { category ->
            CategoryCard(
                category = category,
                progress = uiState.progressByCategory[category.id],
                onClick = { onNavigateToCategory(category.id) },
            )
        }
        item { Spacer(modifier = Modifier.height(16.dp)) }
    }
}
}
}
}
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
private fun CategoryCard(
    category: DictCategory,
    progress: ReciteProgressEntity?,
    onClick: () -> Unit,
) {
    Card(
        modifier = Modifier.fillMaxWidth(),

```

```

        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surfaceVariant,
        ),
        onClick = onClick,
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                // 14 → 10 dp vertical padding: 11 cards now occupy the
                // screen instead of 5, so each card has to give back ~8dp
                // of height to keep the picker manageable.
                .padding(horizontal = 16.dp, vertical = 10.dp),
            verticalAlignment = Alignment.CenterVertically,
        ) {
            // Leading icon — picks one of 11 brand-coherent Material icons
            // based on the category id. (2026-07-05) Removed the
            // primaryContainer background tint — the icon now sits in
            // a transparent 40dp frame and tints with primary (brand
            // purple), reading as a clean outlined glyph rather than
            // a tinted chip. Keeps the 40dp container so the Row's
            // vertical alignment and 12dp gap to the title don't
            // shift.
            Surface(
                shape = RoundedCornerShape(12.dp),
                color = Color.Transparent,
                modifier = Modifier.size(40.dp),
            ) {
                Box(contentAlignment = Alignment.Center) {
                    Icon(
                        imageVector = iconForCategory(category.id),
                        contentDescription = null,
                        tint = MaterialTheme.colorScheme.primary,
                    )
                }
            }
        }
        Spacer(modifier = Modifier.size(12.dp))
        Column(modifier = Modifier.weight(1f)) {
            Row(
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.spacedBy(8.dp),
            ) {
                Text(
                    text = category.name,
                    // titleMedium (16sp) → titleSmall (14sp):
                    // shrunk to match the tighter card — 11 cards
                    // now occupy the screen, so the picker has to
                    // feel denser without losing legibility.
                    style = MaterialTheme.typography.titleSmall,
                    fontWeight = FontWeight.SemiBold,
                    maxLines = 1,
                    overflow = TextOverflow.Ellipsis,
                )
                Surface(
                    shape = RoundedCornerShape(8.dp),
                    // Gray-tinted background to match the new
                    // gray text — a purple background would look
                    // off with a gray foreground. Same #666666
                    // as the vocabulary translation, so the
                    // "secondary text" color is consistent
                    // across both screens.
                    color = Color(0xFF666666).copy(alpha = 0.12f),
                ) {

```

```

        Text(
            text = "${category.size} 词",
            style = MaterialTheme.typography.labelSmall,
            // § 12.31c (cross-screen): primary
            // purple → #666666 gray. Purple used to
            // compete with the category name and the
            // leading icon; gray reads as "data" and
            // lets the category title stay the
            // focal point.
            color = Color(0xFF666666),
            modifier = Modifier.padding(horizontal = 6.dp, vertical = 1.dp),
        )
    }
}
if (category.description.isNotBlank()) {
    Spacer(modifier = Modifier.height(2.dp))
    Text(
        text = category.description,
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
        maxLines = 2,
        overflow = TextOverflow.Ellipsis,
    )
}
// Per-card study sub-label — replaced the static
// description's "X 词" reading with a real progress
// line so the user can see the system IS tracking
// their sessions. Without this the picker always
// showed "未开始学习" / 0 / 0 which felt broken.
val studied = (progress?.knownCount ?: 0) + (progress?.unknownCount ?: 0)
if (studied > 0 || progress != null) {
    Spacer(modifier = Modifier.height(4.dp))
    Text(
        text = progressSubline(progress, category.size),
        style = MaterialTheme.typography.labelMedium,
        // § 12.31c (cross-screen): primary purple →
        // #666666 gray, matching the vocabulary
        // translation and the "X 词" chip above. The
        // progress line is data, not action — gray
        // keeps it visually subordinate to the
        // category name.
        color = Color(0xFF666666),
        maxLines = 1,
        overflow = TextOverflow.Ellipsis,
    )
}
}
Spacer(modifier = Modifier.size(8.dp))
Icon(
    imageVector = Icons.Filled.ArrowForward,
    contentDescription = null,
    tint = MaterialTheme.colorScheme.onSurfaceVariant,
)
}
}
}

/**
 * Render the small primary-tinted line under each category card.
 *
 * - No progress row → "未开始学习"
 * - Otherwise → "已学 N / total 词 • 认识 K • 不认识 U" with an
 * optional " • 上次学习于 X 分钟前" suffix when the timestamp

```

```

*   is recent enough to be meaningful (>= 1 min ago).
*
* Kept in one place so the format is consistent across the 5 cards
* and easy to tweak later (e.g. switch to "X% 已完成" once the
* dataset is large enough that the raw count looks noisy).
*/
private fun progressSubline(
    progress: ReciteProgressEntity?,
    total: Int,
): String {
    if (progress == null) return "未开始学习"
    val studied = progress.knownCount + progress.unknownCount
    if (studied == 0) return "未开始学习"
    val base = "已学 $studied / $total 词 • 认识 ${progress.knownCount} • 不认识 ${progress.unknownCount}"
    val last = relativeTimeAgo(progress.lastStudiedAt)
    return if (last.isNotEmpty()) "$base • $last" else base
}

/**
 * Compact relative-time string for the "last studied" suffix.
 * Returns empty string when the timestamp is missing or in the
 * future (clock skew after restoring a backup) — the caller then
 * omits the suffix.
 */
private fun relativeTimeAgo(epochMs: Long): String {
    if (epochMs <= 0L) return ""
    val deltaMin = (System.currentTimeMillis() - epochMs) / 60_000L
    return when {
        deltaMin < 0L -> "" // clock skew
        deltaMin < 1L -> "刚刚学习"
        deltaMin < 60L -> "上次学习于 $deltaMin 分钟前"
        deltaMin < 24 * 60L -> "上次学习于 ${deltaMin / 60} 小时前"
        else -> "上次学习于 ${deltaMin / (24 * 60)} 天前"
    }
}

private fun iconForCategory(id: String): ImageVector = when (id) {
    // 11-category expansion (2026-07-03) — vocab cards now span
    // primary 小学 → cet8 → 考研 → toefl → gre / ielts / bec. Icons only
    // use symbols confirmed resolvable in this project's
    // `material-icons-core` set (Favorite / MilitaryTech / Language /
    // BusinessCenter are unresolved in 1.5.4 — fall back to a smaller
    // palette of proven icons and accept a few visual duplicates).
    // Unknown ids still fall back to MenuBook.
    "primary" -> Icons.Default.Star // 小学 = 星 = 童趣
    "junior" -> Icons.Default.School
    "senior" -> Icons.Default.MenuBook
    "cet4" -> Icons.Default.MenuBook
    "cet6" -> Icons.Default.AutoStories
    "cet8" -> Icons.Default.AutoStories
    "kaoyan" -> Icons.Default.School
    "toefl" -> Icons.Default.Public
    "gre" -> Icons.Default.Public
    "ielts" -> Icons.Default.Translate
    "bec" -> Icons.Default.Check
    else -> Icons.Default.MenuBook
}

```

```
// === app\src\main\java\com\echoling\app\presentation\ui\screens\recite\ReciteViewModel.kt ===
```

```
package com.echoling.app.presentation.ui.screens.recite
```

```

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.echoling.app.data.local.db.entity.ReciteProgressEntity
import com.echoling.app.domain.model.DictCategory
import com.echoling.app.domain.usecase.GetDictionaryCategoriesUseCase
import com.echoling.app.domain.usecase.ObserveAllReciteProgressUseCase
import com.echoling.app.domain.usecase.WarmupDictionaryUseCase
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import javax.inject.Inject

/**
 * UI state for the "记单词" tab landing.
 *
 * - [categories] — ordered list of bundled vocabulary categories
 *   (id, name, description, entries). Renders one card per entry.
 * - [progressByCategory] — `categoryId → progress row` map of every
 *   persisted study session. Drives the per-card sub-label that
 *   shows "已学 27 / 2340" and "上次学习于..." — without this the
 *   picker always reads "已学 0 / N" which made the user feel the
 *   app wasn't tracking anything. Keyed by category id so the
 *   `ReciteScreen` does a single `categories.associate { ... }` to
 *   pair manifest data with persisted progress.
 * - [isLoading] — true until the first load completes; the screen
 *   shows a centered [CircularProgressIndicator] while loading.
 */
data class ReciteUiState(
    val categories: List<DictCategory> = emptyList(),
    val progressByCategory: Map<String, ReciteProgressEntity> = emptyMap(),
    val isLoading: Boolean = true,
)

/**
 * State holder for the "记单词" tab landing.
 *
 * Loads the manifest's category list once on construction and
 * subscribes to the `recite_progress` table via
 * [ObserveAllReciteProgressUseCase] — every time the user marks a
 * card on a sub-page, the corresponding row is upserted and this
 * flow re-emits, refreshing the picker's sub-labels live.
 *
 * The picker never has to know the DAO/UseCase split — it just
 * reads `uiState.progressByCategory[cat.id]`. When the row is null
 * the card renders "未开始学习"; otherwise it renders the persisted
 * counts and the relative "last studied" timestamp.
 */
@HiltViewModel
class ReciteViewModel @Inject constructor(
    private val getDictionaryCategories: GetDictionaryCategoriesUseCase,
    private val warmupDictionary: WarmupDictionaryUseCase,
    private val observeAllReciteProgress: ObserveAllReciteProgressUseCase,
) : ViewModel() {

    private val _uiState = MutableStateFlow(ReciteUiState())
    val uiState: StateFlow<ReciteUiState> = _uiState.asStateFlow()

    init {
        loadCategories()
        observeProgress()
    }

```

```

    }

    private fun loadCategories() {
        viewModelScope.launch {
            val cats = getDictionaryCategories()
            _uiState.value = _uiState.value.copy(
                categories = cats,
                isLoading = false,
            )
            // After Phase 1 (manifest) is rendered, kick off Phase 3
            // (full lookup build) on a background coroutine so the user
            // — who is currently staring at the picker — doesn't pay
            // the 12 MB parse cost. By the time they tap into a
            // category, the per-category cache is already populated.
            // See `DictionaryRepositoryImpl.warmupAll`.
            warmupDictionary()
        }
    }

    private fun observeProgress() {
        viewModelScope.launch {
            observeAllReciteProgress().collect { rows ->
                _uiState.value = _uiState.value.copy(
                    progressByCategory = rows.associateBy { it.categoryId },
                )
            }
        }
    }
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\splash\SplashScreen.kt ===

package com.echoling.app.presentation.ui.screens.splash

import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import com.echoling.app.R

/**
 * Cold-start splash page. Painted before the rest of the app graph is
 * composed so the user always sees a deliberate hero image for ~1s, then
 * the navigation host takes over. The image already contains its own
 * caption ("不必急于求成，每天进步一点"), so we just paint it once.
 *
 * The 1-second timing itself is enforced by
 * [com.echoling.app.presentation.ui.navigation.MainScaffold] — this
 * composable just renders the image.
 *
 * Sizing strategy matches `splash_background.xml` (the launcher theme's
 * `windowBackground`) so the pre-Compose frame and the Compose splash
 * are visually identical:
 * - The light-blue `Box` background matches the splash image's base
 *   color, so any letterbox margin blends in seamlessly.
 * - The image uses `ContentScale.Fit` (preserve aspect ratio, show

```



```

* full image, possibly with margins) instead of `Crop` so the
* figure and caption never get clipped on phones whose aspect
* ratio differs from 9:16. Centering it with `Alignment.Center`
* keeps the caption where the eye expects it.
*
* The `matchParentSize` modifier makes the Image obey the Box's
* intrinsic 1080×1920 aspect ratio while still respecting its layout
* constraints — same visual outcome as `gravity="center"` in the
* layer-list XML, just in Compose.
*/
@Composable
fun SplashScreen(modifier: Modifier = Modifier) {
    Box(
        modifier = modifier
            .fillMaxSize()
            .background(androidx.compose.ui.graphics.Color(0xFF9C84C2)),
        contentAlignment = Alignment.Center,
    ) {
        Image(
            painter = painterResource(R.drawable.splash_image),
            contentDescription = null,
            // Use the image's natural intrinsic size, then Fit to scale it
            // down only if the box is smaller than the image. Combined
            // with the centered Box, this guarantees no distortion and
            // shows the full image with margin on non-9:16 phones.
            contentScale = ContentScale.Fit,
            modifier = Modifier.fillMaxSize(),
        )
    }
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\statistics\StatisticsScreen.kt ===

package com.echoling.app.presentation.ui.screens.statistics

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyRow
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.ArrowBack
import androidx.compose.material.icons.filled.LocalFireDepartment
import androidx.compose.material.icons.filled.MenuBook
import androidx.compose.material.icons.filled.Schedule
import androidx.compose.material.icons.filled.School
import androidx.compose.material.icons.filled.Star
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.hilt.navigation.compose.hiltViewModel

```

```

import com.echoling.app.presentation.ui.components.PageHeader
import com.echoling.app.presentation.viewmodel.DailyStatistic
import com.echoling.app.presentation.viewmodel.StatisticsViewModel

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun StatisticsScreen(
    onNavigateBack: () -> Unit,
    viewModel: StatisticsViewModel = hiltViewModel()
) {
    val uiState by viewModel.uiState.collectAsState()

    Scaffold(
        // No topBar — back + title in PageHeader (§ 12.18)
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding),
        ) {
            PageHeader(
                onBack = onNavigateBack,
                // § 12.21: left-align the sub-page title (the brand-
                // bar tabs Courses / Me stay centered by default).
                titleAlignment = Alignment.Start,
                title = {
                    Text(
                        text = "Learning Statistics",
                        style = MaterialTheme.typography.titleMedium,
                        fontWeight = FontWeight.SemiBold,
                    )
                },
            )
            if (uiState.isLoading) {
                Box(
                    modifier = Modifier.fillMaxSize(),
                    contentAlignment = Alignment.Center,
                ) {
                    CircularProgressIndicator()
                }
            } else {
                Column(
                    modifier = Modifier
                        .fillMaxSize()
                        // § 12.21: verticalScroll so the user can
                        // see the 7-day chart (and the rest of the
                        // content) on smaller phones where the
                        // 4 stat cards + streak + 30-day heatmap
                        // already exceed the first screen.
                        .verticalScroll(rememberScrollState())
                        .padding(horizontal = 16.dp, vertical = 8.dp),
                    verticalArrangement = Arrangement.spacedBy(12.dp),
                ) {
                    // Streak card
                    StreakCard(streak = uiState.currentStreak)

                    // Stats grid
                    Row(
                        modifier = Modifier.fillMaxWidth(),
                        horizontalArrangement = Arrangement.spacedBy(12.dp)
                    ) {
                        StatCard(
                            modifier = Modifier.weight(1f),

```

```

        icon = Icons.Default.Schedule,
        value = formatDuration(uiState.totalLearnTimeMs),
        label = "Total Time"
    )
    StatCard(
        modifier = Modifier.weight(1f),
        icon = Icons.Default.School,
        value = uiState.totalCoursesLearned.toString(),
        label = "Courses"
    )
}

Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.spacedBy(12.dp)
) {
    StatCard(
        modifier = Modifier.weight(1f),
        icon = Icons.Default.MenuBook,
        value = uiState.totalWordsCollected.toString(),
        label = "Words"
    )
    StatCard(
        modifier = Modifier.weight(1f),
        icon = Icons.Default.Star,
        value = uiState.totalWordsMastered.toString(),
        label = "Mastered"
    )
}

// (2026-07-04) Re-ordered: weekly 7-day bar chart now
// sits ABOVE the 30-day heatmap so the user sees the
// most-recent / most-actionable view first ("how did
// this week go?"), then the longer context below
// ("how did the month look?"). Previously the 30-day
// heatmap was on top per § 12.21's "lands within first
// screen" rationale — that priority no longer wins
// over the recency-first reading order.
WeeklyActivityChart(dailyStats = uiState.dailyStats)

// § 12.21: monthly activity (last 30 days) — rendered
// as a calendar heatmap (GitHub-style). Now below the
// 7-day bar chart; the heatmap is the longer context
// view, and the bars give the precise per-day numbers.
MonthlyActivityChart(monthlyStats = uiState.monthlyStats)
}
}
}

@Composable
private fun StreakCard(streak: Int) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.primaryContainer
        )
    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 16.dp, vertical = 12.dp),

```

```

        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(
            imageVector = Icons.Default.LocalFireDepartment,
            contentDescription = null,
            modifier = Modifier.size(48.dp),
            tint = if (streak > 0)
                MaterialTheme.colorScheme.primary
            else
                MaterialTheme.colorScheme.onSurfaceVariant
        )
        Spacer(modifier = Modifier.width(16.dp))
        Column {
            Text(
                text = "$streak day${if (streak != 1) "s" else ""}",
                // (2026-07-04) was headlineMedium (28sp) — too
                // dominant against the 48dp LocalFireDepartment
                // icon. Dropped one step to headlineSmall (24sp)
                // so the streak number shares visual weight with
                // the icon instead of dwarfing it.
                style = MaterialTheme.typography.headlineSmall,
                color = MaterialTheme.colorScheme.onPrimaryContainer
            )
            Text(
                text = "Current streak",
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.7f)
            )
        }
    }
}

@Composable
private fun StatCard(
    modifier: Modifier = Modifier,
    icon: ImageVector,
    value: String,
    label: String
) {
    Card(
        modifier = modifier,
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surfaceVariant
        )
    ) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(horizontal = 12.dp, vertical = 10.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Icon(
                imageVector = icon,
                contentDescription = null,
                modifier = Modifier.size(24.dp),
                tint = MaterialTheme.colorScheme.primary
            )
            Spacer(modifier = Modifier.height(6.dp))
            Text(
                text = value,
                // (2026-07-04) was headlineSmall (24sp) — the four
                // stat cards' big numbers dominated the page. Dropped

```

```

        // one step to titleLarge (22sp) so each value still
        // reads as the focal point but doesn't overpower the
        // bodySmall (12sp) label underneath.
        style = MaterialTheme.typography.titleLarge
    )
    Text(
        text = label,
        style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant
    )
}
}
}

@Composable
private fun WeeklyActivityChart(dailyStats: List<DailyStatistic>) {
    Card(
        modifier = Modifier.fillMaxWidth()
    ) {
        Column(
            modifier = Modifier.padding(horizontal = 14.dp, vertical = 10.dp)
        ) {
            Text(
                text = "Last 7 Days",
                style = MaterialTheme.typography.titleMedium
            )
            Spacer(modifier = Modifier.height(10.dp))

            LazyRow(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                items(dailyStats) { stat ->
                    DayStatItem(stat = stat, maxTimeMs = dailyStats.maxOfOrNull { it.learnTimeMs } ?: 1L)
                }
            }
        }
    }
}

/**
 * 30-day activity rendered as a calendar heatmap (GitHub-style).
 *
 * Why heatmap instead of bars:
 * - 30 thin bars (the old design) were 6dp wide — at any normal
 *   screen density that's barely a vertical line, so day-to-day
 *   variation was invisible. A heatmap gives each day a real cell
 *   (~32dp) with both a day-of-month number and a color, which makes
 *   the data scannable at a glance.
 * - 30 days don't divide evenly into 7-day weeks, but the user
 *   thinks in weeks. We pad the first row with empty cells so the
 *   first active day lands in its correct Mon-Sun column, then fill
 *   the rest of the grid. The user sees a proper week layout, with
 *   the 30-day window "snapped" to the calendar.
 * - Color intensity (5 levels) is denser information than bar
 *   height — you can read both "how much" and "which days were
 *   active" from a single glance.
 *
 * Today is highlighted with a 2dp primary border so the user can
 * find the right end of the timeline immediately.
 */
@Composable
private fun MonthlyActivityChart(monthlyStats: List<DailyStatistic>) {

```

```

val totalMs = monthlyStats.sumOf { it.learnTimeMs }
val activeDays = monthlyStats.count { it.learnTimeMs > 0 }
val maxTimeMs = monthlyStats.maxOrNull { it.learnTimeMs } ?: 1L

// Compute the 5-week × 7-day grid (35 cells, 4.5 of which may
// be empty padding on the first or last row depending on where
// "30 days back" lands in its week). [remember] keyed on the
// stats identity so it doesn't recompute on every recomposition.
val grid = remember(monthlyStats) { buildMonthlyHeatGrid(monthlyStats) }

Card(modifier = Modifier.fillMaxWidth()) {
    Column(modifier = Modifier.padding(horizontal = 14.dp, vertical = 10.dp)) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically,
        ) {
            Text(
                text = "Last 30 Days",
                style = MaterialTheme.typography.titleMedium,
            )
            Text(
                text = "$activeDays 天 • ${formatDuration(totalMs)}",
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
            )
        }
        Spacer(modifier = Modifier.height(8.dp))

        // Day-of-week header row — Chinese "一 二 三 四 五 六 日" so the
        // first column is Monday, matching the calendar
        // convention in the app's primary locale.
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(4.dp),
        ) {
            listOf("一", "二", "三", "四", "五", "六", "日").forEach { d ->
                Text(
                    text = d,
                    modifier = Modifier.weight(1f),
                    textAlign = androidx.compose.ui.text.style.TextAlign.Center,
                    style = MaterialTheme.typography.labelSmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant,
                )
            }
        }
        Spacer(modifier = Modifier.height(6.dp))

        // 5 weeks of 7 days. Each cell is `weight(1f)` so they
        // divide the row width evenly, and `aspectRatio(1f)` so
        // they stay square. The 4dp spacedBy matches the header
        // row's spacing so the columns line up vertically.
        grid.forEach { row ->
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.spacedBy(4.dp),
            ) {
                row.forEach { cell ->
                    Box(modifier = Modifier.weight(1f).aspectRatio(1f)) {
                        if (cell != null) {
                            MonthlyHeatCell(
                                dayOfMonth = cell.dayOfMonth,
                                learnTimeMs = cell.learnTimeMs,

```

```

        maxTimeMs = maxTimeMs,
        isToday = cell.isToday,
    )
}
// Empty cells (padding at the start of
// row 1, or the end of the last row)
// render as nothing — the parent Box
// takes the same space so the grid stays
// aligned.
}
}
}
Spacer(modifier = Modifier.height(4.dp))
}

// Color legend — explains the 5-level intensity scale
// (None / Low / Medium / High / Max) so the user knows
// that darker = more study time.
Spacer(modifier = Modifier.height(6.dp))
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.End,
    verticalAlignment = Alignment.CenterVertically,
) {
    Text(
        text = "少",
        style = MaterialTheme.typography.labelSmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
    )
    Spacer(modifier = Modifier.width(6.dp))
    listOf(0f, 0.25f, 0.5f, 0.75f, 1f).forEach { alpha ->
        Box(
            modifier = Modifier
                .padding(horizontal = 1.5.dp)
                .size(12.dp)
                .background(
                    color = if (alpha == 0f) {
                        MaterialTheme.colorScheme.surfaceVariant
                    } else {
                        MaterialTheme.colorScheme.primary.copy(alpha = alpha)
                    },
                    shape = RoundedCornerShape(3.dp),
                ),
        )
    }
    Spacer(modifier = Modifier.width(6.dp))
    Text(
        text = "多",
        style = MaterialTheme.typography.labelSmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
    )
}
}
}
}

/**
 * Single cell of the 30-day heatmap. Cell color encodes the
 * learn-time intensity; the day-of-month number sits centered inside
 * so the user can read both at once.
 *
 * Today is rendered specially so the user can spot it at a glance:
 * the rest of the grid is intensity-based, but today gets a fixed

```

```

* filled primary background, a larger bold white number, and a small
* "今天" label below the number. This makes "where am I on this
* timeline?" answerable in one glance — the user does not have to
* compare each cell's day number against today's date in their head,
* and it stays unambiguous even when a day number (e.g. "1") appears
* more than once across a month boundary.
*/
@Composable
private fun MonthlyHeatCell(
    dayOfMonth: Int,
    learnTimeMs: Long,
    maxTimeMs: Long,
    isToday: Boolean,
) {
    val ratio = if (learnTimeMs > 0 && maxTimeMs > 0) {
        (learnTimeMs.toFloat() / maxTimeMs).coerceIn(0f, 1f)
    } else 0f

    // Today uses a soft primaryContainer fill + larger bold dark-violet
    // number, regardless of learn time. This makes today easy to find
    // and removes the need to interpret color intensity on that cell,
    // while keeping visual weight in the same family as the rest of
    // the grid (violet-200 instead of full primary).
    // Other cells: gradient intensity by learn-time ratio.
    val fillColor = if (isToday) {
        MaterialTheme.colorScheme.primaryContainer
    } else when {
        ratio == 0f -> MaterialTheme.colorScheme.surfaceVariant
        ratio < 0.25f -> MaterialTheme.colorScheme.primary.copy(alpha = 0.25f)
        ratio < 0.5f -> MaterialTheme.colorScheme.primary.copy(alpha = 0.5f)
        ratio < 0.75f -> MaterialTheme.colorScheme.primary.copy(alpha = 0.75f)
        else -> MaterialTheme.colorScheme.primary
    }

    // Today is dark-violet-on-light-violet (primaryContainer / onPrimaryContainer
    // pair from §11.2 brand palette). For other cells, white text
    // on the two darkest intensity levels for contrast.
    val numberColor = if (isToday) {
        MaterialTheme.colorScheme.onPrimaryContainer
    } else when {
        ratio >= 0.5f -> MaterialTheme.colorScheme.onPrimary
        else -> MaterialTheme.colorScheme.onSurface
    }

    // Bump today's number up to titleSmall (14sp) Bold; regular
    // cells use labelMedium (12sp) — both larger than the previous
    // labelSmall (11sp) so the day numbers are easy to read in a
    // ~43dp cell.
    val numberStyle = if (isToday) {
        MaterialTheme.typography.titleSmall
    } else {
        MaterialTheme.typography.labelMedium
    }
    val numberFontWeight = if (isToday) FontWeight.Bold else FontWeight.Normal

    Box(
        modifier = Modifier
            .fillMaxSize()
            .clip(RoundedCornerShape(6.dp))
            .background(fillColor),
        contentAlignment = Alignment.Center,
    ) {
        if (isToday) {

```



```

        // Today: number + "今天" label, both centered. The label
        // is the unambiguous "you are here" marker — even if the
        // user can read the day number, the text spells it out
        // so there is zero ambiguity.
        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center,
        ) {
            Text(
                text = dayOfMonth.toString(),
                style = numberStyle,
                color = numberColor,
                fontWeight = numberFontWeight,
            )
            Text(
                text = "今天",
                style = MaterialTheme.typography.labelSmall,
                color = numberColor.copy(alpha = 0.85f),
            )
        }
    } else {
        Text(
            text = dayOfMonth.toString(),
            style = numberStyle,
            color = numberColor,
            fontWeight = numberFontWeight,
        )
    }
}

/**
 * Convert the 30-entry [monthlyStats] (index 0 = today) into a
 * 5-week × 7-day grid of cells. The first row is padded with `null`
 * entries so the first active day lands in the correct Mon-Sun
 * column based on its actual day-of-week. The last row may also be
 * padded if the 30-day window doesn't reach the end of a week.
 *
 * Day-of-month numbers are computed from the index (0 = today, i =
 * today minus i days) so the cell text reads naturally ("22" for
 * "30 days ago" if today is the 21st).
 */
private data class HeatCell(
    val dayOfMonth: Int,
    val learnTimeMs: Long,
    val isToday: Boolean,
)

private fun buildMonthlyHeatGrid(
    monthlyStats: List<DailyStatistic>,
    todayMillis: Long = System.currentTimeMillis(),
): List<List<HeatCell?>> {
    if (monthlyStats.isEmpty()) return emptyList()

    // Day-of-week of "30 days ago" (the last item in the stats list).
    // We convert Calendar.SUNDAY=1..SATURDAY=7 to Mon=0..Sun=6.
    val firstStatCal = java.util.Calendar.getInstance().apply {
        timeInMillis = todayMillis
        add(java.util.Calendar.DAY_OF_YEAR, -(monthlyStats.size - 1))
    }
    val calDow = firstStatCal.get(java.util.Calendar.DAY_OF_WEEK)
    val firstCol = ((calDow - java.util.Calendar.MONDAY + 7) % 7)

```

```

// Build the cells list, then chunk into rows of 7.
val cells = mutableListOf<HeatCell?>()
repeat(firstCol) { cells.add(null) }

// Iterate from oldest (index = size-1) to newest (index = 0) so
// the cells land in chronological order. The stats list from
// the ViewModel is indexed 0=today, ..., 29=29-days-ago — if we
// iterated it in natural order, today would land in the column
// of "29 days ago" instead of its actual day-of-week column,
// making "21" appear in the Saturday slot when today is actually
// a Sunday. Reversing the iteration puts the oldest day in the
// first available cell (after the leading padding) and today at
// the bottom-right of the grid — the "end of the timeline"
// position, which matches the calendar's left-to-right,
// top-to-bottom reading order.
monthlyStats.indices.reversed().forEach { index ->
    val stat = monthlyStats[index]
    val cellCal = java.util.Calendar.getInstance().apply {
        timeInMillis = todayMillis
        add(java.util.Calendar.DAY_OF_YEAR, -index)
    }
    cells.add(
        HeatCell(
            dayOfMonth = cellCal.get(java.util.Calendar.DAY_OF_MONTH),
            learnTimeMs = stat.learnTimeMs,
            isToday = index == 0,
        )
    )
}

// Pad the tail so the grid is exactly 5 rows × 7 cols.
while (cells.size < 35) { cells.add(null) }

return cells.chunked(7)
}

@Composable
private fun DayStatItem(stat: DailyStatistic, maxTimeMs: Long) {
    val maxHeight = 80.dp
    val barHeight = if (stat.learnTimeMs > 0) {
        val ratio = (stat.learnTimeMs.toFloat() / maxTimeMs.coerceAtLeast(1)).coerceIn(0.1f, 1f)
        maxHeight * ratio
    } else {
        4.dp
    }

    Column(
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = stat.date,
            style = MaterialTheme.typography.labelSmall
        )
        Spacer(modifier = Modifier.height(4.dp))
        Box(
            modifier = Modifier
                .width(32.dp)
                .height(maxHeight),
            contentAlignment = Alignment.BottomCenter
        ) {
            Surface(
                modifier = Modifier
                    .fillMaxWidth()

```

```

        .height(barHeight),
        shape = MaterialTheme.shapes.small,
        color = if (stat.learnTimeMs > 0)
            MaterialTheme.colorScheme.primary
        else
            MaterialTheme.colorScheme.surfaceVariant
    ) {}
}
Spacer(modifier = Modifier.height(4.dp))
Text(
    text = formatShortDuration(stat.learnTimeMs),
    style = MaterialTheme.typography.labelSmall,
    color = MaterialTheme.colorScheme.onSurfaceVariant
)
}
}

private fun formatDuration(ms: Long): String {
    if (ms == 0L) return "0m"
    val totalSeconds = ms / 1000
    val hours = totalSeconds / 3600
    val minutes = (totalSeconds % 3600) / 60
    return if (hours > 0) "${hours}h ${minutes}m" else "${minutes}m"
}

private fun formatShortDuration(ms: Long): String {
    val totalMinutes = ms / 60000
    return if (totalMinutes >= 60) "${totalMinutes / 60}h" else "${totalMinutes}m"
}

// === app\src\main\java\com\echoling\app\presentation\ui\screens\vocabulary\VocabularyScreen.kt ===

package com.echoling.app.presentation.ui.screens.vocabulary

import androidx.compose.animation.core.Animatable
import androidx.compose.animation.core.spring
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.gestures.Orientation
import androidx.compose.foundation.gestures.draggable
import androidx.compose.foundation.gestures.rememberDraggableState
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.itemsIndexed
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.VolumeUp
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.Divider
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.SnackbarDuration
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable

```

```

import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalDensity
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontStyle
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.IntOffset
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.hilt.navigation.compose.hiltViewModel
import com.echoling.app.R
import com.echoling.app.domain.model.Word
import com.echoling.app.presentation.ui.components.PageHeader
import com.echoling.app.presentation.viewmodel.VocabularyViewModel
import kotlinx.coroutines.launch
import kotlin.math.roundToInt

/**
 * Vocabulary book — the user's saved words, one per row.
 *
 * § 12.31 layout (beautified 2026-07-03):
 *   Top row:   word (18sp Bold) | phonetic (14sp) | ☐
 *   Mid row:   POS chip + translation (bodyMedium primary)
 *   Last row:  example sentence (bodySmall muted, 1 line) — optional
 *   Bottom:    0.5dp HorizontalDivider (omitted on the last row)
 *
 * Interactions:
 *   - Tap row → pronounce via TTS
 *   - Tap ☐ → pronounce via TTS (same action, visible affordance)
 *   - Swipe left (right→left) → reveal red [删除] background + button.
 *     The row STAYS swiped open (custom SwipeRevealRow with
 *     [Animatable] offset; does NOT auto-dismiss).
 *   - Tap the [删除] button → onDelete(); row disappears.
 *   - Drag the row back to the right past the midpoint → snaps
 *     closed (no commit).
 *
 * Removed in this revision (per user request 2026-07-03):
 *   - The ☐ "Mark mastered" IconButton and its callback chain
 *     (WordRow.onToggleMastered → WordList.onToggleMastered →
 *     VocabularyViewModel.toggleMastered → ToggleWordMasteredUseCase).
 *   - The "全部 / 未掌握" filter dropdown in the page header
 *     (PageHeader actions slot, showFilterMenu state, setShowMastered
 *     method on the ViewModel).
 *   - The domain use case `ToggleWordMasteredUseCase.kt` was deleted
 *     from the codebase (no other callers after the UI was removed).
 *   - The [Word.isMastered] data field is PRESERVED for Room schema
 *     backwards-compatible — already-mastered words still display in the
 *     list, they just can't be toggled from this UI.
 */
@Composable
fun VocabularyScreen(
    onNavigateBack: (() -> Unit)? = null,
    viewModel: VocabularyViewModel = hiltViewModel()
) {
    val uiState by viewModel.uiState.collectAsState()

```

```

val snackbarHostState = remember { SnackbarHostState() }

// Surface TTS unavailability as a snackbar. Long duration so the
// user has time to read the install instructions.
LaunchedEffect(uiState.ttsUnavailableMessage) {
    val msg = uiState.ttsUnavailableMessage
    if (msg != null) {
        snackbarHostState.showSnackbar(msg, duration = SnackbarDuration.Long)
        viewModel.consumeTtsUnavailableMessage()
    }
}

Scaffold(
    snackbarHost = { SnackbarHost(hostState = snackbarHostState) },
    // §12.30: only consume the top status-bar inset, not the bottom
    // navigation-bar inset. The outer MainScaffold already accounts
    // for the bottom tab bar, so a second nav-bar subtraction here
    // would leave a 24dp page-colored strip between the tab bar
    // and the last list item. See CLAUDE.md §12.30.
    contentWindowInsets = WindowInsets.statusBars,
    // No topBar — title sits in PageHeader (§12.18, §12.31)
) { padding ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(padding),
    ) {
        PageHeader(
            onBack = onNavigateBack,
            title = {
                // Two-line brand bar matching the home page (Courses)
                // and Me tabs — Chinese title with 3sp letter-spacing
                // so the three characters don't visually crowd, plus
                // an italic English subtitle below in onSurfaceVariant.
                Column(horizontalAlignment = Alignment.CenterHorizontally) {
                    Text(
                        text = stringResource(R.string.vocabulary_title),
                        style = TextStyle(
                            fontSize = 18.sp,
                            fontWeight = FontWeight.Bold,
                            letterSpacing = 3.sp,
                        ),
                        maxLines = 1,
                    )
                    Text(
                        text = stringResource(R.string.vocabulary_subtitle),
                        style = TextStyle(
                            fontSize = 11.sp,
                            fontStyle = FontStyle.Italic,
                            fontWeight = FontWeight.Medium,
                            letterSpacing = 1.sp,
                        ),
                        color = MaterialTheme.colorScheme.onSurfaceVariant,
                        maxLines = 1,
                    )
                }
            },
            // §12.31: filter dropdown removed (the only path into
            // `setShowMastered` was this menu; with ☐ gone there's
            // no new way to mark words as mastered, so the filter
            // has no purpose). PageHeader has no actions slot now.
        )
        Box(modifier = Modifier.fillMaxSize()) {

```

```

        when {
            uiState.isLoading -> {
                CircularProgressIndicator(
                    modifier = Modifier.align(Alignment.Center)
                )
            }
            uiState.words.isEmpty() -> {
                EmptyVocabulary()
            }
            else -> {
                WordList(
                    words = uiState.words,
                    onDelete = { viewModel.deleteWord(it) },
                    onPronounce = { viewModel.pronounce(it) },
                )
            }
        }
    }
}

@Composable
private fun EmptyVocabulary() {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        Column(
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                text = "单词本还是空的",
                style = MaterialTheme.typography.titleMedium
            )
            Spacer(modifier = Modifier.height(8.dp))
            Text(
                text = "在跟读练习中长按单词，或在学习页点击「不认识 → 加入单词本」来保存",
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant,
                textAlign = androidx.compose.ui.text.style.TextAlign.Center,
                modifier = Modifier.padding(horizontal = 32.dp),
            )
        }
    }
}

@Composable
private fun WordList(
    words: List<Word>,
    onDelete: (Word) -> Unit,
    onPronounce: (Word) -> Unit,
) {
    LazyColumn(
        modifier = Modifier.fillMaxSize(),
        // 4dp top + 0 bottom: the first row's own 12dp vertical
        // padding already gives breathing room from the PageHeader;
        // 0 bottom so the last row sits flush with the bottom
        // edge (the divider is omitted on the last row anyway).
        contentPadding = PaddingValues(vertical = 4.dp),
    ) {
        itemsIndexed(words, key = { _, w -> w.word }) { index, word ->
            WordRow(

```

```

        word = word,
        isLast = index == words.lastIndex,
        onDelete = { onDelete(word) },
        onPronounce = { onPronounce(word) },
    )
}
}

@Composable
private fun WordRow(
    word: Word,
    isLast: Boolean,
    onDelete: () -> Unit,
    onPronounce: () -> Unit,
) {
    SwipeRevealRow(onDelete = onDelete) {
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .background(MaterialTheme.colorScheme.background)
                // Whole row is tappable. The inner IconButton still
                // receives its own click — M3's IconButton routes the
                // click before the parent's clickable in the gesture
                // path, so tapping ☐ speaks without also triggering
                // a duplicate TTS call from the parent.
                .clickable(onClick = onPronounce)
                // § 12.31b: 12 → 8dp vertical padding — tighten the
                // row height so more words fit per scroll, matching
                // the smaller 14sp word + 12sp translation fonts.
                .padding(horizontal = 16.dp, vertical = 8.dp),
        ) {
            // Top row: word (weight=1) | 4dp | phonetic | 8dp | ☐
            Row(
                modifier = Modifier.fillMaxWidth(),
                verticalAlignment = Alignment.CenterVertically,
            ) {
                Text(
                    text = word.word,
                    style = TextStyle(
                        fontSize = 14.sp,
                        fontWeight = FontWeight.Bold,
                    ),
                    color = MaterialTheme.colorScheme.onSurface,
                    maxLines = 1,
                    overflow = TextOverflow.Ellipsis,
                    modifier = Modifier.weight(1f),
                )
                if (word.phonetic.isNotEmpty()) {
                    // § 12.31b: 8 → 4dp — pull the phonetic flush
                    // against the word so they read as a single
                    // "abandon /☐☐b☐nd☐n/" unit. The 8dp gap before
                    // ☐ stays to keep the action visually separate
                    // from the read-only data.
                    Spacer(modifier = Modifier.width(4.dp))
                    Text(
                        text = word.phonetic,
                        style = TextStyle(
                            fontSize = 12.sp,
                        ),
                        color = MaterialTheme.colorScheme.onSurfaceVariant,
                        maxLines = 1,
                        overflow = TextOverflow.Ellipsis,
                    )
                }
            }
        }
    }
}

```

```

    )
  }
  Spacer(modifier = Modifier.width(8. dp))
  IconButton(
    onClick = onPronounce,
    modifier = Modifier.size(32. dp),
  ) {
    Icon(
      imageVector = Icons.Default.VolumeUp,
      contentDescription = "朗读单词",
      tint = MaterialTheme.colorScheme.onSurfaceVariant,
      modifier = Modifier.size(18. dp),
    )
  }
}
// Translation row. POS prefix chip rendered inline ahead
// of the translation text so the part-of-speech reads as
// a label ("vt. 抛弃") rather than getting concatenated.
Spacer(modifier = Modifier.height(2. dp))
if (word.pos.isNotBlank()) {
  Row(
    verticalAlignment = Alignment.CenterVertically,
    horizontalArrangement = Arrangement.spacedBy(8. dp),
  ) {
    Surface(
      shape = RoundedCornerShape(6. dp),
      color = MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.6f),
    ) {
      Text(
        text = word.pos,
        style = MaterialTheme.typography.labelSmall,
        color = MaterialTheme.colorScheme.onPrimaryContainer,
        modifier = Modifier.padding(horizontal = 6. dp, vertical = 1. dp),
      )
    }
    Text(
      text = word.translation,
      // § 12.31b: bodyMedium (14sp) → bodySmall
      // (12sp) — match the phonetic size for a
      // uniform secondary-text scale, and make the
      // row more compact.
      // § 12.31c: color = primary (purple) → #666666
      // medium gray. The purple used to compete with
      // the word text for attention; a softer gray
      // keeps the Chinese reading as the "payload"
      // without pulling focus from the 14sp Bold
      // English word.
      style = MaterialTheme.typography.bodySmall,
      color = Color(0xFF666666),
      maxLines = 2,
      overflow = TextOverflow.Ellipsis,
    )
  }
} else {
  Text(
    text = word.translation,
    // § 12.31b: bodyMedium (14sp) → bodySmall (12sp)
    // § 12.31c: medium gray instead of primary purple
    style = MaterialTheme.typography.bodySmall,
    color = Color(0xFF666666),
    maxLines = 2,
    overflow = TextOverflow.Ellipsis,
  )
}

```



```

    }
    // Example sentence — only when the word came from practice
    // and the user actually has an example attached. Shown in
    // full (no maxLines / no ellipsis) so the user can see the
    // whole context for the word — practice sentences are the
    // whole point of saving a word with a source link, and a
    // 1-line ellipsized version was hiding the meaningful half.
    if (word.exampleSentence.isNotEmpty()) {
        Spacer(modifier = Modifier.height(2.dp))
        Text(
            text = word.exampleSentence,
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
        )
    }
}

// § 12.31: thin gray divider between rows. Omitted on the last
// row so the list visually "ends" cleanly without a dangling
// line at the bottom of the scrollable area.
if (!isLast) {
    Divider(
        thickness = 0.5.dp,
        color = MaterialTheme.colorScheme.onSurfaceVariant.copy(alpha = 0.2f),
    )
}
}

/**
 * Custom swipe-to-reveal-a-button row.
 *
 * Why not M3 `SwipeToDismissBox`? That component auto-dismisses on
 * release (or snaps back if `confirmValueChange` returns false) —
 * neither is what we want. We want the row to STAY swiped open
 * showing the [删除] button until the user either taps the button
 * (commits the delete) or drags the row back to the right past the
 * midpoint (cancels).
 *
 * Implementation: track the foreground offset with an [Animatable]
 * driven by `Modifier.draggable`. On drag end, spring to either the
 * fully-revealed position (-maxReveal) or back to 0 depending on
 * which side of the midpoint we ended on. The red background and
 * [删除] button live behind the foreground at the right edge; they
 * become visible as the foreground slides left.
 */
@Composable
private fun SwipeRevealRow(
    onDelete: () -> Unit,
    content: @Composable () -> Unit,
) {
    val density = LocalDensity.current
    val maxRevealPx = with(density) { 96.dp.toPx() } // width of the [删除] area
    val scope = rememberCoroutineScope()
    val offsetX = remember { Animatable(0f) }

    val draggableState = rememberDraggableState { delta ->
        scope.launch {
            val target = (offsetX.value + delta).coerceIn(-maxRevealPx, 0f)
            offsetX.snapTo(target)
        }
    }
}

```

```

Box(modifier = Modifier.fillMaxWidth()) {
    // Red full-width background. Always present so the row
    // doesn't "collapse" when the foreground slides left. The
    // [删除] button is anchored to the right edge with 16dp
    // padding so it lines up with the row's right margin.
    Box(
        modifier = Modifier
            .matchParentSize()
            .background(MaterialTheme.colorScheme.errorContainer),
        contentAlignment = Alignment.CenterEnd,
    ) {
        TextButton(
            onClick = onDelete,
            modifier = Modifier.padding(horizontal = 16.dp),
        ) {
            Icon(
                imageVector = Icons.Default.Delete,
                contentDescription = null,
                tint = MaterialTheme.colorScheme.onErrorContainer,
                modifier = Modifier.size(20.dp),
            )
            Spacer(modifier = Modifier.width(4.dp))
            Text(
                text = "删除",
                color = MaterialTheme.colorScheme.onErrorContainer,
                style = MaterialTheme.typography.labelLarge,
            )
        }
    }
}
// Foreground: the actual row content. Slides left as the
// user drags right→left. Background color = page background
// so the red area is hidden by the row when offset = 0.
Box(
    modifier = Modifier
        .fillMaxWidth()
        .offset { IntOffset(offsetX.value.roundToInt(), 0) }
        .draggable(
            state = draggableState,
            orientation = Orientation.Horizontal,
            onDragStopped = { velocity ->
                scope.launch {
                    // Snap rule: if past halfway, fully open;
                    // otherwise snap back closed. The `velocity`
                    // sign determines flick direction (a quick
                    // left flick from -10px opens; a quick right
                    // flick from -80px closes).
                    val shouldOpen = offsetX.value < -maxRevealPx / 2 ||
                        velocity < -800f
                    val shouldClose = offsetX.value > -maxRevealPx / 2 ||
                        velocity > 800f
                    val target = when {
                        shouldOpen && !shouldClose -> -maxRevealPx
                        else -> 0f
                    }
                    offsetX.animateTo(
                        targetValue = target,
                        animationSpec = spring(
                            dampingRatio = 0.85f,
                            stiffness = 1200f,
                        ),
                        initialVelocity = velocity,
                    )
                }
            }
        )
}

```

```

        },
    ),
) {
    content()
}
}
}

// === app\src\main\java\com\echoling\app\presentation\ui\theme\Color.kt ===

package com.echoling.app.presentation.ui.theme

import androidx.compose.ui.graphics.Color

// Primary colors - Deep Purple (matches app icon gradient bottom: #7A2BE0)
val Primary = Color(0xFF7C3AED) // violet-600, deep vibrant purple
val OnPrimary = Color(0xFFFFFFFF)
val PrimaryContainer = Color(0xFFD66FE) // light purple tint (container for contrast)
val OnPrimaryContainer = Color(0xFF2E1065) // very dark purple text on container

// Secondary colors - deep purple, slightly different hue
val Secondary = Color(0xFF8B5CF6) // violet-500
val OnSecondary = Color(0xFFFFFFFF)
val SecondaryContainer = Color(0xFFE99FE)
val OnSecondaryContainer = Color(0xFF2E1065)

// Tertiary colors - even deeper purple
val Tertiary = Color(0xFF6D28D9) // violet-700
val OnTertiary = Color(0xFFFFFFFF)
val TertiaryContainer = Color(0xFFC4B5FD)
val OnTertiaryContainer = Color(0xFF1E147)

// Error colors
val Error = Color(0xFFB00020)
val OnError = Color(0xFFFFFFFF)
val ErrorContainer = Color(0xFFCD8DF)
val OnErrorContainer = Color(0xFF370617)

// Surface colors - Light
val Surface = Color(0xFFFFFBFE)
val OnSurface = Color(0xFF1C1B1F)
val SurfaceVariant = Color(0xFFE7E0EC)
val OnSurfaceVariant = Color(0xFF49454F)
val Outline = Color(0xFF79747E)
val OutlineVariant = Color(0xFFCAC4D0)

// Surface colors - Dark
val SurfaceDark = Color(0xFF1C1B1F)
val OnSurfaceDark = Color(0xFFE6E1E5)
val SurfaceVariantDark = Color(0xFF49454F)
val OnSurfaceVariantDark = Color(0xFFCAC4D0)
val OutlineDark = Color(0xFF938F99)
val OutlineVariantDark = Color(0xFF49454F)

// Background
val Background = Color(0xFFFFFBFE)
val OnBackground = Color(0xFF1C1B1F)
val BackgroundDark = Color(0xFF1C1B1F)
val OnBackgroundDark = Color(0xFFE6E1E5)

// Inverse

```

```
val InverseSurface = Color(0xFF313033)
val InverseOnSurface = Color(0xFFFF4EFF4)
val InversePrimary = Color(0xFFA78BFA)    // lighter purple for dark theme contrast

// Scrim
val Scrim = Color(0xFF000000)

// Seed color for dynamic theming (kept for reference; not used when dynamicColor=false)
val SeedColor = Color(0xFF7C3AED)

// === app\src\main\java\com\echoling\app\presentation\ui\theme\Theme.kt ===

package com.echoling.app.presentation.ui.theme

import android.app.Activity
import android.os.Build
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.dynamicDarkColorScheme
import androidx.compose.material3.dynamicLightColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.runtime.SideEffect
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalView
import androidx.core.view.WindowCompat

private val LightColorScheme = lightColorScheme(
    primary = Primary,
    onPrimary = OnPrimary,
    primaryContainer = PrimaryContainer,
    onPrimaryContainer = OnPrimaryContainer,
    secondary = Secondary,
    onSecondary = OnSecondary,
    secondaryContainer = SecondaryContainer,
    onSecondaryContainer = OnSecondaryContainer,
    tertiary = Tertiary,
    onTertiary = OnTertiary,
    tertiaryContainer = TertiaryContainer,
    onTertiaryContainer = OnTertiaryContainer,
    error = Error,
    onError = OnError,
    errorContainer = ErrorContainer,
    onErrorContainer = OnErrorContainer,
    background = Background,
    onBackground = OnBackground,
    surface = Surface,
    onSurface = OnSurface,
    surfaceVariant = SurfaceVariant,
    onSurfaceVariant = OnSurfaceVariant,
    outline = Outline,
    outlineVariant = OutlineVariant,
    inverseSurface = InverseSurface,
    inverseOnSurface = InverseOnSurface,
    inversePrimary = InversePrimary,
    scrim = Scrim
)

private val DarkColorScheme = darkColorScheme(
    primary = InversePrimary,
```

```

        onPrimary = OnPrimaryContainer,
        primaryContainer = Primary,
        onPrimaryContainer = OnPrimary,
        secondary = Secondary,
        onSecondary = OnSecondary,
        secondaryContainer = SecondaryContainer,
        onSecondaryContainer = OnSecondaryContainer,
        tertiary = Tertiary,
        onTertiary = OnTertiary,
        tertiaryContainer = TertiaryContainer,
        onTertiaryContainer = OnTertiaryContainer,
        error = Error,
        onError = OnError,
        errorContainer = ErrorContainer,
        onErrorContainer = OnErrorContainer,
        background = BackgroundDark,
        onBackground = OnBackgroundDark,
        surface = SurfaceDark,
        onSurface = OnSurfaceDark,
        surfaceVariant = SurfaceVariantDark,
        onSurfaceVariant = OnSurfaceVariantDark,
        outline = OutlineDark,
        outlineVariant = OutlineVariantDark,
        inverseSurface = Surface,
        inverseOnSurface = OnSurface,
        inversePrimary = Primary,
        scrim = Scrim
    )

@Composable
fun EchoLingTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Use static light-purple theme by default so the app matches the
    // app-icon gradient on all Android versions (Material You dynamic
    // colors would otherwise pick a blue tint from the user's wallpaper).
    dynamicColor: Boolean = false,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
        }
        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }

    val view = LocalView.current
    if (!view.isInEditMode) {
        SideEffect {
            val window = (view.context as Activity).window
            // (2026-06-28) Do NOT set `window.statusBarColor` here. We
            // call `enableEdgeToEdge()` in MainActivity which already
            // makes the status bar transparent and configures the
            // appropriate scrim. Re-painting it with the surface color
            // undoes the edge-to-edge setup and makes the status bar
            // look like a separate "band" at the top of the screen,
            // visually separated from the page content below — the
            // user reports the page is not "adjacent to the top of
            // the phone screen", but in fact the content IS correctly
            // positioned right below the status bar inset; it's the
            // opaque status bar background that creates the visible
            // step. Letting the bar stay transparent lets the

```

```

        // activity's Surface (drawn by `setContent`) fill the
        // status-bar area too, so the user sees one continuous
        // color from the very top of the screen down to the
        // body content, with the system icons floating on top.
        //
        // We still configure the icon tint (dark icons on light
        // backgrounds, light icons on dark backgrounds) — that's
        // the only thing that's safe to set after enableEdgeToEdge.
        WindowCompat.getInsetsController(window, view).isAppearanceLightStatusBars = !darkTheme
    }
}

MaterialTheme(
    colorScheme = colorScheme,
    typography = Typography,
    content = content
)
}

```

```
// === app\src\main\java\com\echoling\app\presentation\ui\theme\Type.kt ===
```

```

package com.echoling.app.presentation.ui.theme

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp

val Typography = Typography(
    // Display styles
    displayLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 57.sp,
        lineHeight = 64.sp,
        letterSpacing = (-0.25).sp
    ),
    displayMedium = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 45.sp,
        lineHeight = 52.sp,
        letterSpacing = 0.sp
    ),
    displaySmall = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 36.sp,
        lineHeight = 44.sp,
        letterSpacing = 0.sp
    ),

    // Headline styles
    headlineLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.SemiBold,
        fontSize = 32.sp,
        lineHeight = 40.sp,
        letterSpacing = 0.sp
    ),

```

```
headlineMedium = TextStyle(  
    fontFamily = FontFamily.Default,  
    fontWeight = FontWeight.SemiBold,  
    fontSize = 28.sp,  
    lineHeight = 36.sp,  
    letterSpacing = 0.sp  
),  
headlineSmall = TextStyle(  
    fontFamily = FontFamily.Default,  
    fontWeight = FontWeight.SemiBold,  
    fontSize = 24.sp,  
    lineHeight = 32.sp,  
    letterSpacing = 0.sp  
),  
  
// Title styles  
titleLarge = TextStyle(  
    fontFamily = FontFamily.Default,  
    fontWeight = FontWeight.SemiBold,  
    fontSize = 22.sp,  
    lineHeight = 28.sp,  
    letterSpacing = 0.sp  
),  
titleMedium = TextStyle(  
    fontFamily = FontFamily.Default,  
    fontWeight = FontWeight.SemiBold,  
    fontSize = 16.sp,  
    lineHeight = 24.sp,  
    letterSpacing = 0.15.sp  
),  
titleSmall = TextStyle(  
    fontFamily = FontFamily.Default,  
    fontWeight = FontWeight.Medium,  
    fontSize = 14.sp,  
    lineHeight = 20.sp,  
    letterSpacing = 0.1.sp  
),  
  
// Body styles  
bodyLarge = TextStyle(  
    fontFamily = FontFamily.Default,  
    fontWeight = FontWeight.Normal,  
    fontSize = 16.sp,  
    lineHeight = 24.sp,  
    letterSpacing = 0.5.sp  
),  
bodyMedium = TextStyle(  
    fontFamily = FontFamily.Default,  
    fontWeight = FontWeight.Normal,  
    fontSize = 14.sp,  
    lineHeight = 20.sp,  
    letterSpacing = 0.25.sp  
),  
bodySmall = TextStyle(  
    fontFamily = FontFamily.Default,  
    fontWeight = FontWeight.Normal,  
    fontSize = 12.sp,  
    lineHeight = 16.sp,  
    letterSpacing = 0.4.sp  
),  
  
// Label styles  
labelLarge = TextStyle(  

```

```

        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Medium,
        fontSize = 14. sp,
        lineHeight = 20. sp,
        letterSpacing = 0.1. sp
    ),
    labelMedium = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Medium,
        fontSize = 12. sp,
        lineHeight = 16. sp,
        letterSpacing = 0.5. sp
    ),
    labelSmall = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Medium,
        fontSize = 11. sp,
        lineHeight = 16. sp,
        letterSpacing = 0.5. sp
    )
)

// === app\src\main\java\com\echoling\app\presentation\viewmodel\CourseDetailViewModel.kt ===

package com.echoling.app.presentation.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.echoling.app.domain.model.Course
import com.echoling.app.domain.usecase.DeleteCourseUseCase
import com.echoling.app.domain.usecase.GetCourseGroupsUseCase
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.launch
import javax.inject.Inject

/**
 * State for the category-detail screen (§12.19). The screen shows the
 * list of lessons inside one courseName "folder". The delete action
 * moved here from the home page so the user can prune individual
 * lessons.
 */
data class CourseDetailUiState(
    val courseName: String = "",
    val courses: List<Course> = emptyList(),
    val isLoading: Boolean = true,
    val error: String? = null,
)

@HiltViewModel
class CourseDetailViewModel @Inject constructor(
    private val getCourseGroups: GetCourseGroupsUseCase,
    // Renamed from `deleteCourse` to avoid a name collision with the
    // public `fun deleteCourse(courseId)` below — Kotlin resolves
    // `deleteCourse(id)` to the function (invocation syntax wins over
    // property access), which would otherwise be infinite recursion.
    private val deleteCourseUseCase: DeleteCourseUseCase,
) : ViewModel() {

```



```

private val _uiState = MutableStateFlow(CourseDetailUiState())
val uiState: StateFlow<CourseDetailUiState> = _uiState.asStateFlow()

fun loadGroup(courseName: String) {
    viewModelScope.launch {
        _uiState.value = CourseDetailUiState(isLoading = true, courseName = courseName)
        try {
            // Reuse the same flow that powers the home page so the
            // grouping rules are guaranteed to agree. We snapshot
            // the first emission — the screen will refresh on the
            // next emit (e.g. after a delete) because the screen
            // calls `loadGroup` again on returning from Practice.
            val groups = getCourseGroups().first()
            val match = groups.firstOrNull { it.courseName == courseName }
            if (match == null) {
                _uiState.value = CourseDetailUiState(
                    courseName = courseName,
                    isLoading = false,
                    error = "Group not found",
                )
            } else {
                _uiState.value = CourseDetailUiState(
                    courseName = courseName,
                    courses = match.courses,
                    isLoading = false,
                    error = null,
                )
            }
        } catch (e: Exception) {
            _uiState.value = CourseDetailUiState(
                courseName = courseName,
                isLoading = false,
                error = e.message ?: "Failed to load group",
            )
        }
    }
}

fun deleteCourse(courseId: String) {
    viewModelScope.launch {
        deleteCourseUseCase(courseId)
        // Re-emit the freshly-filtered list so the row disappears
        // without a screen-level reload.
        val name = _uiState.value.courseName
        if (name.isNotBlank()) loadGroup(name)
    }
}
}

```

```
// === app\src\main\java\com\echoling\app\presentation\viewmodel\CoursesViewModel.kt ===
```

```

package com.echoling.app.presentation.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.echoling.app.domain.model.CourseGroup
import com.echoling.app.domain.usecase.ContinueLearningItem
import com.echoling.app.domain.usecase.DeleteCourseUseCase
import com.echoling.app.domain.usecase.GetCourseGroupsUseCase
import com.echoling.app.domain.usecase.GetStatisticsUseCase

```

```

import com.echoling.app.domain.usecase.GetContinueLearningUseCase
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.combine
import kotlinx.coroutines.launch
import javax.inject.Inject

/**
 * Aggregate UI state for the Courses tab. As of § 12.19 the home page
 * shows *groups* of courses (folders), not individual lessons — tapping
 * a group navigates to the repurposed `CourseDetailScreen` which lists
 * the lessons in that group.
 */
data class CoursesUiState(
    val groups: List<CourseGroup> = emptyList(),
    val totalLearnTimeMs: Long = 0L,
    val totalWordsCollected: Int = 0,
    val continueLearning: ContinueLearningItem? = null,
    /**
     * Flips to true the first time Room emits into the combine. Used
     * to avoid flashing the "暂无练习" empty state during the first
     * frame, when the ViewModel's `init` coroutine hasn't yet started
     * collecting and the groups list is still the default empty list.
     */
    val hasLoadedOnce: Boolean = false,
)

@HiltViewModel
class CoursesViewModel @Inject constructor(
    private val getCourseGroups: GetCourseGroupsUseCase,
    private val getStatistics: GetStatisticsUseCase,
    private val getContinueLearning: GetContinueLearningUseCase,
    // The delete use case is kept here so it can be re-exposed later
    // (e.g. a long-press on a group in the future). Today delete lives
    // inside the category-detail screen — see `CourseDetailViewModel`.
    @Suppress("unused")
    private val deleteCourseUseCase: DeleteCourseUseCase,
) : ViewModel() {

    private val _uiState = MutableStateFlow(CoursesUiState())
    val uiState: StateFlow<CoursesUiState> = _uiState.asStateFlow()

    init {
        load()
    }

    private fun load() {
        viewModelScope.launch {
            combine(
                getCourseGroups(),
                getStatistics.getStatisticsFlow(),
                getContinueLearning(),
            ) { groups, stats, continueItem ->
                val totalTime = stats.progressList.sumOf { it.totalLearnTimeMs }
                CoursesUiState(
                    groups = groups,
                    totalLearnTimeMs = totalTime,
                    totalWordsCollected = stats.words.size,
                    continueLearning = continueItem,
                    hasLoadedOnce = true,
                )
            }
        }
    }
}

```

```

        }.collect { state ->
            _uiState.value = state
        }
    }
}
}

```

```
// === app\src\main\java\com\echoling\app\presentation\viewmodel\ImportViewModel.kt ===
```

```
package com.echoling.app.presentation.viewmodel
```

```
import android.app.Application
import android.content.Context
import android.net.Uri
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.ViewModelScope
import com.echoling.app.domain.model.Course
import com.echoling.app.domain.usecase.ImportCourseUseCase
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import java.io.File
import java.io.FileOutputStream
import javax.inject.Inject
```

```
enum class ImportState {
    IDLE,
    IMPORTING,
    SUCCESS,
    ERROR
}
```

```
@HiltViewModel
```

```
class ImportViewModel @Inject constructor(
    private val application: Application,
    private val importCourseUseCase: ImportCourseUseCase
) : AndroidViewModel(application) {
```

```
    private val _importState = MutableStateFlow(ImportState.IDLE)
    val importState: StateFlow<ImportState> = _importState.asStateFlow()
```

```
    private val _errorMessage = MutableStateFlow<String?>(null)
    val errorMessage: StateFlow<String?> = _errorMessage.asStateFlow()
```

```
    fun importCourse(
        courseName: String,
        title: String,
        difficulty: String,
        audioUri: Uri?,
        videoUri: Uri?,
        subtitleUri: Uri?,
        durationMs: Long = 0L
    ) {
        viewModelScope.launch {
            _importState.value = ImportState.IMPORTING

            try {
                val context = application.applicationContext
            }
        }
    }
}
```

```

// Must have either audio or video
if (audioUri == null && videoUri == null) {
    _errorMessage.value = "Please select an audio or video file"
    _importState.value = ImportState.ERROR
    return@launch
}

// Resolve the parent-group name. A blank entry falls back
// to the lesson title so we never persist a row with both
// fields empty — that matches the "old data uses title"
// invariant the rest of the app relies on.
val resolvedCourseName = courseName.trim().ifBlank { title.trim() }

// Copy audio file to app's internal storage
var audioFile: File? = null
if (audioUri != null) {
    val extension = getMediaExtension(audioUri, "mp3")
    val audioFileName = "course_${System.currentTimeMillis()}_audio.$extension"
    audioFile = copyUriToInternalStorage(context, audioUri, audioFileName)
}

// Copy video file to app's internal storage
var videoFile: File? = null
if (videoUri != null) {
    val extension = getMediaExtension(videoUri, "mp4")
    val videoFileName = "course_${System.currentTimeMillis()}_video.$extension"
    videoFile = copyUriToInternalStorage(context, videoUri, videoFileName)
}

// Copy subtitle file if provided
var subtitleFile: File? = null
if (subtitleUri != null) {
    val extension = getFileExtension(subtitleUri)
    val subtitleFileName = "course_${System.currentTimeMillis()}_subtitle.$extension"
    subtitleFile = copyUriToInternalStorage(context, subtitleUri, subtitleFileName)
}

// Get duration from audio or video
val sourceUri = audioUri ?: videoUri
val duration = if (sourceUri != null) getMediaDuration(context, sourceUri) else 0L

// Create course entity
val course = Course(
    courseId = "course_${System.currentTimeMillis()}",
    courseName = resolvedCourseName,
    title = title,
    description = "Imported course: $title",
    difficulty = difficulty,
    audioUri = audioFile?.absolutePath,
    videoUri = videoFile?.absolutePath,
    subtitleUri = subtitleFile?.absolutePath,
    durationMs = duration,
    totalSentences = 0,
    thumbnailUri = null,
    createdAt = System.currentTimeMillis(),
    updatedAt = System.currentTimeMillis()
)

importCourseUseCase(course)

_importState.value = ImportState.SUCCESS
} catch (e: Exception) {

```

```

        _errorMessage.value = e.message ?: "Unknown error occurred"
        _importState.value = ImportState.ERROR
    }
}

private fun copyUriToInternalStorage(context: Context, uri: Uri, fileName: String): File? {
    return try {
        val outputDir = File(context.filesDir, "courses")
        if (!outputDir.exists()) {
            outputDir.mkdirs()
        }

        val outputFile = File(outputDir, fileName)

        context.contentResolver.openInputStream(uri)?.use { input ->
            FileOutputStream(outputFile).use { output ->
                input.copyTo(output)
            }
        }

        outputFile
    } catch (e: Exception) {
        e.printStackTrace()
        null
    }
}

private fun getMediaDuration(context: Context, uri: Uri): Long {
    return try {
        val retriever = android.media.MediaMetadataRetriever()
        retriever.setDataSource(context, uri)
        val durationStr = retriever.extractMetadata(android.media.MediaMetadataRetriever.METADATA_KEY_DURATION)
        retriever.release()
        durationStr?.toLongOrNull() ?: 0L
    } catch (e: Exception) {
        e.printStackTrace()
        0L
    }
}

private fun getFileExtension(uri: Uri): String {
    val mimeType = application.contentResolver.getType(uri)
    return when {
        mimeType?.contains("srt") == true -> "srt"
        mimeType?.contains("ass") == true -> "ass"
        mimeType?.contains("ssa") == true -> "ssa"
        else -> uri.lastPathSegment?.substringAfterLast('.', "srt") ?: "srt"
    }
}

private fun getMediaExtension(uri: Uri, defaultExt: String): String {
    val mimeType = application.contentResolver.getType(uri)
    return when {
        mimeType?.contains("mp3") == true -> "mp3"
        mimeType?.contains("mpeg") == true -> "mp3"
        mimeType?.contains("mp4") == true -> "mp4"
        mimeType?.contains("mkv") == true -> "mkv"
        mimeType?.contains("webm") == true -> "webm"
        mimeType?.contains("avi") == true -> "avi"
        mimeType?.contains("m4a") == true -> "m4a"
        mimeType?.contains("wav") == true -> "wav"
        mimeType?.contains("flac") == true -> "flac"
    }
}

```

```
        else -> uri.lastPathSegment?.substringAfterLast('.', defaultExt) ?: defaultExt
    }
}

fun resetState() {
    _importState.value = ImportState.IDLE
    _errorMessage.value = null
}
}
```

```
// === app\src\main\java\com\echoling\app\presentation\viewmodel\PracticeViewModel.kt ===
```

```
package com.echoling.app.presentation.viewmodel

import android.app.Application
import android.media.MediaPlayer
import android.net.Uri
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.ViewModelScope
import androidx.media3.common.MediaItem
import androidx.media3.common.Player
import androidx.media3.exoplayer.ExoPlayer
import com.echoling.app.domain.model.Course
import com.echoling.app.domain.model.LearningProgress
import com.echoling.app.domain.model.Sentence
import com.echoling.app.domain.model.Word
import com.echoling.app.domain.usecase.GetCourseDetailUseCase
import com.echoling.app.domain.usecase.GetCourseSentencesUseCase
import com.echoling.app.domain.usecase.LookupWordUseCase
import com.echoling.app.domain.usecase.SaveProgressUseCase
import com.echoling.app.domain.usecase.SaveWordUseCase
import com.echoling.app.domain.usecase.SyncSentencesUseCase
import com.echoling.app.domain.usecase.UpdateSentenceCompletedUseCase
import com.echoling.app.domain.usecase.UpdateSentenceTestedUseCase
import com.echoling.app.player.AudioPlayer
import com.echoling.app.player.PlaybackState
import com.echoling.app.player.subtitle.Subtitle
import com.echoling.app.player.subtitle.SubtitleParserFactory
import com.echoling.app.speech.RecordingResult
import com.echoling.app.speech.RecordingState
import com.echoling.app.speech.VoiceRecorder
import com.echoling.app.speech.WordMatcher
import com.echoling.app.speech.WavRecorder
import com.echoling.app.speech.VoskSpeechRecognizer
import com.echoling.app.speech.ModelManager
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.combine
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.isActive
import kotlinx.coroutines.launch
import java.io.File
import java.util.concurrent.atomic.AtomicLong
import javax.inject.Inject
```

```

data class SentenceState(
    val sentenceId: Int,
    val isCompleted: Boolean = false,
    val isTested: Boolean = false
)

data class TestState(
    val isActive: Boolean = false,
    val testItems: List<Subtitle> = emptyList(),
    val currentTestIndex: Int = 0,
    val revealedWords: Set<Int> = emptySet(),
    val testedCount: Int = 0
)

/**
 * One-shot word-translation request. Reset to [Idle] when the dialog
 * closes so a fresh long-press can issue a new request.
 */
sealed class WordTranslationState {
    object Idle : WordTranslationState()
    data class Loading(val word: String) : WordTranslationState()
    /**
     * A translation arrived from the bundled local dictionary. The
     * practice flow no longer falls back to a network translation API
     * — the local dictionary is the sole source. [phonetic] and [pos]
     * are always present (may be empty strings for entries that lack
     * one).
     */
    data class Loaded(
        val word: String,
        val translation: String,
        val phonetic: String,
        val pos: String,
    ) : WordTranslationState()
    data class Failed(val word: String, val message: String) : WordTranslationState()
}

/**
 * Testing-tab STT state machine.
 * Idle → Listening → Transcribed → (Passed | Failed).
 * User can cancel from Listening back to Idle, or reset from any state.
 */
sealed class SttTestState {
    data object Idle : SttTestState()
    data class Listening(val elapsedMs: Long = 0L) : SttTestState()
    /** Vosk is transcribing the recorded WAV file. Shown briefly
     * after the user releases the mic, before the editor appears. */
    data class Transcribing(val recordingPath: String) : SttTestState()
    data class Transcribed(val text: String) : SttTestState()
    data class Passed(val text: String) : SttTestState()
    data class Failed(
        val transcribed: String,
        val original: String,
        val origWords: List<String>,
        val transWords: List<String>,
        // (2026-06-28) Per-word match flags from WordMatcher.
        // Index-aligned with origWords / transWords. Drives the
        // per-chip color in TestResultCard.WordChipsRow — the
        // previous version did strict position-by-position
        // comparison, which caused a single wrong word to mark
        // all subsequent (correct) words as wrong too. With
        // these flags the UI can mark each word independently.
        val origMatched: BooleanArray = BooleanArray(0),
    )
}

```

```

        val transMatched: BooleanArray = BooleanArray(0),
        val reason: String
    ) : SttTestState()
}

@HiltViewModel
class PracticeViewModel @Inject constructor(
    private val application: Application,
    private val getCourseDetailUseCase: GetCourseDetailUseCase,
    private val getCourseSentencesUseCase: GetCourseSentencesUseCase,
    private val saveProgressUseCase: SaveProgressUseCase,
    private val updateSentenceCompletedUseCase: UpdateSentenceCompletedUseCase,
    private val updateSentenceTestedUseCase: UpdateSentenceTestedUseCase,
    private val syncSentencesUseCase: SyncSentencesUseCase,
    private val saveWordUseCase: SaveWordUseCase,
    private val audioPlayer: AudioPlayer,
    private val subtitleParserFactory: SubtitleParserFactory,
    private val voiceRecorder: VoiceRecorder,
    private val lookupWordUseCase: LookupWordUseCase,
    private val wavRecorder: WavRecorder,
    private val voskRecognizer: VoskSpeechRecognizer,
    private val modelManager: ModelManager,
) : AndroidViewModel(application) {

    val playbackState = audioPlayer.playbackState
    val recordingState = voiceRecorder.recordingState

    /** Vosk model download state — surfaced to TestingPage so the user
     *  sees a "downloading model" message on first use. */
    val modelDownloadState = modelManager.downloadState
    val wavRecordingState = wavRecorder.recordingState
    val wavAmplitude = wavRecorder.amplitude

    private val _subtitles = MutableStateFlow<List<Subtitle>>(emptyList())
    val subtitles: StateFlow<List<Subtitle>> = _subtitles.asStateFlow()

    // Page switching state. Declared before [_subtitleIndexByPage] /
    // [currentSubtitleIndex] so the derived StateFlow below can
    // reference [_currentPage] (forward references aren't allowed in
    // class bodies).
    enum class PracticePage { LISTENING, SPEAKING, TESTING }
    private val _currentPage = MutableStateFlow<PracticePage>(PracticePage.LISTENING)
    val currentPage: StateFlow<PracticePage> = _currentPage.asStateFlow()

    // Per-page subtitle index. v2: the three practice pages (泛听 / 精听 /
    // 测试) each track their own position in the subtitle list, so
    // switching tabs no longer makes one page jump to another page's
    // last position.
    //
    // TESTING actually uses [_testState]'s `currentTestIndex` for its
    // own progress (it has a separate test-item list, see
    // [startTestMode]). We still include TESTING in the map for
    // symmetry — its entry just stays at -1 since TESTING never
    // updates it.
    //
    // Initial value -1 means "no subtitle focused yet" (before any
    // playback or skip action).
    private val _subtitleIndexByPage = MutableStateFlow<Map<PracticePage, Int>>(
        mapOf(
            PracticePage.LISTENING to -1,
            PracticePage.SPEAKING to -1,
            PracticePage.TESTING to -1,
        )
    )
}

```



```

)
val subtitleIndexByPage: StateFlow<Map<PracticePage, Int>> = _subtitleIndexByPage.asStateFlow()

/**
 * UI-bound: the current page's subtitle index.
 *
 * The UI doesn't need to know there's a per-page map — it just
 * reads the right value for the page it's on. Switching pages
 * (via [setCurrentPage]) automatically updates this derived flow
 * because [combine] reacts to [_currentPage] changes.
 */
val currentSubtitleIndex: StateFlow<Int> = combine(
    _currentPage, _subtitleIndexByPage
) { page, map -> map[page] ?: -1 }
    .stateIn(viewModelScope, SharingStarted.Eagerly, -1)

private var currentSubtitleObj: Subtitle? = null
private var positionUpdateJob: Job? = null
private var currentCourseId: String = ""
private var currentSentenceId: Int = 0

// § 12.21: learn-time accounting.
//
// Old design (pre-§ 12.21) used a single `sessionStartTimeMs` wall-
// clock and recorded the full elapsed time on every save. Combined
// with the 10s periodic save, that meant the user got ~10s of
// "learning time" credited for every 10s they had the Practice
// screen open — even while paused, in a test mode dialog, or just
// looking at the subtitle list. Result: 5 minutes of actual
// listening inflated to 30+ minutes of credited time.
//
// New design: only count time when `playbackState.isPlaying` is
// true. The position loop ticks every 50ms; on each tick we add
// the delta-since-last-tick to [accumulatedPlayMs] only if the
// player is currently producing audio. The save body reads the
// counter via getAndSet(0) so each window is consumed exactly
// once even if a periodic save and a pause-save race.
private val accumulatedPlayMs = AtomicLong(0L)
private val lastTickMs = AtomicLong(0L)

// Wall-clock of the last periodic save. Used by
// startPositionUpdates() to throttle the auto-save to once per
// [PERIODIC_SAVE_INTERVAL_MS] so a crash mid-session loses at
// most that much progress.
private var lastPeriodicSaveTimeMs: Long = 0L

private companion object {
    /** Auto-save cadence while Practice is open. */
    const val PERIODIC_SAVE_INTERVAL_MS: Long = 10_000L

    /**
     * Minimum press-and-hold duration to start STT, in milliseconds.
     * Below this, we treat the press as accidental and return to Idle
     * without invoking the recognizer — Android's SpeechRecognizer needs
     * ~300ms to initialize, and stopping it before that fires
     * ERROR_NO_MATCH which the user perceives as a broken button.
     */
    const val STT_MIN_HOLD_MS: Long = 300L
}

// For playing single subtitle once
private var singleSubtitleIndex: Int = -1
private var singleSubtitleEndMs: Long = 0L

```

```

private var isSinglePlayMode: Boolean = false
private var skipTargetListIndex: Int = -1
private var skipTargetTimeoutMs: Long = 0L

// Recording playback
private var mediaPlayer: MediaPlayer? = null
// (2026-06-28) Per-page recording paths. Previously a single
// _recordingPath was shared between SpeakingPage (via
// stopRecording → VoiceRecorder) and TestingPage (via
// stopStt → WavRecorder for Vosk). The shared flow meant the
// recording made on one page leaked into the other page's
// "播放录音" / 回放录音 button — e.g. recording on the test
// page, switching to the speaking page, the speaking page
// would show "我的录音 / 点击播放" with the test page's audio.
// Each page now owns its own path; [playRecording] takes the
// path as a parameter so the caller is explicit about which
// recording to play.
private val _speakingRecordingPath = MutableStateFlow<String?>(null)
val speakingRecordingPath: StateFlow<String?> = _speakingRecordingPath.asStateFlow()

private val _testRecordingPath = MutableStateFlow<String?>(null)
val testRecordingPath: StateFlow<String?> = _testRecordingPath.asStateFlow()

private val _isPlayingRecording = MutableStateFlow(false)
val isPlayingRecording: StateFlow<Boolean> = _isPlayingRecording.asStateFlow()

// Video player for video playback
private var videoPlayer: ExoPlayer? = null
private val _videoPlayer = MutableStateFlow<ExoPlayer?>(null)
val videoPlayerState: StateFlow<ExoPlayer?> = _videoPlayer.asStateFlow()

private val _isVideoMode = MutableStateFlow(false)
val isVideoMode: StateFlow<Boolean> = _isVideoMode.asStateFlow()

// Sentence completion/test states (in-memory cache)
private val _sentenceStates = MutableStateFlow<Map<Int, SentenceState>>(emptyMap())
val sentenceStates: StateFlow<Map<Int, SentenceState>> = _sentenceStates.asStateFlow()

// Test mode state
private val _testState = MutableStateFlow<TestState>()
val testState: StateFlow<TestState> = _testState.asStateFlow()

// Long-press word translation state — driven by the long-press handler
// on ListeningPage / SpeakingPage. [WordTranslationState.Loading] is
// emitted immediately so the dialog can show a spinner, then either
// [Loaded] or [Failed] once the network call completes.
private val _wordTranslation = MutableStateFlow<WordTranslationState>(WordTranslationState.Idle)
val wordTranslation: StateFlow<WordTranslationState> = _wordTranslation.asStateFlow()

private var wordTranslationJob: Job? = null

// ——— STT word-match testing state (跟读测试) ———
// Phases: Idle → Listening → Transcribed → (Passed | Failed).
private val _sttTestState = MutableStateFlow<SttTestState>(SttTestState.Idle)
val sttTestState: StateFlow<SttTestState> = _sttTestState.asStateFlow()

// 5 random amplitude bars for v1 recording overlay animation.
private val _sttAmplitudeBars = MutableStateFlow<List<Float>>(List(5) { 0.4f })
val sttAmplitudeBars: StateFlow<List<Float>> = _sttAmplitudeBars.asStateFlow()

private var sttElapsedJob: Job? = null
private var sttEventCollectionJob: Job? = null
private var sttWaitJob: Job? = null

```

```

private var sttPressStartTimeMs: Long = 0L

/**
 * Slot for the latest STT result text. The Vosk transcribe
 * coroutine writes here when it completes; it does NOT change
 * the state machine directly. [stopStt] is the only place that
 * reads this and performs the Transcribed(...) transition.
 *
 * v2: the recognizer is Vosk (offline, file-based), not the
 * system SpeechRecognizer. The "early error races the user's
 * release" failure mode of v1 is gone because Vosk runs after
 * the user releases, not in parallel with the recording.
 *
 * null = result not yet arrived (Vosk still running).
 * "" = Vosk returned empty / errored, treat as no recognition.
 * "..." = Vosk returned the recognized text.
 */
private var sttResult: String? = null

fun setCurrentPage(page: PracticePage) {
    if (_currentPage.value != page) {
        // Tab-switch bug fix (2026-06-27): reset audioPlayer state
        // and clear per-tab playback context so each tab has
        // independent playback state. Without this reset, the
        // 50ms position-update loop keeps ticking against the
        // previous tab's frozen audioPlayer position; the
        // subtitleProvider callback (see loadSubtitles) then
        // finds the matching subtitle at that stale position
        // and writes its index into _subtitleIndexByPage[page]
        // — the NEW tab's slot — leaking the previous tab's
        // context into the new tab's UI. Symptom: 泛听 played
        // sentence 5 → switch to 精听 → 精听 highlights
        // sentence 5 as currently playing. The isPlaying guard
        // inside the subtitleProvider is the belt-and-suspenders
        // backup: even if a future refactor removes the
        // seekTo(0) here, paused ticks won't leak.
        resetPlaybackForTabSwitch()
    }
    _currentPage.value = page
}

/**
 * Update the current page's subtitle index. Used by the playback
 * auto-advance and by [skipToSubtitle] / [skipToPreviousSubtitle] /
 * [skipToNextSubtitle]. Page-aware: only the active page's slot
 * changes, so switching tabs preserves each page's position.
 */
private fun setCurrentSubtitleIndex(newIndex: Int) {
    val page = _currentPage.value
    _subtitleIndexByPage.update { current ->
        current.toMutableMap().apply { this[page] = newIndex }
    }
}

/**
 * Reset playback state when switching between tabs.
 *
 * Critical: the audioPlayer is a @Singleton, so its position is
 * "sticky" across tabs. After pause(), the 50ms position-update
 * loop (startPositionUpdates, line 649) still calls
 * audioPlayer.updatePosition() with the frozen position from
 * the previous tab. The subtitleProvider callback (see
 * loadSubtitles) then finds the matching subtitle at that

```

```

* stale position and would write its index into
* _subtitleIndexByPage[page] — the NEW tab's slot — leaking
* the previous tab's context.
*
* Belt-and-suspenders with the `playbackState.value.isPlaying`
* guard inside the subtitleProvider: this method resets
* audioPlayer to a clean state (seekTo(0) etc.) so the next
* tick doesn't see a stale position; the isPlaying guard
* catches any future refactor that forgets to seekTo(0).
*
* Order matters: save progress BEFORE seekTo(0) so the user's
* last position from the previous tab is persisted (Continue
* Learning card on home still picks it up).
*/
private fun resetPlaybackForTabSwitch() {
    // 1) Persist the previous tab's position before we lose it.
    saveProgress()
    // 2) Pause audio/video.
    if (_isVideoMode.value) {
        pauseVideo()
    } else {
        audioPlayer.pause()
    }
    // 3) Clear single-play / skip state.
    singleSubtitleIndex = -1
    singleSubtitleEndMs = 0L
    isSinglePlayMode = false
    skipTargetListIndex = -1
    skipTargetTimeoutMs = 0L
    // 4) Clear "current subtitle" so the new tab doesn't show
    // the previous tab's subtitle as its context.
    currentSubtitleObj = null
    currentSentenceId = 0
    // 5) Seek to 0 so the position-update loop's next tick
    // doesn't re-trigger the bug with the previous tab's
    // position. (subtitleProvider's isPlaying guard also
    // blocks auto-advance while paused — this seekTo(0) is
    // belt-and-suspenders.)
    if (_isVideoMode.value) {
        seekVideoTo(0L)
    } else {
        audioPlayer.seekTo(0L)
    }
}

fun loadSentenceStates(courseId: String) {
    if (courseId.isBlank()) return
    viewModelScope.launch {
        val sentences = getCourseSentencesUseCase.getByCourseIdSync(courseId)
        val states = sentences.associate { it.sentenceId to SentenceState(
            sentenceId = it.sentenceId,
            isCompleted = it.isCompleted,
            isTested = it.isTested
        ) }
        _sentenceStates.value = states
    }
}

fun markSentenceCompleted(sentenceId: Int, completed: Boolean) {
    // (2026-07-04) Bug 2 fix: do the in-memory state update
    // SYNCHRONOUSLY, before the DB write. The previous ordering
    // put the StateFlow write inside the coroutine, after
    // `updateSentenceCompletedUseCase(...)` (which suspends for

```

```

// the Room write). That meant the dropdown's cell color
// didn't update until the DB write finished — usually
// <10ms but observable to the user as "I tapped the check,
// opened the dropdown, and the cell isn't green yet". Worse,
// if the user clicked "下一句" before the DB write finished,
// Compose batched the sentenceStates update with the
// currentSubtitleIndex update from the next-button click,
// making it LOOK like the green only appeared after clicking
// next — but it was actually just a delayed snapshot apply.
//
// Doing the StateFlow write synchronously ensures the change
// is visible in the next Compose frame, regardless of when
// the DB write lands. The DB write still happens on
// viewModelScope as before; if it fails, the in-memory state
// is already correct for the current session, and the next
// loadSentenceStates() will reconcile from the DB.
val current = _sentenceStates.value.toMutableMap()
current[sentenceId] = current[sentenceId]?.copy(isCompleted = completed)
    ?: SentenceState(sentenceId, isCompleted = completed)
_sentenceStates.value = current
viewModelScope.launch {
    updateSentenceCompletedUseCase(currentCourseId, sentenceId, completed)
}
}

fun markSentenceTested(sentenceId: Int, tested: Boolean) {
    // (2026-07-04) Same Bug 2 fix as [markSentenceCompleted]:
    // hoist the StateFlow writes out of the coroutine so the UI
    // reflects the change in the next Compose frame instead of
    // after the DB write completes. Read the prior isTested
    // BEFORE the synchronous write so the increment-testCount
    // logic still detects "newly tested".
    val wasTested = _sentenceStates.value[sentenceId]?.isTested == true
    val current = _sentenceStates.value.toMutableMap()
    current[sentenceId] = current[sentenceId]?.copy(isTested = tested)
        ?: SentenceState(sentenceId, isTested = tested)
    _sentenceStates.value = current
    // Sync testedCount in test state. (2026-06-28) Only
    // increment when this call actually newly-tests a
    // sentence that is in the current test items. Prevents:
    // - Double-tap "标记完成" on the same test item
    //   incrementing testedCount twice (e.g., 5 → 6 even
    //   though we only have 5 test items → "6 / 5 已完成"
    //   display bug).
    // - Marking a sentence NOT in the current test items
    //   inflating testedCount beyond testItems.size
    //   (e.g., from a future feature that lets users
    //   mark sentences tested outside test mode).
    if (tested && !wasTested) {
        val ts = _testState.value
        if (ts.testItems.any { it.index == sentenceId }) {
            _testState.value = ts.copy(testedCount = ts.testedCount + 1)
        }
    }
    viewModelScope.launch {
        updateSentenceTestedUseCase(currentCourseId, sentenceId, tested)
    }
}

fun startTestMode() {
    val allSubtitles = _subtitles.value.filter { it.index >= 0 }
    // (2026-06-28) Per user spec: test items = a random 1/3 of
    // sentences in the current course, with a per-sentence

```

```

// minimum of 3 words.
//   "从当前所学素材中所有句子中随机收取三分之一的句子
//   用于测试，要求每个所选的句子长度必须满足至少三个单词"
//
// The 3-word minimum is a sanity check on the test sample:
// - Sentences shorter than 3 words are usually
//   interjections / fragments ("Yes.", "Oh, no.", "Sure.")
//   that don't exercise comprehension meaningfully.
// - They also visually look broken in the test page's
//   word-hide UI (fewer than 3 boxes to reveal — the user
//   can guess the sentence from 1-2 revealed words).
//
// We filter BEFORE the random pick so the 1/3 sample isn't
// biased: a 30-sentence course with 5 short interjections
// would otherwise have ~17% of its pool be untestable;
// filtering first gives us a clean 1/3 sample of the
// testable subset.
//
// Word count uses the same whitespace-split heuristic as
// [TestingWordsFlowRow] (each visible word block = one
// non-blank token) so the filter matches what the user
// actually sees on screen.
val eligibleSentences = allSubtitles.filter { sub ->
    val wordCount = sub.contentEn
        .split(Regex("\\s+"))
        .count { it.isNotBlank() }
    wordCount >= 3
}
// (2026-06-28) Edge case: if NO sentence in the course has
// 3+ words (very rare — could happen on a single-line
// import or a course with only short phrases), fall back
// to all sentences so the test mode can still produce at
// least 1 test item instead of leaving the user staring at
// "X / 0 已完成".
val pool = if (eligibleSentences.isNotEmpty()) eligibleSentences else allSubtitles
val shuffled = pool.shuffled()
// maxOf(1, ...) preserves the old behavior of always giving
// the user at least 1 test item (e.g., a 2-sentence course
// → size/3 = 0 → would be 0 test items without this guard).
val testCount = maxOf(1, shuffled.size / 3)
val testItems = shuffled.take(testCount)
// (2026-06-28) Only count test items that are already marked
// tested in the DB. Previously this counted all sentences
// in the course, which could exceed testItems.size. The
// more-specific count guards against the display ever
// showing "X / Y 已完成" with X > Y (the user-reported
// testedCount > totalCount bug).
val testedSoFar = testItems.count { item ->
    _sentenceStates.value[item.index]?.isTested == true
}
_testState.value = TestState(
    isActive = true,
    testItems = testItems,
    currentTestIndex = 0,
    revealedWords = emptySet(),
    testedCount = testedSoFar
)
// (2026-06-28) Make sure no leftover STT / recording state
// bleeds into the new session — a previous mid-test STT
// attempt (Listening / Transcribing / Transcribed / etc.)
// would otherwise show up on the freshly-started session's
// first sentence and confuse the user.
resetStt()

```

```

    // (2026-06-28) Stop any in-progress audio playback from the
    // previous test attempt — typically the user is starting a
    // fresh round, audio from the old session shouldn't continue
    // playing into the new one.
    if (_isVideoMode.value) pauseVideo() else audioPlayer.pause()
}

/**
 * (2026-06-28) "重新测试" button action — user wants to start a
 * brand-new test round from X=1. Per user spec:
 *   "用户也可以再进行新一轮测试，所有需要增加一个按钮
 *   重新测试/刷新图标，放在测试页面的右上角"
 *
 * Steps:
 *   1. Un-mark all current test items in _sentenceStates so
 *      the counter starts fresh at 0. (Otherwise re-using
 *      pre-tested sentences wouldn't increment testedCount —
 *      [markSentenceTested] is idempotent for already-tested
 *      items, and testedCount would never reach testItems.size
 *      → "round complete" state never triggers.)
 *   2. Persist the un-marking to DB so a process kill doesn't
 *      leave the DB out of sync with what the UI shows.
 *   3. Reset STT state (any leftover Listening/Transcribed UI
 *      from the old round should be cleared).
 *   4. Delegate to [startTestMode] to re-initialize the test
 *      state with X=1.
 *
 * Order matters: un-mark in _sentenceStates FIRST so
 * [startTestMode]'s `testedSoFar` computation reads the cleared
 * state and starts at 0. If we called startTestMode first,
 * `testedSoFar` would be Y (all tested), and un-marking later
 * would only lower the in-memory state — the counter would stay
 * at Y.
 */
fun restartTest() {
    val current = _testState.value
    if (current.isActive && current.testItems.isNotEmpty()) {
        val newStates = _sentenceStates.value.toMutableMap()
        current.testItems.forEach { item ->
            val existing = newStates[item.index]
            ?: SentenceState(item.index)
            newStates[item.index] = existing.copy(isTested = false)
        }
        _sentenceStates.value = newStates
        // Persist un-marking to DB (async). Same viewModelScope
        // channel as markSentenceTested uses, so the two don't
        // race: this launches AFTER the synchronous state flip
        // above, and any subsequent markSentenceTested call from
        // the UI fires its own launch.
        val courseId = currentCourseId
        if (courseId.isNotBlank()) {
            viewModelScope.launch {
                current.testItems.forEach { item ->
                    updateSentenceTestedUseCase(courseId, item.index, false)
                }
            }
        }
    }
    // Cancel any in-progress STT (recording / transcribing /
    // transcribed) and audio from the old round.
    cancelStt()
    if (_isVideoMode.value) pauseVideo() else audioPlayer.pause()
    // Re-initialize test state (X=1, testedCount=0).

```

```

        startTestMode()
    }

    fun nextTestItem() {
        val current = _testState.value
        if (current.currentTestIndex < current.testItems.size - 1) {
            _testState.value = current.copy(
                currentTestIndex = current.currentTestIndex + 1,
                revealedWords = emptySet()
            )
        }
    }

    /**
     * Advance to the next test item AND clear any STT result state.
     *
     * (2026-07-05) Bug fix: previously the ControlBar's 下一题
     * button only called [nextTestItem], leaving [_sttTestState]
     * at `Passed` / `Failed` so the result card from the just-
     * finished sentence stayed on screen while the new sentence's
     * subtitle card appeared above it. Users saw what looked like
     * "next sentence already tested + showing the passed card"
     * because both cards rendered simultaneously.
     *
     * The TestResultCard's own 下一题 button already called
     * `nextTestItem(); resetStt()` to avoid this — the ControlBar's
     * 下一题 just didn't. Routing both callsites through this
     * helper keeps them in sync.
     *
     * Also called from the Failed card's 下一题 for the same
     * reason.
     */
    fun advanceToNextTestItem() {
        nextTestItem()
        resetStt()
    }

    fun previousTestItem() {
        val current = _testState.value
        if (current.currentTestIndex > 0) {
            _testState.value = current.copy(
                currentTestIndex = current.currentTestIndex - 1,
                revealedWords = emptySet()
            )
        }
    }

    fun revealTestWord(wordIndex: Int) {
        val current = _testState.value
        _testState.value = current.copy(revealedWords = current.revealedWords + wordIndex)
    }

    fun initializePlayer() {
        audioPlayer.initialize()
    }

    fun loadCourse(courseId: String) {
        if (courseId.isBlank()) return
        currentCourseId = courseId
        // §12.21: counter starts at 0 — we only count time when
        // audio is actually playing, not when the user has the screen
        // open. Don't reset accumulatedPlayMs here in case the user
        // navigates away and comes back: the periodic save (10s) and

```



```

// pause() / onCleared() saves already consume (getAndSet(0))
// the counter, so a re-entry of loadCourse inherits a clean
// 0 from those.
lastTickMs.set(0L)

viewModelScope.launch {
    try {
        val detail = getCourseDetailUseCase(courseId)
        val course = detail.course
        if (course == null) {
            android.util.Log.e("PracticeViewModel", "Course not found: $courseId")
            return@launch
        }

        val savedProgress = detail.progress
        android.util.Log.d("PracticeViewModel", "Loaded saved progress: ${savedProgress?.currentPositionMs}m
s")

        if (course.hasVideoContent() && course.videoUri.isNullOrBlank().not()) {
            loadVideo(course.videoUri!!, course.subtitleUri, courseId)
        } else if (course.hasAudioContent() && course.audioUri.isNullOrBlank().not()) {
            loadMedia(course.audioUri!!, course.subtitleUri, courseId)
        } else {
            android.util.Log.e("PracticeViewModel", "No media available for course: $courseId")
        }

        if (savedProgress != null && savedProgress.currentPositionMs > 0) {
            android.util.Log.d("PracticeViewModel", "Restoring position: ${savedProgress.currentPositionMs}m
s")

            delay(1000)
            seekTo(savedProgress.currentPositionMs)
            delay(200)
            seekTo(savedProgress.currentPositionMs)
        }

        // §12.20: eager save so the Continue Learning card on
        // home picks up this course even if the user kills the
        // app / process dies before pause() or onCleared()
        // get a chance to fire. sessionStartTimeMs was set at
        // the top of loadCourse(), so this initial save writes
        // a 0ms session window — fine, the row exists.
        saveProgress()
    } catch (e: Exception) {
        android.util.Log.e("PracticeViewModel", "Error loading course: ${e.message}")
        e.printStackTrace()
    }
}

fun loadMedia(audioUri: String, subtitleUri: String?, courseId: String = "") {
    if (audioUri.isBlank()) return
    currentCourseId = courseId
    _isVideoMode.value = false

    try {
        audioPlayer.setMediaUri(audioUri)
    } catch (e: Exception) {
        e.printStackTrace()
        return
    }

    subtitleUri?.let { uri ->
        if (uri.isNotBlank()) {

```

```

        loadSubtitles(uri)
    }
}

startPositionUpdates()
}

fun loadVideo(videoUri: String, subtitleUri: String?, courseId: String = "") {
    if (videoUri.isBlank()) return
    currentCourseId = courseId
    _isVideoMode.value = true

    try {
        if (videoPlayer == null) {
            videoPlayer = ExoPlayer.Builder(application).build().apply {
                addListener(object : Player.Listener {
                    override fun onIsPlayingChanged(isPlaying: Boolean) {
                        // Sync with audio player state if needed
                    }

                    override fun onPlaybackStateChanged(state: Int) {
                        if (state == Player.STATE_READY) {
                            // Video is ready
                        }
                    }
                })
            }
            _videoPlayer.value = videoPlayer
        }

        videoPlayer?.apply {
            val mediaItem = MediaItem.fromUri(Uri.parse(videoUri))
            setMediaItem(mediaItem)
            prepare()
        }
        audioPlayer.setVideoPlayer(videoPlayer)
    } catch (e: Exception) {
        e.printStackTrace()
        return
    }

    subtitleUri?.let { uri ->
        if (uri.isNotBlank()) {
            loadSubtitles(uri)
        }
    }

    startPositionUpdates()
}

fun playVideo() {
    // In video mode, audioPlayer is a UI-state holder only — the actual
    // playback engine is videoPlayer (which carries the audio track of the
    // MKV itself). Calling audioPlayer.play() here is a no-op for the
    // empty audioPlayer ExoPlayer but used to cause two ExoPlayers to
    // race for audio focus. The position-update loop mirrors videoPlayer
    // state into audioPlayer.playbackState via updatePositionFromExternal.
    videoPlayer?.play()
}

fun pauseVideo() {
    videoPlayer?.pause()
    singleSubtitleIndex = -1
}

```

```

    }

    fun seekVideoTo(positionMs: Long) {
        // Seek only the actual playback engine. audioPlayer's UI state is
        // updated on the next 50ms tick via updatePositionFromExternal.
        videoPlayer?.seekTo(positionMs)
    }

    fun getVideoCurrentPosition(): Long = videoPlayer?.currentPosition ?: 0L

    fun getVideoDuration(): Long = videoPlayer?.duration?.coerceAtLeast(0) ?: 0L

    private fun syncSentencesToDb(subtitles: List<Subtitle>) {
        if (currentCourseId.isBlank()) return
        viewModelScope.launch {
            val sentences = subtitles.map { sub ->
                com.echoling.app.domain.model.Sentence(
                    courseId = currentCourseId,
                    sentenceId = sub.index,
                    contentEn = sub.contentEn,
                    contentCn = sub.contentCn,
                    startTimeMs = sub.startTimeMs,
                    endTimeMs = sub.endTimeMs
                )
            }
            syncSentencesUseCase(sentences)
        }
    }

    private fun loadSubtitles(uri: String) {
        viewModelScope.launch {
            try {
                val content: String?
                val fileName: String

                android.util.Log.d("PracticeViewModel", "loadSubtitles uri: $uri")

                val parsedUri = android.net.Uri.parse(uri)
                if (parsedUri.scheme == "content") {
                    content = application.applicationContext.contentResolver.openInputStream(parsedUri)?.use {
                        it.bufferedReader().readText()
                    }
                    fileName = uri.substringAfterLast("/")
                } else {
                    val file = File(uri)
                    android.util.Log.d("PracticeViewModel", "File exists: ${file.exists()}, isFile: ${file.isFile}")
                    content = if (file.exists()) file.readText() else null
                    fileName = file.name
                }

                android.util.Log.d("PracticeViewModel", "Subtitle content length: ${content?.length}, fileName: $fil
eName")

                if (content != null && content.isNotEmpty()) {
                    val parsed = subtitleParserFactory.parseSubtitles(content, fileName)
                    android.util.Log.d("PracticeViewModel", "Parsed ${parsed.size} subtitles")
                    _subtitles.value = parsed

                    // Sync sentences to database so completion/test status persists
                    syncSentencesToDb(parsed)

                    audioPlayer.setSubtitleProvider { positionMs ->
                        val sub = parsed.find { sub ->

```

```

        positionMs >= sub.startTimeMs && positionMs <= sub.endTimeMs
    } ?: parsed.find { sub ->
        positionMs >= sub.startTimeMs - 100 && positionMs <= sub.endTimeMs + 100
    }
    if (sub != null) {
        val listIndex = parsed.indexOf(sub)
        android.util.Log.d("PracticeViewModel", "subtitleProvider: pos=$positionMs, matched index=${sub.index}, listIndex=$listIndex, isSinglePlayMode=$isSinglePlayMode, singleSubtitleIndex=$singleSubtitleIndex, skipTargetListIndex=$skipTargetListIndex")
        currentSubtitleObj = sub
        currentSentenceId = sub.index
        if (isSinglePlayMode && listIndex > singleSubtitleIndex) {
            val currentSub = parsed.getOrNull(singleSubtitleIndex)
            if (currentSub != null) {
                android.util.Log.d("PracticeViewModel", "single play mode: staying on subtitle $singleSubtitleIndex")

                return@setSubtitleProvider currentSub.contentEnd
            }
        }
        // When skipTargetListIndex is set, only update index when we reach the target
        if (skipTargetListIndex >= 0) {
            if (listIndex == skipTargetListIndex) {
                setCurrentSubtitleIndex(listIndex)
                skipTargetListIndex = -1 // Target reached, clear guard
                skipTargetTimeoutMs = 0L
                android.util.Log.d("PracticeViewModel", "skip target reached: listIndex=$listIndex")
            } else if (System.currentTimeMillis() > skipTargetTimeoutMs) {
                // (2026-07-04) Bug 1 fix: previous behavior
                // called setCurrentSubtitleIndex(listIndex)
                // here, which forced the UI back to
                // whatever subtitle the audio happened
                // to be at — usually the OLD one if
                // seek hadn't completed (paused state,
                // or audio position reading returning
                // stale exoPlayer.currentPosition while
                // ExoPlayer's seek was still in
                // flight). Symptom: click "下一句"
                // → UI shows new sentence → ~1 second
                // later UI jumps back to previous.
                //
                // New behavior: clear the skipTarget
                // guard so the next tick can run its
                // natural isPlaying branch, but do
                // NOT overwrite the user's explicit
                // navigation. If audio truly never
                // reached the target, the playing
                // branch will eventually align UI to
                // the audio's actual position — that
                // path is honest because it reflects
                // what the user is hearing. The 3-second
                // timeout here was conflating "seek
                // never finished" with "user changed
                // their mind", which is wrong.
                skipTargetListIndex = -1
                skipTargetTimeoutMs = 0L
                android.util.Log.d("PracticeViewModel", "skip target timeout: clearing guard without reverting UI (audio still at listIndex=$listIndex)")
            }
            // else: still seeking, don't update index
        } else if (!isSinglePlayMode && playbackState.value.isPlaying) {
            // Tab-switch bug fix (2026-06-27): when
            // paused, don't auto-advance

```

```

        // _subtitleIndexByPage. The audioPlayer
        // position freezes on pause but the
        // 50ms tick keeps firing — without this
        // guard the residual ticks would write
        // the previous tab's subtitle into the
        // new tab's per-page slot. User actions
        // (skipTo* / seekToSubtitle) write to
        // setCurrentSubtitleIndex directly and
        // bypass this guard, so explicit
        // navigation still works.
        setCurrentSubtitleIndex(listIndex)
    }
    sub.contentEn
} else {
    android.util.Log.d("PracticeViewModel", "subtitleProvider: pos=$positionMs, no match fou
nd")

    null
}
}
} catch (e: Exception) {
    e.printStackTrace()
}
}
}

private fun startPositionUpdates() {
    positionUpdateJob?.cancel()
    // § 12.20: anchor the periodic-save clock to "now" so the
    // first auto-save fires ~PERIODIC_SAVE_INTERVAL_MS after the
    // player starts, not whenever the previous run of this loop
    // happened to last save. Also reset here (not in the eager
    // loadCourse() save) so a fresh media load starts the
    // cadence over.
    lastPeriodicSaveTimeMs = System.currentTimeMillis()
    // § 12.21: reset the per-tick wall-clock so the very first
    // tick after media starts doesn't credit the time between
    // loadCourse() and now as "playing" (it wasn't — the player
    // is paused until the user hits play).
    lastTickMs.set(0L)
    positionUpdateJob = viewModelScope.launch {
        while (isActive) {
            if (_isVideoMode.value) {
                videoPlayer?.let { player ->
                    val position = player.currentPosition
                    val duration = player.duration.coerceAtLeast(0)
                    audioPlayer.updatePositionFromExternal(position, duration, player.isPlaying)
                    if (singleSubtitleIndex >= 0 && position >= singleSubtitleEndMs - 500) {
                        pauseVideo()
                        android.util.Log.d("PracticeViewModel", "Video single play ended: position=$position, en
dMs=$singleSubtitleEndMs")
                        isSinglePlayMode = false
                    }
                }
            } else {
                audioPlayer.updatePosition()
                if (singleSubtitleIndex >= 0) {
                    val currentPos = audioPlayer.getCurrentPosition()
                    val threshold = singleSubtitleEndMs - 500
                    // § 12.36: diagnostic log on every gate check
                    // while single-play is active. If the
                    // "continues past subtitle end" symptom
                    // recurs, this line is what we need in

```

```

        // logcat — it shows the exact
        // currentPos / endMs / singleSubtitleIndex
        // at every 50ms tick. Filter logcat with
        // `tag:PracticeViewModel` to capture.
        android.util.Log.d("PracticeViewModel", "single-play gate: currentPos=$currentPos, endMs=$singleSubtitleEndMs, threshold=$threshold, singleSubtitleIndex=$singleSubtitleIndex, isPlaying=${playbackState.value.isPlaying}")

        if (currentPos >= threshold) {
            audioPlayer.pause()
            android.util.Log.d("PracticeViewModel", "Single play ended: currentPos=$currentPos, endMs=$singleSubtitleEndMs")

            singleSubtitleIndex = -1
            isSinglePlayMode = false
        }
    }

    val now = System.currentTimeMillis()

    // §12.21: accumulate playing time. Only add the tick
    // delta when the player is actively producing audio.
    // The 5000ms ceiling guards against pathological
    // deltas (process sleep, debugger break) that would
    // otherwise credit huge chunks of wall-clock time.
    val isPlaying = playbackState.value.isPlaying
    val prevTick = lastTickMs.get()
    if (isPlaying && prevTick > 0L) {
        val delta = now - prevTick
        if (delta in 1L..5_000L) {
            accumulatedPlayMs.addAndGet(delta)
        }
    }
    lastTickMs.set(now)

    // §12.20: periodic auto-save. Throttled by
    // PERIODIC_SAVE_INTERVAL_MS so a crash mid-session
    // loses at most that much progress. saveProgress()
    // consumes accumulatedPlayMs atomically (getAndSet(0))
    // so periodic vs pause-saves never double-count.
    if (now - lastPeriodicSaveTimeMs >= PERIODIC_SAVE_INTERVAL_MS) {
        lastPeriodicSaveTimeMs = now
        saveProgress()
    }
    delay(50)
}

}

fun playSubtitleOnce(subtitle: Subtitle) {
    val index = _subtitles.value.indexOf(subtitle)
    android.util.Log.d("PracticeViewModel", "playSubtitleOnce: subtitle.index=${subtitle.index}, listIndex=$index, total=${_subtitles.value.size}")
    // §12.36: set the single-play fields BEFORE starting playback
    // and bypass play()'s clear-state contract. The previous
    // design (see git history for §12.34) called play() between
    // two field-set blocks: play() would wipe singleSubtitleIndex /
    // singleSubtitleEndMs / isSinglePlayMode to (-1, 0L, false),
    // and the second block would re-arm them. Same-thread analysis
    // showed the gap between play() returning and the re-arm was
    // microseconds — well within a single statement's window — so
    // the position-update loop could not fire during it. The
    // residual "continues past subtitle end" failure that survived
    // §12.34 (user reported 2026-07-04: "多数情况下会自动暂停,有时候会

```

```

        // 出现继续往后播放") suggested the race window was real on some
        // devices — possibly under thread contention that let the
        // coroutine scheduler advance past one statement boundary. To
        // eliminate the race class entirely we now set the fields
        // FIRST and call audioPlayer.play() directly, sidestepping
        // play()'s clear-state contract (which is meant for the
        // continuous-mode play button, not for entering single-play).
        singleSubtitleIndex = index
        singleSubtitleEndMs = subtitle.endTimeMs
        isSinglePlayMode = true
        skipTargetListIndex = -1
        seekToSubtitle(subtitle)
        if (_isVideoMode.value) {
            playVideo()
        } else {
            // Bypass play() — its contract is "exit single-play mode"
            // for the play button, the opposite of what we want here.
            audioPlayer.play()
        }
    }

fun play() {
    if (_isVideoMode.value) {
        playVideo()
    } else {
        audioPlayer.play()
    }
    // §12.34: the play button (ListeningPage's progress-bar /
    // bottom-controls play) is the ONLY way to resume continuous
    // auto-advance from a single-play state. Tapping a sentence
    // row calls playSubtitleOnce() (which re-arms the flag after
    // this call returns); tapping prev/next also clears the flag
    // in its own skip flow. So the play button is the canonical
    // "exit single-play" gesture.
    isSinglePlayMode = false
    singleSubtitleIndex = -1
    singleSubtitleEndMs = 0L
}

fun pause() {
    if (_isVideoMode.value) {
        pauseVideo()
    } else {
        audioPlayer.pause()
    }
    singleSubtitleIndex = -1
    skipTargetListIndex = -1
    saveProgress()
}

fun seekTo(positionMs: Long) {
    if (_isVideoMode.value) {
        seekVideoTo(positionMs)
    } else {
        audioPlayer.seekTo(positionMs)
    }
}

fun seekToSubtitle(subtitle: Subtitle) {
    android.util.Log.d("PracticeViewModel", "seekToSubtitle: index=${subtitle.index}, startTime=${subtitle.start
    TimeMs}")
    if (_isVideoMode.value) {
        seekVideoTo(subtitle.startTimeMs)
    }
}

```

```

    } else {
        audioPlayer.seekTo(subtitle.startTimeMs)
    }
    currentSubtitleObj = subtitle
    currentSentenceId = subtitle.index
    setCurrentSubtitleIndex(_subtitles.value.indexOf(subtitle))
}

fun setPlaybackSpeed(speed: Float) {
    audioPlayer.setPlaybackSpeed(speed)
    videoPlayer?.setPlaybackSpeed(speed)
}

fun toggleLooping() {
    val newLooping = !playbackState.value.isLooping
    audioPlayer.setLooping(newLooping)
    videoPlayer?.repeatMode = if (newLooping) Player.REPEAT_MODE_ONE else Player.REPEAT_MODE_OFF
}

fun seekBackward(ms: Long = 5000) {
    val currentPos = if (_isVideoMode.value) getVideoCurrentPosition() else audioPlayer.getCurrentPosition()
    val newPosition = (currentPos - ms).coerceAtLeast(0)
    seekTo(newPosition)
}

fun seekForward(ms: Long = 5000) {
    val currentPos = if (_isVideoMode.value) getVideoCurrentPosition() else audioPlayer.getCurrentPosition()
    val newPosition = (currentPos + ms).coerceAtLeast(0)
    seekTo(newPosition)
}

fun skipToSubtitle(index: Int? = null) {
    // (2026-07-05) Bug fix: see skipToPreviousSubtitle /
    // skipToNextSubtitle — re-arm single-play for the target
    // sentence. Without this, jumping to a sentence via the
    // dropdown menu would also fall into continuous-play and
    // run through every following subtitle.
    val targetIndex = index ?: currentSubtitleIndex.value
    if (targetIndex >= 0 && targetIndex < _subtitles.value.size) {
        val targetSubtitle = _subtitles.value[targetIndex]
        currentSubtitleObj = targetSubtitle
        currentSentenceId = targetSubtitle.index
        setCurrentSubtitleIndex(targetIndex)
        singleSubtitleIndex = targetIndex
        singleSubtitleEndMs = targetSubtitle.endTimeMs
        isSinglePlayMode = true
        skipTargetListIndex = targetIndex
        skipTargetTimeoutMs = System.currentTimeMillis() + 3000
        seekTo(targetSubtitle.startTimeMs)
    }
}

fun skipToPreviousSubtitle() {
    // (2026-07-05) Bug fix: the skip-* paths used to clear
    // single-play state and never re-arm it for the new
    // sentence, so after a skip the player fell into normal
    // continuous-play mode and ran through every following
    // sentence. Now re-arm single-play for the target
    // sentence so playback stops at its endTimeMs.
    val currentIndex = currentSubtitleIndex.value
    val newIndex = if (currentIndex > 0) currentIndex - 1 else 0
    if (_subtitles.value.isNotEmpty()) {
        val targetSubtitle = _subtitles.value[newIndex]
    }
}

```



```

        android.util.Log.d("PracticeViewModel", "skipToPreviousSubtitle: currentIndex=$currentIndex, newIndex=$newIndex, targetSubtitleIndex=${targetSubtitle.index}")
        currentSubtitleObj = targetSubtitle
        currentSentenceId = targetSubtitle.index
        setCurrentSubtitleIndex(newIndex)
        singleSubtitleIndex = newIndex
        singleSubtitleEndMs = targetSubtitle.endTimeMs
        isSinglePlayMode = true
        skipTargetListIndex = newIndex
        skipTargetTimeoutMs = System.currentTimeMillis() + 3000
        seekTo(targetSubtitle.startTimeMs)
    }
}

fun skipToNextSubtitle() {
    // (2026-07-05) Bug fix: see skipToPreviousSubtitle —
    // re-arm single-play for the target sentence so playback
    // stops at its endTimeMs instead of rolling into the next
    // subtitle. Without this, clicking "下一句" during
    // playback of A would jump to B then keep going through
    // C, D, E...
    val currentIndex = currentSubtitleIndex.value
    val newIndex = if (currentIndex < _subtitles.value.size - 1) currentIndex + 1 else _subtitles.value.size - 1
    if (_subtitles.value.isNotEmpty()) {
        val targetSubtitle = _subtitles.value[newIndex.coerceAtLeast(0)]
        android.util.Log.d("PracticeViewModel", "skipToNextSubtitle: currentIndex=$currentIndex, newIndex=$newIndex, targetSubtitleIndex=${targetSubtitle.index}")
        currentSubtitleObj = targetSubtitle
        currentSentenceId = targetSubtitle.index
        setCurrentSubtitleIndex(newIndex)
        singleSubtitleIndex = newIndex
        singleSubtitleEndMs = targetSubtitle.endTimeMs
        isSinglePlayMode = true
        skipTargetListIndex = newIndex
        skipTargetTimeoutMs = System.currentTimeMillis() + 3000
        seekTo(targetSubtitle.startTimeMs)
    }
}

fun saveWord(
    word: String,
    translation: String,
    exampleSentence: String,
    phonetic: String = "",
    pos: String = "",
) {
    viewModelScope.launch {
        val newWord = Word(
            word = word,
            phonetic = phonetic,
            pos = pos,
            translation = translation,
            exampleSentence = exampleSentence,
            sourceCourseId = currentCourseId,
            sourceSentenceId = currentSentenceId,
            collectedAt = System.currentTimeMillis(),
            nextReviewTime = System.currentTimeMillis() + 86400000
        )
        saveWordUseCase(newWord)
    }
}

/**

```

```

* Look up a single word in the bundled local dictionary
* (CET-6 / Gaokao list). The result is published to
* [wordTranslation] as a [WordTranslationState] flow. If a previous
* request is still in flight it is cancelled — only the latest
* long-press wins.
*
* The practice flow is local-only: there is no network translation
* fallback. If the word is not in the bundled list, we surface a
* [WordTranslationState.Failed] with a "未在本词典中找到本词"
* message — the user can still type a translation manually in the
* dialog and save.
*/
fun requestWordTranslation(word: String) {
    val clean = word.trim()
    if (clean.isEmpty()) return
    wordTranslationJob?.cancel()
    _wordTranslation.value = WordTranslationState.Loading(clean)
    wordTranslationJob = viewModelScope.launch {
        try {
            val local = lookupWordUseCase(clean)
            if (local != null) {
                _wordTranslation.value = WordTranslationState.Loaded(
                    word = clean,
                    translation = local.translation,
                    phonetic = local.phonetic,
                    pos = local.pos,
                )
            } else {
                _wordTranslation.value = WordTranslationState.Failed(
                    clean,
                    "未在本词典中找到本词，请手动输入翻译",
                )
            }
        } catch (e: Exception) {
            android.util.Log.e("PracticeViewModel", "Local dictionary exception", e)
            _wordTranslation.value = WordTranslationState.Failed(
                clean,
                "翻译异常: ${e.message ?: e.javaClass.simpleName}",
            )
        }
    }
}

/** Reset the translation state — call when the dialog dismisses. */
fun clearWordTranslation() {
    wordTranslationJob?.cancel()
    _wordTranslation.value = WordTranslationState.Idle
}

// Voice recording functions for 跟读 (shadowing)
fun startRecording(): Boolean {
    stopPlayingRecording()
    return try {
        voiceRecorder.startRecording()
        true
    } catch (e: Exception) {
        false
    }
}

fun pauseRecording() {
    voiceRecorder.pauseRecording()
}

```

```

fun resumeRecording() {
    voiceRecorder.resumeRecording()
}

fun stopRecording(): RecordingResult? {
    val result = voiceRecorder.stopRecording()
    result?.let {
        // (2026-06-28) Write to the speaking-page path. Does
        // NOT touch _testRecordingPath — recordings made in
        // the speaking page should not appear in the test
        // page's 回放 button, and vice versa.
        _speakingRecordingPath.value = it.filePath
    }
    return result
}

fun cancelRecording() {
    voiceRecorder.cancelRecording()
}

// ----- STT word-match testing -----
//
// v2 flow (Vosk-based, replaces the system SpeechRecognizer that
// the previous version used). Why the change:
// - System SpeechRecognizer + MediaRecorder cannot share the mic
//   (Android audio HAL allows only one consumer per stream), so
//   any attempt to give the user both a recording file AND a
//   transcription failed with one side getting silence.
// - Vosk is offline, accepts a WAV file, and gives us text from
//   the exact audio we recorded. So we record with WavRecorder
//   (AudioRecord → 16 kHz mono 16-bit PCM WAV file) and then
//   run Vosk on that file after the user releases the mic.
// - The recording file is preserved for playback (sets
//   _recordingPath so the existing 回放 button works), AND Vosk
//   gives us text → "显示文字" button works too.
//
// State machine:
// Idle → (press) → Listening → (release) → Transcribing(path)
//       → (vosk done) → Transcribed(text)
//       → (submit) → Passed / Failed
//       → (重录) → Idle

/**
 * Start recording via WavRecorder. Kicks off the Vosk model
 * download in the background (no-op if already present) so the
 * first transcribe is fast on subsequent uses.
 */
fun startStt() {
    if (_sttTestState.value is SttTestState.Listening) return
    sttPressStartTimeMs = System.currentTimeMillis()
    sttWaitJob?.cancel()
    sttWaitJob = null
    sttResult = null
    cancelSttEventCollection()
    _sttTestState.value = SttTestState.Listening(0L)

    // Start the recording. WavRecorder writes a 16 kHz mono WAV
    // file under cacheDir/recordings/recording_<ts>.wav.
    try {
        val path = wavRecorder.start()
        android.util.Log.d("SttTest", "startStt: wavRecorder started at $path")
    } catch (e: Throwable) {

```

```

        android.util.Log.e("SttTest", "startStt: wavRecorder.start failed", e)
        // Couldn't start recording — bounce back to Idle.
        _sttTestState.value = SttTestState.Idle
        return
    }

    // Kick off model download in background. If the model is
    // already present this short-circuits to Ready in a few ms.
    // If the user is on a slow connection on first use, the
    // download may still be in flight when they release the mic
    // — in that case stopStt() will await the download before
    // transcribing.
    if (!modelManager.isModelReady()) {
        viewModelScope.launch {
            modelManager.ensureModelReady()
        }
    }

    startSttTimers()
}

/**
 * Stop recording. Called by UI onRelease of mic button.
 *
 * Flow:
 * 1. If the user released too quickly, treat as accidental and
 *    bounce back to Idle (matches the previous v1 behavior so
 *    "I brushed the mic" doesn't pollute the editor).
 * 2. Otherwise, stop the WavRecorder → we have a .wav file.
 *    Publish its path to _recordingPath so the 回放 button works.
 * 3. Transition to Transcribing(path) and run Vosk in a
 *    coroutine. Vosk may block briefly on first use while
 *    ModelManager downloads the model. On any failure we fall
 *    back to Transcribed("") — the user can still type manually.
 */
fun stopStt() {
    if (_sttTestState.value !is SttTestState.Listening) return
    val holdMs = System.currentTimeMillis() - sttPressStartTimeMs
    if (holdMs < STT_MIN_HOLD_MS) {
        // Accidental press — discard the recording, return to Idle.
        try { wavRecorder.cancel() } catch (_: Throwable) {}
        sttWaitJob?.cancel()
        sttWaitJob = null
        stopSttElapsedTimer()
        _sttTestState.value = SttTestState.Idle
        _sttAmplitudeBars.value = List(5) { 0.4f }
        sttResult = null
        return
    }
    // Stop the recorder and capture the file path.
    val rec = try {
        wavRecorder.stop()
    } catch (e: Throwable) {
        android.util.Log.e("SttTest", "stopStt: wavRecorder.stop failed", e)
        null
    }
    stopSttElapsedTimer()
    val path = rec?.filePath
    if (path == null) {
        // Recorder didn't yield a file — treat as no input.
        _sttTestState.value = SttTestState.Idle
        return
    }
}

```

```

        // Publish the file for playback (回放 button reads this).
        // (2026-06-28) Write to the test-page path. Does NOT touch
        // _speakingRecordingPath — STT recordings made in the test
        // page should not appear in the speaking page's "我的录音"
        // playback card, and vice versa.
        _testRecordingPath.value = path

        // Show a "正在识别" intermediate state, then run Vosk.
        _sttTestState.value = SttTestState.Transcribing(path)
        sttWaitJob?.cancel()
        sttWaitJob = viewModelScope.launch {
            // Ensure model is downloaded before transcribing. This
            // may block on first use for ~40MB download. Subsequent
            // uses short-circuit in milliseconds.
            val modelReady = modelManager.ensureModelReady()
            if (modelReady.isFailure) {
                android.util.Log.e("SttTest", "stopStt: model not ready, manual entry", modelReady.exceptionOrNull())
            }

            _sttTestState.value = SttTestState.Transcribed("")
            return@launch
        }
        val text = try {
            voskRecognizer.transcribeFile(path).getOrElse {
                android.util.Log.e("SttTest", "stopStt: transcribeFile failed", it)
                // Surface a hint to the user instead of silently
                // returning "". Common cause is the Vosk/JNA
                // native library failing to load — a Toast / log
                // here helps the next debugging round.
                "[识别失败: ${it.javaClass.simpleName}]"
            }
        } catch (e: Throwable) {
            android.util.Log.e("SttTest", "stopStt: transcribeFile threw", e)
            "[识别失败: ${e.javaClass.simpleName}]"
        }
        android.util.Log.d("SttTest", "stopStt: vosk result='$text'")
        _sttTestState.value = SttTestState.Transcribed(text)
    }
}

/**
 * Suspend until [sttResult] is filled by the event collection job,
 * or up to 2s. Returns the result (or "" on timeout). Reads and
 * clears [sttResult] as a side-effect.
 *
 * Safe to call only from [stopStt], which has already signalled
 * end-of-input via [SttRecognizer.stop]. Both the wait loop and
 * the event collector run on the main thread (default for
 * viewModelScope.launch), so the read/write to [sttResult] is
 * serialized — no extra synchronization needed.
 */
private suspend fun waitForSttResult(): String {
    val timeoutMs = 2000L
    val start = System.currentTimeMillis()
    while (sttResult == null && System.currentTimeMillis() - start < timeoutMs) {
        delay(50)
    }
    val result = sttResult ?: ""
    sttResult = null
    return result
}

private fun startSttTimers() {
    val startTime = System.currentTimeMillis()

```

```

        sttElapsedJob = viewModelScope.launch {
            while (isActive && _sttTestState.value is SttTestState.Listening) {
                val elapsed = System.currentTimeMillis() - startTime
                (_sttTestState.value as? SttTestState.Listening)?.let {
                    _sttTestState.value = it.copy(elapsedMs = elapsed)
                }
                _sttAmplitudeBars.value = List(5) { (Math.random().toFloat() * 0.7f + 0.3f) }
                delay(100)
            }
        }
    }

    /**
     * Cancel only the elapsed-time ticker. The event collection job
     * stays alive — we still need to receive the final onResults/
     * onError from SpeechRecognizer after stopListening() is called.
     * The event collection job is cancelled in onSttResults() (when
     * we get the result) or cancelStt() (when the user aborts).
     */
    private fun stopSttElapsedTimer() {
        sttElapsedJob?.cancel()
        sttElapsedJob = null
    }

    private fun cancelSttEventCollection() {
        sttEventCollectionJob?.cancel()
        sttEventCollectionJob = null
    }

    /**
     * Cancel STT (user taps "取消" in overlay, or backs out of the page).
     * v2: stop the WavRecorder if it's running, and discard any
     * in-flight transcribe coroutine. Doesn't touch the model cache.
     */
    fun cancelStt() {
        try { wavRecorder.cancel() } catch (_: Throwable) {}
        sttWaitJob?.cancel()
        sttWaitJob = null
        stopSttElapsedTimer()
        cancelSttEventCollection()
        _sttTestState.value = SttTestState.Idle
        _sttAmplitudeBars.value = List(5) { 0.4f }
        sttResult = null
    }

    /** User submitted the transcription for matching. */
    fun submitTranscription(text: String) {
        val currentTest = _testState.value.testItems.getOrNull(_testState.value.currentTestIndex)
        ?: run {
            _sttTestState.value = SttTestState.Failed(
                transcribed = text,
                original = "",
                origWords = emptyList(),
                transWords = emptyList(),
                // (2026-06-28) Empty match arrays — there's no
                // orig sentence to align against in this branch.
                origMatched = BooleanArray(0),
                transMatched = BooleanArray(0),
                reason = "no_test_item"
            )
            return
        }
        val result = WordMatcher.match(currentTest.contentEn, text)
    }

```

```

        if (result.passed) {
            _sttTestState.value = SttTestState.Passed(text)
            markSentenceTested(currentTest.index, true)
        } else {
            _sttTestState.value = SttTestState.Failed(
                transcribed = text,
                original = currentTest.contentEn,
                origWords = result.origWords,
                transWords = result.transWords,
                // (2026-06-28) Pass the per-word match flags
                // through so WordChipsRow can color each chip
                // independently. See WordMatcher for the
                // position-independent alignment logic.
                origMatched = result.origMatched,
                transMatched = result.transMatched,
                reason = result.reason
            )
        }
    }

    /** Reset to Idle for next attempt. */
    fun resetStt() {
        _sttTestState.value = SttTestState.Idle
        _sttAmplitudeBars.value = List(5) { 0.4f }
    }

    /**
     * (2026-06-28) Plays a recording by file path. The caller
     * supplies the path explicitly instead of the ViewModel
     * reading from a shared field — this is what enforces the
     * per-page isolation (SpeakingPage passes
     * [speakingRecordingPath], TestingPage passes
     * [testRecordingPath], so a speaking-page recording can never
     * be played from the test page's "回放录音" button and vice
     * versa).
     *
     * @param path the file path to play. Null / blank → no-op,
     *             matches the old behavior so the UI can call this
     *             unconditionally from any "播放录音" button.
     */
    fun playRecording(path: String?) {
        val p = path ?: return

        stopPlayingRecording()

        mediaPlayer = MediaPlayer().apply {
            setDataSource(p)
            prepare()
            setOnCompletionListener {
                _isPlayingRecording.value = false
            }
            start()
        }
        _isPlayingRecording.value = true
    }

    fun stopPlayingRecording() {
        mediaPlayer?.apply {
            if (isPlaying) {
                stop()
            }
            release()
        }
    }

```

```

        mediaPlayer = null
        _isPlayingRecording.value = false
    }

    fun isRecordingPlaying(): Boolean = _isPlayingRecording.value

    fun saveProgress() {
        viewModelScope.launch {
            try {
                val currentPos = if (_isVideoMode.value) getVideoCurrentPosition() else audioPlayer.getCurrentPositi
on()

                val duration = if (_isVideoMode.value) getVideoDuration() else audioPlayer.getDuration()

                // §12.21: read-and-reset the playing-time counter
                // atomically. Even if a periodic save (from the
                // position loop) and a pause-save (from the user
                // hitting back / home button) fire at the same time,
                // getAndSet(0) hands the full accumulated window to
                // exactly one of them — no double-count, no mutex
                // needed. (The old design needed a Mutex + wall-clock
                // anchor reset; the atomic counter replaces both.)
                val sessionTimeMs = accumulatedPlayMs.getAndSet(0L)

                val existingProgress = saveProgressUseCase.getProgressByCourseId(currentCourseId)
                val newProgress = LearningProgress(
                    courseId = currentCourseId,
                    currentPositionMs = currentPos,
                    currentSentenceId = currentSentenceId,
                    learnedSentences = existingProgress?.learnedSentences ?: 0,
                    totalLearnTimeMs = (existingProgress?.totalLearnTimeMs ?: 0) + sessionTimeMs,
                    lastLearnTime = System.currentTimeMillis(),
                    finishRate = if (duration > 0) currentPos.toFloat() / duration.toFloat() else 0f
                )
                saveProgressUseCase(newProgress)
                // Reset the per-tick anchor so the next session's
                // first tick doesn't credit any pre-save time.
                lastTickMs.set(0L)
                android.util.Log.d("PracticeViewModel", "Progress saved: courseId=$currentCourseId, pos=$currentPos,
duration=$duration, sessionTime=$sessionTimeMs")
            } catch (e: Exception) {
                android.util.Log.e("PracticeViewModel", "Error saving progress: ${e.message}")
            }
        }
    }

    override fun onCleared() {
        super.onCleared()
        positionUpdateJob?.cancel()
        audioPlayer.release()
        videoPlayer?.release()
        voiceRecorder.release()
        wavRecorder.release()
        voskRecognizer.shutdown()
        stopPlayingRecording()
        sttElapsedJob?.cancel()
        sttEventCollectionJob?.cancel()
        sttWaitJob?.cancel()
        saveProgress()
    }
}

```



```
// === app\src\main\java\com\echoling\app\presentation\viewmodel\StatisticsViewModel.kt ===
```

```
package com.echoling.app.presentation.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.echoling.app.domain.model.LearningProgress
import com.echoling.app.domain.usecase.GetStatisticsUseCase
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import java.util.Calendar
import javax.inject.Inject

data class DailyStatistic(
    val date: String,
    val learnTimeMs: Long,
    val sentencesLearned: Int
)

data class StatisticsUiState(
    val totalLearnTimeMs: Long = 0L,
    val totalCoursesLearned: Int = 0,
    val totalWordsCollected: Int = 0,
    val totalWordsMastered: Int = 0,
    val averageDailyTimeMs: Long = 0L,
    val currentStreak: Int = 0,
    val dailyStats: List<DailyStatistic> = emptyList(),
    /**
     * 30-day activity, day 0 = today. Same shape as [dailyStats] but
     * without "今天"/"昨天" labels — days 2..29 use the weekday
     * short name (Mon, Tue, ...) so the labels fit in the thin bar
     * chart.
     */
    val monthlyStats: List<DailyStatistic> = emptyList(),
    val isLoading: Boolean = true
)

@HiltViewModel
class StatisticsViewModel @Inject constructor(
    private val getStatisticsUseCase: GetStatisticsUseCase
) : ViewModel() {

    private val _uiState = MutableStateFlow(StatisticsUiState())
    val uiState: StateFlow<StatisticsUiState> = _uiState.asStateFlow()

    init {
        loadStatistics()
    }

    private fun loadStatistics() {
        viewModelScope.launch {
            getStatisticsUseCase.getStatisticsFlow().collect { (progressList, words) ->
                val totalLearnTime = progressList.sumOf { it.totalLearnTimeMs }
                // §12.21: count any course that has a progress row as
                // "learned". The old `finishRate >= 1.0f` rule was
                // almost never satisfied (a user rarely plays to the
                // exact last ms), so the counter was stuck at 0 for
                // every user. Combined with the eager-save on
                // loadCourse() (see PracticeViewModel §12.20), the
                // counter now reflects "courses I've started
            }
        }
    }
}
```

```

        // practicing" — which matches the user's intent.
        val coursesLearned = progressList.size
        val wordsCollected = words.size
        val wordsMastered = words.count { it.isMastered }

        // Calculate streak
        val sortedProgress = progressList.sortedByDescending { it.lastLearnTime }
        val streak = calculateStreak(sortedProgress)

        // Calculate daily stats (last 7 days)
        val dailyStats = calculateDailyStats(progressList, days = 7)
        // §12.21: monthly stats (last 30 days) for the wider
        // chart. Same shape as [dailyStats] but a longer
        // window; the screen renders it as a thinner bar
        // chart with day labels every 5th day.
        val monthlyStats = calculateDailyStats(progressList, days = 30, showTodayLabel = false)

        // Calculate average daily time (over the 7-day window)
        val avgDailyTime = if (dailyStats.isNotEmpty()) {
            dailyStats.sumOf { it.learnTimeMs } / dailyStats.size
        } else 0L

        _uiState.value = StatisticsUiState(
            totalLearnTimeMs = totalLearnTime,
            totalCoursesLearned = coursesLearned,
            totalWordsCollected = wordsCollected,
            totalWordsMastered = wordsMastered,
            averageDailyTimeMs = avgDailyTime,
            currentStreak = streak,
            dailyStats = dailyStats,
            monthlyStats = monthlyStats,
            isLoading = false
        )
    }
}

private fun calculateStreak(progressList: List<LearningProgress>): Int {
    if (progressList.isEmpty()) return 0

    val calendar = Calendar.getInstance()
    val today = calendar.timeInMillis

    var streak = 0
    var currentDay = calendar.apply {
        timeInMillis = today
        set(Calendar.HOUR_OF_DAY, 0)
        set(Calendar.MINUTE, 0)
        set(Calendar.SECOND, 0)
        set(Calendar.MILLISECOND, 0)
    }.timeInMillis

    val learnedDays = progressList
        .filter { it.totalLearnTimeMs > 0 }
        .map { it.lastLearnTime }
        .toSet()

    // Check consecutive days
    while (true) {
        val dayStart = Calendar.getInstance().apply {
            timeInMillis = currentDay
        }.timeInMillis
    }
}

```

```

        val dayEnd = dayStart + 86400000 // 24 hours

        if (learnedDays.any { it in dayStart until dayEnd }) {
            streak++
            currentDay -= 86400000
        } else {
            break
        }
    }

    return streak
}

private fun calculateDailyStats(
    progressList: List<LearningProgress>,
    days: Int,
    showTodayLabel: Boolean = true,
): List<DailyStatistic> {
    val stats = mutableListOf<DailyStatistic>()
    val calendar = Calendar.getInstance()

    for (i in (days - 1) downTo 0) {
        val dayCalendar = Calendar.getInstance().apply {
            add(Calendar.DAY_OF_YEAR, -i)
            set(Calendar.HOUR_OF_DAY, 0)
            set(Calendar.MINUTE, 0)
            set(Calendar.SECOND, 0)
            set(Calendar.MILLISECOND, 0)
        }
        val dayStart = dayCalendar.timeInMillis
        val dayEnd = dayStart + 86400000

        val dayProgress = progressList.filter { it.lastLearnTime in dayStart until dayEnd }
        val learnTime = dayProgress.sumOf { it.totalLearnTimeMs }
        val sentencesLearned = dayProgress.sumOf { it.learnedSentences }

        val dayLabel = when {
            showTodayLabel && i == 0 -> "今天"
            showTodayLabel && i == 1 -> "昨天"
            else -> {
                calendar.timeInMillis = dayStart
                calendar.displayName(Calendar.DAY_OF_WEEK, Calendar.SHORT, java.util.Locale.getDefault()) ?:
""
            }
        }

        stats.add(DailyStatistic(dayLabel, learnTime, sentencesLearned))
    }

    return stats
}

// === app\src\main\java\com\echoling\app\presentation\viewmodel\VocabularyViewModel.kt ===

package com.echoling.app.presentation.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelScope
import com.echoling.app.domain.model.Word
import com.echoling.app.domain.usecase.DeleteWordUseCase

```

```

import com.echoling.app.domain.usecase.GetWordsUseCase
import com.echoling.app.player.TtsManager
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch
import javax.inject.Inject

data class VocabularyUiState(
    val words: List<Word> = emptyList(),
    val isLoading: Boolean = true,
    /**
     * Non-null when the user just tapped a speaker button but the
     * device has no TTS engine installed. Screen reads this and shows
     * a snackbar instructing the user to install Google TTS / iFlytek.
     * Cleared by [VocabularyViewModel.consumeTtsUnavailableMessage].
     */
    val ttsUnavailableMessage: String? = null,
)

@HiltViewModel
class VocabularyViewModel @Inject constructor(
    private val getWordsUseCase: GetWordsUseCase,
    private val deleteWordUseCase: DeleteWordUseCase,
    private val ttsManager: TtsManager,
) : ViewModel() {

    private val _uiState = MutableStateFlow(VocabularyUiState())
    val uiState: StateFlow<VocabularyUiState> = _uiState.asStateFlow()

    init {
        loadWords()
    }

    private fun loadWords() {
        viewModelScope.launch {
            getWordsUseCase.getAllWords().collect { words ->
                _uiState.value = _uiState.value.copy(
                    words = words,
                    isLoading = false
                )
            }
        }
    }

    fun deleteWord(word: Word) {
        viewModelScope.launch {
            deleteWordUseCase(word.word)
        }
    }

    /**
     * Speak [word] aloud via the system TTS engine. Side-effect —
     * no DB / no UseCase. Safe to call before TTS init completes;
     * [TtsManager] queues the call internally.
     *
     * If no TTS engine is installed on the device (e.g. CN Xiaomi
     * without Google services), sets
     * [VocabularyUiState.ttsUnavailableMessage] instead of speaking —
     * the screen observes this and shows a snackbar telling the user
     * how to install a TTS engine.
     */
}

```

```

    fun pronounce(word: Word) {
        if (!ttsManager.isAvailable.value) {
            _uiState.value = _uiState.value.copy(
                ttsUnavailableMessage = "设备未安装 TTS 引擎，请到应用商店安装「Google 文字转语音」或「讯飞语音」后
再试"
            )
            return
        }
        ttsManager.speak(word.word)
    }

    /**
     * Clear [VocabularyUiState.ttsUnavailableMessage] after the screen
     * has shown the snackbar, so a subsequent tap re-fires it.
     */
    fun consumeTtsUnavailableMessage() {
        if (_uiState.value.ttsUnavailableMessage != null) {
            _uiState.value = _uiState.value.copy(ttsUnavailableMessage = null)
        }
    }
}

```

```
// === app\src\main\java\com\echoling\app\speech\ModelManager.kt ===
```

```
package com.echoling.app.speech
```

```

import android.content.Context
import android.util.Log
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.withContext
import java.io.File
import java.io.FileOutputStream
import javax.inject.Inject
import javax.inject.Singleton

/**
 * Manages the on-disk Vosk acoustic model.
 *
 * **Source priority** — [ensureModelReady] walks these in order:
 *
 * 1. **Local filesDir.** If [filesDir]/models/vosk-model-small-en-us-0.15/
   already contains a valid model (validated by checking the key
   files), use it. This is the fast path for every launch after the
   first.
 *
 * 2. **Bundled assets.** If the model is missing locally, copy the
   bundled copy from `assets/models/vosk-model-small-en-us-0.15/`
   into `filesDir/`. The bundled copy ships with the APK so users
   don't need internet access at any point.
 *
 * **Why copy from assets to filesDir?** Vosk's `Model` constructor
 * takes a real filesystem directory — Android `assets/` is not a real
 * filesystem path, the files are accessed through AssetManager. So we
 * unpack once into internal storage and feed Vosk that path on
 * subsequent runs.
 *
 * **Where it lives:** `filesDir/models/vosk-model-small-en-us-0.15/`.
 *
 */

```

```

* **Validation:** [isModelPresent] checks for the key files that any
* Vosk model needs — `am/final.mdl` (the acoustic model binary) and
* the `conf/` directory (mfcc config). If either is missing we treat
* the install as corrupt and re-copy from assets.
*
* (2026-07-04) Removed the previous alphacephei.com network fallback
* (was ModelManager.downloadAndExtract). The Vosk model is bundled in
* `assets/models/vosk-model-small-en-us-0.15/` (~68 MB) and shipped
* with every release APK, so the network path is dead code in
* production. Keeping it would have required `android.permission.INTERNET`
* to stay declared in the manifest — which fails the
* 隐私政策 / "用不到的网络权限" review that the Chinese
* 应用商店 (华为 / 小米 / OPPO / vivo / 应用宝) enforce for an
* otherwise fully-offline app. If you ever switch to a lean APK
* strategy that ships the model separately, the download path
* needs to come back along with the INTERNET permission.
*/
@Singleton
class ModelManager @Inject constructor(
    @ApplicationContext private val context: Context
) {
    sealed class DownloadState {
        data object NotStarted : DownloadState()
        data class Deploying(val progress: Float) : DownloadState() // 0..1
        data object Ready : DownloadState()
        data class Failed(val message: String) : DownloadState()
    }

    private val _downloadState = MutableStateFlow<DownloadState>(DownloadState.NotStarted)
    val downloadState: StateFlow<DownloadState> = _downloadState.asStateFlow()

    private val mutex = Any()

    /** Returns the absolute path to the model dir, deploying it if needed. */
    suspend fun ensureModelReady(): Result<String> = withContext(Dispatchers.IO) {
        synchronized(mutex) {
            val modelDir = modelDir()
            if (isModelPresent(modelDir)) {
                _downloadState.value = DownloadState.Ready
                return@withContext Result.success(modelDir.absolutePath)
            }
            // Bundled assets are the only source we use. If they're
            // missing the APK was built wrong — fail loudly so the
            // problem is reported rather than silently retrying a
            // network path that no longer exists.
            if (!hasBundledModel()) {
                val msg = "Vosk model missing from APK assets/models/. " +
                    "Release builds must package the model."
                Log.e(TAG, msg)
                _downloadState.value = DownloadState.Failed(msg)
                return@withContext Result.failure(IllegalStateException(msg))
            }
            try {
                _downloadState.value = DownloadState.Deploying(0f)
                copyFromAssets(modelDir)
                _downloadState.value = DownloadState.Ready
                Result.success(modelDir.absolutePath)
            } catch (e: Exception) {
                Log.e(TAG, "copy from assets failed", e)
                _downloadState.value = DownloadState.Failed(e.message ?: e.javaClass.simpleName)
                Result.failure(e)
            }
        }
    }
}

```

```

    }

    fun isModelReady(): Boolean = isModelPresent(modelDir())

    fun modelDir(): File = File(context.filesDir, "models/vosk-model-small-en-us-0.15")

    private fun isModelPresent(dir: File): Boolean {
        if (!dir.isDirectory) return false
        // Vosk model dir must contain the acoustic model binary and
        // the mfcc config directory. A partial copy fails this check
        // and forces a re-copy from assets.
        val am = File(dir, "am/final.mdl")
        val conf = File(dir, "conf")
        return am.exists() && conf.isDirectory
    }

    private fun hasBundledModel(): Boolean {
        // We only need to know whether assets/models/vosk-model-small-en-us-0.15/
        // was packaged into the APK. Listing assets is cheap enough
        // to do synchronously; copying only happens if needed.
        return try {
            context.assets.list("models/vosk-model-small-en-us-0.15")?.isNotEmpty() == true
        } catch (e: Exception) {
            Log.w(TAG, "assets.list failed", e)
            false
        }
    }

    private fun copyFromAssets(targetDir: File) {
        targetDir.parentFile?.mkdirs()
        // Walk the asset tree depth-first. AssetManager.list() gives
        // us the entries at one level — files vs subdirs are
        // distinguished by re-listing a subdir.
        copyAssetTree("models/vosk-model-small-en-us-0.15", targetDir)
    }

    private fun copyAssetTree(assetPath: String, outDir: File) {
        val entries = context.assets.list(assetPath)
        if (entries.isNullOrEmpty()) {
            // No children — this could be either an empty dir or a file.
            // Probe by trying to open it; if it works, it's a file.
            val outFile = File(outDir, File(assetPath).name)
            outFile.parentFile?.mkdirs()
            context.assets.open(assetPath).use { input ->
                FileOutputStream(outFile).use { output -> input.copyTo(output) }
            }
            return
        }
        outDir.mkdirs()
        for (name in entries) {
            val childAssetPath = "$assetPath/$name"
            val childEntries = context.assets.list(childAssetPath)
            if (childEntries.isNullOrEmpty()) {
                // File — leaf
                val outFile = File(outDir, name)
                outFile.parentFile?.mkdirs()
                context.assets.open(childAssetPath).use { input ->
                    FileOutputStream(outFile).use { output -> input.copyTo(output) }
                }
            } else {
                // Subdirectory — recurse
                copyAssetTree(childAssetPath, File(outDir, name))
            }
        }
    }

```

```

    }
}

/** Reset state — used in tests / for "redownload model" UI. */
fun resetForRedownload() {
    synchronized(mutex) {
        _downloadState.value = DownloadState.NotStarted
        // Don't delete the model dir here — the UI calls
        // ensureModelReady() next, which short-circuits if present.
    }
}

private companion object {
    const val TAG = "ModelManager"
}
}

// === app\src\main\java\com\echoling\app\speech\SttRecognizer.kt ===

package com.echoling.app.speech

import android.content.Context
import android.content.Intent
import android.os.Bundle
import android.speech.RecognitionListener
import android.speech.RecognizerIntent
import android.speech.SpeechRecognizer
import android.util.Log
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.channels.BufferOverflow
import kotlinx.coroutines.flow.MutableSharedFlow
import kotlinx.coroutines.flow.SharedFlow
import kotlinx.coroutines.flow.asSharedFlow
import javax.inject.Inject
import javax.inject.Singleton

/**
 * System SpeechRecognizer wrapper for the testing tab's STT flow.
 *
 * Wraps Android's [SpeechRecognizer] with a [SharedFlow] of [SttEvent] so
 * callers don't need to manage [RecognitionListener] lifecycle. Uses
 * [MutableSharedFlow] with extraBufferCapacity=4 so events survive a brief
 * collector start race.
 *
 * Not thread-safe for concurrent start() calls — callers must serialize
 * (the ViewModel does this via state checks).
 */
@Singleton
class SttRecognizer @Inject constructor(
    @ApplicationContext private val context: Context
) {
    sealed class SttEvent {
        data class PartialResults(val text: String) : SttEvent()
        data class Results(val text: String) : SttEvent()
        data class Error(val code: Int, val message: String) : SttEvent()
    }

    private val _events = MutableSharedFlow<SttEvent>(
        replay = 0,
        extraBufferCapacity = 4,
        onBufferOverflow = BufferOverflow.DROP_OLDEST,
    )

```



```

val events: SharedFlow<SttEvent> = _events.asSharedFlow()

private var recognizer: SpeechRecognizer? = null

fun isAvailable(): Boolean = SpeechRecognizer.isRecognitionAvailable(context)

fun start(language: String = "en-US") {
    // Abandon any previous recognizer — we're starting a fresh session
    // and don't need its result. Don't call stop() here because that
    // would emit a final Results event for a session the caller has
    // already given up on.
    teardown()
    recognizer = SpeechRecognizer.createSpeechRecognizer(context).apply {
        val intent = Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH).apply {
            putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL, RecognizerIntent.LANGUAGE_MODEL_FREE_FORM)
            putExtra(RecognizerIntent.EXTRA_LANGUAGE, language)
            putExtra(RecognizerIntent.EXTRA_PARTIAL_RESULTS, false)
            putExtra(RecognizerIntent.EXTRA_MAX_RESULTS, 1)
        }
        setRecognitionListener(object : RecognitionListener {
            override fun onReadyForSpeech(params: Bundle?) {
                Log.d(TAG, "onReadyForSpeech")
            }
            override fun onBeginningOfSpeech() {
                Log.d(TAG, "onBeginningOfSpeech")
            }
            override fun onRmsChanged(rmsdB: Float) {
                // v2: emit real amplitude for waveform animation
            }
            override fun onBufferReceived(buffer: ByteArray?) {}
            override fun onEndOfSpeech() {
                Log.d(TAG, "onEndOfSpeech")
            }
            override fun onError(error: Int) {
                Log.w(TAG, "onError: $error (this listener attached to recognizer=${this@apply})")
                _events.tryEmit(SttEvent.Error(error, "SpeechRecognizer error code: $error"))
                teardown()
            }
            override fun onResults(results: Bundle?) {
                val text = results
                    ?.getStringArrayList(SpeechRecognizer.RESULTS_RECOGNITION)
                    ?.firstOrNull()
                    .orEmpty()
                Log.d(TAG, "onResults: '$text'")
                _events.tryEmit(SttEvent.Results(text))
                teardown() // emit first, then destroy — so a collector that's
                           // still attached can process the event before teardown
            }
            override fun onPartialResults(partial: Bundle?) {
                // v1: ignore partial results
            }
            override fun onEvent(eventType: Int, params: Bundle?) {}
        })
        startListening(intent)
    }
}

/**
 * Signal end-of-input to the active recognizer. The recognizer will
 * finalize asynchronously and fire either [RecognitionListener.onResults]
 * or [RecognitionListener.onError]; both callbacks call [teardown]
 * which actually destroys the recognizer.
 */

```

```

    * Do NOT call [SpeechRecognizer.destroy] here — doing so synchronously
    * cancels pending callbacks and the result is lost. The previous
    * recognizer is also abandoned (left for GC) in that case, since the
    * caller is choosing to discard it.
    */
fun stop() {
    try {
        recognizer?.stopListening()
    } catch (e: Throwable) {
        Log.w(TAG, "stopListening threw", e)
        // If stopListening itself throws, we still need to release
        // the recognizer and surface an error so the UI can recover.
        _events.tryEmit(
            SttEvent.Error(SpeechRecognizer.ERROR_CLIENT, "stopListening failed: ${e.message}")
        )
        teardown()
    }
}

/**
 * Destroy the recognizer and clear the reference. Safe to call
 * multiple times. Called from the result/error callbacks after the
 * event has been emitted, and from [start] when abandoning a stale
 * recognizer.
 */
private fun teardown() {
    try {
        recognizer?.destroy()
    } catch (e: Throwable) {
        Log.w(TAG, "destroy threw", e)
    }
    recognizer = null
}

private companion object {
    const val TAG = "SttRecognizer"
}
}

```

```
// === app\src\main\java\com\echoling\app\speech\VoiceRecorder.kt ===
```

```

package com.echoling.app.speech

import android.content.Context
import android.media.MediaRecorder
import android.os.Build
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import java.io.File
import javax.inject.Inject
import javax.inject.Singleton

enum class RecordingState {
    IDLE,
    RECORDING,
    PAUSED,
    STOPPED
}

```

```

data class RecordingResult(
    val filePath: String,
    val durationMs: Long
)

sealed class RecordingException(message: String) : Exception(message) {
    class PrepareFailed(message: String) : RecordingException(message)
    class StartFailed(message: String) : RecordingException(message)
    class OutputDirectoryFailed : RecordingException("Failed to create output directory")
}

@Singleton
class VoiceRecorder @Inject constructor(
    @ApplicationContext private val context: Context
) {
    private var mediaRecorder: MediaRecorder? = null
    private var currentFilePath: String? = null
    private var recordingStartTime: Long = 0L

    private val _recordingState = MutableStateFlow(RecordingState.IDLE)
    val recordingState: StateFlow<RecordingState> = _recordingState.asStateFlow()

    private val _amplitude = MutableStateFlow(0)
    val amplitude: StateFlow<Int> = _amplitude.asStateFlow()

    fun startRecording(): String {
        val outputDir = File(context.cacheDir, "recordings")
        if (!outputDir.exists() && !outputDir.mkdirs()) {
            throw RecordingException.OutputDirectoryFailed()
        }

        val outputFile = File(outputDir, "recording_${System.currentTimeMillis()}.m4a")
        currentFilePath = outputFile.absolutePath

        mediaRecorder = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            MediaRecorder(context)
        } else {
            @Suppress("DEPRECATION")
            MediaRecorder()
        }.apply {
            setAudioSource(MediaRecorder.AudioSource.MIC)
            setOutputFormat(MediaRecorder.OutputFormat.MPEG_4)
            setAudioEncoder(MediaRecorder.AudioEncoder.AAC)
            setAudioEncodingBitRate(128000)
            setAudioSamplingRate(44100)
            setOutputFile(currentFilePath)

            try {
                prepare()
                start()
                recordingStartTime = System.currentTimeMillis()
                _recordingState.value = RecordingState.RECORDING
            } catch (e: Exception) {
                release()
                mediaRecorder = null
                throw RecordingException.StartFailed(e.message ?: "Unknown error during recording start")
            }
        }

        return currentFilePath ?: error("File path not set")
    }

    fun pauseRecording() {

```

```
        if (_recordingState.value == RecordingState.RECORDING && Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
            mediaRecorder?.pause()
            _recordingState.value = RecordingState.PAUSED
        }
    }

    fun resumeRecording() {
        if (_recordingState.value == RecordingState.PAUSED && Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
            mediaRecorder?.resume()
            _recordingState.value = RecordingState.RECORDING
        }
    }

    fun stopRecording(): RecordingResult? {
        val filePath = currentFilePath ?: return null
        val duration = System.currentTimeMillis() - recordingStartTime

        try {
            mediaRecorder?.apply {
                stop()
                release()
            }
        } catch (e: Exception) {
            e.printStackTrace()
        }

        mediaRecorder = null
        _recordingState.value = RecordingState.STOPPED
        _amplitude.value = 0

        return RecordingResult(filePath, duration)
    }

    fun cancelRecording() {
        try {
            mediaRecorder?.apply {
                stop()
                release()
            }
        } catch (e: Exception) {
            e.printStackTrace()
        }

        mediaRecorder = null
        currentFilePath?.let { File(it).delete() }
        currentFilePath = null
        _recordingState.value = RecordingState.IDLE
        _amplitude.value = 0
    }

    fun getMaxAmplitude(): Int {
        return try {
            if (_recordingState.value == RecordingState.RECORDING) {
                val amp = mediaRecorder?.maxAmplitude ?: 0
                _amplitude.value = amp
                amp
            } else {
                0
            }
        } catch (e: Exception) {
            0
        }
    }
}
```

```

        fun release() {
            try {
                mediaRecorder?.release()
            } catch (e: Exception) {
                e.printStackTrace()
            }
            mediaRecorder = null
            _recordingState.value = RecordingState.IDLE
        }
    }
}

// === app\src\main\java\com\echoling\app\speech\VoskSpeechRecognizer.kt ===

package com.echoling.app.speech

import android.util.Log
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import org.json.JSONObject
import org.vosk.Model
import org.vosk.Recognizer
import java.io.FileInputStream
import java.nio.ByteBuffer
import java.nio.ByteOrder
import javax.inject.Inject
import javax.inject.Singleton

/**
 * File-based offline STT using Vosk. Takes a 16 kHz mono 16-bit PCM
 * WAV file (as written by [WavRecorder]) and returns the recognized
 * English text.
 *
 * **Why file-based, not live?** The Testing page (测试) needs both
 * a recording file (for playback) AND a transcription. On Android,
 * the system [android.speech.SpeechRecognizer] gives text but writes
 * no file; [android.media.MediaRecorder] writes a file but does no
 * transcription. They share the mic, so they cannot run in parallel.
 * Vosk accepts a file, which lets us keep the recording AND get text
 * from the same audio — no mic conflict, no extra library, no cloud.
 *
 * **Threading:** Model load is expensive (~1 s on first use after
 * install). The [Model] is cached in-process and reused for subsequent
 * transcriptions. Transcription itself is CPU-bound; we run on
 * [Dispatchers.IO] to keep the main thread free.
 *
 * **Format constraints:** Vosk requires 16 kHz mono 16-bit little-endian
 * PCM. The [WavRecorder] writes exactly that. If you ever feed this
 * from a different source (e.g. a file the user picks), validate the
 * WAV header first.
 */
@Singleton
class VoskSpeechRecognizer @Inject constructor(
    @ApplicationContext private val context: android.content.Context,
    private val modelManager: ModelManager,
) {
    @Volatile private var cachedModel: Model? = null
    private val modelLock = Any()

    /**

```

```

* Transcribe the given WAV file. Returns "" if the file is empty
* or no speech was recognized (Vosk returns an empty `text` field
* in that case).
*
* Performs the full pipeline: ensures model is downloaded, loads
* it (cached after first call), reads the WAV file, feeds PCM to
* the recognizer, parses the JSON result.
*/
suspend fun transcribeFile(wavPath: String): Result<String> = withContext(Dispatchers.IO) {
    try {
        val model = getOrLoadModel().getOrNull { return@withContext Result.failure(it) }
        val text = transcribeWith(model, wavPath)
        Result.success(text)
    } catch (e: Throwable) {
        Log.e(TAG, "transcribeFile failed", e)
        Result.failure(e)
    }
}

private fun getOrLoadModel(): Result<Model> {
    cachedModel?.let { return Result.success(it) }
    synchronized(modelLock) {
        cachedModel?.let { return Result.success(it) }
        val ready = modelManager.isModelReady()
        if (!ready) {
            return Result.failure(
                IllegalStateException("Vosk model not downloaded. Call ModelManager.ensureModelReady() first.")
            )
        }
        return try {
            val m = Model(modelManager.modelDir().absolutePath)
            cachedModel = m
            Result.success(m)
        } catch (e: Throwable) {
            Log.e(TAG, "Model load failed", e)
            Result.failure(e)
        }
    }
}

private fun transcribeWith(model: Model, wavPath: String): String {
    // Skip the 44-byte RIFF header — read sample-rate etc. from
    // the header just enough to validate, but we know WavRecorder
    // writes 16kHz mono 16-bit so the recognizer config matches.
    val sampleRate = 16_000f
    val recognizer = Recognizer(model, sampleRate)
    // (2026-07-05) Bug fix: Vosk's recognizer has internal endpoint
    // detection — when it sees ~1s of silence it commits the audio
    // up to that pause as a finished segment and treats anything
    // afterwards as a new segment. The previous code only called
    // `recognizer.finalResult` at EOF, which returns ONLY the
    // trailing un-committed segment's text. A user who said half
    // a sentence, paused, then finished the second half would get
    // back only the second half (the first half was committed into
    // a separate segment that the old code threw away).
    //
    // Fix: every time `acceptWaveForm` returns true (endpoint
    // detected), parse `recognizer.result` for that committed
    // segment's text and accumulate it. Then concatenate with
    // `finalResult` at end for the trailing segment. Spaces
    // separate adjacent segments so the joined transcript reads
    // naturally — Vosk's per-segment text never contains spaces
    // except where the user actually said them.

```

```

val allText = StringBuilder()
try {
    FileInputStream(wavPath).use { fis ->
        val header = ByteArray(44)
        val headerRead = fis.read(header)
        require(headerRead == 44) { "WAV file too short: $wavPath" }
        // Sanity check the header — WavRecorder must produce this.
        val riff = String(header.copyOfRange(0, 4), Charsets.US_ASCII)
        val wave = String(header.copyOfRange(8, 12), Charsets.US_ASCII)
        require(riff == "RIFF" && wave == "WAVE") { "Not a WAV file: $wavPath" }
        val audioFormat = ByteBuffer.wrap(header.copyOfRange(20, 22))
            .order(ByteOrder.LITTLE_ENDIAN).short.toInt()
        val channels = ByteBuffer.wrap(header.copyOfRange(22, 24))
            .order(ByteOrder.LITTLE_ENDIAN).short.toInt()
        val sr = ByteBuffer.wrap(header.copyOfRange(24, 28))
            .order(ByteOrder.LITTLE_ENDIAN).int
        require(audioFormat == 1) { "WAV is not PCM (format=$audioFormat)" }
        require(channels == 1) { "WAV is not mono (channels=$channels)" }
        require(sr == 16_000) { "WAV sample rate is $sr, need 16000" }

        // Stream PCM in 8 KB chunks. Each int16 sample is 2 bytes.
        val chunk = ByteArray(8 * 1024)
        while (true) {
            val n = fis.read(chunk)
            if (n <= 0) break
            val shortLen = n / 2
            val shorts = ShortArray(shortLen)
            val bb = ByteBuffer.wrap(chunk, 0, n).order(ByteOrder.LITTLE_ENDIAN)
            for (i in 0 until shortLen) shorts[i] = bb.short
            if (recognizer.acceptWaveForm(shorts, shortLen)) {
                // Endpoint detected — this segment is now
                // "committed". `recognizer.result` returns
                // its text in JSON form ({ "text": "..."}).
                // Append it so a multi-segment recording
                // (e.g. mid-sentence pause) keeps every
                // segment instead of dropping the earlier
                // ones.
                val partialJson = recognizer.result
                Log.d(TAG, "endpoint partial: $partialJson")
                val partialText = JSONObject(partialJson)
                    .optString("text", "").trim()
                if (partialText.isNotEmpty()) {
                    if (allText.isNotEmpty()) allText.append(' ')
                    allText.append(partialText)
                }
            }
        }
    }
    // Vosk returns final result on getFinalResult() after EOF.
    // This carries the trailing un-committed segment. The
    // committed segments above are already in [allText].
    val finalJson = recognizer.finalResult
    Log.d(TAG, "final: $finalJson")
    val finalText = JSONObject(finalJson).optString("text", "").trim()
    if (finalText.isNotEmpty()) {
        if (allText.isNotEmpty()) allText.append(' ')
        allText.append(finalText)
    }
    return allText.toString().trim()
} finally {
    try { recognizer.close() } catch (_: Throwable) {}
}
}

```

```

/** Release the cached model — call when STT is no longer needed. */
fun shutdown() {
    synchronized(modelLock) {
        try { cachedModel?.close() } catch (_: Throwable) {}
        cachedModel = null
    }
}

private companion object {
    const val TAG = "VoskSpeechRecognizer"
}
}

```

```
// === app\src\main\java\com\echoling\app\speech\WavRecorder.kt ===
```

```

package com.echoling.app.speech

import android.content.Context
import android.media.AudioFormat
import android.media.AudioRecord
import android.media.MediaRecorder
import android.util.Log
import dagger.hilt.android.qualifiers.ApplicationContext
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import java.io.File
import java.io.FileOutputStream
import java.io.RandomAccessFile
import java.nio.ByteBuffer
import java.nio.ByteOrder
import java.util.concurrent.atomic.AtomicBoolean
import javax.inject.Inject
import javax.inject.Singleton

/**
 * 16 kHz mono 16-bit PCM WAV recorder. Built on [AudioRecord] (not
 * [android.media.MediaRecorder]) because Vosk needs raw PCM at a
 * fixed sample rate, and Android's MediaRecorder cannot output raw
 * PCM in a portable way across devices.
 *
 * The two audio backends in this app have distinct roles:
 * - [VoiceRecorder] (m4a/AAC, 44.1 kHz) — used by SpeakingPage
 *   (跟读). Quality > size.
 * - [WavRecorder] (PCM 16 kHz) — used by TestingPage (测试). The
 *   file must be 16 kHz mono 16-bit PCM so Vosk can transcribe it
 *   directly without resampling. File is also used for playback.
 *
 * Recording state is exposed via [recordingState] / [amplitude] for
 * UI binding (similar shape to VoiceRecorder). The actual capture
 * happens on a dedicated background thread; [start] / [stop] are
 * non-blocking.
 *
 * WAV format: 44-byte RIFF header + 16-bit PCM data, little-endian.
 * Standard for Vosk / most STT engines.
 */
@Singleton
class WavRecorder @Inject constructor(
    @ApplicationContext private val context: Context
) {

```



```

private val _recordingState = MutableStateFlow(RecordingState.IDLE)
val recordingState: StateFlow<RecordingState> = _recordingState.asStateFlow()

private val _amplitude = MutableStateFlow(0)
val amplitude: StateFlow<Int> = _amplitude.asStateFlow()

private var audioRecord: AudioRecord? = null
private var captureThread: Thread? = null
private val isCapturing = AtomicBoolean(false)
private var currentFile: File? = null
private var recordingStartTimeMs: Long = 0L
private var totalSamplesWritten: Long = 0L

/**
 * Begin recording into a fresh .wav file under
 * `cacheDir/recordings/recording_<ts>.wav`. Returns the file path.
 */
fun start(): String {
    if (isCapturing.get()) return currentFile?.absolutePath.orEmpty()

    val outputDir = File(context.cacheDir, "recordings")
    if (!outputDir.exists() && !outputDir.mkdirs()) {
        throw IllegalStateException("Failed to create recordings dir")
    }
    val outputFile = File(outputDir, "recording_${System.currentTimeMillis()}.wav")
    currentFile = outputFile
    totalSamplesWritten = 0L
    recordingStartTimeMs = System.currentTimeMillis()

    val minBuffer = AudioRecord.getMinBufferSize(SAMPLE_RATE, CHANNEL_CONFIG, AUDIO_FORMAT)
    if (minBuffer <= 0) {
        throw IllegalStateException("AudioRecord.getMinBufferSize returned $minBuffer")
    }
    // 2x min to avoid underruns on slow devices.
    val bufferSize = minBuffer * 2

    audioRecord = AudioRecord(
        MediaRecorder.AudioSource.MIC,
        SAMPLE_RATE,
        CHANNEL_CONFIG,
        AUDIO_FORMAT,
        bufferSize
    )
    if (audioRecord?.state != AudioRecord.STATE_INITIALIZED) {
        audioRecord?.release()
        audioRecord = null
        throw IllegalStateException("AudioRecord init failed")
    }
    audioRecord?.startRecording()
    isCapturing.set(true)
    _recordingState.value = RecordingState.RECORDING

    captureThread = Thread({ captureLoop(bufferSize) }, "WavRecorder-capture").also {
        it.start()
    }
    return outputFile.absolutePath
}

private fun captureLoop(bufferSize: Int) {
    val out = try {
        FileOutputStream(currentFile)
    } catch (e: Exception) {
        Log.e(TAG, "open output failed", e)
    }
}

```

```

        isCapturing.set(false)
        _recordingState.value = RecordingState.IDLE
        return
    }
    // Reserve 44-byte header; backfill size fields on stop.
    try {
        out.write(ByteArray(HEADER_SIZE))
    } catch (e: Exception) {
        Log.e(TAG, "write header reserve failed", e)
        closeQuietly(out)
        isCapturing.set(false)
        _recordingState.value = RecordingState.IDLE
        return
    }

    val shortBuf = ShortArray(bufferSize / 2)
    val record = audioRecord?.run {
        closeQuietly(out)
        isCapturing.set(false)
        _recordingState.value = RecordingState.IDLE
        return
    }

    while (isCapturing.get()) {
        val read = try {
            record.read(shortBuf, 0, shortBuf.size)
        } catch (e: Exception) {
            Log.e(TAG, "AudioRecord.read failed", e)
            break
        }
        if (read <= 0) continue
        // Track peak amplitude for the in-overlay meter (0..32767).
        var peak = 0
        for (i in 0 until read) {
            val s = shortBuf[i].toInt()
            val abs = if (s < 0) -s else s
            if (abs > peak) peak = abs
        }
        _amplitude.value = peak

        try {
            val byteBuf = ByteBuffer.allocate(read * 2).order(ByteOrder.LITTLE_ENDIAN)
            for (i in 0 until read) byteBuf.putShort(shortBuf[i])
            out.write(byteBuf.array(), 0, read * 2)
            totalSamplesWritten += read
        } catch (e: Exception) {
            Log.e(TAG, "write PCM failed", e)
            break
        }
    }

    closeQuietly(out)
    // Backfill header now that we know the data size.
    currentFile?.let { f -> writeWavHeader(f, totalSamplesWritten) }
}

/** Stop recording. Returns (filePath, durationMs) or null if not recording. */
fun stop(): RecordingResult? {
    if (!isCapturing.get()) return null
    isCapturing.set(false)
    try {
        audioRecord?.stop()
    } catch (e: Exception) {

```

```

        Log.w(TAG, "AudioRecord.stop failed", e)
    }
    try {
        audioRecord?.release()
    } catch (e: Exception) {
        Log.w(TAG, "AudioRecord.release failed", e)
    }
    audioRecord = null
    captureThread?.join(500)
    captureThread = null
    val path = currentFile?.absolutePath
    val duration = System.currentTimeMillis() - recordingStartTimeMs
    _recordingState.value = RecordingState.STOPPED
    _amplitude.value = 0
    if (path == null) return null
    return RecordingResult(path, duration)
}

/** Discard the current recording. */
fun cancel() {
    if (!isCapturing.get() && _recordingState.value != RecordingState.RECORDING) {
        // still clean up any leftover file
        currentFile?.delete()
        currentFile = null
        return
    }
    isCapturing.set(false)
    try { audioRecord?.stop() } catch (_: Exception) {}
    try { audioRecord?.release() } catch (_: Exception) {}
    audioRecord = null
    captureThread?.join(500)
    captureThread = null
    currentFile?.delete()
    currentFile = null
    _recordingState.value = RecordingState.IDLE
    _amplitude.value = 0
}

fun release() {
    cancel()
}

private fun closeQuietly(fos: FileOutputStream) {
    try { fos.close() } catch (_: Exception) {}
}

/**
 * Write a standard 16-bit PCM mono WAV header at the head of [file].
 * [numSamples] is the count of int16 samples (per-channel) in the
 * data section. Writes the header in-place by reopening the file in
 * rw mode — that's why the capture loop reserves the 44 bytes up
 * front instead of writing the header at start.
 */
private fun writeWavHeader(file: File, numSamples: Long) {
    val byteRate = SAMPLE_RATE * CHANNELS * BITS_PER_SAMPLE / 8
    val dataSize = numSamples * CHANNELS * BITS_PER_SAMPLE / 8
    val totalSize = (dataSize + HEADER_SIZE - 8).toInt()

    val header = ByteBuffer.allocate(HEADER_SIZE).order(ByteOrder.LITTLE_ENDIAN)
    header.put("RIFF".toByteArray(Charsets.US_ASCII))
    header.putInt(totalSize)
    header.put("WAVE".toByteArray(Charsets.US_ASCII))
    header.put("fmt ".toByteArray(Charsets.US_ASCII))

```

```

        header.putInt(16) // subchunk1 size (PCM)
        header.putShort(1) // audio format (1 = PCM)
        header.putShort(CHANNELS.toShort())
        header.putInt(SAMPLE_RATE)
        header.putInt(byteRate)
        header.putShort((CHANNELS * BITS_PER_SAMPLE / 8).toShort()) // block align
        header.putShort(BITS_PER_SAMPLE.toShort())
        header.put("data".toByteArray(Charsets.US_ASCII))
        header.putInt(dataSize.toInt())

        try {
            RandomAccessFile(file, "rw").use { raf ->
                raf.seek(0)
                raf.write(header.array(), 0, HEADER_SIZE)
            }
        } catch (e: Exception) {
            Log.e(TAG, "writeWavHeader failed", e)
        }
    }

    private companion object {
        const val TAG = "WavRecorder"
        const val SAMPLE_RATE = 16_000
        const val CHANNELS = 1
        const val BITS_PER_SAMPLE = 16
        const val CHANNEL_CONFIG = AudioFormat.CHANNEL_IN_MONO
        const val AUDIO_FORMAT = AudioFormat.ENCODING_PCM_16BIT
        const val HEADER_SIZE = 44
    }
}

```

```
// === app\src\main\java\com\echoling\app\speech\WordMatcher.kt ===
```

```
package com.echoling.app.speech
```

```

/**
 * Position-independent fuzzy word matcher for STT transcription comparison.
 *
 * Compares an expected (original) sentence against the user's transcription
 * using a greedy "any orig word can match any trans word" strategy with
 * Levenshtein-based fuzzy equivalence, so a single misrecognized word does
 * NOT cascade into marking subsequent correct words as wrong.
 *
 * ## Why position-independent?
 *
 * The previous version of this matcher required (a) exact same word count
 * and (b) exact equality at every position. The user reported that this
 * was too strict in two ways:
 *
 * 1. Subjunctive mood (e.g. "If I were you" vs. "If I was you") and
 *    similar inflection differences were marked as wrong.
 *
 * 2. Once the first word didn't match, every subsequent (correct) word
 *    was ALSO marked wrong because the loop returned early on first
 *    mismatch.
 *
 * This rewrite addresses both:
 *
 * 1. Fuzzy equivalence via Levenshtein distance (threshold scales with
 *    word length) plus a small [COMMON_ALTERNATES] set for cases that
 *    pure edit distance would reject (were□was).
 *
 * 2. Greedy multiset-style alignment: for every orig word, find the
 *    earliest unused trans word that's fuzzy-equal, and mark both as
 *    "matched". An orig word that finds no match is wrong; a trans word

```

```

*      that isn't consumed by any orig is extra. Either way, a wrong
*      word at position i no longer affects the verdict for position j.
*
* No stemming, no synonym matching, no real fuzzy phonetics — the user
* can still edit the transcription in [TranscriptionEditor] if STT
* misrecognized a word.
*/
object WordMatcher {
    data class MatchResult(
        val passed: Boolean,
        val origWords: List<String>,
        val transWords: List<String>,
        /**
         * Per-orig-word match status. `origMatched[i] == true` means
         * `origWords[i]` found an equivalent unused trans word during
         * alignment. Index aligns with [origWords].
         */
        val origMatched: BooleanArray,
        /**
         * Per-trans-word match status. `transMatched[j] == true` means
         * `transWords[j]` was consumed by some orig word. Index aligns
         * with [transWords]. Trans words that are `false` are "extra"
         * (no corresponding orig word).
         */
        val transMatched: BooleanArray,
        /** "ok" | "empty_transcription" | "wrong_word" */
        val reason: String
    ) {
        // BooleanArray doesn't have structural equals / hashCode,
        // so override to keep value semantics on the data class —
        // otherwise a state diff in a StateFlow wouldn't notice
        // when the matcher produces a fresh result with identical
        // contents, leading to spurious UI re-renders.
        override fun equals(other: Any?): Boolean {
            if (this === other) return true
            if (other !is MatchResult) return false
            return passed == other.passed
                && origWords == other.origWords
                && transWords == other.transWords
                && origMatched.contentEquals(other.origMatched)
                && transMatched.contentEquals(other.transMatched)
                && reason == other.reason
        }

        override fun hashCode(): Int {
            var result = passed.hashCode()
            result = 31 * result + origWords.hashCode()
            result = 31 * result + transWords.hashCode()
            result = 31 * result + origMatched.contentHashCode()
            result = 31 * result + transMatched.contentHashCode()
            result = 31 * result + reason.hashCode()
            return result
        }
    }

    private val NORMALIZE_REGEX = Regex("[^a-z0-9\\s']")

    /**
     * Common alternates that aren't close enough under Levenshtein
     * to be considered equivalent, but ARE semantically interchangeable
     * in spoken English. Both directions are checked.
     */
    private val COMMON_ALTERNATES: Set<Pair<String, String>> = setOf(

```

```

    "were" to "was",    // subjunctive "If I were" vs. indicative "If I was"
    "was" to "were",
    "gotten" to "got", // AmE got vs. BrE gotten
    "got" to "gotten",
)

fun match(original: String, transcribed: String): MatchResult {
    val orig = normalize(original)
    val trans = normalize(transcribed)
    if (trans.isEmpty()) {
        return MatchResult(
            passed = false,
            origWords = orig,
            transWords = trans,
            origMatched = BooleanArray(orig.size), // all false
            transMatched = BooleanArray(0),
            reason = "empty_transcription"
        )
    }

    // Greedy alignment: scan origWords in order, for each one find
    // the earliest unused trans word that's fuzzy-equal. Mark both
    // as matched. If we can't find any, the orig word stays
    // unmatched (and will render as red on the orig row).
    val origMatched = BooleanArray(orig.size)
    val transMatched = BooleanArray(trans.size)
    for (i in orig.indices) {
        for (j in trans.indices) {
            if (!transMatched[j] && fuzzyEqual(orig[i], trans[j])) {
                origMatched[i] = true
                transMatched[j] = true
                break
            }
        }
    }

    val passed = origMatched.all { it }
    return MatchResult(
        passed = passed,
        origWords = orig,
        transWords = trans,
        origMatched = origMatched,
        transMatched = transMatched,
        reason = if (passed) "ok" else "wrong_word"
    )
}

/**
 * Two words are considered equivalent if:
 * - they're identical, OR
 * - they appear in [COMMON_ALTERNATES] (either order), OR
 * - their Levenshtein distance is within a length-scaled
 *   threshold (1 edit for short words, up to 2 for longer ones).
 */
private fun fuzzyEqual(a: String, b: String): Boolean {
    if (a == b) return true
    if (a.length < 2 || b.length < 2) return false
    if ((a to b) in COMMON_ALTERNATES) return true
    if ((b to a) in COMMON_ALTERNATES) return true
    val threshold = when (maxOf(a.length, b.length)) {
        in 0..3 -> 0    // "to", "a", "I" — must match exactly
        in 4..5 -> 1    // "were", "have" — tolerate 1 edit
        else -> 2       // "checks", "running" — tolerate 2 edits
    }

```

```

    }
    if (threshold == 0) return false
    return levenshtein(a, b) <= threshold
}

/**
 * Iterative two-row Levenshtein. O(len(a) * len(b)) time, O(b)
 * space. Words in STT output are short (typically < 20 chars) so
 * the O(n²) worst case is fine.
 */
private fun levenshtein(a: String, b: String): Int {
    if (a == b) return 0
    if (a.isEmpty()) return b.length
    if (b.isEmpty()) return a.length
    var prev = IntArray(b.length + 1) { it }
    val curr = IntArray(b.length + 1)
    for (i in 1..a.length) {
        curr[0] = i
        for (j in 1..b.length) {
            val cost = if (a[i - 1] == b[j - 1]) 0 else 1
            curr[j] = minOf(
                curr[j - 1] + 1,      // insertion
                prev[j] + 1,          // deletion
                prev[j - 1] + cost    // substitution
            )
        }
        // Swap rows
        for (k in prev.indices) prev[k] = curr[k]
    }
    return prev[b.length]
}

private fun normalize(s: String): List<String> =
    s.lowercase()
        .replace(NORMALIZE_REGEX, " ")
        .split(Regex("\\s+"))
        .filter { it.isNotEmpty() }
}

```